



Machine Learning Using SAS® Viya®

Slides and Notes

Machine Learning Using SAS® Viya® was developed by Andy Ravenna, Sharad Saxena, Jeff Thompson, and Carlos Pinheiro. Instructional design was provided by Julia Johnson.

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration. Other brand and product names are trademarks of their respective companies.

Machine Learning Using SAS® Viya® Slides and Notes

Content based on SAS® Viya® Release Viya LTS 2025.09

Copyright © 2026 SAS Institute Inc. Cary, NC, USA. All rights reserved. Printed in the United States of America. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, or otherwise, without the prior written permission of the publisher, SAS Institute Inc.

Course code CPML52, prepared date March 30, 2026.

<http://support.sas.com/training/>



For information about other courses in the curriculum, contact the SAS Education Division at 1-800-333-7660, or send e-mail to training@sas.com. You can also find this information on the web at <http://support.sas.com/training/> as well as in the Training Course Catalog.

For a list of SAS books (including e-books) that relate to the topics covered in this course notes, visit <https://www.sas.com/sas/books.html> or call 1-800-727-0025. US customers receive free shipping to US addresses.

Course Overview



Welcome from the Instructors

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Practicing in this Course (Required)

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Lesson 1: Getting Started with Machine Learning and SAS Viya



1.0 Lesson Overview



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



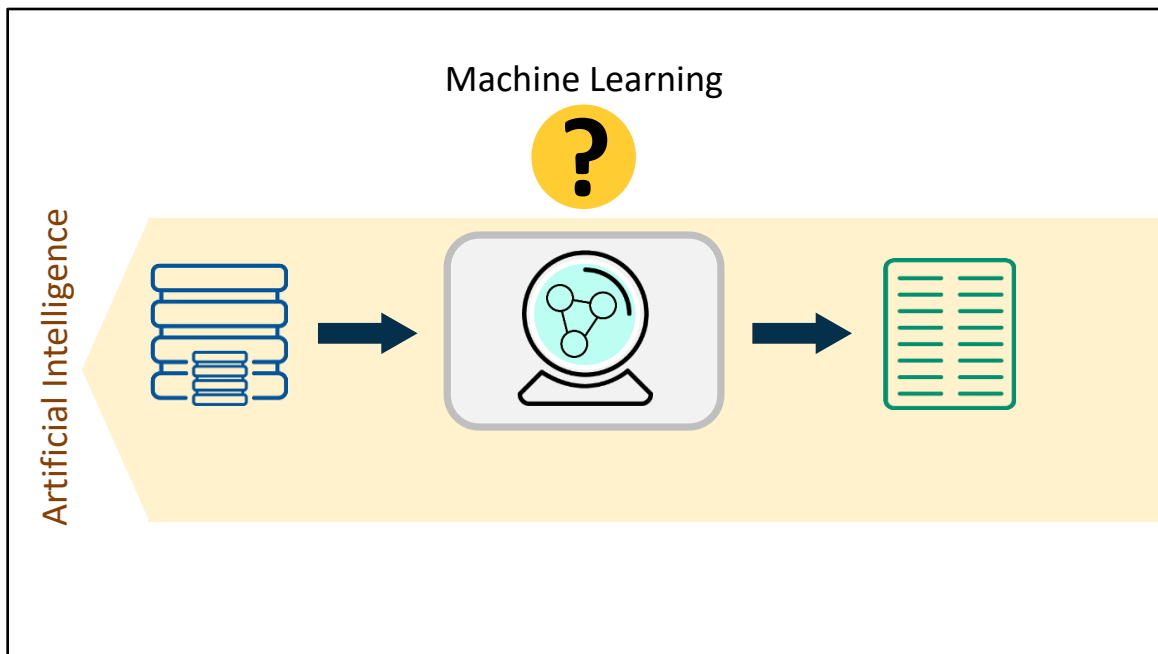
1.1 Machine Learning in Business Decision Making



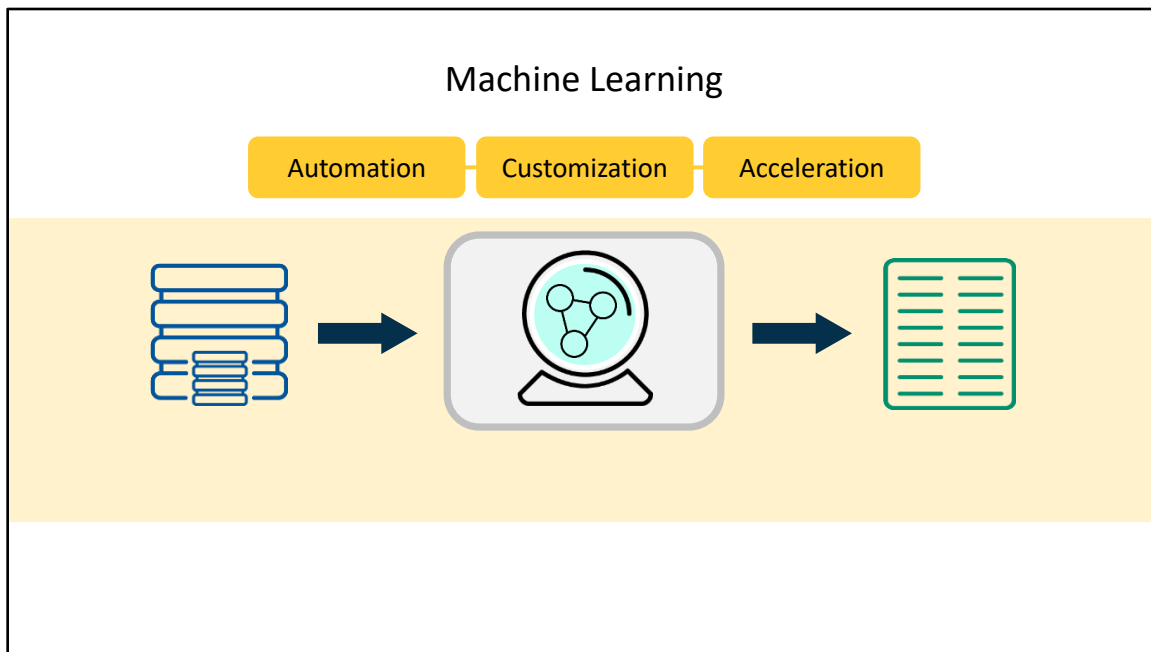
Machine Learning in SAS Viya



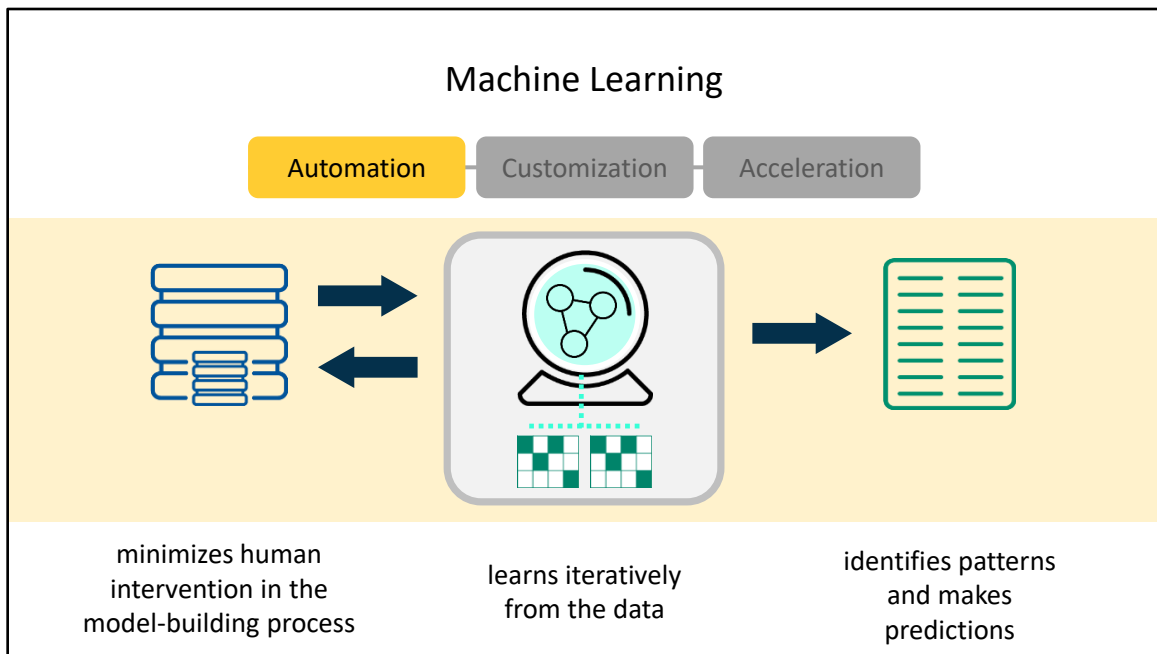
Copyright © SAS Institute Inc. All rights reserved.



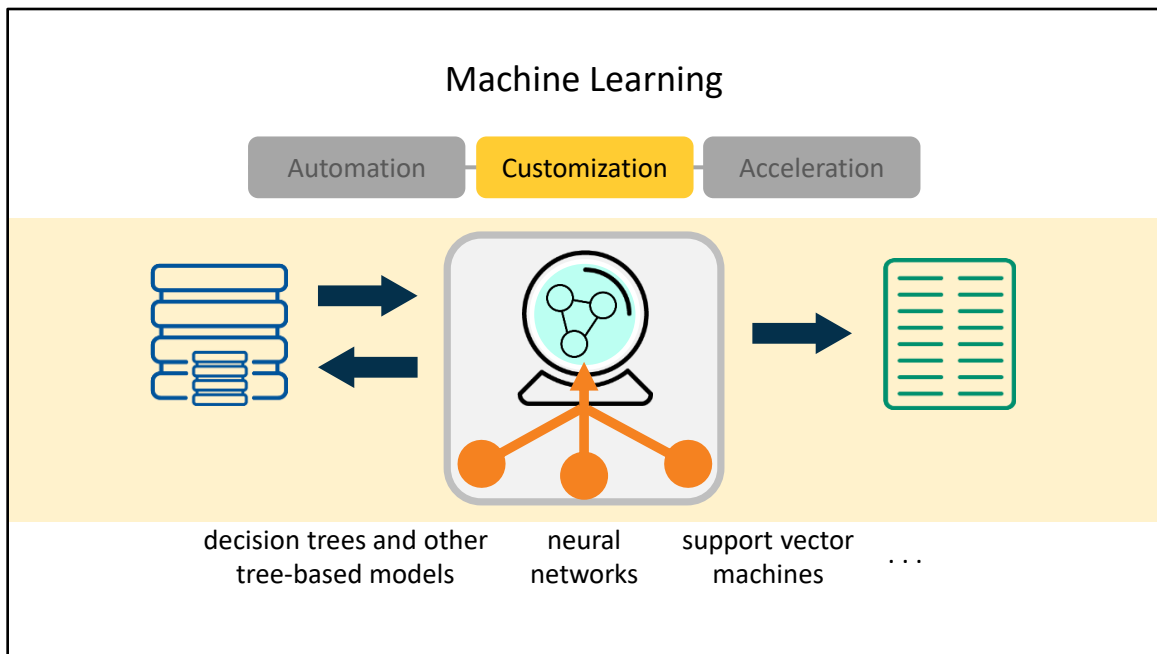
So, what is machine learning? Machine learning is a branch of artificial intelligence that builds systems that learn iteratively from data and make predictions.



Three main characteristics of machine learning are automation, customization, and acceleration.



Machine learning is largely an **automated** process. Model building requires minimal human intervention. Click Run and the system learns iteratively from the data, based on algorithms instead of programming. With each pass through the data, the system identifies patterns and makes predictions, and continues to learn and improve. A neural network is an example of a machine learning model. In a neural network, the weights start close to zero. With each pass through the data, the neural network learns more and refines the weights.



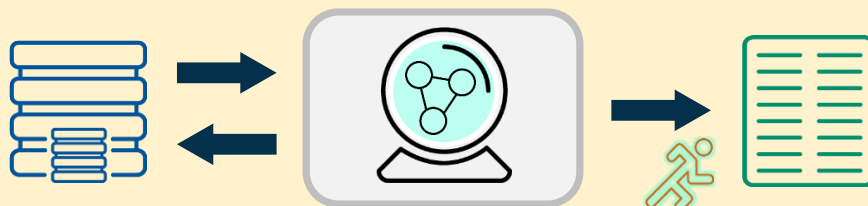
The machine learning process is highly **customized**. Machine learning can draw from many algorithms for training the data, depending on the requirements of the situation. In this course, you learn about several types of machine learning models: decision trees and other tree-based models, neural networks, and support vector machines.

Machine Learning

Automation

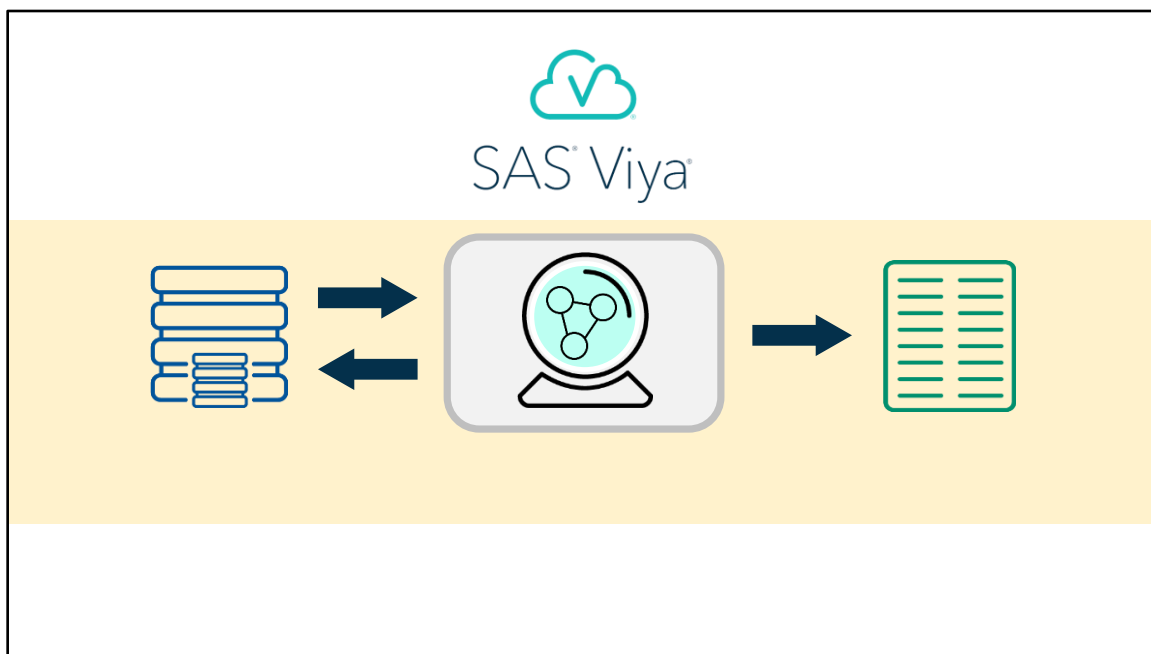
Customization

Acceleration

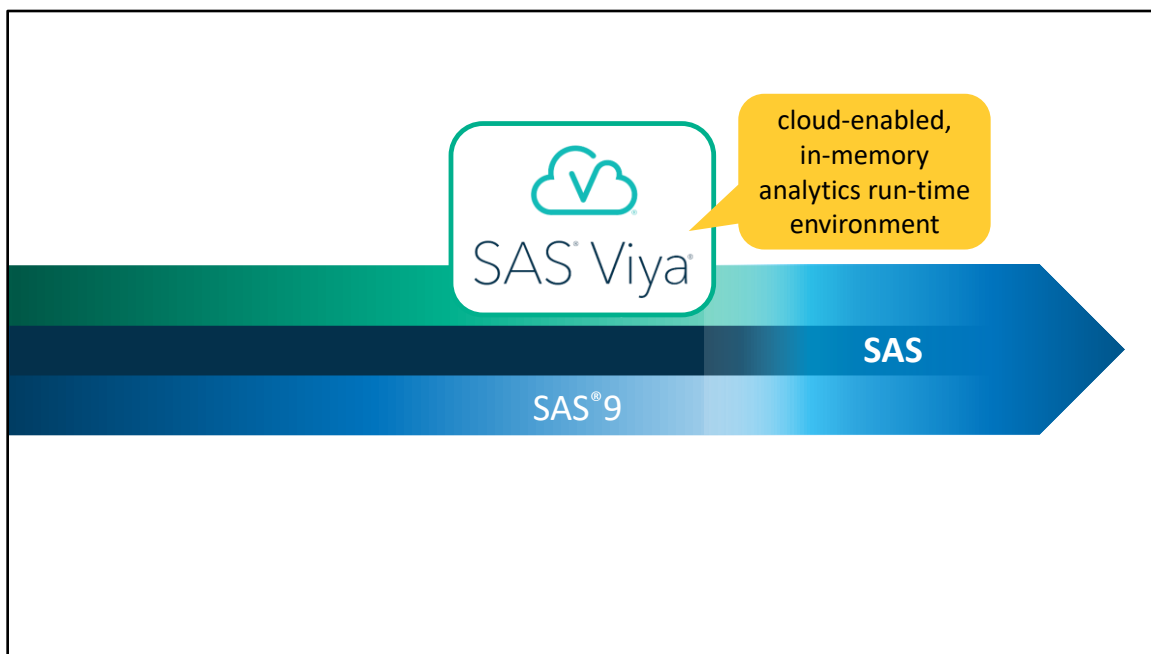


fast response time for sophisticated analytics
applied to data of any size or complexity

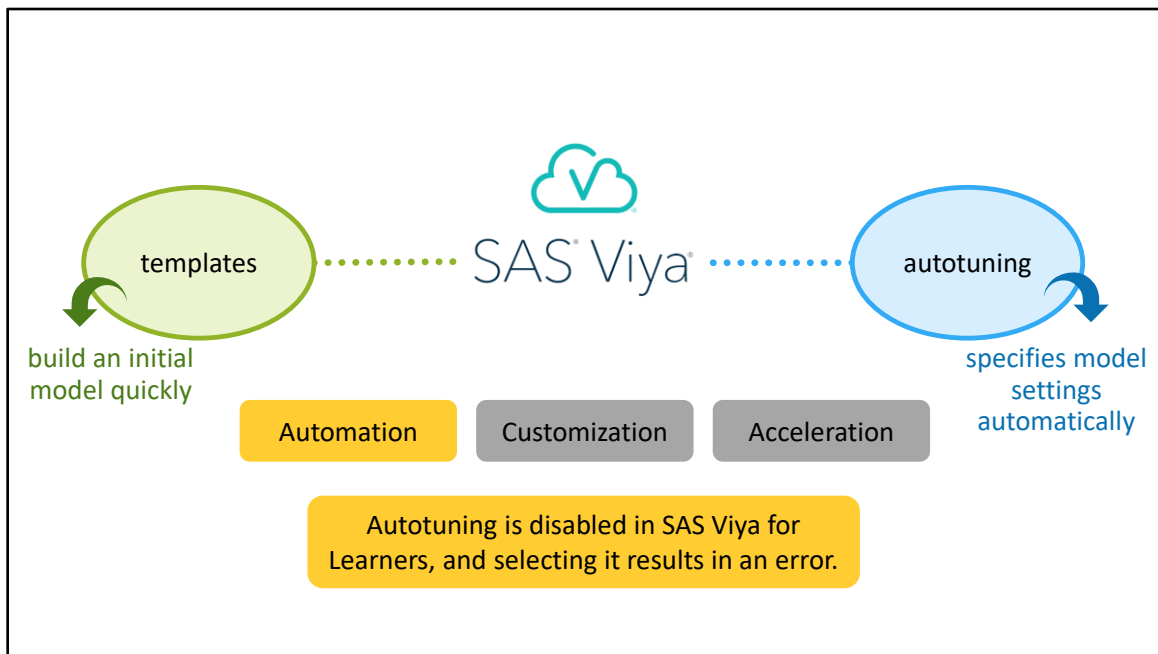
Finally, machine learning **accelerates** the process of running sophisticated analytics on big data so that results are generated quickly.



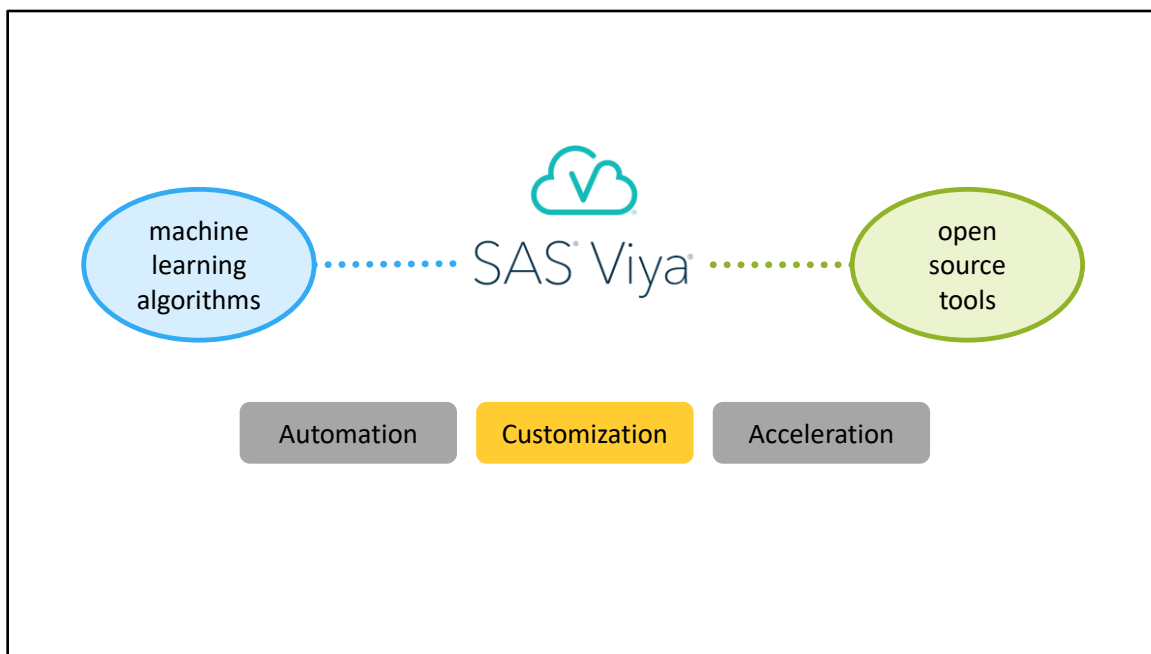
Machine learning shares many approaches with other related fields. SAS Viya offers the tools that you need for machine learning.



SAS Viya is a cloud-enabled, in-memory analytics run-time environment that seamlessly scales for data of any size, type, speed, and complexity.



Templates enable you to build an initial model quickly. The autotuning feature automates the process of specifying model settings. You can then apply your business knowledge and experience to improve on the automated results.



And you can **customize** your machine learning models by combining the state-of-the-art machine learning algorithms in SAS Viya with open source tools.

Analytics Life Cycle



Copyright © SAS Institute Inc. All rights reserved.

Analytics Life Cycle

Analysis Goal

Business
Challenge

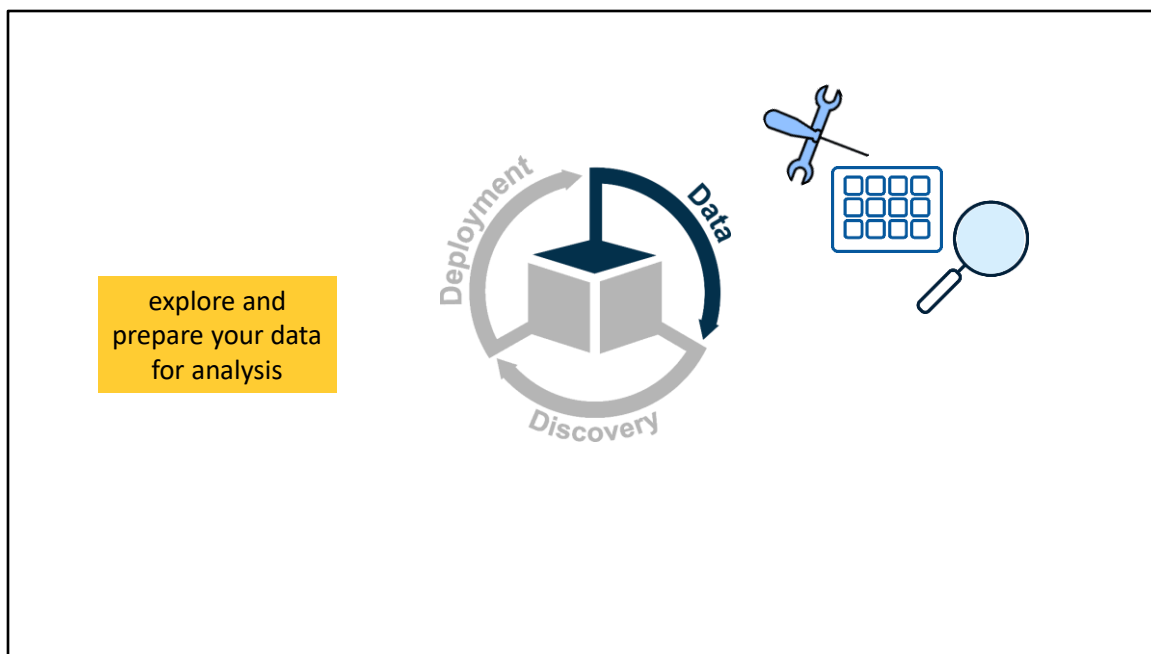
The analytics process always starts with a business challenge. In response, the organization identifies a precise and measurable analysis goal.



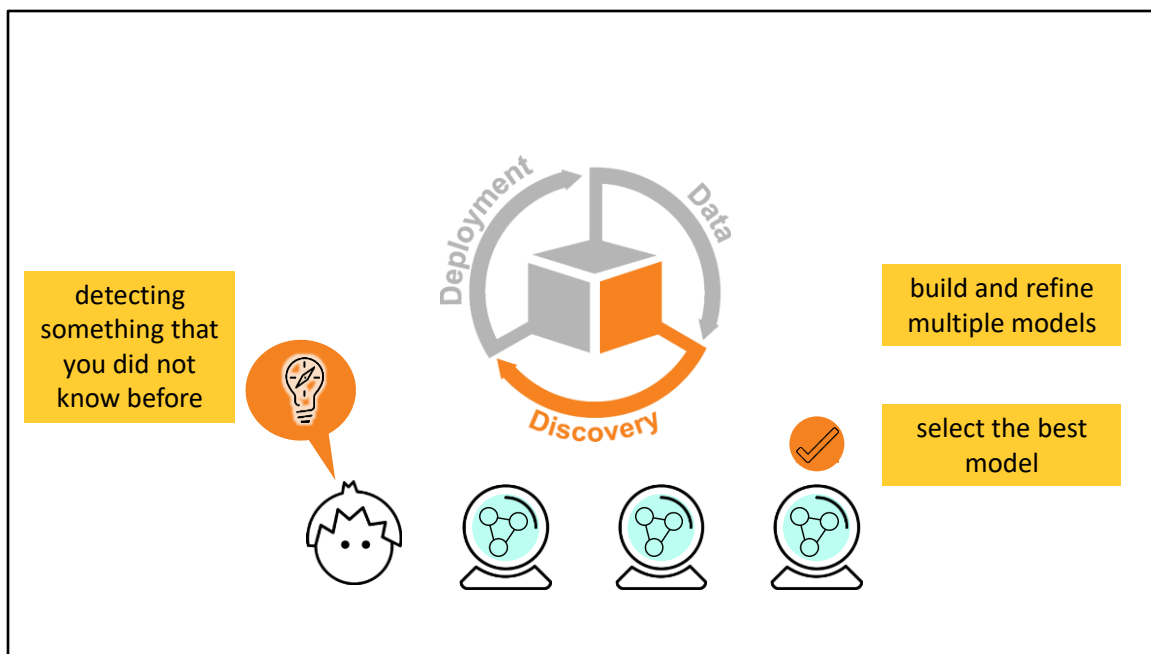
The analytics life cycle represents a series of activities to extract value from raw data.

The analytics life cycle represents a series of activities whose goal is to extract value from raw data. Of course, the definition of “value” is specific to each organization’s goals and objectives.

The analytics life cycle has three phases: Data, Discovery, and Deployment.



Data are the foundation of everything you do. At the Data phase, you explore and prepare your data for analysis.



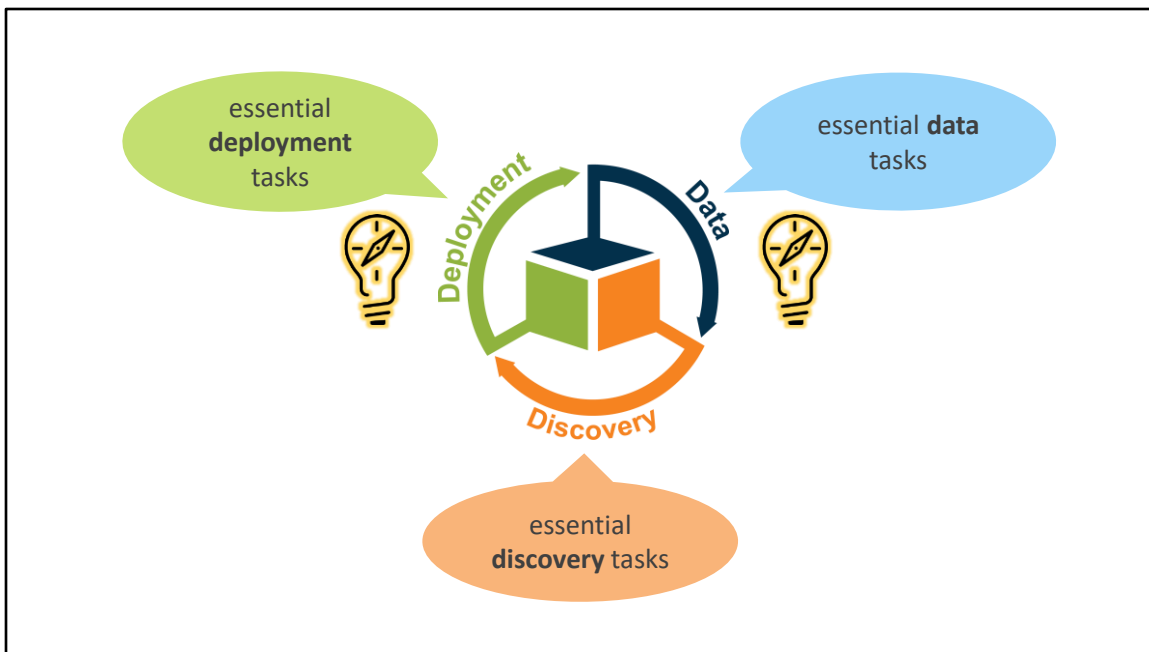
Discovery is the act of detecting something that you did not know before. You build and refine multiple models with the goal of selecting the best model for your analysis.



Scoring



Deployment is where you put the model to work. You apply the model to new data, which is a process called scoring.



In fact, the value of machine learning is evident throughout the analytics life cycle, with actionable insights at each phase.

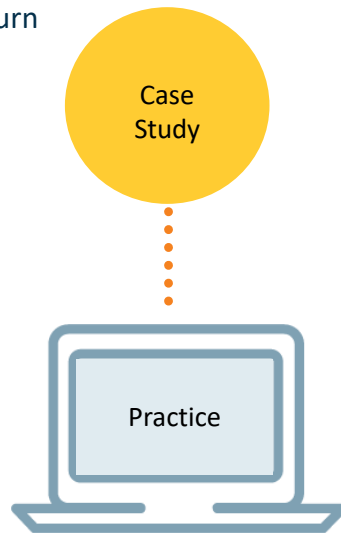
As you create machine learning models with SAS Viya tools in this course, you'll learn to perform the essential tasks that are defined for each phase. It's good to know that the analytics life cycle is flexible. When you apply what you've learned to your own projects in the future, you might do things in a different sequence.

Case Study: Customer Churn

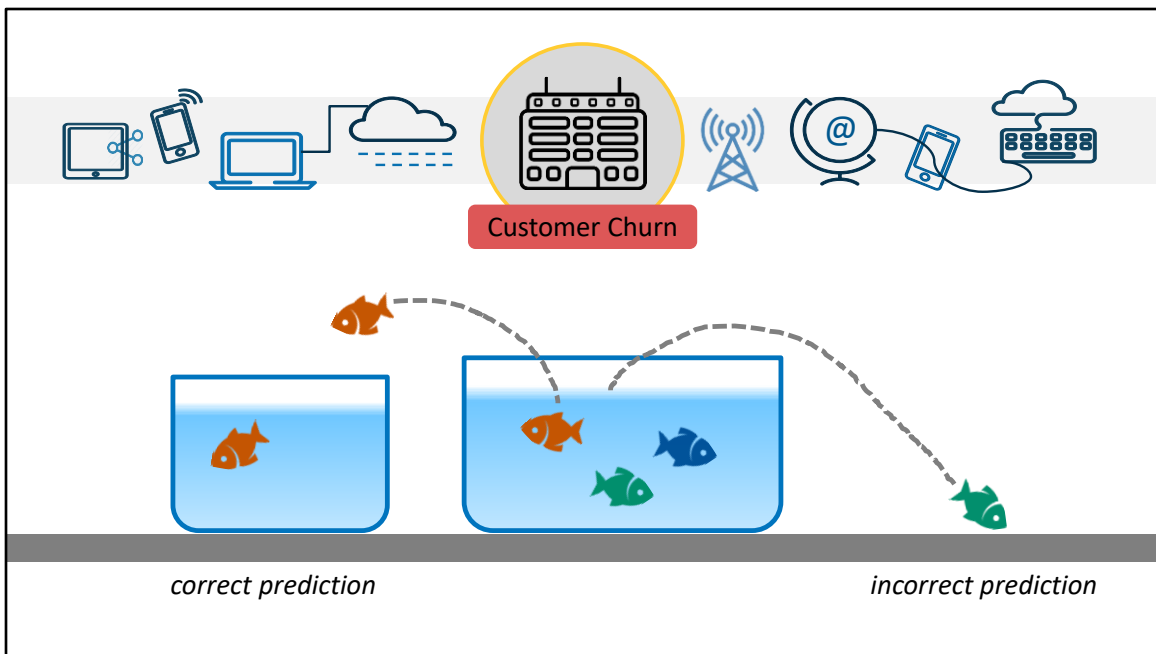


Copyright © SAS Institute Inc. All rights reserved.

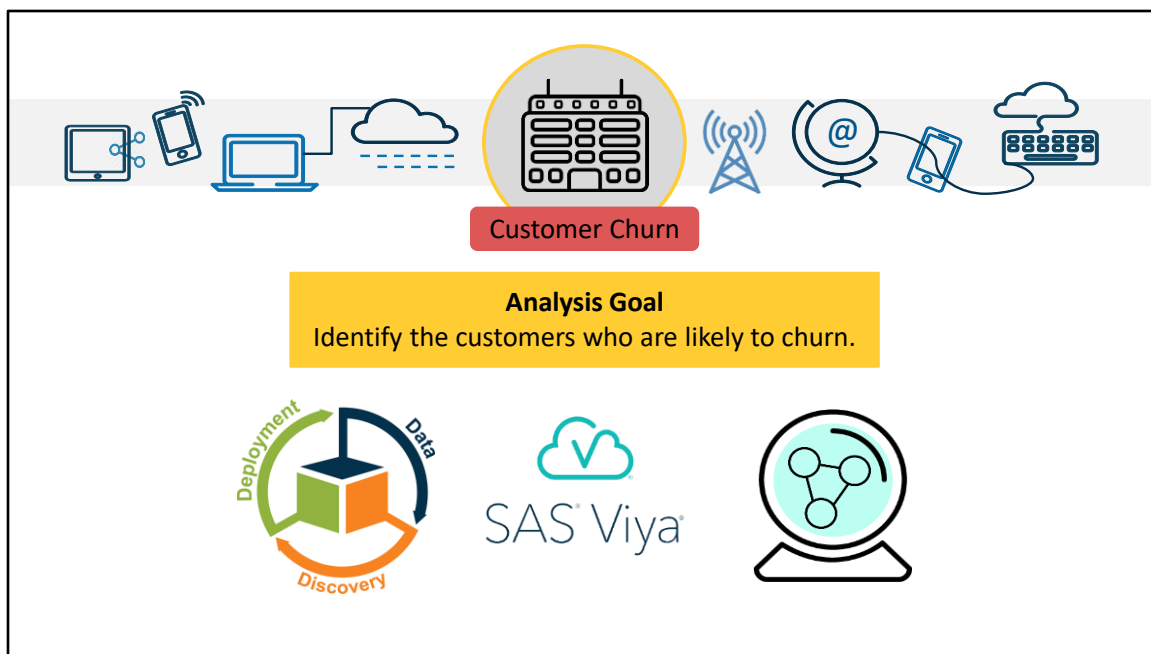
Case Study: Customer Churn



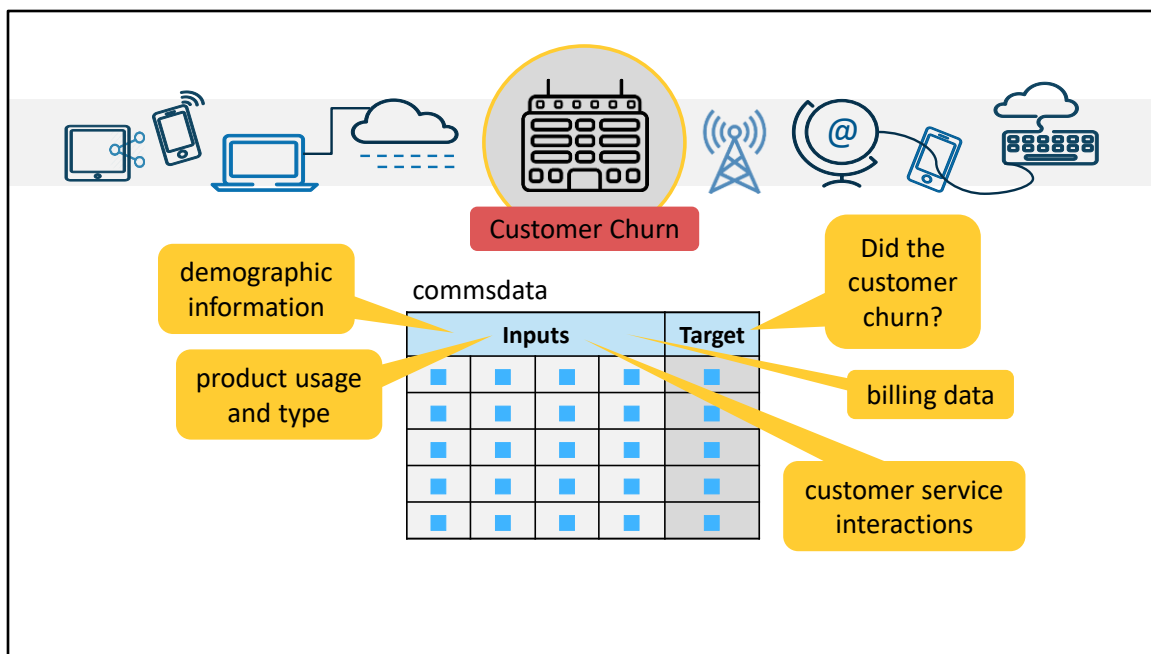
Throughout this course, you will practice what you learn with a case study.



A fictitious telecommunications company has a problem with customer churn. When existing customers or subscribers stop doing business or otherwise end a relationship with a company, they are said to churn, and this is bad for the bottom line. It is more costly to attract new customers than to keep existing customers.



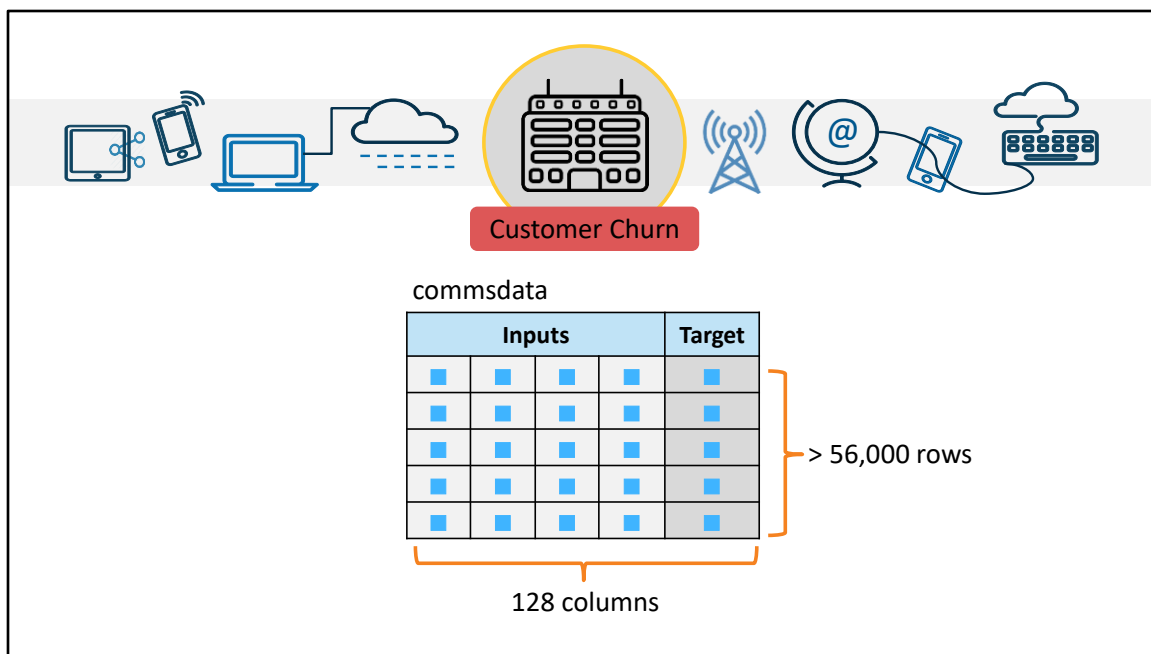
Many companies use analytics to identify customers at risk for churn and to understand how to retain them. Telecommunications providers routinely use predictive models to reduce the churn rate for post-paid subscribers, or customers who have a contract. This is the analysis goal for our case study. The results of your analysis could help management deploy effective retention and loyalty programs.



A little more about your case study: The company has raw data in a SAS table named `commsdata`, that contains information about customer behavior.

The input variables include demographic information, product usage and type, billing data, and interactions with customer service.

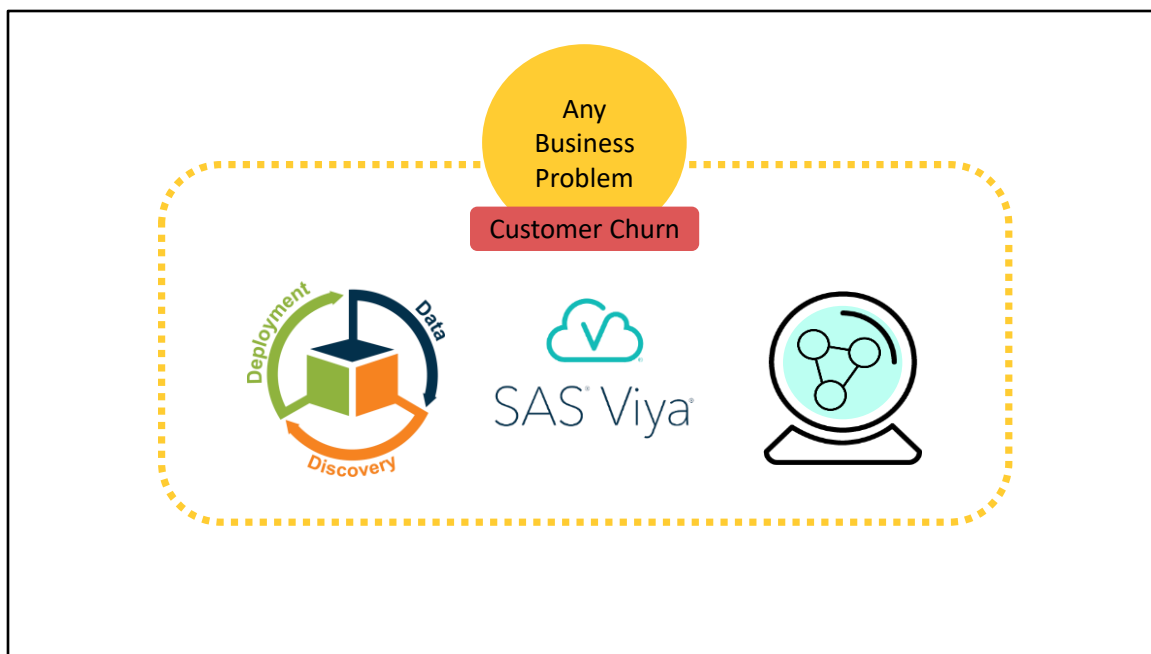
There is also a binary variable that indicates whether customers churned, and this is the target that you are trying to predict.



The commsdata data set contains 128 columns and more than 56,000 rows.

Don't worry if some of the terminology is unfamiliar right now—we'll explain everything as you go through the course.

Your assignment is to use this data set to create supervised models that predict whether customers will churn. Your instructors will use the same data to teach concepts—and your practice is to perform the same analyses as your instructor. Then, you will have some extra practice beyond what the instructor shows you in the demonstration videos.



Although this course uses customer churn to practice with machine learning models, you can apply what you learn here to many other business problems across a vast array of domains.

Applications of Prediction-Based Decision Making

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Case Study: Data Dictionary

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



SAS Viya Applications Menu

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.

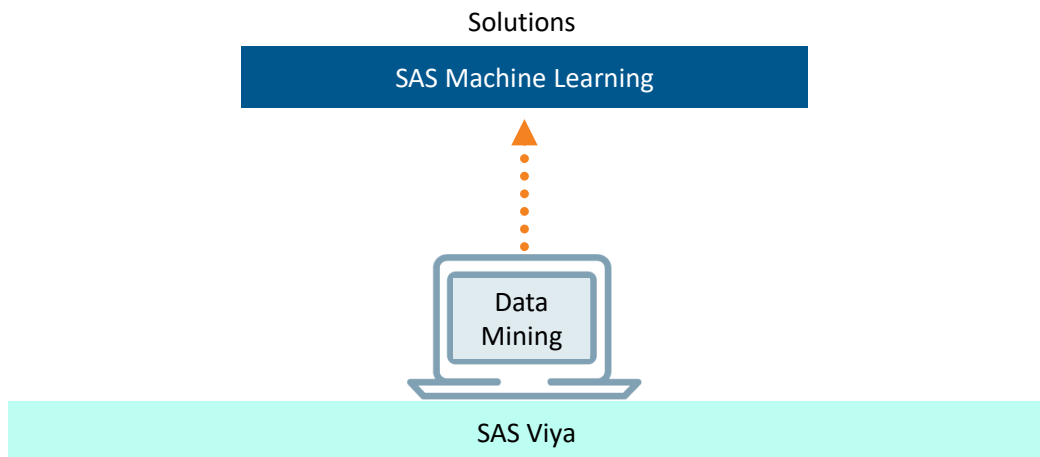


SAS Viya Tools for Machine Learning



Copyright © SAS Institute Inc. All rights reserved.

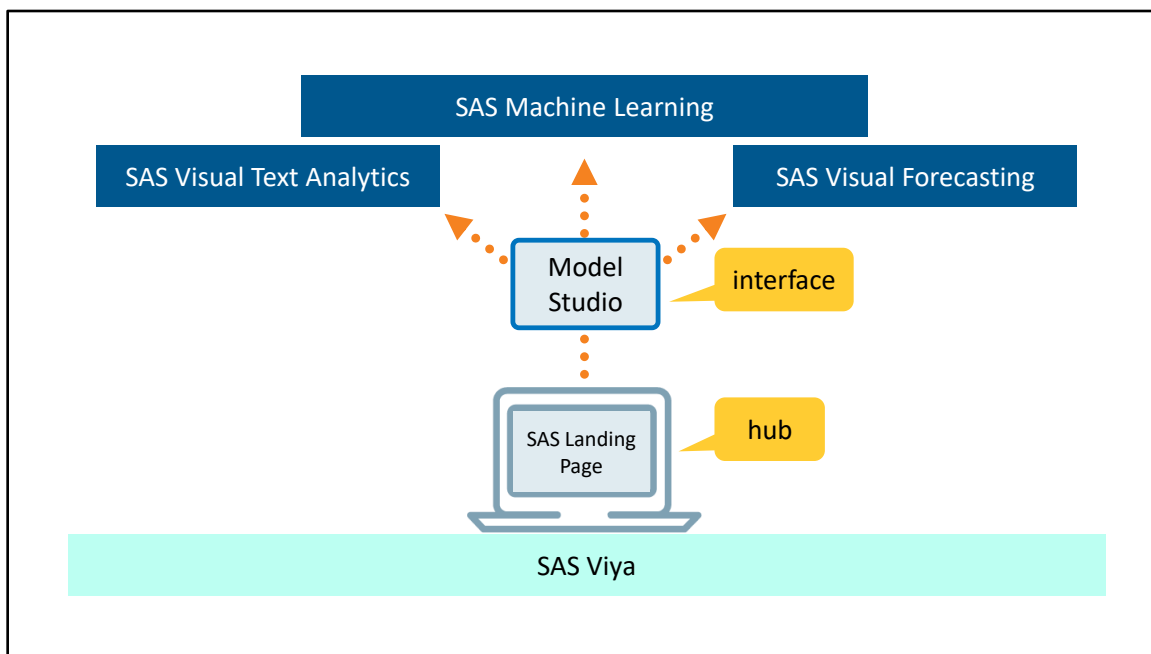
SAS Viya Tools for Machine Learning



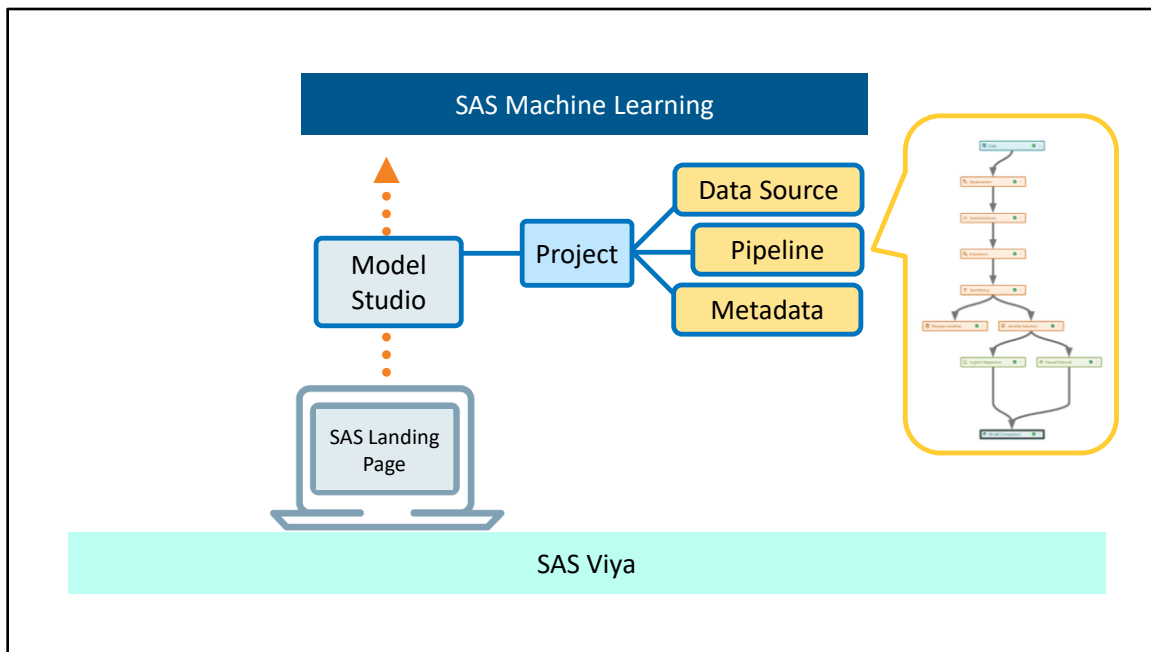
Here is a quick introduction to the SAS Viya tools that you learn about in this course, and how those tools work together.

The SAS Viya foundation supports a variety of software solutions for data mining.

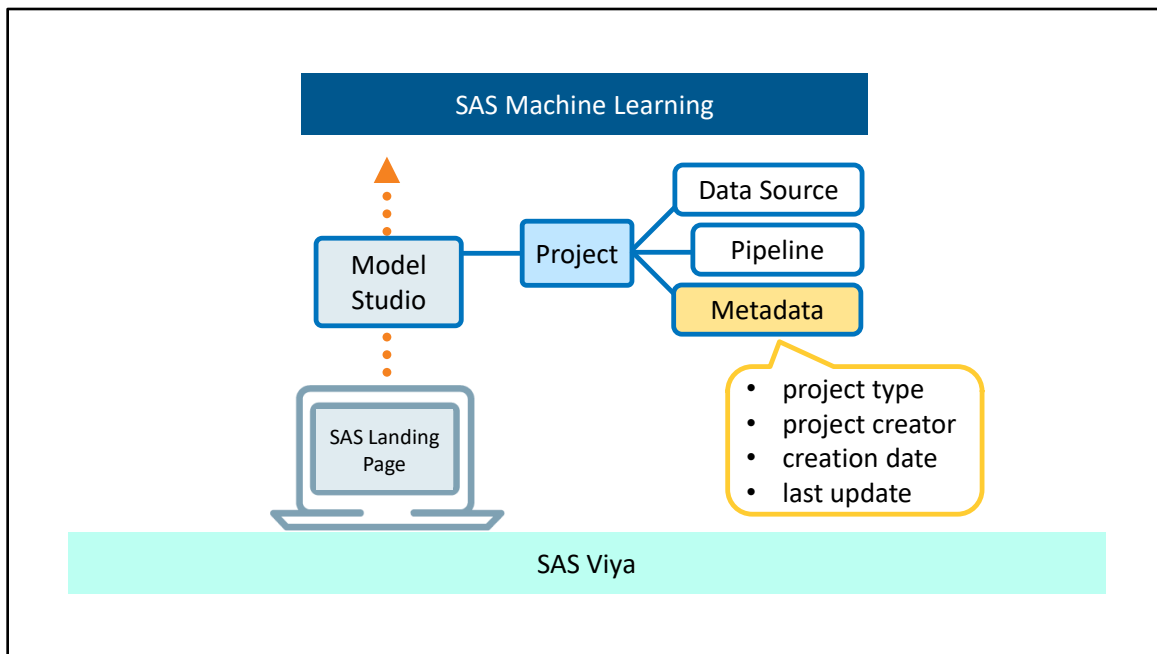
This course shows you how to use analytic data mining tools that are available in the SAS Machine Learning solution. These tools take advantage of SAS Viya programming and cloud processing environments for building predictive models and putting them into production.



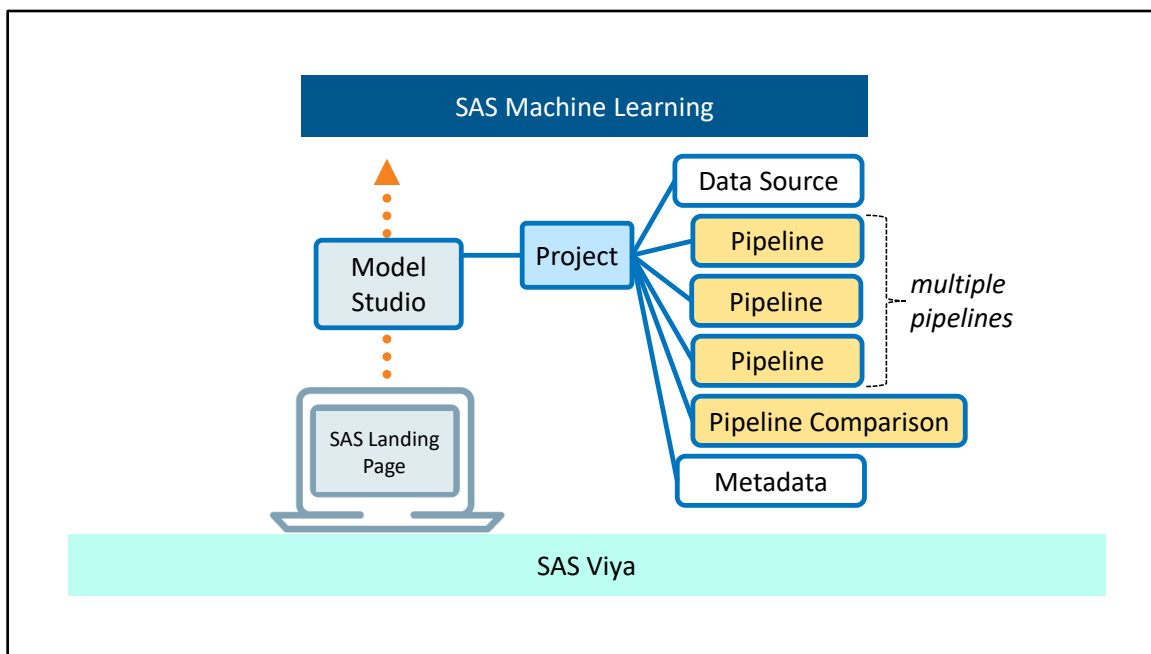
SAS Landing Page is the web-based hub (that is, the single point of access) for SAS Viya solutions. From SAS Landing Page, you access the features of SAS Machine Learning through Model Studio, a web-based visual interface. Model Studio is also an interface for two other SAS Viya solutions: SAS Visual Text Analytics and SAS Visual Forecasting. Your site's licensing agreement determines which solutions you can access from Model Studio.



In Model Studio, the main container for your analytic work is a project. A basic Model Studio project contains a data source, a pipeline that you create, and related project metadata. Remember that a pipeline is a structured workflow of analytic actions that is represented as a diagram. In the pipeline diagram, a node represents each action.



Some examples of project metadata are the project type, project creator, creation date, and last update.



A project can also have multiple pipelines and pipeline comparison results.

SAS Landing

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Creating a Project and Loading Data

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Model Studio



Copyright © SAS Institute Inc. All rights reserved.

Question: Model Studio

Model Studio is a web-based platform that includes a suite of integrated data mining tools to facilitate end-to-end data mining analysis. Model Studio can be used with SAS Machine Learning in SAS Viya and two other visual tools: SAS Visual Text Analytics and SAS Visual Forecasting.

- a. True
- b. False

Answer: Model Studio

Model Studio is a web-based platform that includes a suite of integrated data mining tools to facilitate end-to-end data mining analysis. Model Studio can be used with SAS Machine Learning in SAS Viya and two other visual tools: SAS Visual Text Analytics and SAS Visual Forecasting.

- a. True
- b. False

CA: True

FB: Model Studio can be used with SAS Machine Learning in SAS Viya and two other visual tools: SAS Visual Text Analytics and SAS Visual Forecasting.

1.2 Supervised Prediction: Preparing the Data and Building the Initial Model



Predictive Modeling



Copyright © SAS Institute Inc. All rights reserved.

Predictive Modeling

Predictive Modeling

Supervised Prediction

Supervised Learning

Training Data

observations

Inputs				Target
■	■	■	■	■
■	■	■	■	■
■	■	■	■	■
■	■	■	■	■
■	■	■	■	■

training cases

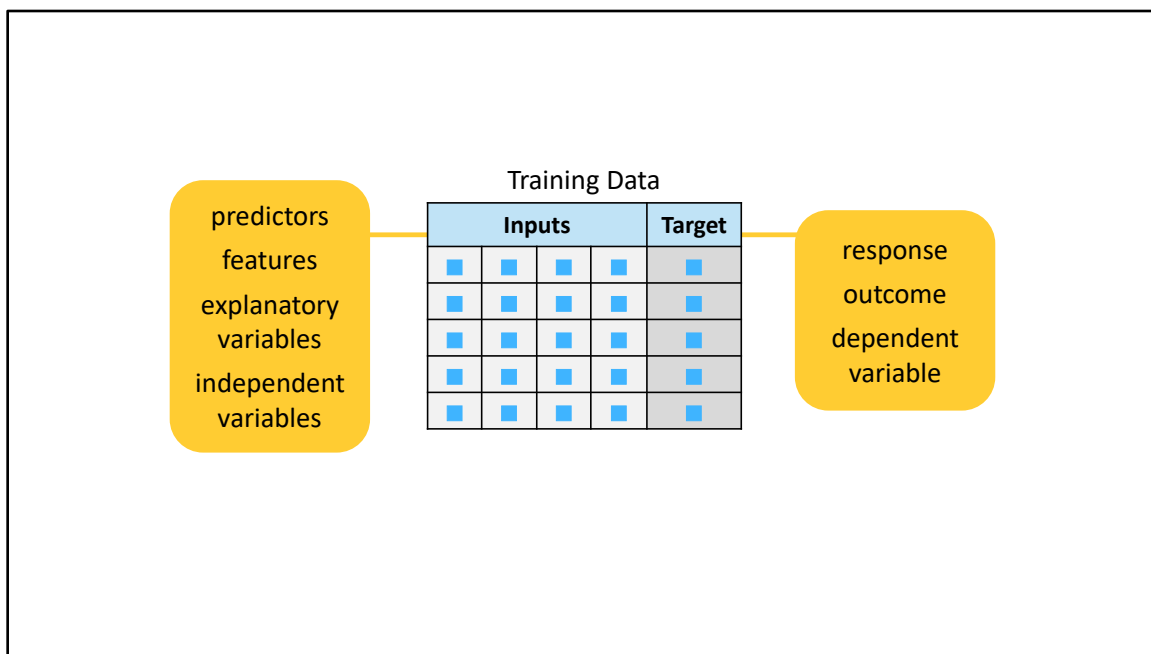
examples

instances

records

Predictive modeling (also known as *supervised prediction* or *supervised learning*) starts with a training data set.

The observations in a training data set are known as *training cases* (and are also known as *examples*, *instances*, or *records*).

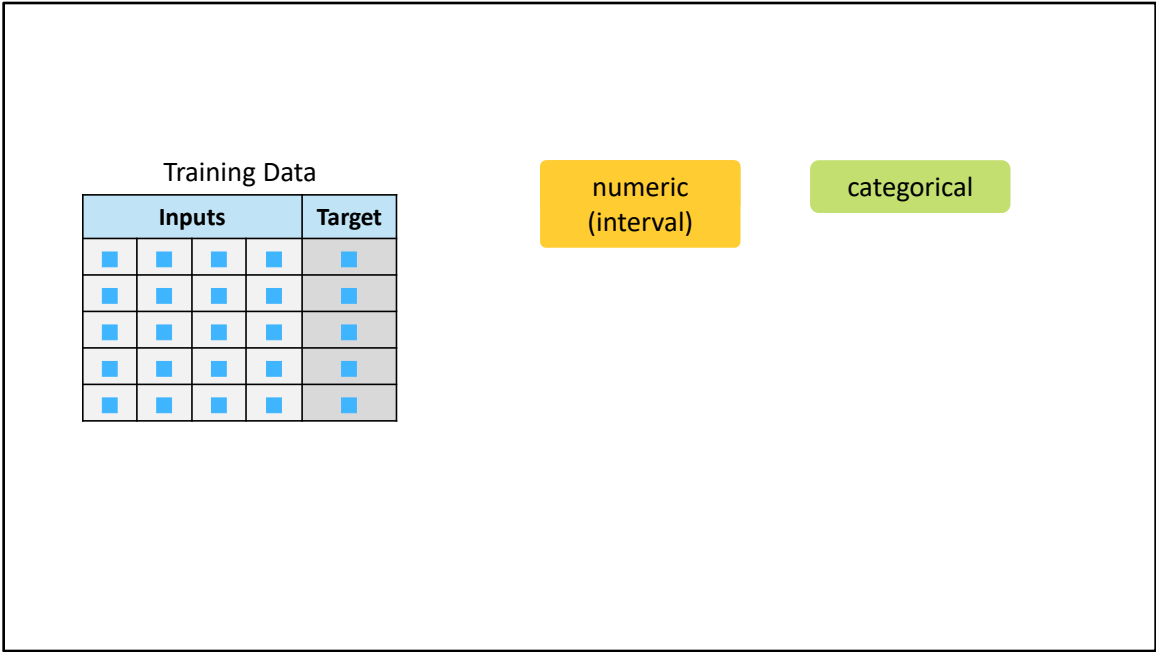


The variables include *inputs* and a *target*. In the data set used for the churn business challenge, some of the inputs are number of calls to technical support, account tenure, age of customer's device, and device manufacturer. The target indicates whether customers churned.

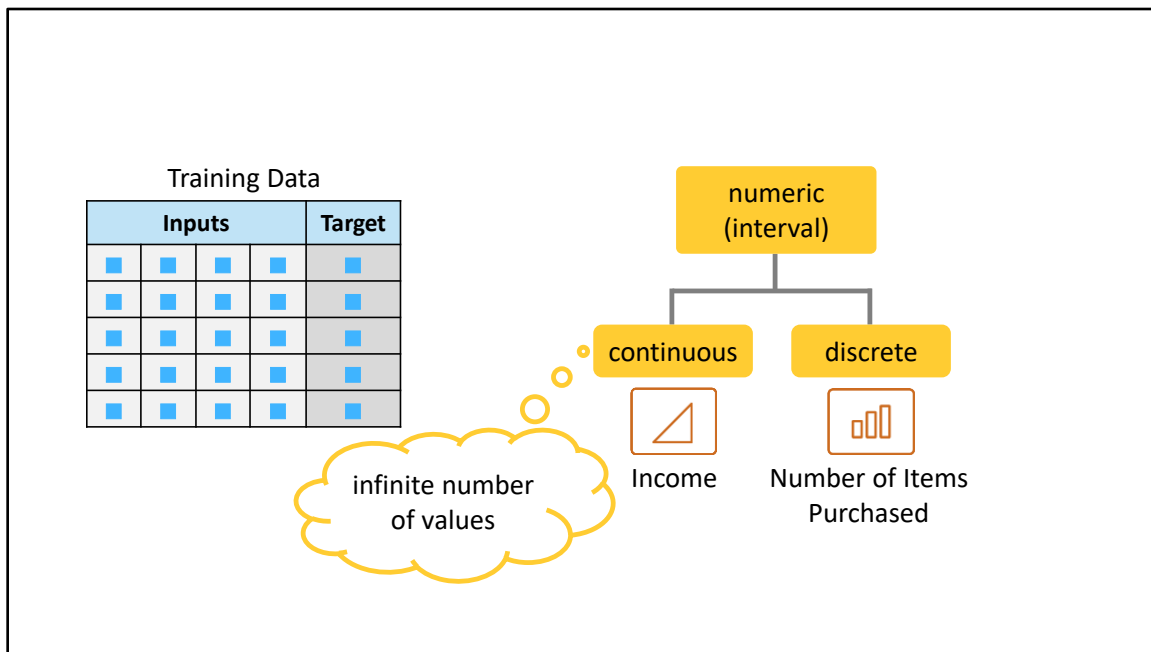
Inputs are also known as *predictors*, *features*, *explanatory variables*, or *independent variables*.

The *target* is also known as a *response*, *outcome*, or *dependent variable*.

For a given case, the inputs reflect your knowledge before measuring the target.

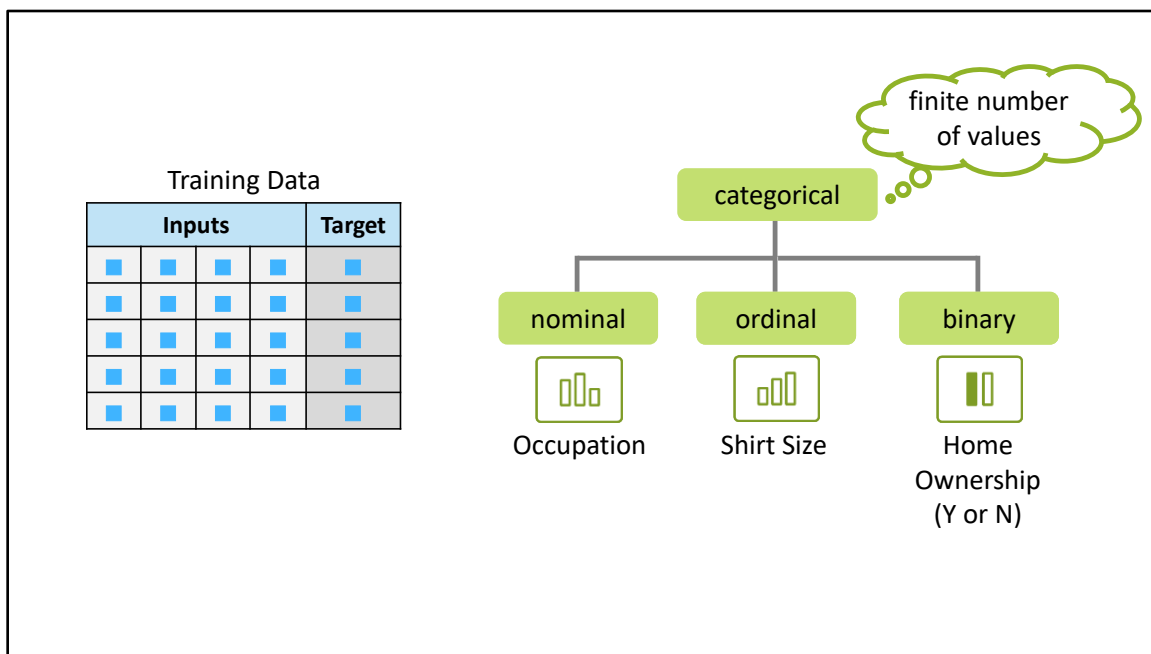


A variable can be numeric or categorical. Numeric variables are also known as interval variables.



Numeric variables can further be classified as continuous or discrete. Continuous variables can take on any one of an infinite number of values within a given range. Income is an example of a continuous variable. Of course, for practical reasons, income is typically rounded to no more than two decimal places.

Discrete variables are counts. The number of items purchased is an example of a discrete variable.

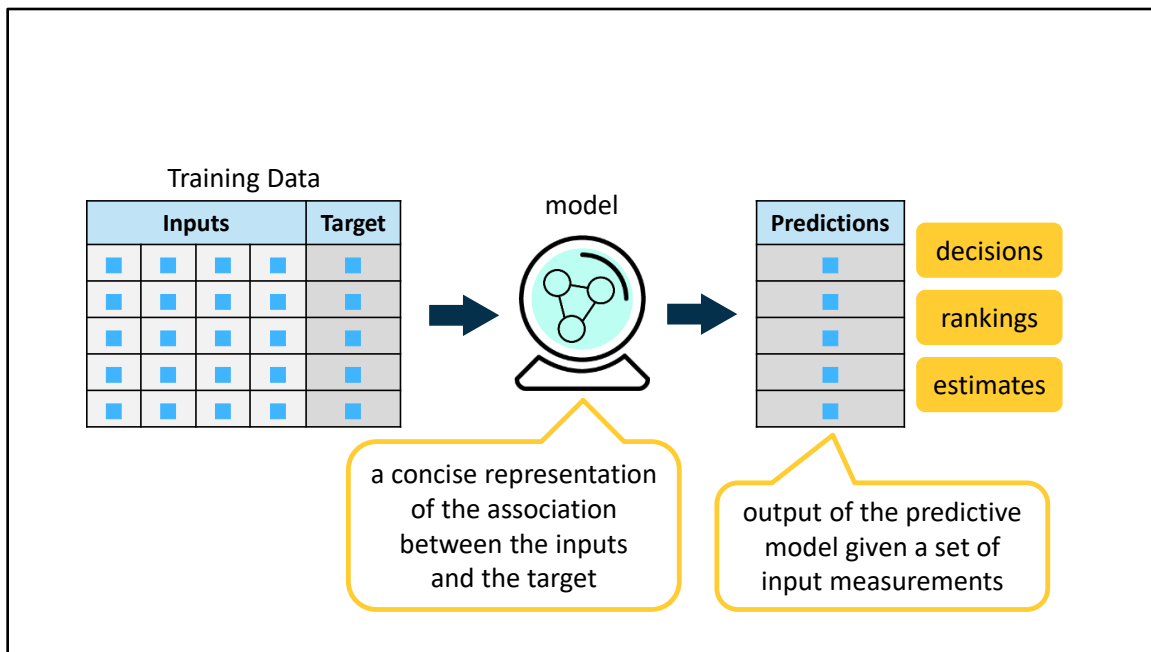


Categorical variables take on qualitative values, each value representing a group or category. The number of possible values of a categorical variable is typically finite.

Categorical variables are often nominal, which means that the categories have no particular order. Occupation is an example of a nominal variable.

Ordinal variables have categories with an inherent order (for example, Shirt Size).

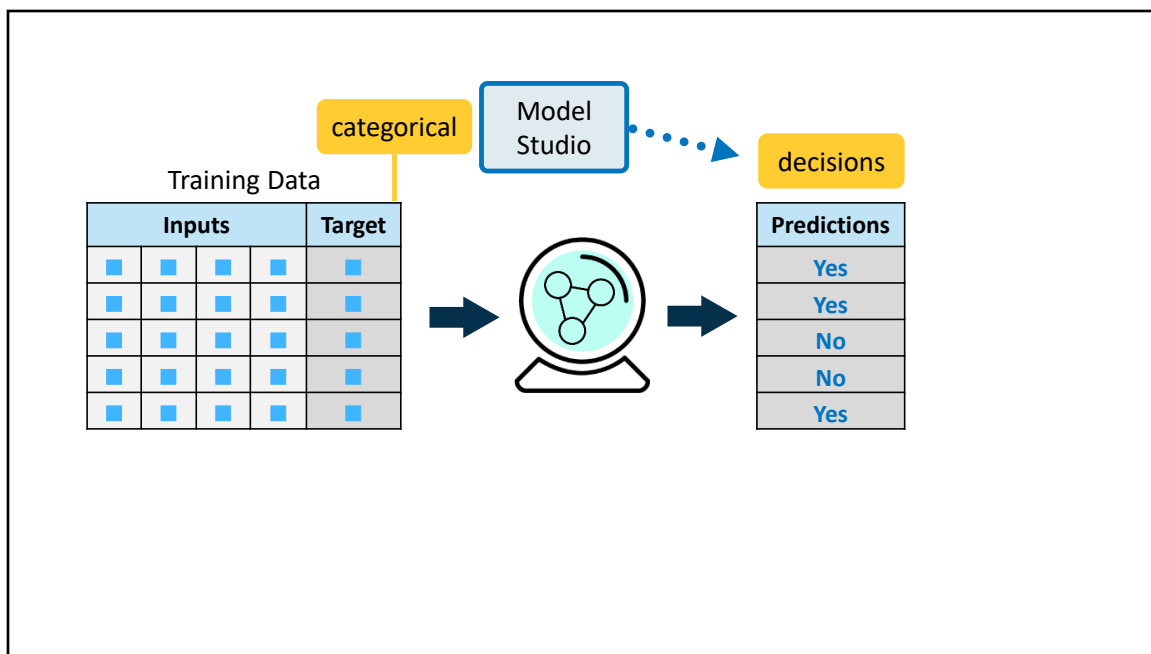
Binary variables have two categories, such as a positive or negative response concerning home ownership.



The purpose of the training data is to construct a predictive model (that is, a rule) that relates the inputs to the target. The *predictive model* is a concise representation of the association between the inputs and the target.

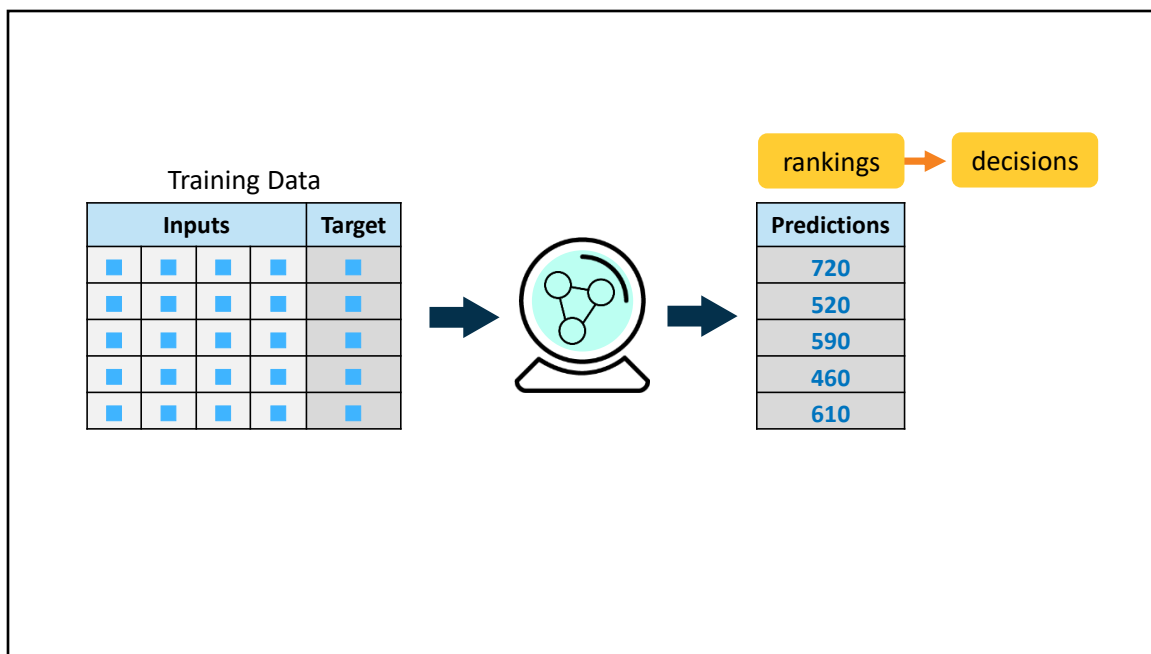
The outputs of the predictive model are known as *predictions*. Predictions represent the model's best guess about the target given a set of input measurements. The predictive model makes predictions based on what it learns from the source data.

A predictive model can generate one of three types of predictions: decisions, rankings, and estimates.



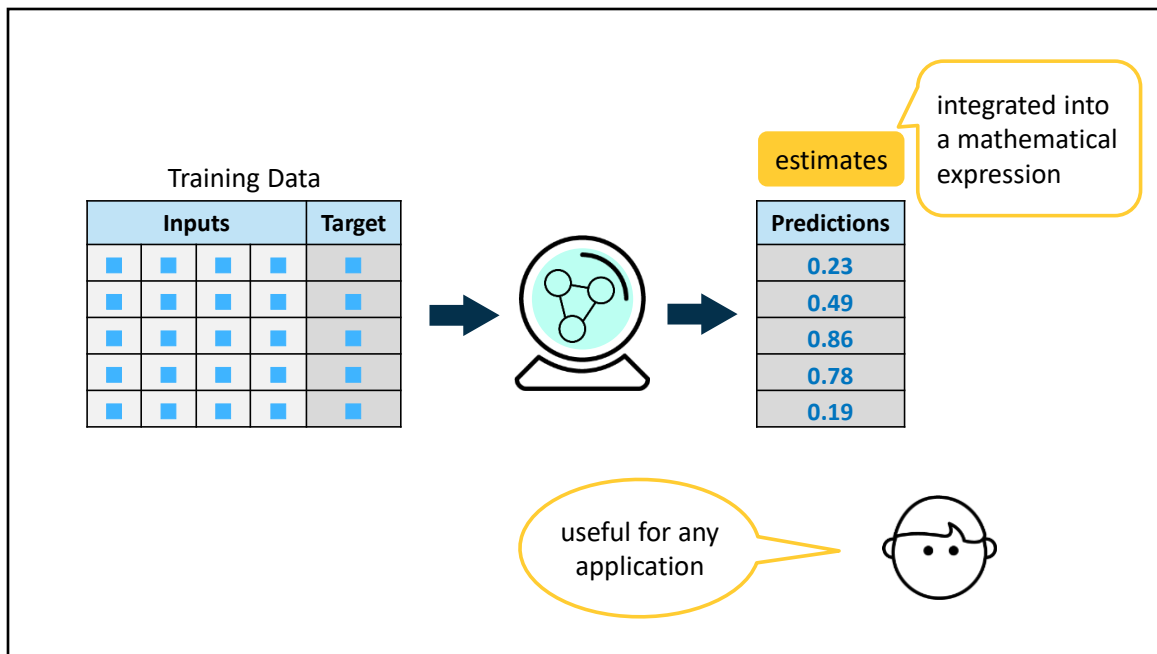
Decision predictions are the simplest type of prediction. Decisions are usually associated with some type of action (for example, classifying a case as a churn or a no-churn). For this reason, decisions are also known as *classifications*. Decision predictions are used, for example, in handwriting recognition, fraud detection, and direct mail solicitation.

Decision predictions usually relate to a categorical target variable. By default, Model Studio assumes decision predictions when the target variable is categorical (binary, nominal, or ordinal).



Ranking predictions order the cases based on the relationships between the input variables and the target variable. The most common example of a ranking prediction is a credit score. Using the training data, the model attempts to rank the high-value cases higher than the low-value cases. For example, the model might produce a list of customers ranked from highest priority to lowest priority. The actual rank values are irrelevant; only the relative order is important.

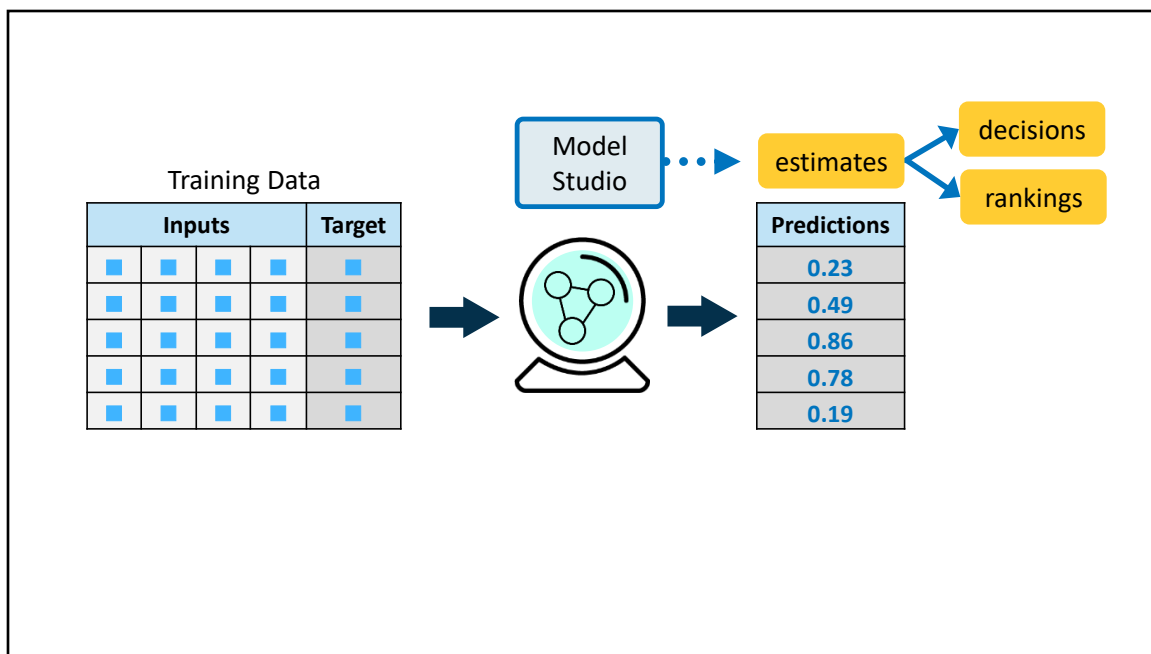
Note that ranking predictions can be transformed into decision predictions if cutoffs are defined for the ranks. For example, it is possible to make a binary decision by classifying all ranks above a cutoff as event cases and all ranks below the cutoff as nonevent cases.



Estimate predictions approximate the expected value of the target, given the input values. For cases with numeric targets, this estimate can be thought of as the average value of the target for all cases that have the observed input measurements. When the target is categorical, the estimate might equal the probability of a target outcome.

Prediction estimates are most commonly used when their values are integrated into a mathematical expression. An example is two-stage modeling, where the probability of an event is combined with an estimate of profit or loss to form an estimate of unconditional expected profit or loss.

Prediction estimates are also useful when you are not certain about the ultimate application of the model.



Estimate predictions are versatile because they can be transformed into both decisions and ranking predictions. Thus, if you are not sure which type of prediction to use, it is best to go with estimates.

In Model Studio, most types of models can be configured to produce estimate predictions.

Data Preparation and Preprocessing



Copyright © SAS Institute Inc. All rights reserved.

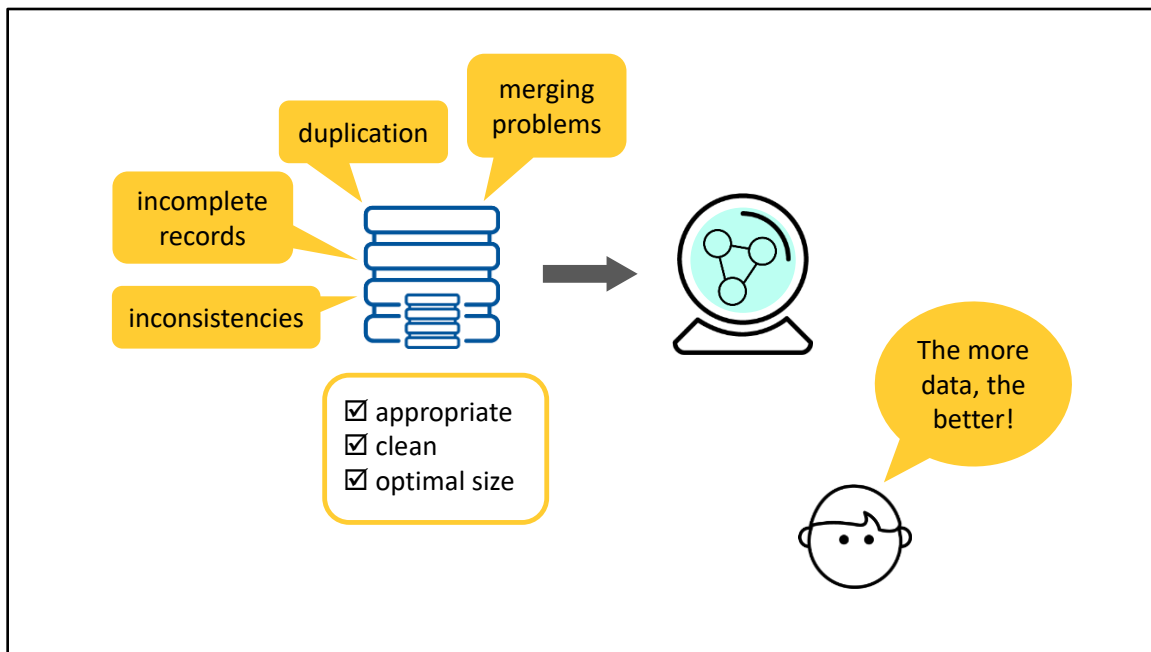
Data Preparation and Preprocessing



You start your analytics project at the Data phase of the analytics life cycle. This phase is also known as data preparation or data preprocessing.

Data preparation is the process of converting raw data into a wide table by parsing and joining data. At the end of data preparation, data are aggregated and summarized to the right granularity. Extraneous columns have been dropped, and invalid records have been filtered out. However, you are yet not ready to apply machine learning models.

The data now need to be preprocessed to achieve certain modeling requirements. Some common data preprocessing tasks are case filtering, value replacement, imputing missing values, feature scaling, truncating values, and one-hot-encoding categorical features. In this course, we focus on data preprocessing tasks.



Analysts generally agree that the more data you have, the better your models will be.

However, data preprocessing involves more than simply collecting a large amount of data. For one thing, data are usually dirty and noisy. It's typical to encounter inconsistencies, incomplete records, duplication, and merging problems.

Good data are the foundation for good models, so it is important to make sure that your data are appropriate, clean, and reduced to the optimal size for analysis. In fact, it is not unusual for analysts to spend the majority of their project time on data preprocessing.



Essential Data Tasks

- Divide the data.
- Address rare events.
- Manage missing values.
- Replace incorrect values.
- Add unstructured data.
- Extract features.
- Handle extreme or unusual values.
- Select useful inputs.

To prepare your data, you perform several essential data tasks. In this lesson, you learn about the first three tasks in the list: dividing the data, addressing rare events, and managing missing values. You perform these tasks while you create your Model Studio project and your first few pipelines. In the next lesson you learn about the remaining essential data tasks: replacing incorrect values, adding unstructured data, extracting features, handling extreme or unusual values, and selecting useful inputs.

Although you perform these tasks in the listed sequence in this course, keep in mind that you can vary the order in your own projects, as needed.

SAS Viya Tools for Data Preparation

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Dividing the Data



Copyright © SAS Institute Inc. All rights reserved.

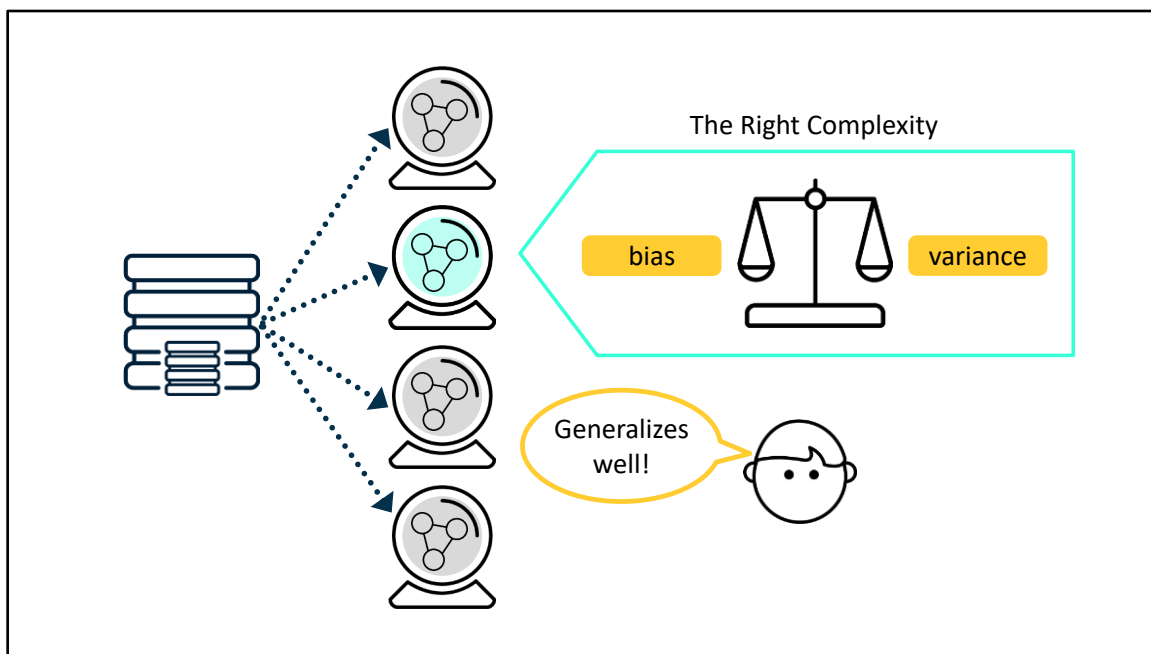
Dividing the Data



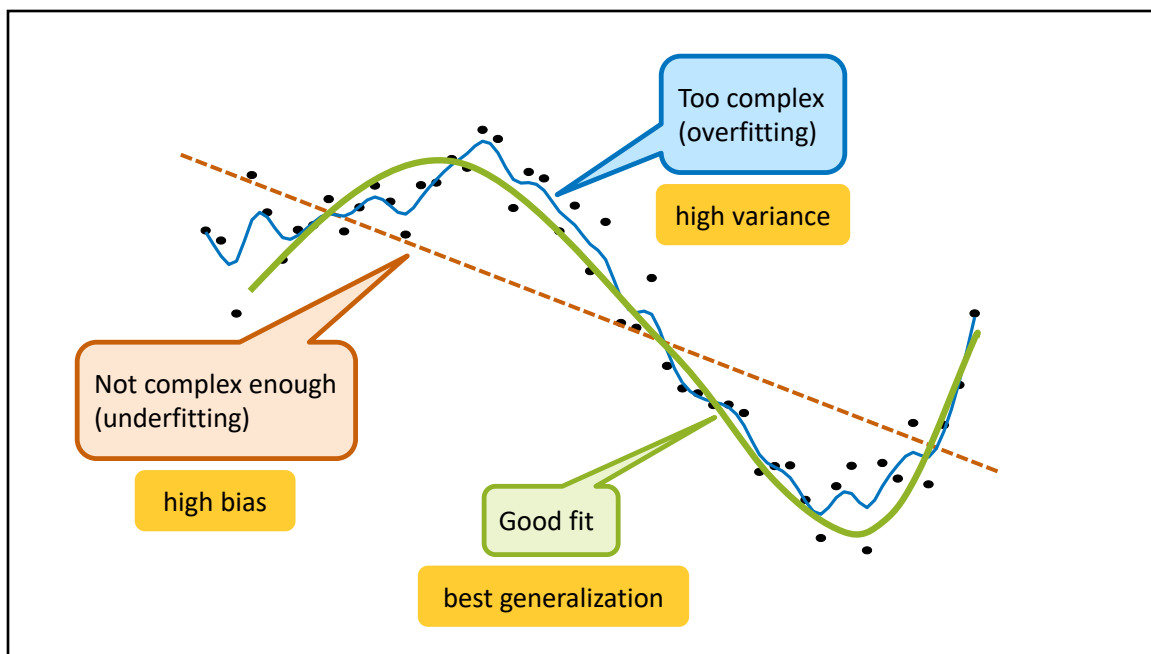
Essential Data Tasks

- Divide the data.
- Address rare events.
- Manage missing values.
- Replace incorrect values.
- Add unstructured data.
- Extract features.
- Handle extreme or unusual values.
- Select useful inputs.

We're ready for the first essential data task: dividing the data. To start, let's see why it's a good idea to divide the data.



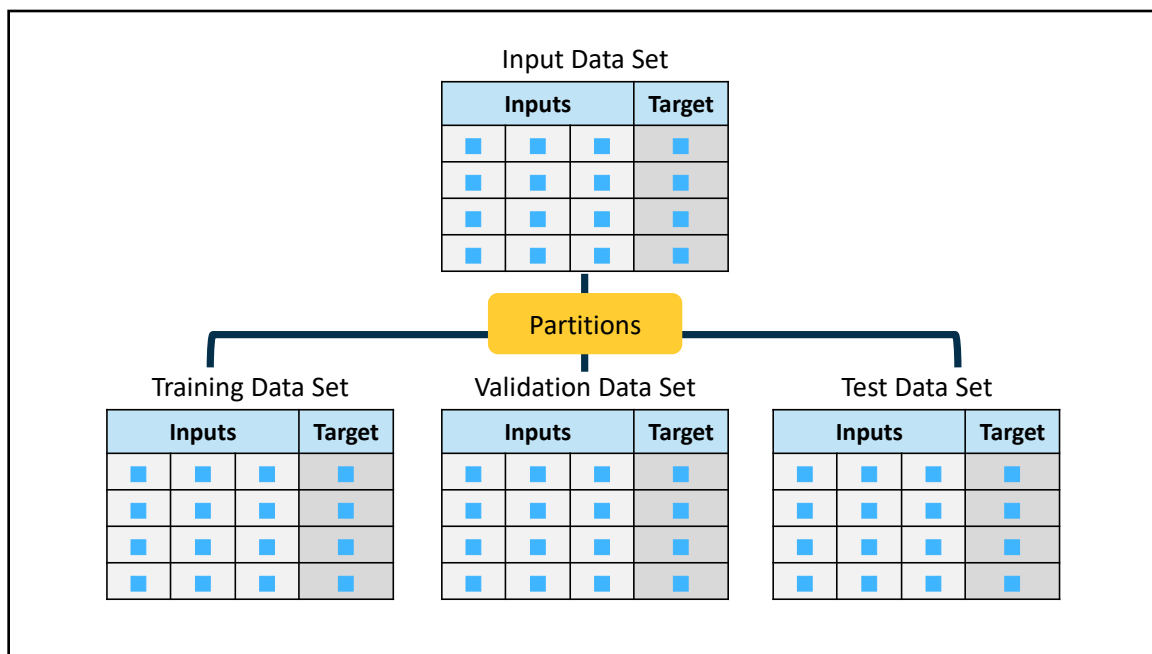
Fitting a model to data requires searching through the space of possible models, building a series of models of increasing complexity, and selecting a model that generalizes well. You want to choose the right complexity, which involves a trade-off between bias and variance.



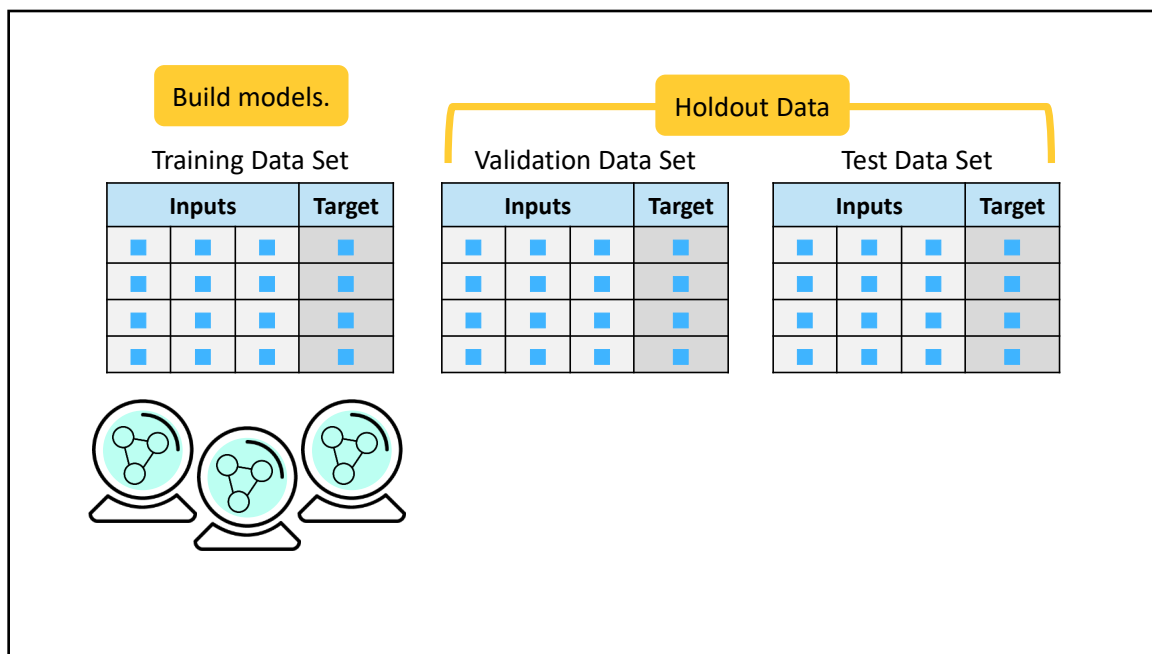
Suppose you have some data points, as shown here. An insufficiently complex model might not be flexible enough, which can lead to underfitting (that is, systematically missing the signal). In this situation, the model has high bias.

A naïve modeler might assume that the most complex model should always outperform the others, but this is not the case. An overly complex model might be too flexible, which can lead to overfitting (that is, accommodating nuances of the random noise in the sample). In this situation, the model has high variance.

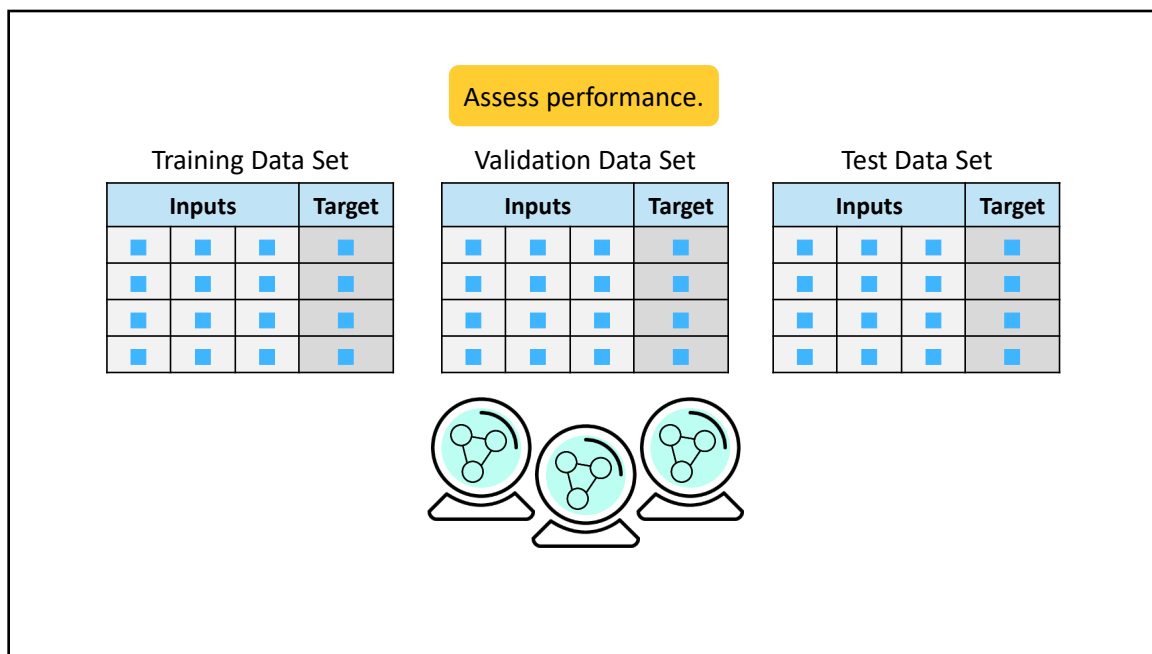
The goal of model selection is to choose a model that neither overfits nor underfits the data. A model with the right amount of flexibility gives the best generalization.



To allow for honest assessment of model performance, the data are typically divided into two or three non-overlapping sets, which are called partitions.



You use the first partition, the *training set*, to build models. Remaining partitions of the data can be referred to as holdout data.



Training Data Set

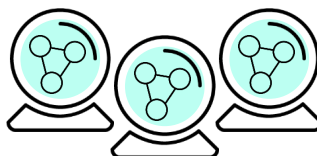
Inputs			Target
■	■	■	■
■	■	■	■
■	■	■	■
■	■	■	■

Validation Data Set

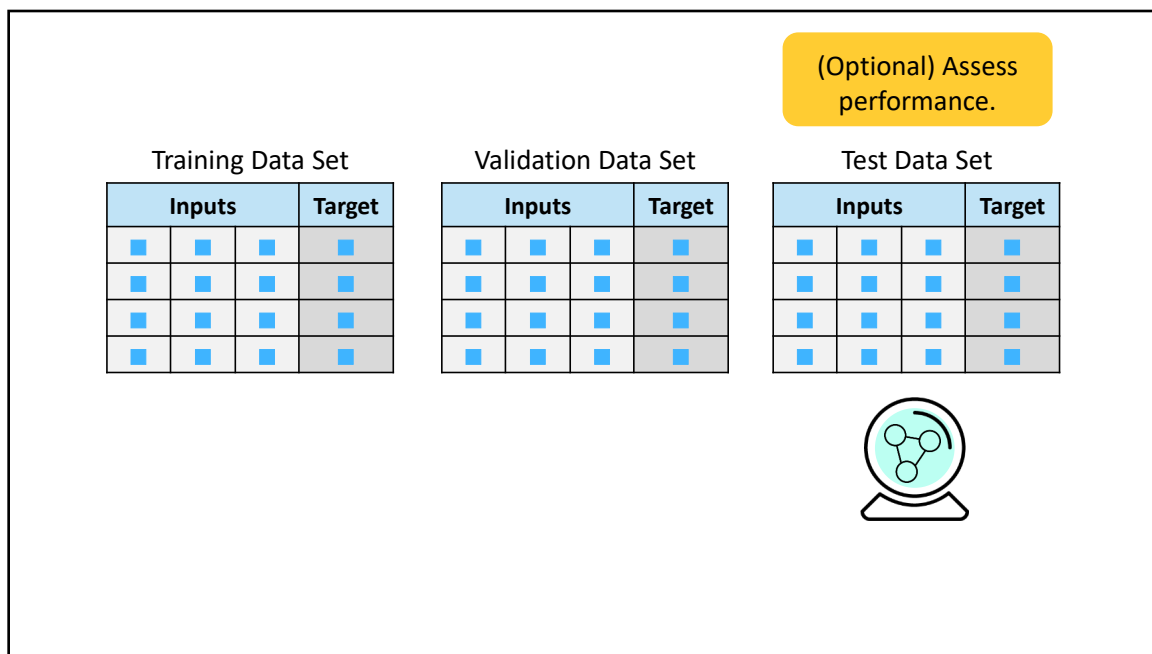
Inputs			Target
■	■	■	■
■	■	■	■
■	■	■	■
■	■	■	■

Test Data Set

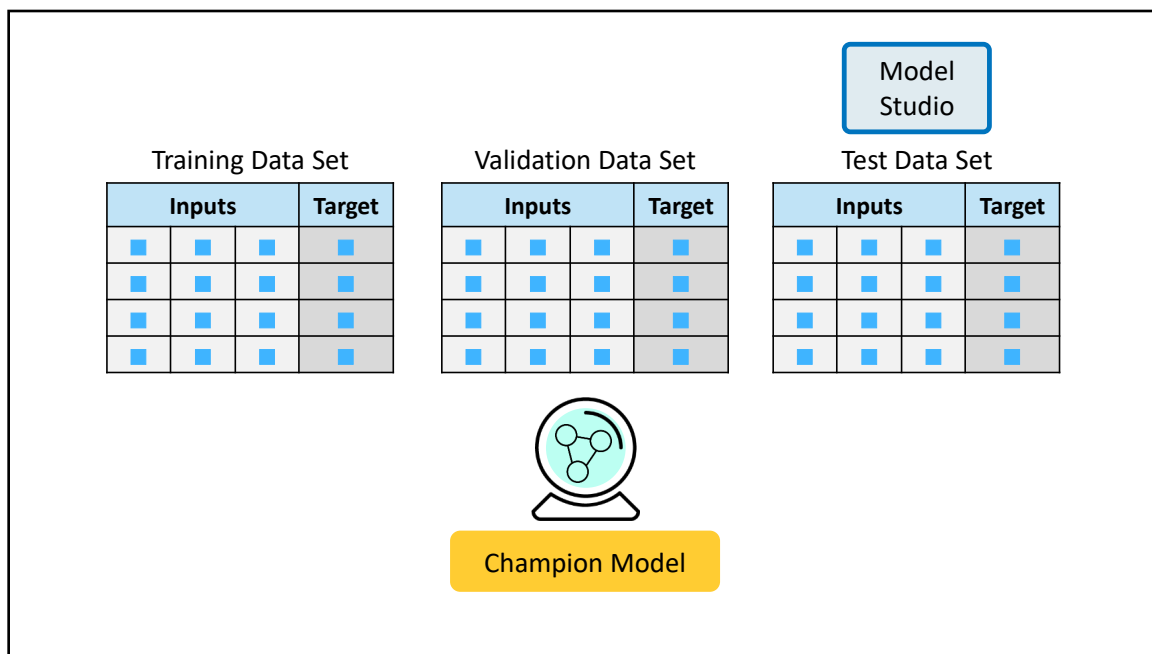
Inputs			Target
■	■	■	■
■	■	■	■
■	■	■	■
■	■	■	■



After model training, you can assess the performance of each model on the second partition of the data, the *validation set*. The validation data set is used to optimize the complexity of the model and find the sweet spot between bias and variance. Based on the validation data, you tune the models that were built on the training data and determine whether additional training is required.



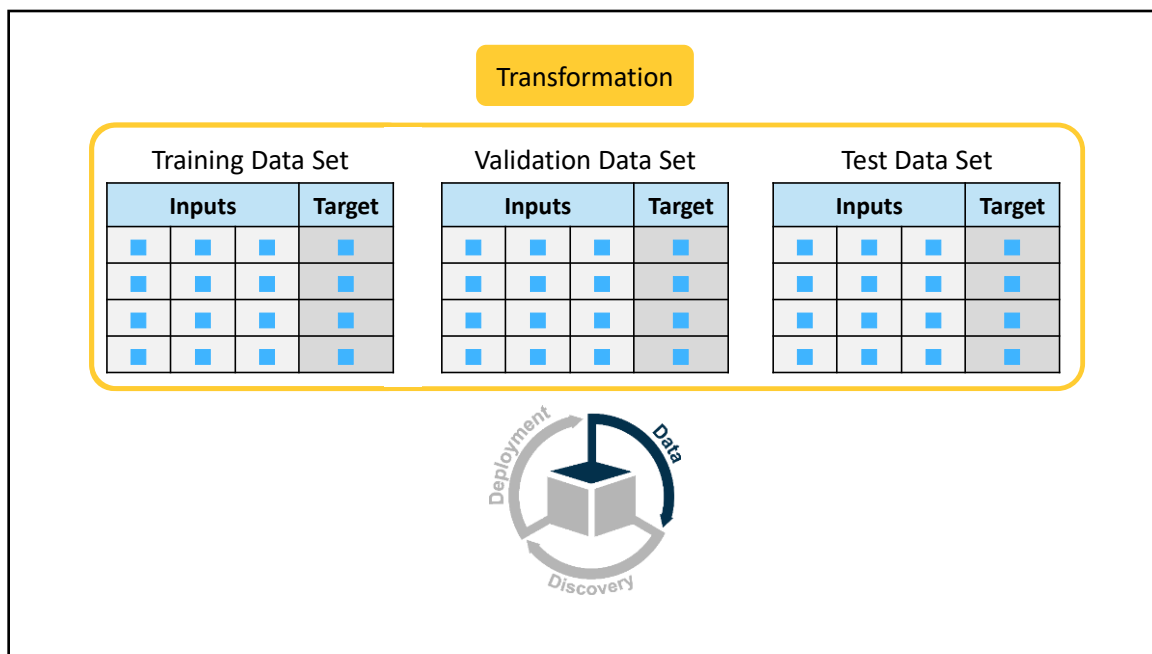
The *test data set* is an optional partition for the model building process, but some industries might require it as a source of unbiased model performance. The test data set gives the honest, unbiased estimates of the model's performance. The test data set provides one final measure of how the model performs on new data, before the model is put into production.



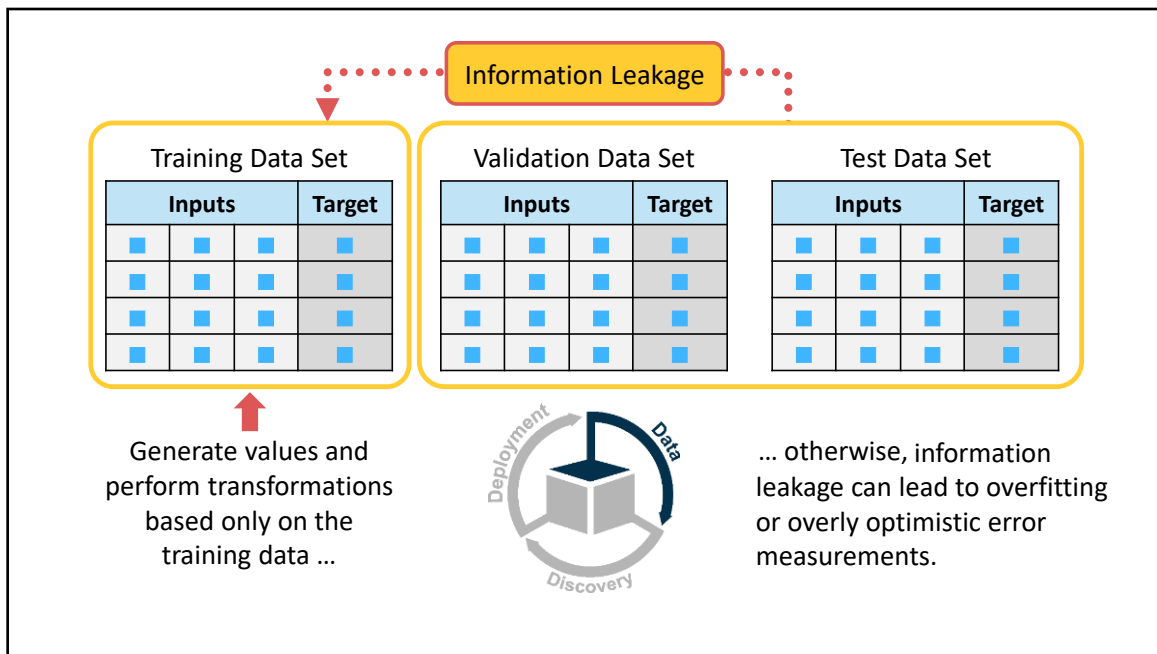
By default, Model Studio uses the test data to select the champion model, if a test data set exists.

Otherwise, Model Studio uses the validation data to select the champion model.

If a validation data set does not exist, Model Studio uses the training data to select a champion model.



Many of the essential data tasks involve transforming the data. It is critical that all transformations that are used to prepare the training data are also applied to any holdout data.



In other words, it is critical that information from holdout data does not leak into the training data. Information leakage can occur in many ways and can potentially lead to overfitting or overly optimistic error measurements. For example, think about taking a mean or median across your data before partitioning and then later using this mean or median to replace missing values across all partitions of your data. In this case, your training data would be aware of the information from your holdout partitions. To avoid this type of leakage, make sure to generate values and perform other basic transformations based only on the training data.

Cross Validation for Small Data Sets

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Addressing Rare Events Using Event-Based Sampling



Copyright © SAS Institute Inc. All rights reserved.

Addressing Rare Events Using Event-Based Sampling



Essential Data Tasks

- ✓ Divide the data.
- Address rare events.
- Manage missing values.
- Replace incorrect values.
- Add unstructured data.
- Extract features.
- Handle extreme or unusual values.
- Select useful inputs.

It is important to determine whether your target can be characterized as a rare event relative to the total number of cases.

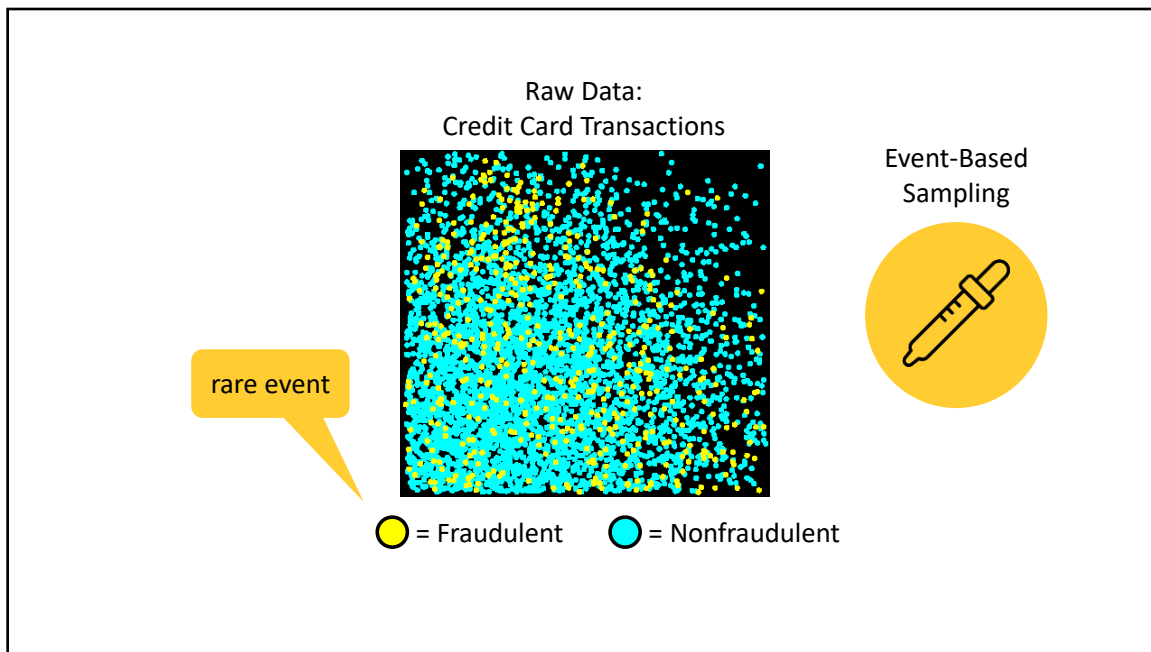


Training Data

Inputs				Target
■	■	■	■	■
■	■	■	■	■
■	■	■	■	■
■	■	■	■	■
■	■	■	■	■

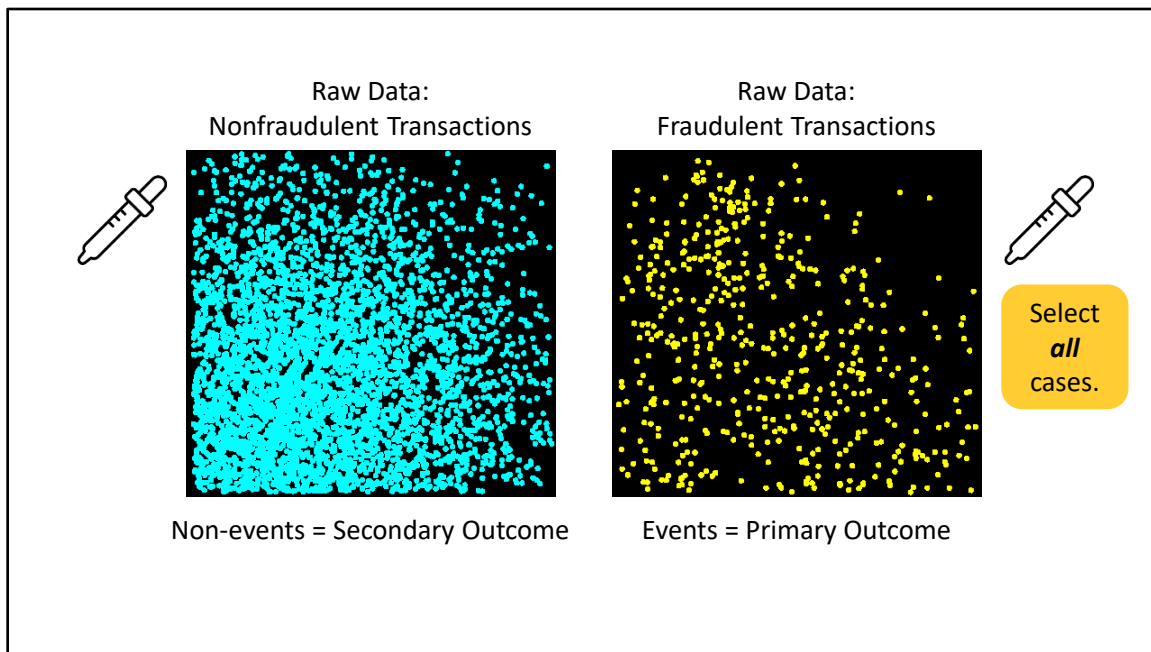
Include a representative number of fraudulent cases.

For example, in the detection of fraudulent activity, it is necessary to ensure that the training data include a representative number of fraudulent cases so as to capture the event sufficiently.

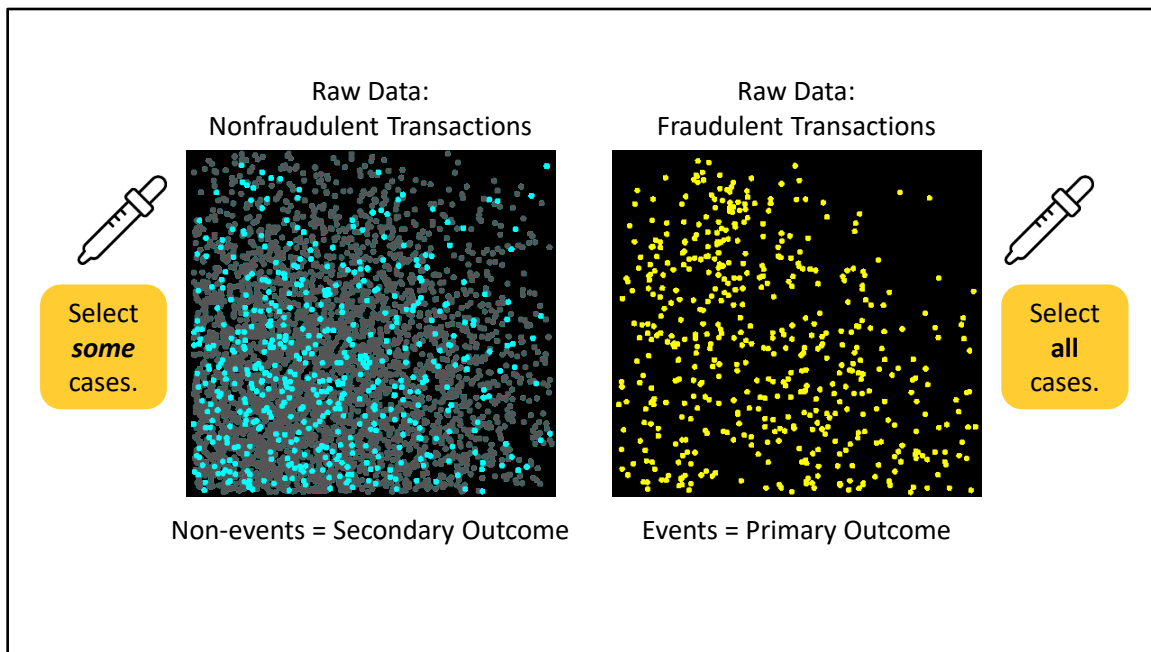


One out of every 1,000 credit card transactions is fraudulent, so fraudulent transactions are a rare event. Fitting a model to such data without accounting for the extreme imbalance in the occurrence of the event does not generate useful results.

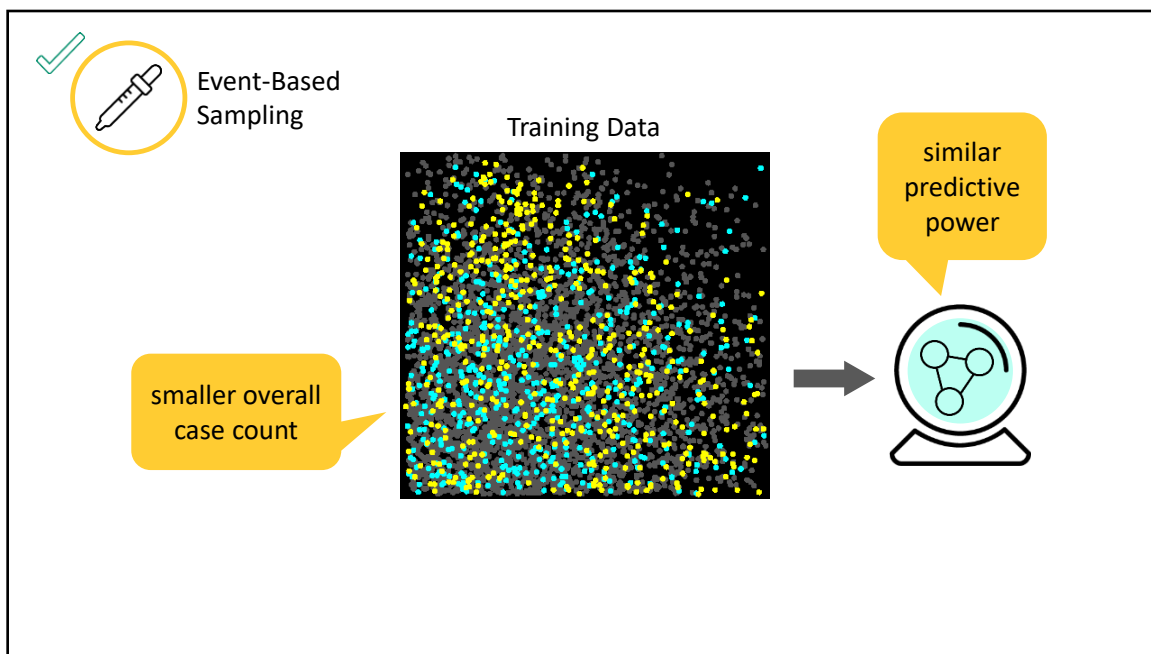
Special sampling methods are commonly used to modify an imbalanced data set (that is, a data set with a rare event) and provide a more balanced distribution. To address a rare target, a common predictive modeling practice is to build models from a sample with a primary outcome proportion that is different from the true population proportion. This technique, called event-based sampling, is typically used when the ratio of primary to secondary outcome cases is small.



In event-based sampling, samples are drawn separately based on the target events (that is, the primary outcome cases) and the non-events (that is, the secondary outcome cases). If the event is rare, usually all events are selected.

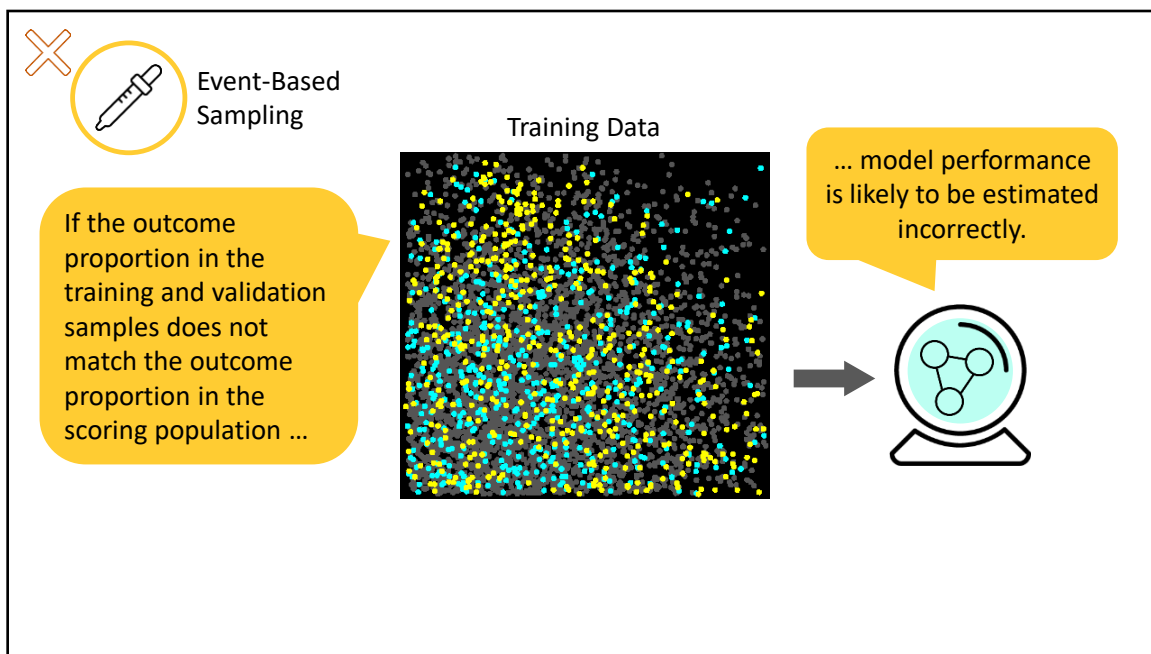


Then each event outcome is matched by one or more non-event outcomes. In other words, only some of the non-event cases are selected.



The advantage of event-based sampling is that you can obtain (on the average) a model of similar predictive power with a smaller overall case count. This is in concordance with the idea that the amount of information in a data set with a categorical outcome is determined not by the total number of cases in the data set itself, but instead by the number of cases in the rarest outcome category. For binary target data sets, this is usually the event outcome.

This advantage might seem of minimal importance in the age of extremely fast computers. However, model fitting occurs only after the completion of a long, tedious, and error-prone data preparation process. Smaller sample sizes for data preparation are usually welcome.



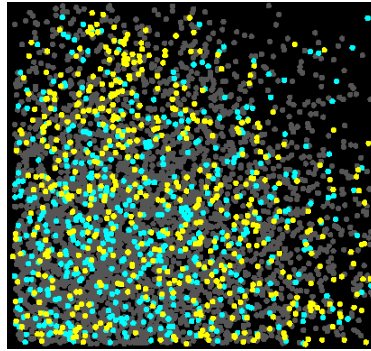
Although it reduces analysis time, event-based sampling also introduces some analysis complications.

Most model fit statistics (especially those related to prediction decisions) and most of the assessment plots are closely tied to the outcome proportions in the training samples. If the outcome proportion in the training and validation samples does not match the outcome proportion in the scoring population, model performance is likely to be estimated incorrectly. If the outcome proportions in the training sample and scoring population do not match, model prediction estimates are biased.



Event-Based
Sampling

Training Data



adjusts scoring
results for bias

Model
Studio



Fortunately, Model Studio automatically adjusts assessment measures, assessment graphs, and prediction estimates for bias if you enable event-based sampling.

Demo: Modifying the Data Partition

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Selecting Variables on the Data Tab

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Data Partitions



Copyright © SAS Institute Inc. All rights reserved.

Question: Data Partitions

When there is a training data set, a validation data set, and a test data set, which data partition is used by default to select the champion model?

- a. training data set
- b. validation data set
- c. test data set

Answer: Data Partitions

When there is a training data set, a validation data set, and a test data set, which data partition is used by default to select the champion model?

- a. training data set
- b. validation data set
- ☒ c. test data set

CA: c

FB: If the test data set exists, it is used by default to select the champion model. If only training and validation data sets exist, then the validation data set is used to select the champion model. If only a training data set exists, it is used to select the champion.

Global Metadata

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Managing Missing Values



Copyright © SAS Institute Inc. All rights reserved.

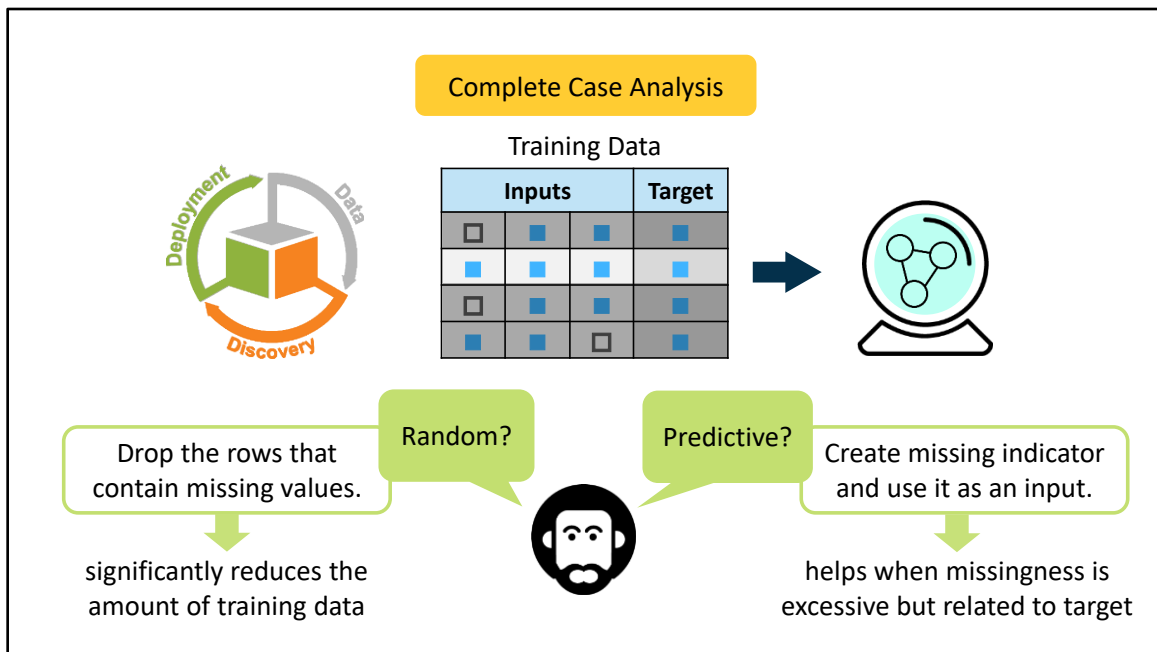
Managing Missing Values



Essential Data Tasks

- ✓ Divide the data.
- ✓ Address rare events.
- Manage missing values.
- Replace incorrect values.
- Add unstructured data.
- Extract features.
- Handle extreme or unusual values.
- Select useful inputs.

The data that you use for predictive modeling are likely to have missing values. It's best to handle the problem of missing values early in data preprocessing.



Missing values might occur in the target variable or the input variables. It is especially problematic for your analysis when the target has missing values. However, in this course, you learn about the more common scenario of missing values in the input variables.

Missing values can be a problem during both model building (in the Discovery phase) and model deployment. Let's see why.

To build a model, several machine learning algorithms in SAS Viya use only the complete cases (that is, the cases with no missing values). With this approach, which is called complete case analysis, the model might be based on only a small subset of the original data.

It is important to consider whether missing values are distributed randomly or whether missingness is somehow predictive of the target. If missing values appear at random in the input data, you can drop the rows that contain missing values without introducing bias into the model. However, complete case analysis can remove a tremendous amount of information from the training data and reduce the predictive accuracy of the model.

Cases often contain missing values for a reason. If the missingness itself is predictive, that is, if the missing value is in some way related to the target variable, then the variable might be better handled by creating the binary missing indicator and use it as an input to the model.



Scoring Data

Inputs			Target

It is very likely that your scoring data will also have the same type of missingness.

If your training data has missing values, it is very likely that your scoring data will also have missing values with the same type of missingness (that is, either random or predictive).



Scoring Data

Inputs			Target

$$\text{logit}(p) = -0.81 + 0.92 \cdot x_1 + 1.11 \cdot x_2$$

Predict: $(x_1, x_2) = (0.3, ?)$

Prediction
formula cannot
score cases with
missing values.

In deployment, the model cannot score a case that has missing values.



Training Data

Inputs			Target
□	■	■	■
■	■	■	■
□	■	■	■
■	■	□	■



managing missing values when training a model



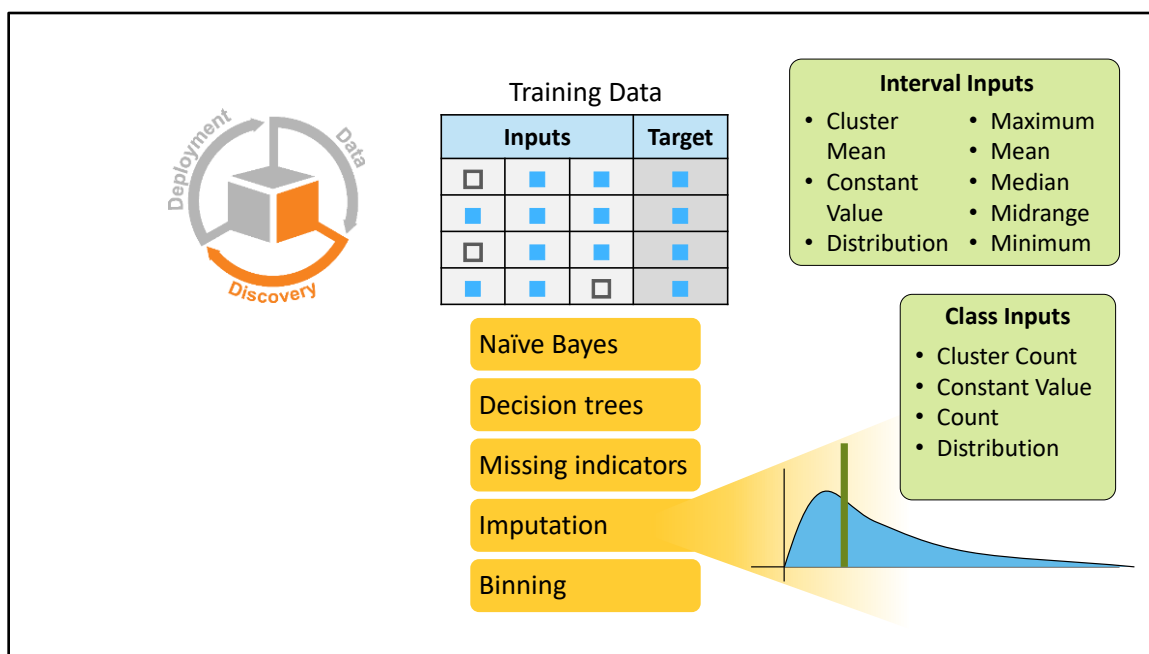
Scoring Data

Inputs			Target
□	■	■	■
■	■	■	■
□	■	■	■
■	■	□	■



handling missing values when scoring a new data

In this course, you learn how to manage missingness in training a machine learning model. You also learn guidelines for handling missing values when scoring data.



Techniques for managing missingness during model building include naïve Bayes, decision trees, missing indicators, imputation, and binning. Remember that missingness can be predictive. Thus, these techniques retain information that is associated with missing values, including the missing values themselves, which increases the predictive accuracy of the model.

In Model Studio, the naïve Bayes technique and decision tree models do not use complete case analysis; these modeling approaches can incorporate missingness.

Missing indicators are new inputs that indicate whether a value of a related variable is missing.

Imputation refers to replacing a missing value with information that is derived from nonmissing values in the training data. This is the approach that we'll use in this course.

Binning is helpful for handling interval input variables. In binning, the original numeric values are grouped into discrete categories called bins. The missing values are assigned to their own bin. The interval variable then becomes a categorical variable.

Next, you can learn additional details about these methods.

Managing Missing Values: Details

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Imputation Methods in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Missing Values



Copyright © SAS Institute Inc. All rights reserved.

Question: Missing Values

What is it called when the model-building process ignores training data cases with missing values of inputs?

- a. event-based sampling
- b. complete case analysis
- c. honest assessment
- d. imputation

Answer: Missing Values

What is it called when the model-building process ignores training data cases with missing values of inputs?

- a. event-based sampling
- ☒ b. complete case analysis
- c. honest assessment
- d. imputation

CA: b

FB: Complete case analysis means that training data cases with missing values of inputs are ignored.

Demo: Building a Pipeline from a Basic Template

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Automated Pipeline Creation

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Pipeline Templates in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Pipeline Templates



Copyright © SAS Institute Inc. All rights reserved.

Question: Pipeline Templates

Which default model is in the basic template for class target?

- a. decision tree
- b. linear regression
- c. logistic regression
- d. neural network

Answer: Pipeline Templates

Which default model is in the basic template for class target?

- a. decision tree
- b. linear regression
- ☒ c. logistic regression
- d. neural network

CA: c

FB: The Basic template for class target uses a logistic regression model by default.

Logistic Regression

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



1.3 A Closer Look at SAS Viya



SAS Cloud Analytic Services

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



SAS Viya: A Shift in Mindset

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Data Sources and CAS

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: CASLIBs



Copyright © SAS Institute Inc. All rights reserved.

Question: Caslibs

What does a caslib do?

- a. enables model content and score code to be integrated with applications outside of SAS
- b. provides access to files in a data source and to in-memory tables
- c. secures network connections
- d. compares pipelines

Answer: Caslibs

What does a caslib do?

- a. enables model content and score code to be integrated with applications outside of SAS
- ☒ b. provides access to files in a data source and to in-memory tables
- c. secures network connections
- d. compares pipelines

CA: b

FB: A CASLIB provides access to files in a data source, such as a database or a file system directory, and to in-memory tables.

Interfaces and Products

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



1.4 Lesson Conclusion



Lesson Quiz

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Lesson 2: Data Preprocessing and Algorithm Selection



2.0 Lesson Overview



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



2.1 Exploring the Data and Replacing Incorrect Values



Exploring the Data



Copyright © SAS Institute Inc. All rights reserved.

Exploring the Data

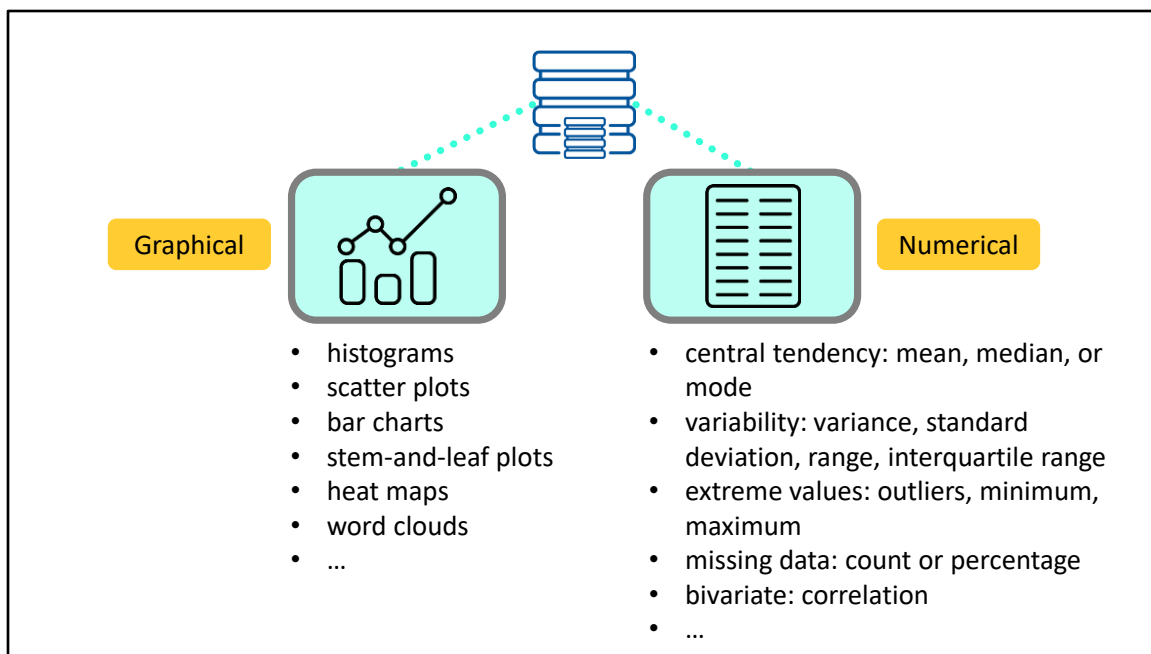


Essential Data Tasks

- ✓ Divide the data.
- ✓ Address rare events.
- ✓ Manage missing values.
 - Replace incorrect values.
 - Add unstructured data.
 - Extract features.
 - Handle extreme or unusual values.
 - Select useful inputs.



In any data analysis, it's important to get to know your data. Exploring the data is one of the most important steps of an analytical project, and it can be time-consuming. However, this is time well-spent. By gaining an intimate understanding of the variables and how they relate to one another, you can make better decisions about how to prepare the data.



You can explore the data using both graphical and numerical methods.

Common graphical tools include histograms, scatter plots, bar charts, and stem-and-leaf plots. These have been popular for decades, because they are easy to create by hand or with a computer, and they're easy to understand. More modern graphical tools, such as heat maps and word clouds, scale well to large data sets and help you make sense of bigger, more complex data.

Numerical methods used for data exploration include summary statistics for measures of central tendency, such as the mean, median, or mode; and statistics that measure variability, such as variance, standard deviation, range, or interquartile-range. You can investigate extreme values such as outliers, the minimum, or the maximum, as well as counts or percentages of missing data. And you can use bivariate measures such as correlation to estimate the association between two variables.

Demo: Exploring the Data

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Data Exploration Node



Copyright © SAS Institute Inc. All rights reserved.

Question: Data Exploration Node

In Model Studio, the Data Exploration node is in which group?

- a. Data Preprocessing
- b. Miscellaneous
- c. Postprocessing
- d. Supervised Learning

Answer: Data Exploration Node

In Model Studio, the Data Exploration node is in which group?

- a. Data Preprocessing
- ☒ b. Miscellaneous
- c. Postprocessing
- d. Supervised Learning

CA: b

FB: The Data Exploration node is in the Miscellaneous group.

Data Preprocessing Nodes in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Risk Modeling Add-on for SAS Viya

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Replacing Incorrect Values



Copyright © SAS Institute Inc. All rights reserved.

Replacing Incorrect Values

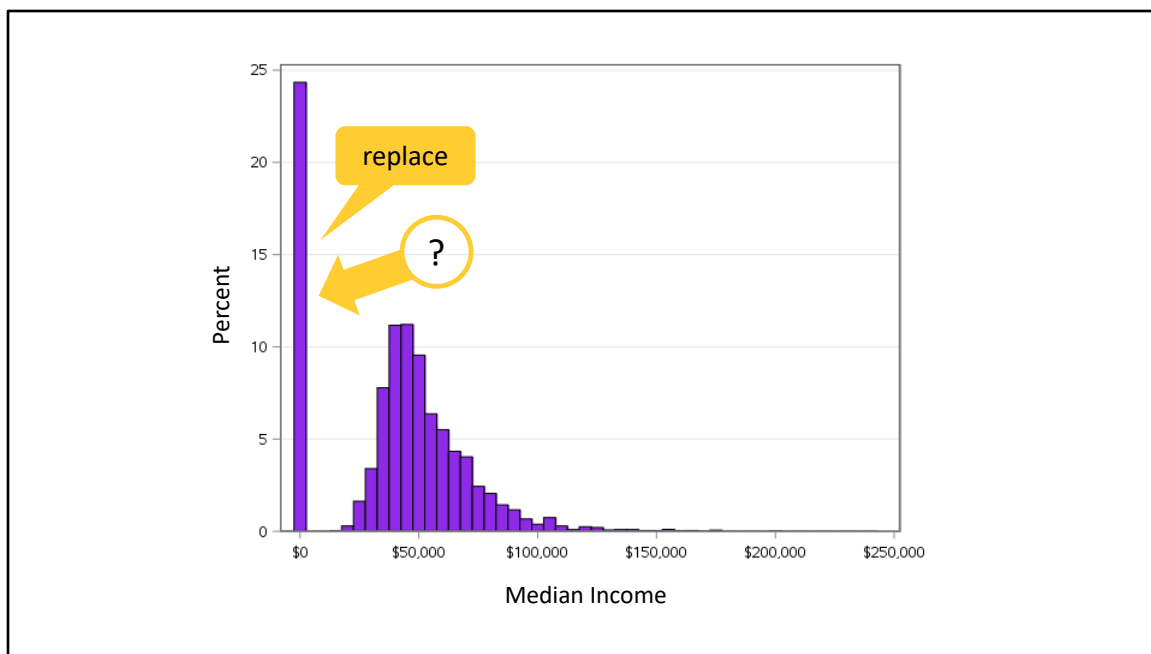


Essential Data Tasks

- ✓ Divide the data.
- ✓ Address rare events.
- ✓ Manage missing values.
 - **Replace incorrect values.**
 - Add unstructured data.
 - Extract features.
 - Handle extreme or unusual values.
 - Select useful inputs.



During data exploration, it is common to find that some data values are incorrect. Perhaps they don't make sense, or they are unrealistic. There are many ways to address this issue. The one we show you is to replace the incorrect values with more appropriate values.



Here is a histogram of income. Notice that there is a large spike of income values at 0. Could that many households in the data set actually have a value of 0 for income? Most likely, these values represent people who did not want to report their income. Instead of leaving a blank income field on a survey, those people might have written the value 0. Or, the data entry policy might have been to use a value of 0 for blank numeric fields.

In this situation, it makes sense to replace the 0 income values with more realistic numbers. Determining the best replacement values relies on business knowledge and familiarity with the data.

Demo: Replacing Incorrect Values Starting on the Data Tab

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Replacing Incorrect Values Starting with the Manage Variables Node

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



2.2 Extracting Features



Feature Extraction



Copyright © SAS Institute Inc. All rights reserved.

Feature Extraction



Essential Data Tasks

- ✓ Divide the data.
- ✓ Address rare events.
- ✓ Manage missing values.
- ✓ Replace incorrect values.
- Add unstructured data.
- Extract features.
- Handle extreme or unusual values.
- Select useful inputs.

feature
extraction

The next two essential data tasks (adding unstructured data and extracting features) are both methods of feature extraction. Feature extraction means creating new input variables based on inputs that are already in the data.

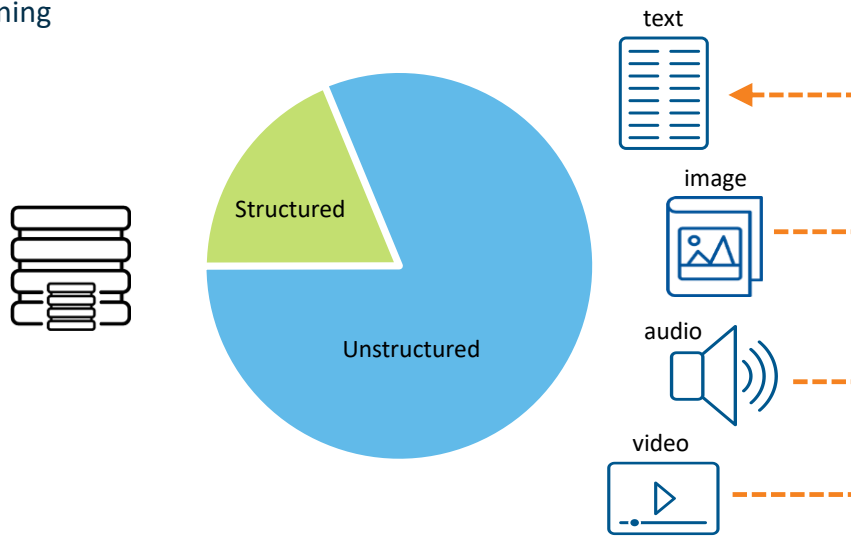
To add unstructured data, we convert unstructured text variables to usable numeric inputs. This involves text mining.

Text Mining



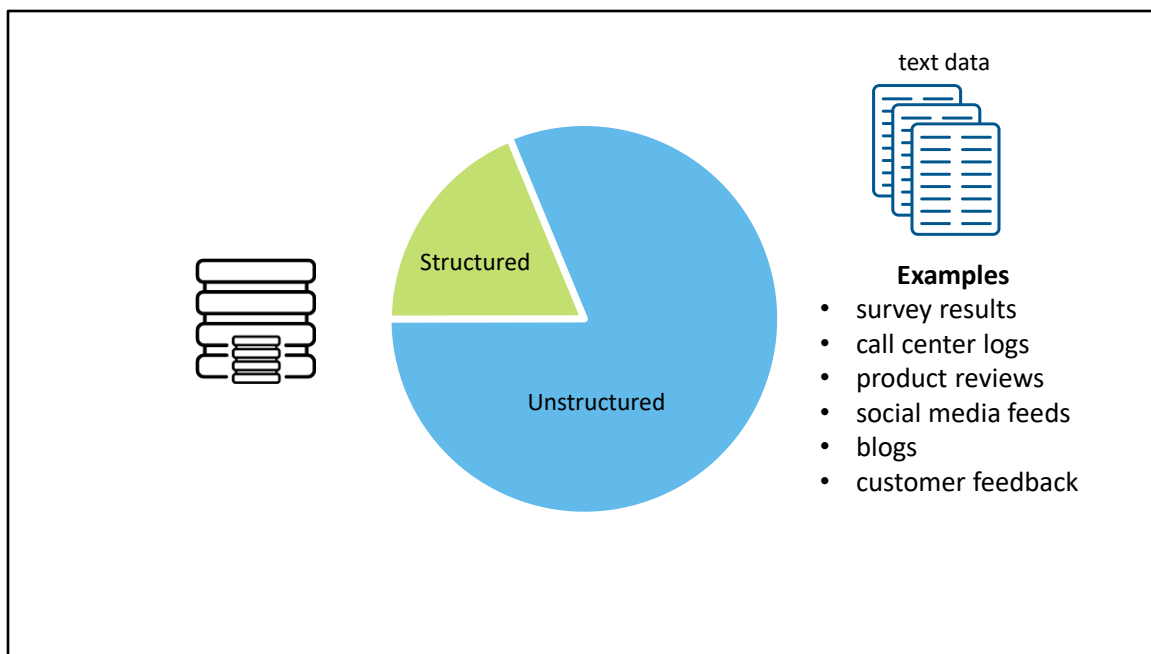
Copyright © SAS Institute Inc. All rights reserved.

Text Mining

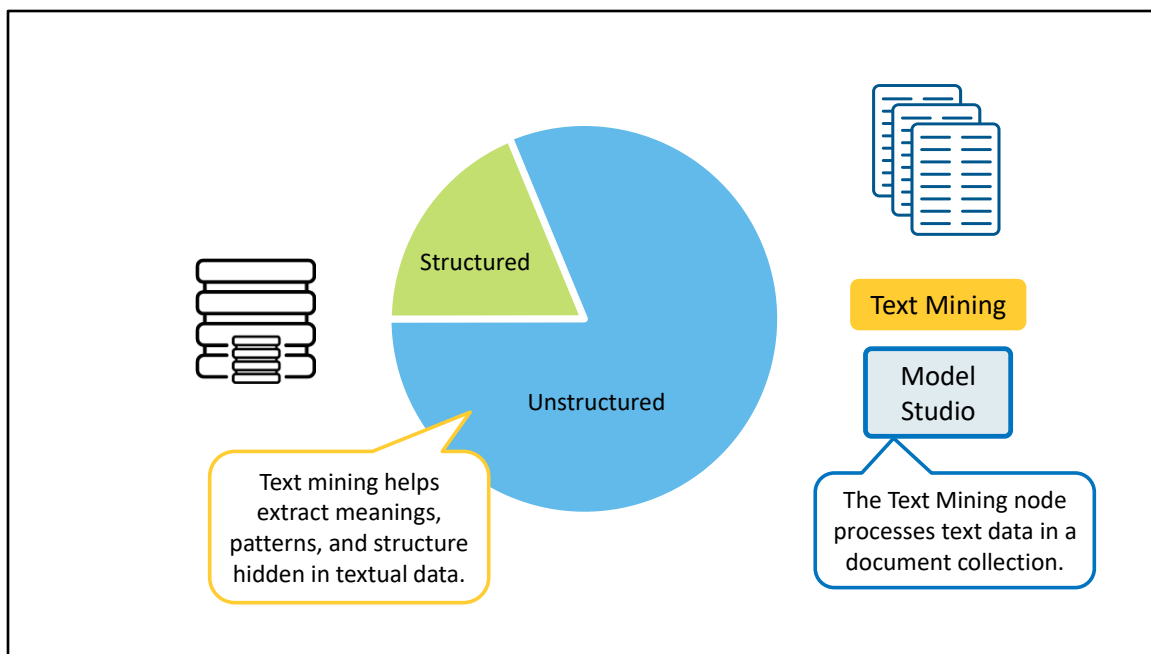


Organizations today are generating and storing tremendous amounts of data. A majority of that data is *unstructured* (that is, information that either does not have a predefined data model or is not organized in a predefined way).

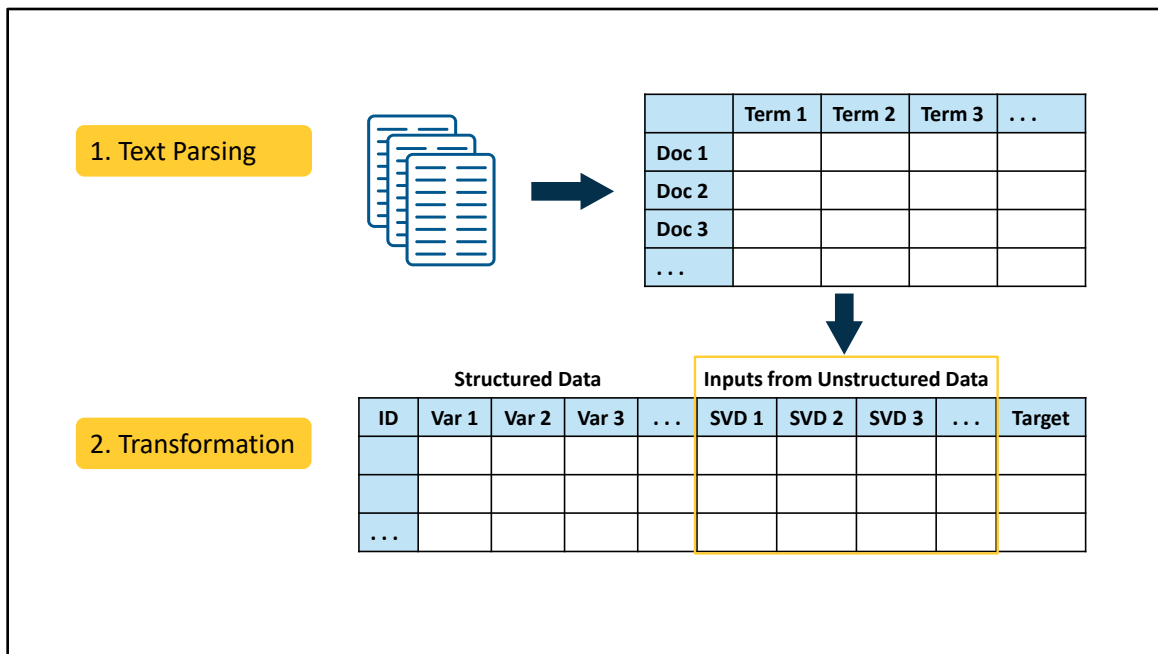
Unstructured data can be text or non-text (such as images, audio, or video). Non-textual data are often converted to text data, so we focus on text data in this course.



Unstructured text data are commonly available in business (for example, survey results, call center logs, product reviews, social media feeds, blogs, and customer feedback). These data contain information that is useful for predictive modeling outcomes, so it is important to analyze the unstructured text data sources along with the structured data.



Unlocking the full potential within these complex data sources can be tricky, but it is possible using text mining techniques. In Model Studio, the Text Mining node enables you to process text data in a document collection. Adding text mining results can improve the predictive ability of models that are based only on structured data.



In text mining, data are processed in two phases: text parsing and transformation. Text parsing processes textual data into a term-by-document frequency matrix. Transformations such as singular value decomposition (or SVD) change this matrix into a data set that is suitable for data mining purposes. As a result of text mining, a document collection with thousands of documents and terms can be represented in a compact and efficient form.

Singular Value Decomposition and the Text Mining Node

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Adding Text Mining Features

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Text Mining Node



Copyright © SAS Institute Inc. All rights reserved.

Question: Text Mining Node

Which statement is true about the new input columns that the Text Mining node creates?

- a. The Text Mining node creates a new input column for each topic that it discovers.
- b. The Text Mining node creates a new input column for each term within each topic.
- c. The measurement scale of each new input column is ordinal.
- d. Each new input column's name starts with a plus sign (+)

Answer: Text Mining Node

Which statement is true about the new input columns that the Text Mining node creates?

- a. The Text Mining node creates a new input column for each topic that it discovers.
- b. The Text Mining node creates a new input column for each term within each topic.
- c. The measurement scale of each new input column is ordinal.
- d. Each new input column's name starts with a plus sign (+)

CA: a

FB: The Text Mining node creates a new input column for each topic it discovers.

Feature Extraction Node

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



2.3 Transforming Inputs



Using Transformations to Handle Extreme or Unusual Values



Copyright © SAS Institute Inc. All rights reserved.

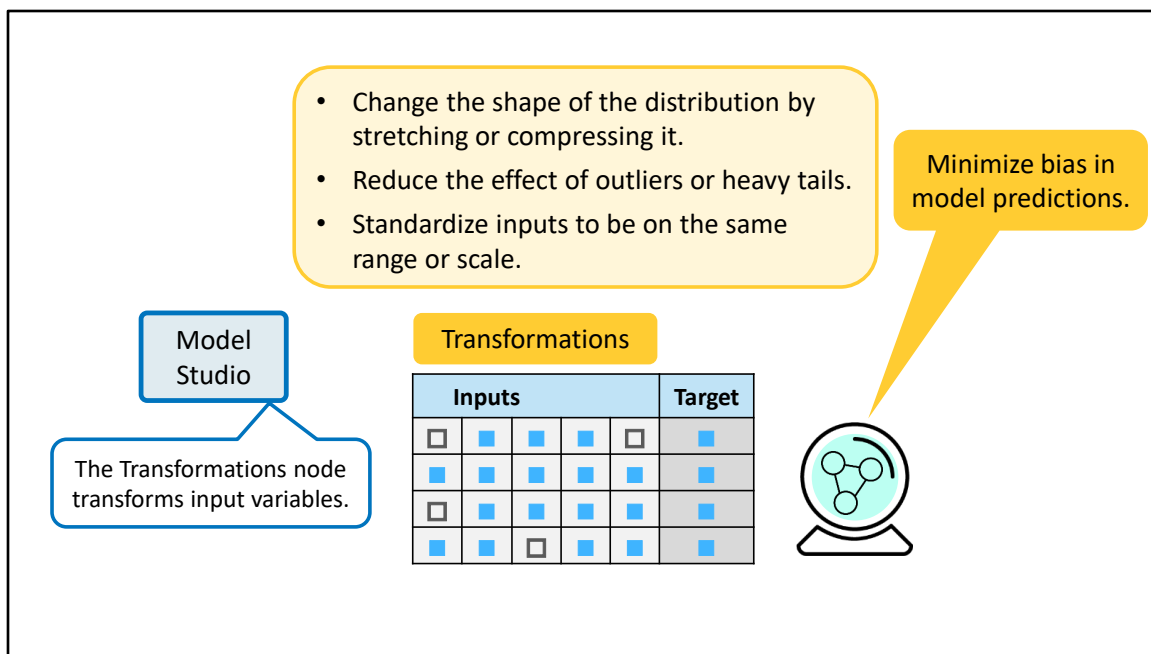
Using Transformations to Handle Extreme or Unusual Values



Essential Data Tasks

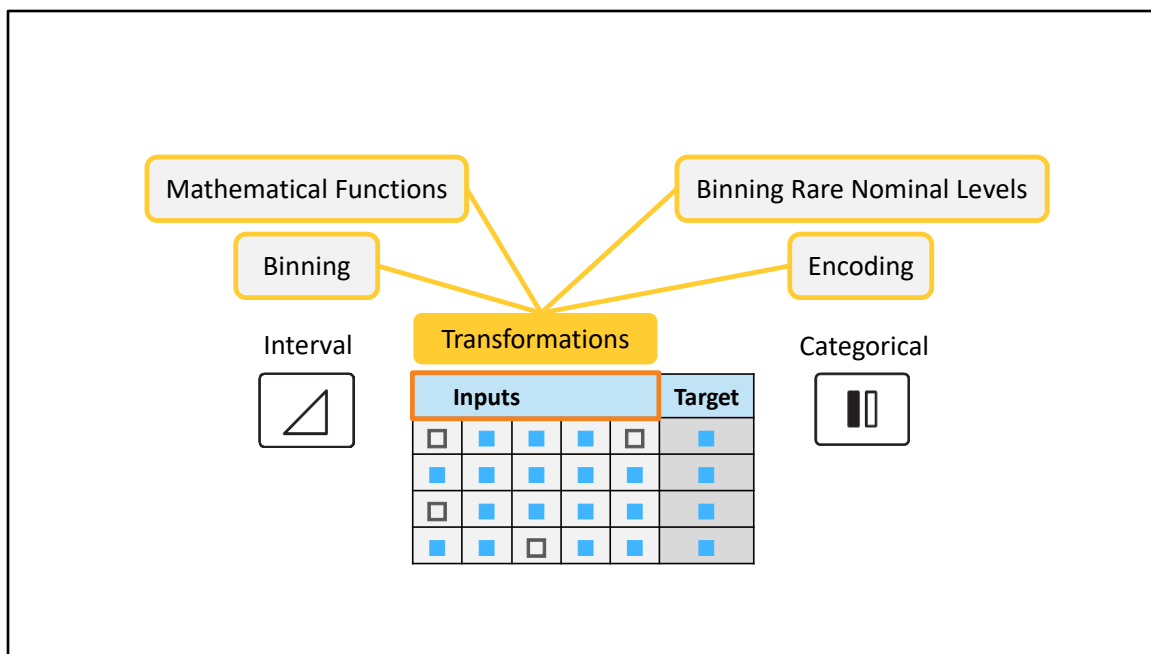
- ✓ Divide the data.
 - ✓ Address rare events.
 - ✓ Manage missing values.
 - ✓ Replace incorrect values.
 - ✓ Add unstructured data.
 - ✓ Extract features.
 - **Handle extreme or unusual values.**
 - Select useful inputs.
- transformations

Extreme or unusual values of the input variables can lead to problems in predictive modeling. These values might be outliers or they might occur in variables whose distributions are highly skewed (that is, not symmetric). Such values can lead to biased predictions from machine learning models, so it is a common practice to manage these values during data preprocessing. The solution is to apply transformations to the input variables.



Transformations can minimize bias in model predictions in various ways. Transformations are most commonly used to change the shape of the distribution of a variable by stretching or compressing it, reduce the effect of outliers or heavy tails, and standardize inputs to be on the same range or scale.

When you prepare data for modeling, you can use the Transformations node to transform input variables.



You choose from different transformation methods, depending on whether the input variable is interval or categorical. For interval inputs it is common to use transformations based on either mathematical functions or binning. For categorical inputs, you can bin rare nominal levels or apply various types of encoding.

Let's take a closer look at the transformations for interval inputs.

Mathematical Functions

- Centering
- Exponential
- Inverse
- Inverse Square
- Inverse Square Root
- $\text{Log}_e / \text{Log}_{10}$
- Range
- Square
- Square Root
- Standardize

Interval



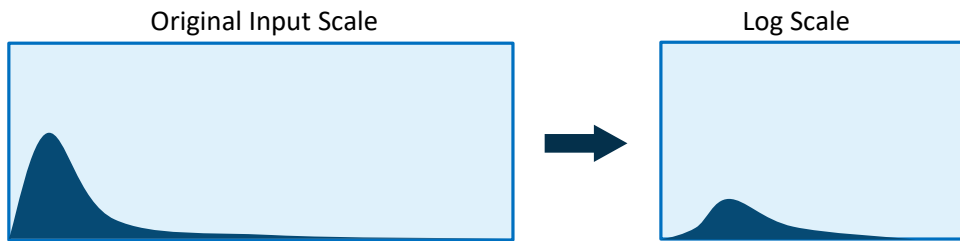
Transformations

Inputs					Target
☐	☒	☒	☒	☐	☒
☒	☒	☒	☒	☒	☒
☐	☒	☒	☒	☒	☒
☒	☒	☐	☒	☒	☒

Examples of mathematical functions are centering, exponential, inverse, log, range, square, square root, and standardize.

Mathematical Functions

Log

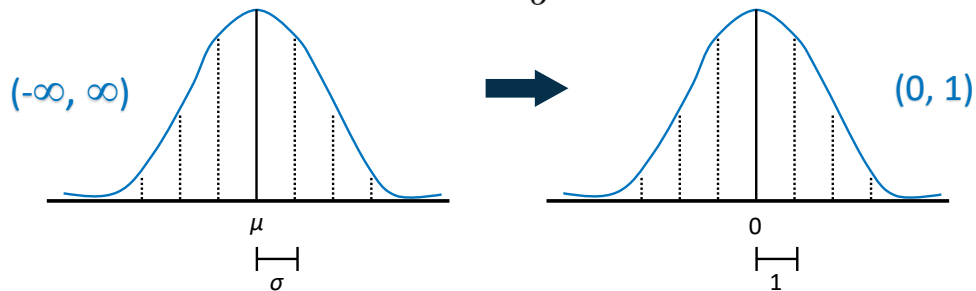


A distribution that is symmetric, or nearly so, is often easier to handle and interpret than a skewed distribution. In the distribution shown here, the variable is skewed to the right, meaning the right-hand tail is stretched out. Applying a log transformation to the input can reduce the skewness, thus making the shape more symmetric.

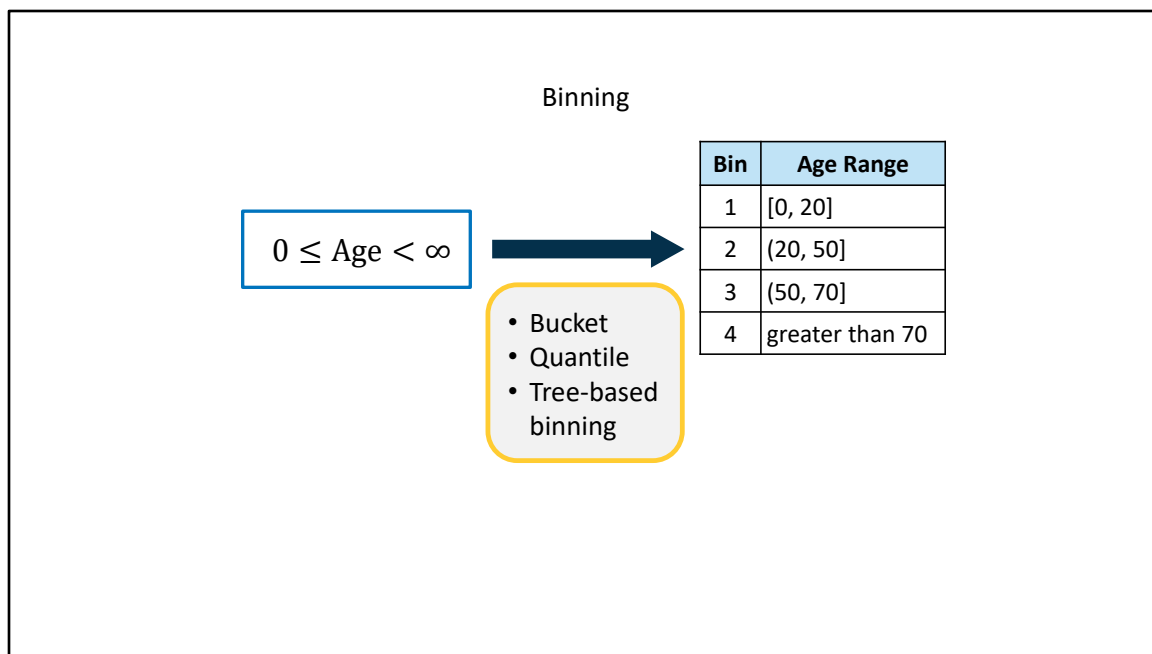
Mathematical Functions

Standardize

$$Z = \frac{x - \mu}{\sigma}$$

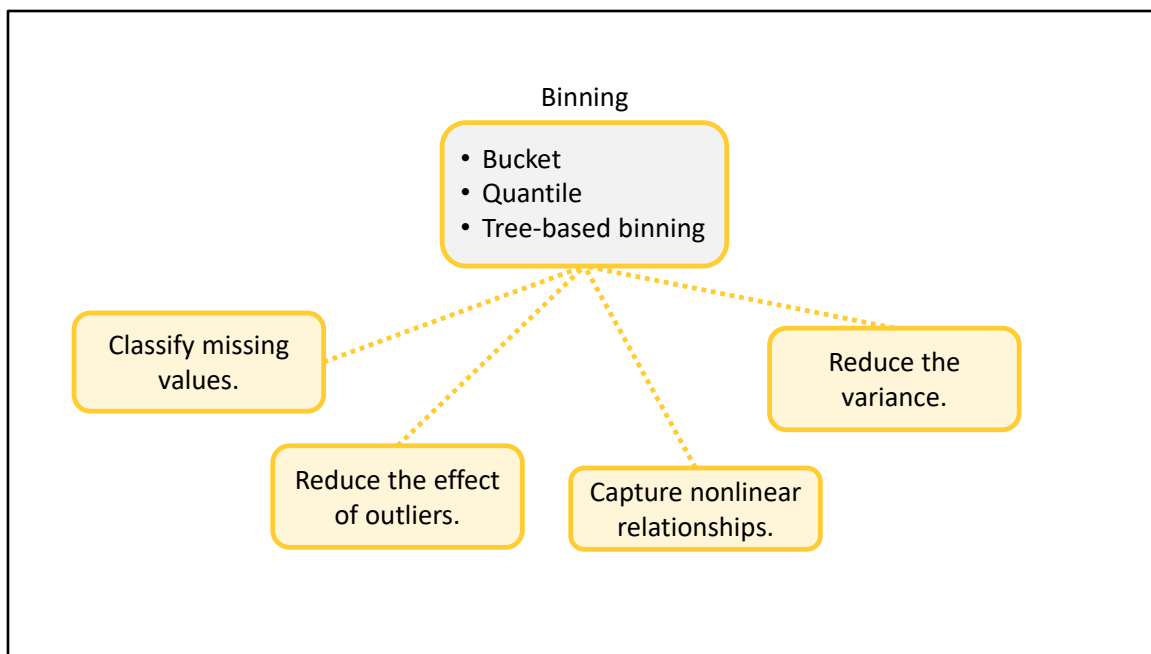


Another method for handling extreme distributions is to use a mathematical function to standardize the inputs. A statistical Z-standardization removes the scale of an input so that it is centered at zero and has a variance of 1. A range standardization transforms an input so that the values range from zero to 1.



Binning is a method of transformation that converts numeric inputs to categories or groups the levels of a high-cardinality input (that is, a categorical input with many levels). In the example shown here, Age is a variable that might theoretically range from zero to infinity. A binning transformation converts Age to an ordinal variable that has only four values, which are represented by bins one through four.

There are many types of binning transformations. In bucketing, the bins themselves are of equal width but the frequency count within each bin varies. Quantile binning creates bins of different widths, but the frequency count of observations is consistent, or at least similar, across the bins. Binning can even be done using a decision tree.



You can use binning to classify missing values, reduce the effect of outliers on a model, or capture nonlinear relationships. A binned version of a variable also has a smaller variance than the original numeric variable.

Transformations in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Transformations



Copyright © SAS Institute Inc. All rights reserved.

Question: Transformations

Transformations can do which of the following? (Select all that apply.)

- a. change the shape of the distribution of a variable
- b. reduce the effect of outliers or heavy tails.
- c. standardize inputs to be on the same range or scale.
- d. reduce the bias in model predictions

Answer: Transformations

Transformations can do which of the following? (Select all that apply.)

- ☒ a. change the shape of the distribution of a variable
- ☒ b. reduce the effect of outliers or heavy tails.
- ☒ c. standardize inputs to be on the same range or scale.
- ☒ d. reduce the bias in model predictions

CA: a,b,c,d

FB: Transformations can do all of these things.

Demo: Transforming Inputs

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Transformation Metadata



Copyright © SAS Institute Inc. All rights reserved.

Question: Transformation Metadata

When you specify a transformation as metadata for specific variables on the Data tab, that metadata rule overrides the default method specified in the Default interval inputs method property (None) for the Transformations node.

- a. True
- b. False

Answer: Transformation Metadata

When you specify a transformation as metadata for specific variables on the Data tab, that metadata rule overrides the default method specified in the Default interval inputs method property (None) for the Transformations node.

- ☒ a. True
- ☐ b. False

CA: a

FB: This statement is true. It is recommended that you specify transformations for specific variables on the Data tab (or in the Manage Variables node) instead of updating the method in the Transformation node's properties.

2.4 Selecting Features



Selecting Useful Inputs



Copyright © SAS Institute Inc. All rights reserved.

Selecting Useful Inputs



Essential Data Tasks

- ✓ Divide the data.
- ✓ Address rare events.
- ✓ Manage missing values.
- ✓ Replace incorrect values.
- ✓ Add unstructured data.
- ✓ Extract features.
- ✓ Handle extreme or unusual values.
- **Select useful inputs.** feature selection

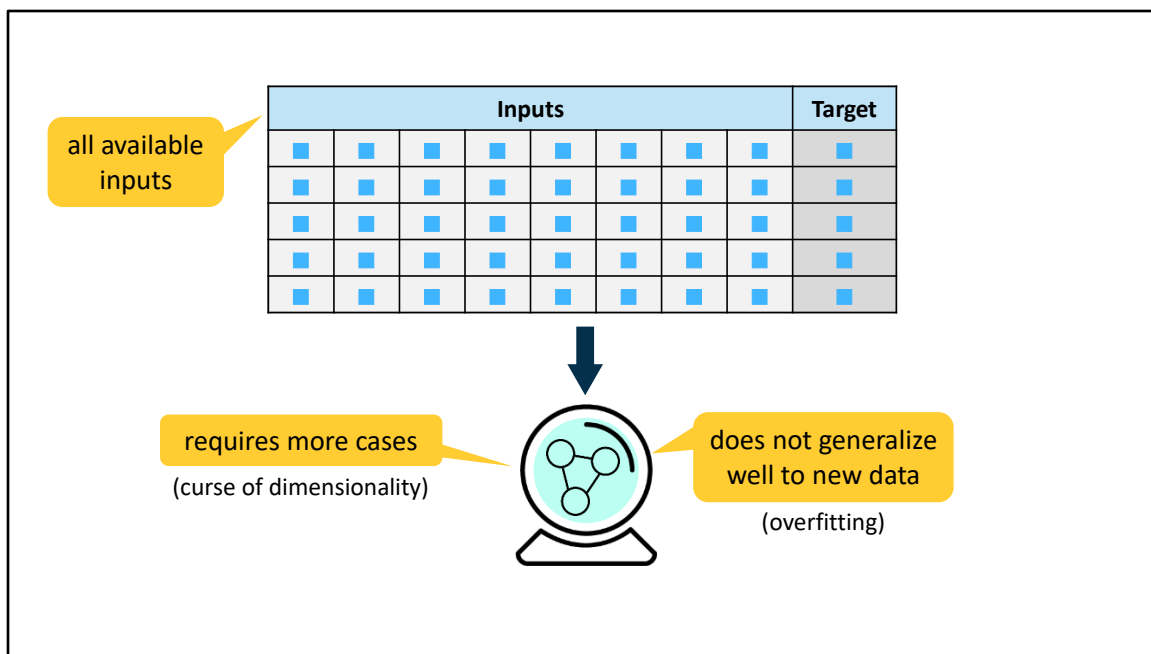
Selecting useful inputs (a process called feature selection) is an essential step in data preprocessing.

Inputs								Target
■	■	■	■	■	■	■	■	■
■	■	■	■	■	■	■	■	■
■	■	■	■	■	■	■	■	■
■	■	■	■	■	■	■	■	■
■	■	■	■	■	■	■	■	■

Use a subset of the input variables in the model.

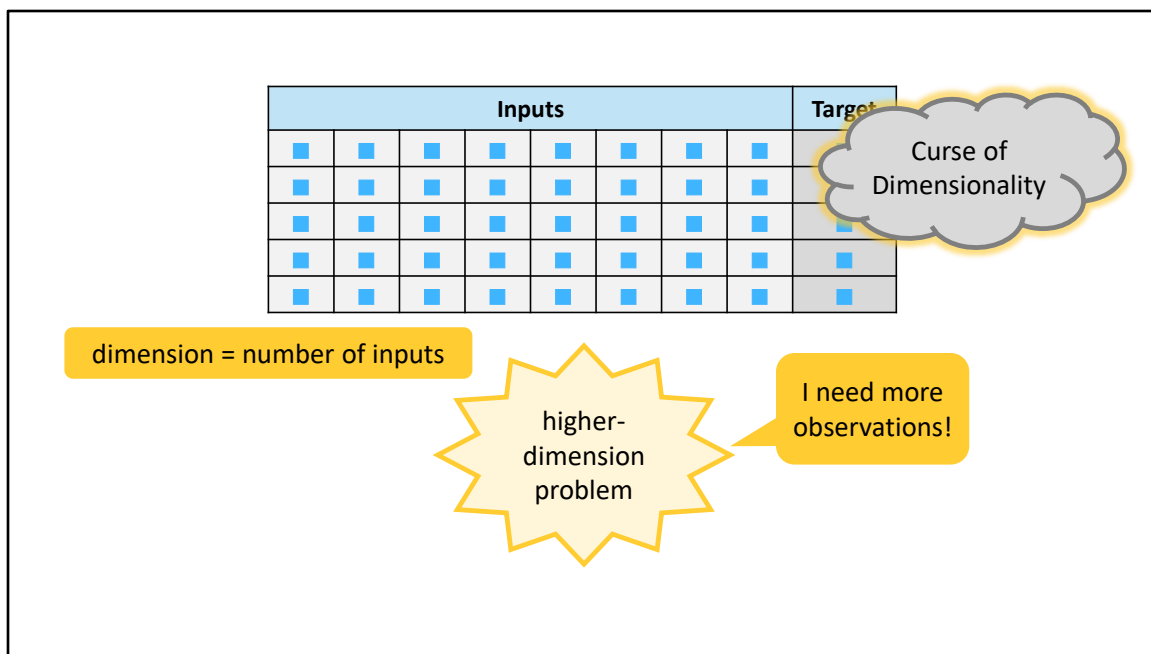


It is important to use a subset of the input variables in the final model for several reasons.



First, remember that using all available inputs almost always results in a predictive model that is overfit to the training data and does not generalize well to new data.

Also, the more inputs you use to build the model, the more cases are required to discover the relationship between the inputs and the target. Let's see why.



The dimension of a data mining problem refers to the number of input variables in the data. The raw data often have a large number of input variables, so data mining problems often have massive dimensions to start.

Higher-dimension problems require a larger number of observations to densely populate the space that represents the relationship between the inputs and the target. This is known as the curse of dimensionality.

Inputs								Target
■	■	■	■	■	■	■	■	■
■	■	■	■	■	■	■	■	■
■	■	■	■	■	■	■	■	■
■	■	■	■	■	■	■	■	■
■	■	■	■	■	■	■	■	■

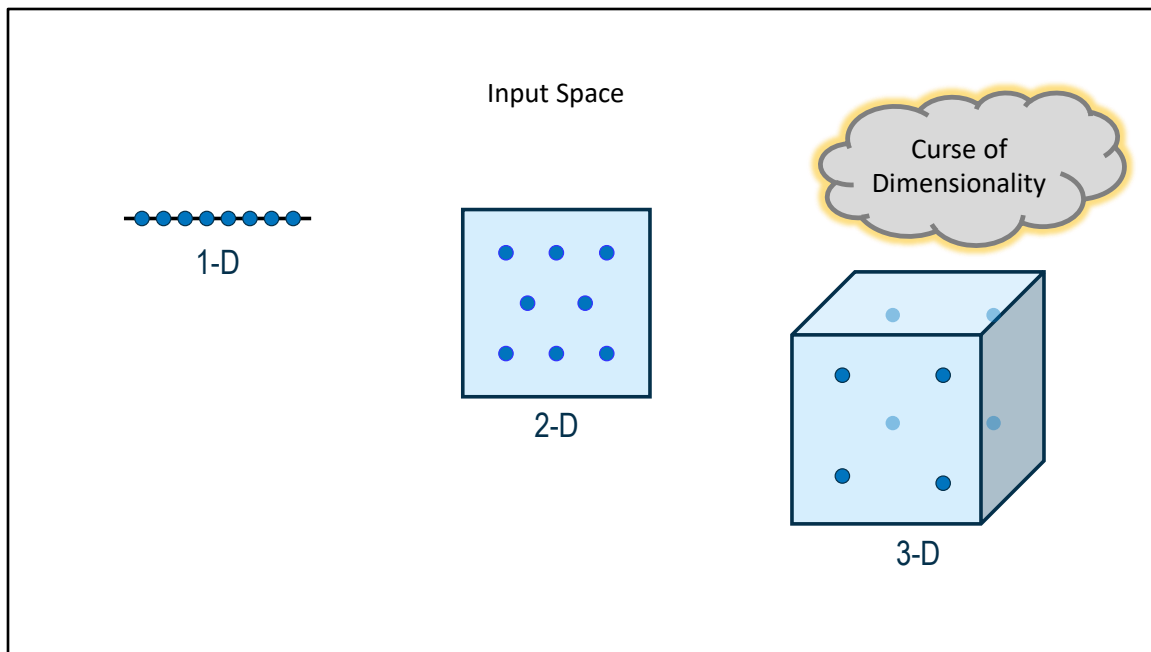
Curse of Dimensionality

highly complex model

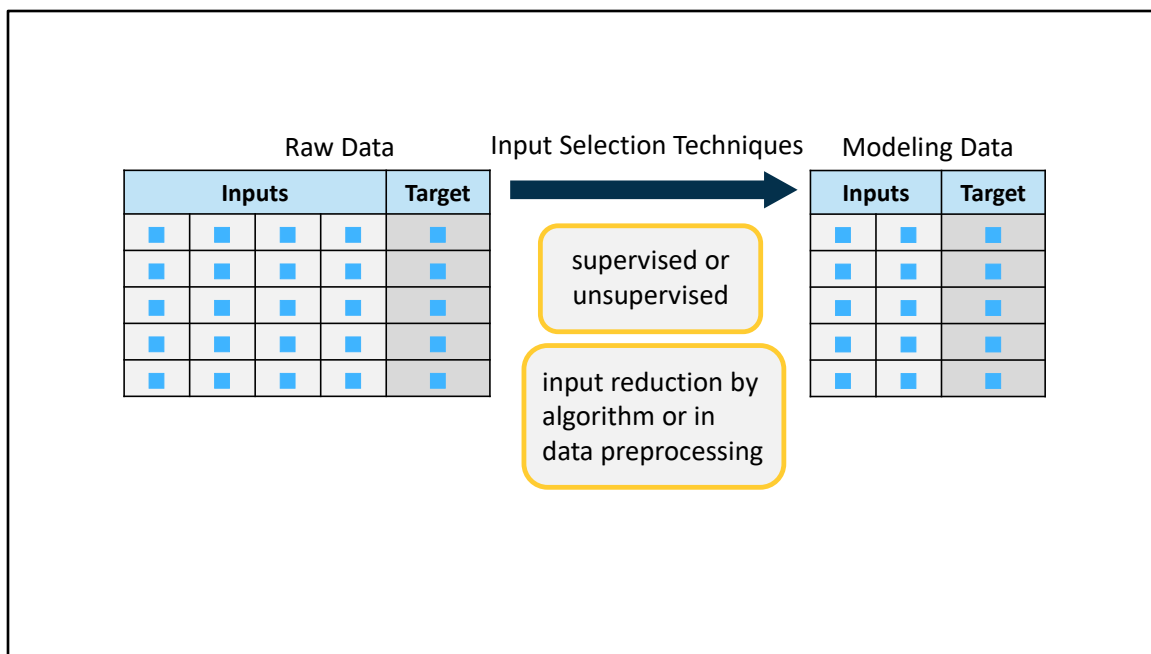


I need a densely populated input space!

It is especially important to have a densely populated input space when fitting highly complex models.

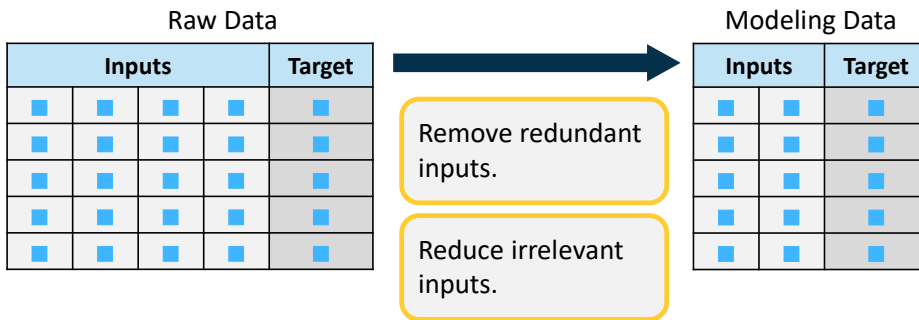


The basic example shown here assumes eight observations. The eight data points very densely fill a one-dimensional input space (that is, a line). These eight points less densely fill a two-dimensional input space (that is, a plane), and even less densely fill a three-dimensional input space (that is, a cube). Imagine how sparse the same eight data points would be in a 10-dimensional, or even 100-dimensional input space. This example shows why it is important to select a subset of inputs.

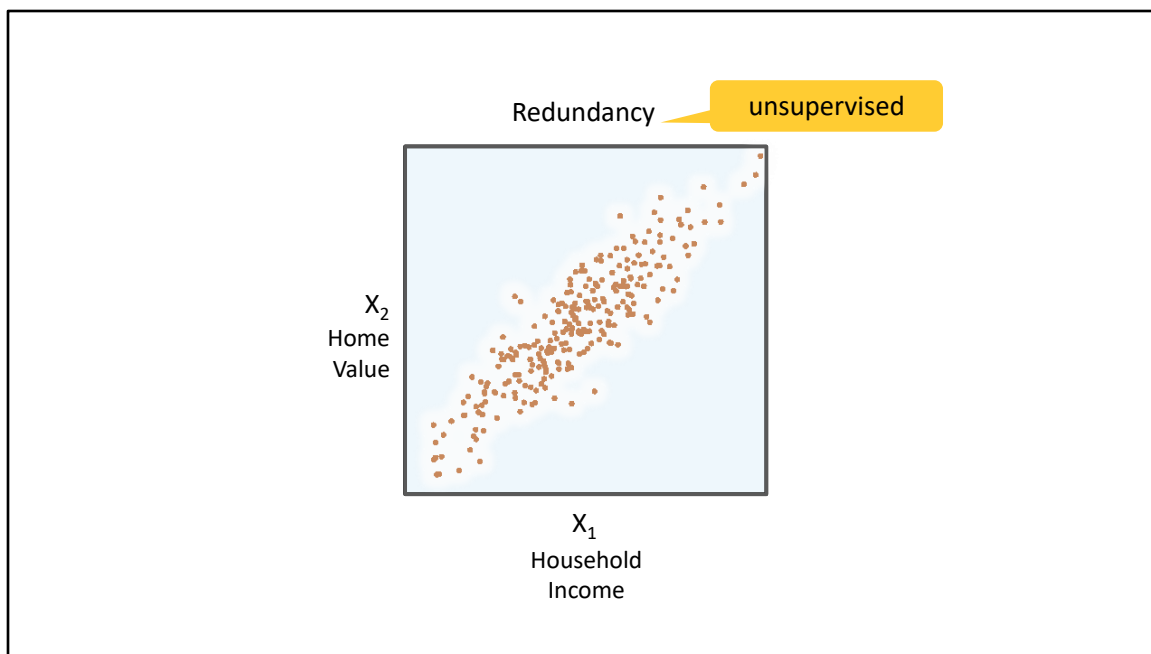


There are many techniques for selecting inputs for a model. Some of these methods are supervised, which means that the target variable is used in the selection process. Other methods are unsupervised and ignore the target. Further, during model building, some modeling algorithms reduce the number of inputs (for example, decision trees), while other algorithms do not (for example, neural networks). For algorithms that do not reduce the number of inputs, it is especially important to reduce the number of inputs during data preprocessing.

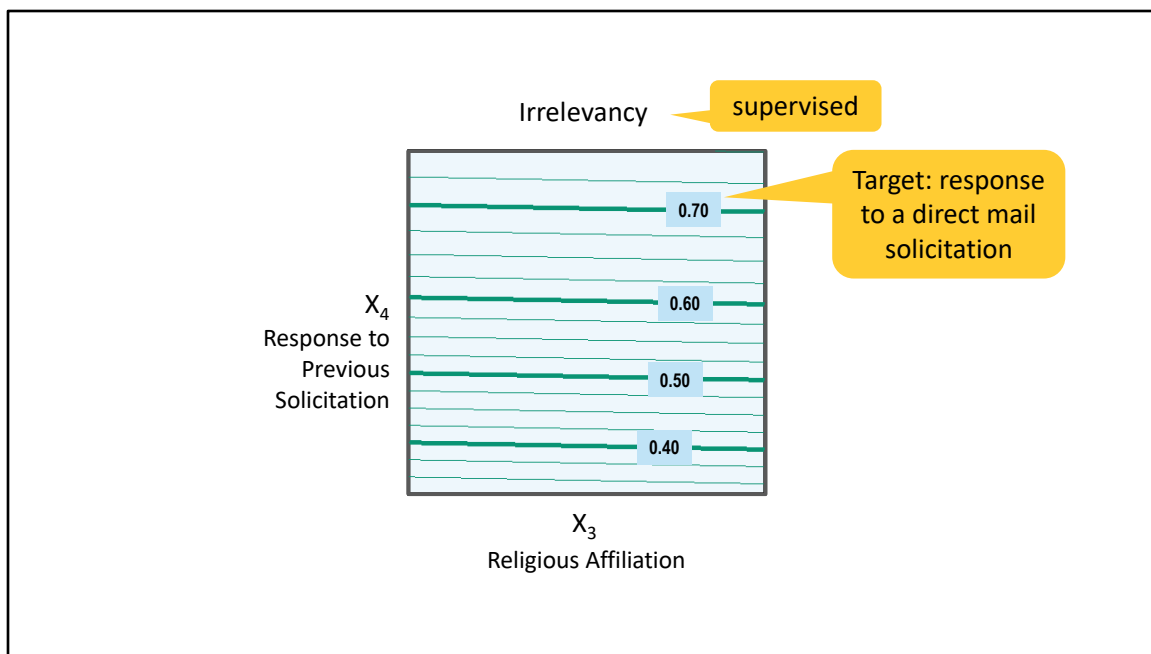
Do not disregard important information!



When we reduce the dimensionality of a problem by selecting inputs, we need to be careful not to disregard important information. Two ways to avoid this are to remove redundant inputs or reduce irrelevant inputs. Let's take a quick look at each method.



A *redundant* input does not give any new information (that is, information that was not already explained by other inputs). In the example shown here, the inputs X_1 and X_2 are highly correlated, so the value of input X_1 gives you a good idea of the value of X_2 . In a business context, two such variables might be household income and home value. Inputs that are highly correlated should not appear in the same model. The target is not used to determine redundant inputs, so redundancy is an unsupervised concept.



An *irrelevant* input does not provide information about the target. In the example shown here, the target values change with changes in X_4 , but not with changes in X_3 .

A business example of irrelevancy is the relationship between religious affiliation and response to a direct mail solicitation. Religious affiliation does not change the probability of response to a direct mail solicitation. However, the response to a previous solicitation does change the probability of responding to a current mail solicitation. If an input is irrelevant to a given target, there is clearly no reason to have it in a model. Irrelevancy relates to whether a variable affects the target, so that it is a supervised concept.

Demo: Selecting Features

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Variable Selection Node



Copyright © SAS Institute Inc. All rights reserved.

Question: Variable Selection Node

In the results from the Variable Selection node is a Variable Selection table with a Reason column. What does it mean when the Reason column for an input is blank?

- a. The input was selected.
- b. The input was rejected.
- c. The node did not have enough information to decide whether to select or reject the input.

Answer: Variable Selection Node

In the results from the Variable Selection node is a Variable Selection table with a Reason column. What does it mean when the Reason column for an input is blank?

- a. The input was selected.
- b. The input was rejected.
- c. The node did not have enough information to decide whether to select or reject the input.

CA: a

FB: A blank **Reason** column indicates that the input was selected.

Feature Selection and the Variable Selection Node: Details

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Saving a Pipeline to the Exchange

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



2.5 Best Practices in Data Preparation



Best Practices for Common Data Preparation Challenges

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Data Collection Challenges



Copyright © SAS Institute Inc. All rights reserved.

Question: Data Collection Challenges

Biased and incomplete data, sparsity, and high-dimensionality are common data collection challenges. What should you do to overcome these challenges? (Select all that apply.)

Hint: Refer to **Best Practices for Common Data Preparation Challenges**.

- a. Take time to understand the business problem and its context.
- b. Enrich the data.
- c. Select or extract features (or do both) to reduce data dimensions.
- d. Change the representation of the data by applying appropriate transformations.

Answer: Data Collection Challenges

Biased and incomplete data, sparsity, and high-dimensionality are common data collection challenges. What should you do to overcome these challenges? (Select all that apply.)

Hint: Refer to **Best Practices for Common Data Preparation Challenges**.

- ☒ a. Take time to understand the business problem and its context.
- ☒ b. Enrich the data.
- ☒ c. Select or extract features (or do both) to reduce data dimensions.
- ☒ d. Change the representation of the data by applying appropriate transformations.

CA: a,b,c,d

FB: All of the listed actions are recommended for these common data collection challenges.

Feature Engineering in SAS Viya

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



2.6 Selecting an Algorithm



Starting the Discovery Phase of the Analytics Life Cycle



Copyright © SAS Institute Inc. All rights reserved.

Starting the Discovery Phase of the Analytics Life Cycle



Essential Data Tasks

- ✓ Divide the data.
- ✓ Address rare events.
- ✓ Manage missing values.
- ✓ Replace incorrect values.
- ✓ Add unstructured data.
- ✓ Extract features.
- ✓ Handle extreme or unusual values.
- ✓ Select useful inputs.

You prepared the data for analysis.



Essential Discovery Tasks

- Select an algorithm.
- Improve the model.
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.
- Build ensemble models.
- Attempt other algorithms.

Now you're moving into the Discovery phase of the analytics life cycle. There are several essential tasks for discovery. The first one is to select an algorithm. Keep in mind there is no single model that works best for every problem. If that were the case, you would need to use only one algorithm! Take the time to experiment with different algorithms and different modeling options until you find the best model for your particular application.

In this course, you learn how to build several types of machine learning models. For each model, you get to try out different settings in your search for the best model. You also have a chance to compare the models that you built.

The next few resources provide general considerations for selecting an algorithm as well as details about a variety of predictive models. You'll likely want to skim these resources now, and then return to them later, after you've had experience building each type of model. This course can serve as a reference later.

Supervised Learning Algorithms in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Considerations for Selecting an Algorithm

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Algorithm Selection



Copyright © SAS Institute Inc. All rights reserved.

Discussion

Suppose you are assigned to build a model for the marketing team in your company. They want to use a data-driven approach to design an effective direct marketing campaign. They also want to understand the relationship between various marketing attributes and the likelihood that a customer will purchase a product. You will work with a large data set that contains information for all customers who have bought a product since the company was started five years ago. You have not seen the data yet. The project deadline is the end of the second quarter, which is less than two months away.

What are some key considerations for selecting algorithms? Which algorithms might be appropriate for this project, based on what you know?



FB: When selecting algorithms for this project, you might want to consider the following:

Size and nature of the data: You know that the data set is large. Linear models can be a good choice for large data sets because of their training efficiency and the curse of dimensionality. However, you have not had a chance to explore the data, so you do not know whether the relationship between the inputs and the target is linear. Therefore, you cannot rule out the more sophisticated machine learning algorithms.

Analysis goal: The marketing team wants to design an effective direct marketing campaign, which involves predicting whether customers will make a purchase (the target). If prediction were the only goal, and barring any time constraints, you would want to consider the most sophisticated algorithm that you could afford in terms of time and computational resources. However, remember that the project also has a second goal: to understand the relationship between the inputs (various marketing attributes) and the target. This second goal requires a model that is interpretable, which narrows your choices to regression or decision trees.

Time to build the model: The deadline for the project is less than two months away. After you explore and prepare the data, you will have a better idea of how much time is left to build models.

These are only a few of the considerations for selecting algorithms. As you learn more about a project, other considerations might arise.

Comparison of Modeling Algorithms

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



2.7 Lesson Conclusion



Lesson Quiz

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Lesson 3: Decision Trees and Ensembles of Trees



3.0 Lesson Overview



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



3.1 Building a Default Decision Tree Model



Basics of Decision Trees



Copyright © SAS Institute Inc. All rights reserved.

Basics of Decision Trees



Essential Discovery Tasks

- ✓ Select an algorithm.
- Improve the model.
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.
- Build ensemble models.



For the first task in the Discovery phase of the analytics lifecycle, we've selected the *decision tree* algorithm. In this lesson, you learn to build individual decision trees as well as tree-based ensemble models. To start, you learn the basics about decision trees.

Analysis Goal: Predict the outcome based on the location in the plot.

IF $X_2 < 0.63$ THEN branch left.
ELSE IF $X_2 \geq 0.63$ THEN
branch right.

rules

root node

$X_2 < 0.63$

yes

no

interior node

$X_1 < 0.52$

$X_1 < 0.51$

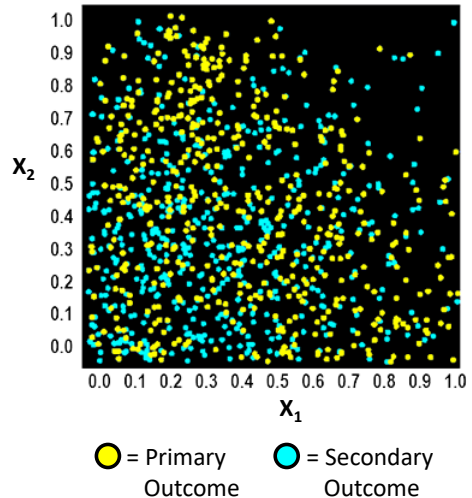
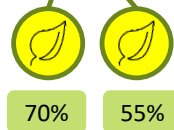
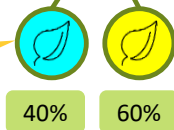
yes

no

yes

no

leaf node



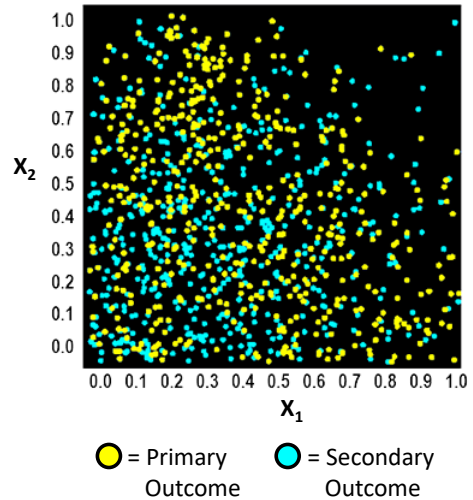
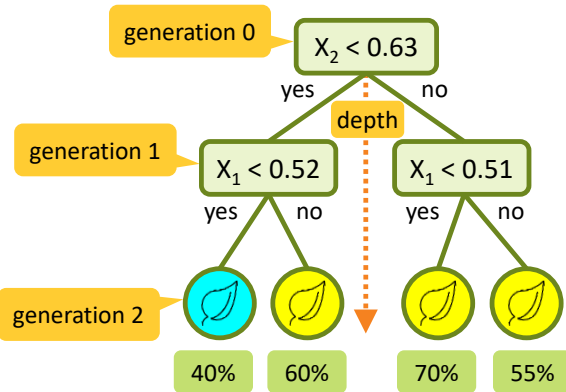
Let's look at a simple example. In this plot, the input X_1 is on the horizontal axis and the input X_2 is on the vertical axis. The values of the binary target are plotted according to the corresponding values of the two inputs. The target outcome is represented by a color: yellow is the primary outcome and blue is the secondary outcome. The analysis goal is to predict the outcome based on the location in the plot.

To predict cases (that is, to score data), decision trees use rules that specify a decision based on the values of the input variables. The rules are expressed in Boolean logic, which means that they are if-then-else statements. The rules are arranged hierarchically in a tree-like structure with nodes connected by lines. The nodes represent the rules, and the lines order the rules.

The first rule, at the base (that is, the top) of the tree, is named the *root node*. For example, the rule in the root node shown here is: If the value of X_2 is less than 0.63, branch left; else if the value of X_2 is greater than or equal to 0.63, branch right.

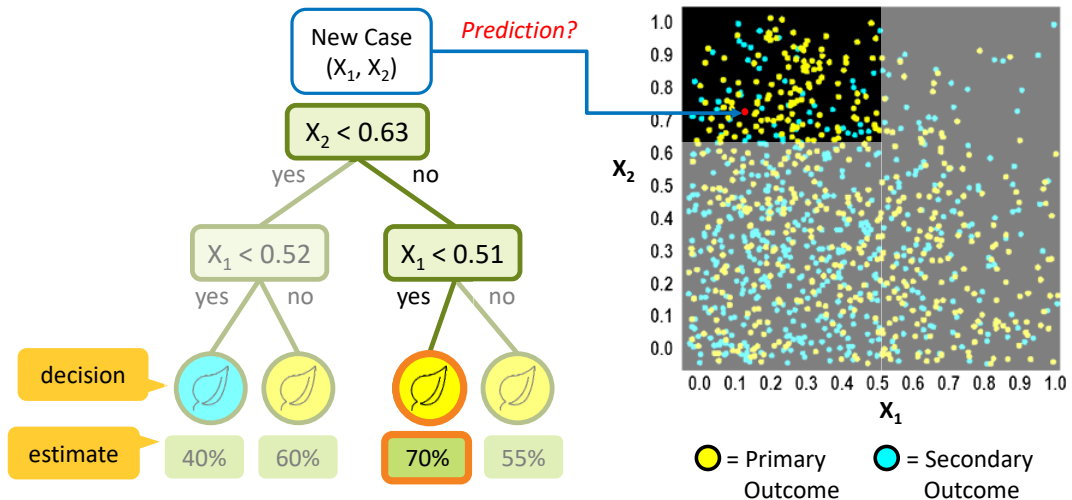
Subsequent rules are named *interior nodes*. Nodes with only one connection are *leaf nodes*.

Analysis Goal: Predict the outcome based on the location in the plot.



The *depth* of a tree specifies the number of generations of nodes. The root node is generation 0. The children of the root node are the first generation, and so on. The tree shown here stops at the second generation.

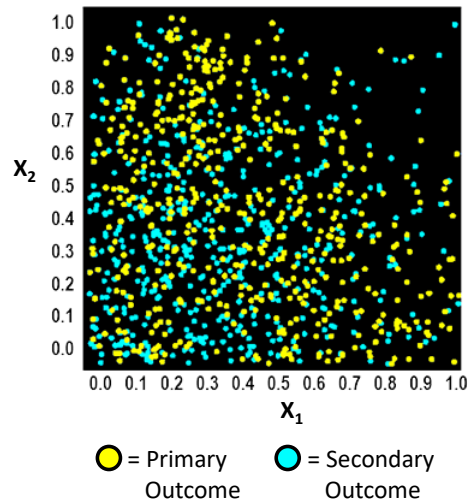
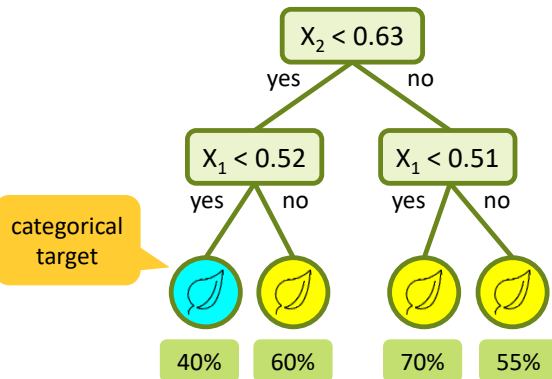
Analysis Goal: Predict the outcome based on the location in the plot.



To score a new case, we examine the input values and apply the rules defined by the decision tree. The input values of a new case eventually lead to a single leaf in the tree. A tree leaf provides a prediction that typically includes a decision (for example, to classify the outcome as yellow) and an estimate (for example, the primary-target proportion of 70%).

Analysis Goal: Predict the outcome based on the location in the plot.

✓
An interval target can also be used.



In this example, the target is a categorical target. However, as you'll see later, decision trees can also be used for interval targets.

Question: Decision Tree Nodes



Copyright © SAS Institute Inc. All rights reserved.

Question: Decision Tree Nodes

Which component of a decision tree provides the predictions?

- a. child nodes
- b. interior nodes
- c. leaf nodes
- d. nodes in generation 0
- e. root nodes

Answer: Decision Tree Nodes

Which component of a decision tree provides the predictions?

- a. child nodes
- b. interior nodes
- ☒ c. leaf nodes
- d. nodes in generation 0
- e. root nodes

CA: c

FB: A tree's leaf nodes provide the predictions. Leaf nodes are child nodes, but not all child nodes are leaves. Generation 0 is the root level of the tree, which has one root node. The root node does not make predictions. The interior nodes are all the nodes between the root node and the leaves.

Demo: Building a Decision Tree Model Using the Default Settings

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Supervised Learning Node Results Window

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Score Code in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Interactively Edit a Decision Tree

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



3.2 Modifying the Model: Tree Structure

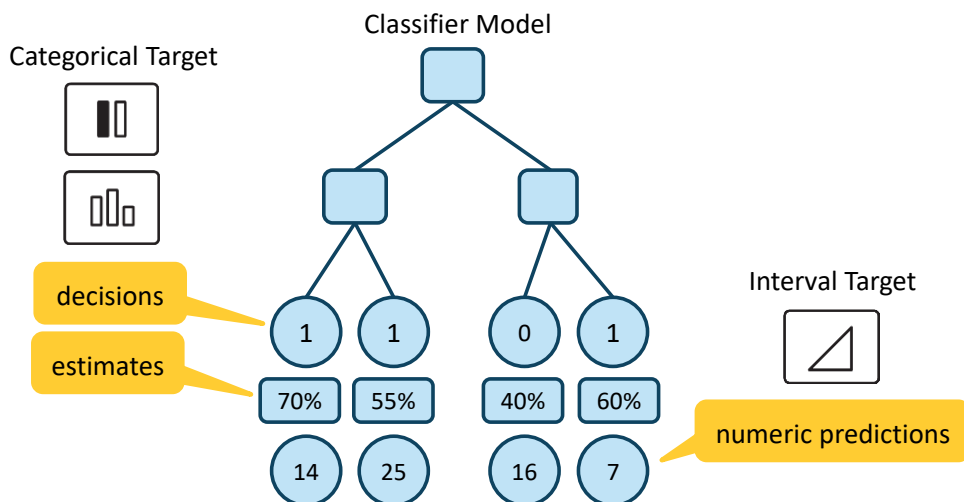


Decision Trees for Categorical Targets: Classification Trees

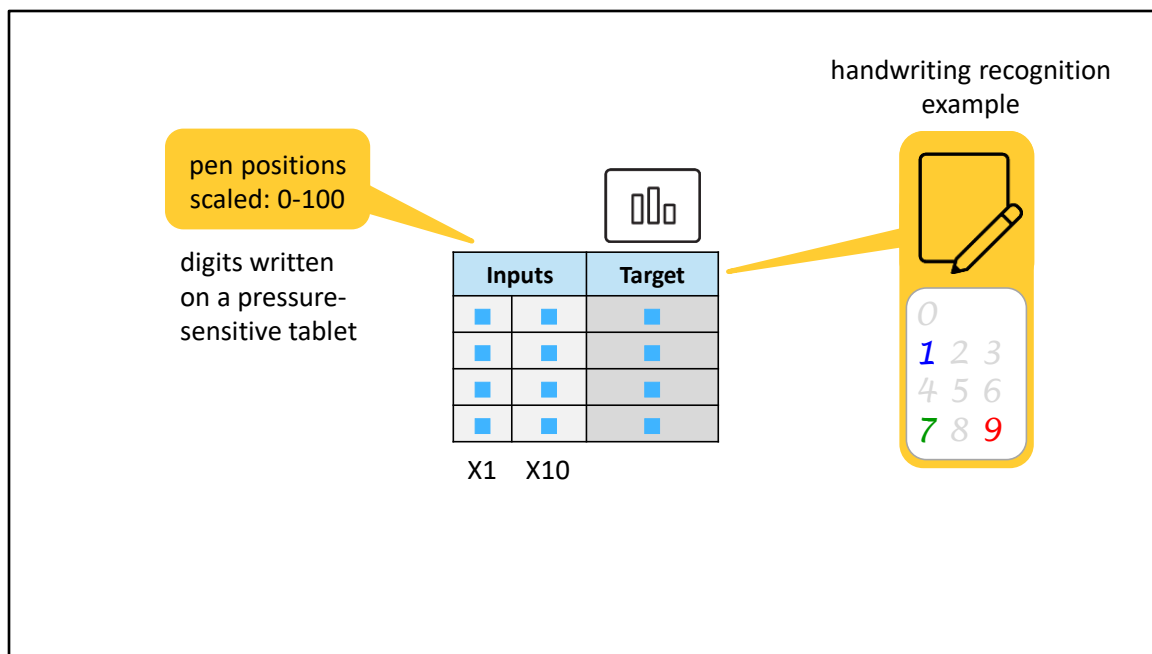


Copyright © SAS Institute Inc. All rights reserved.

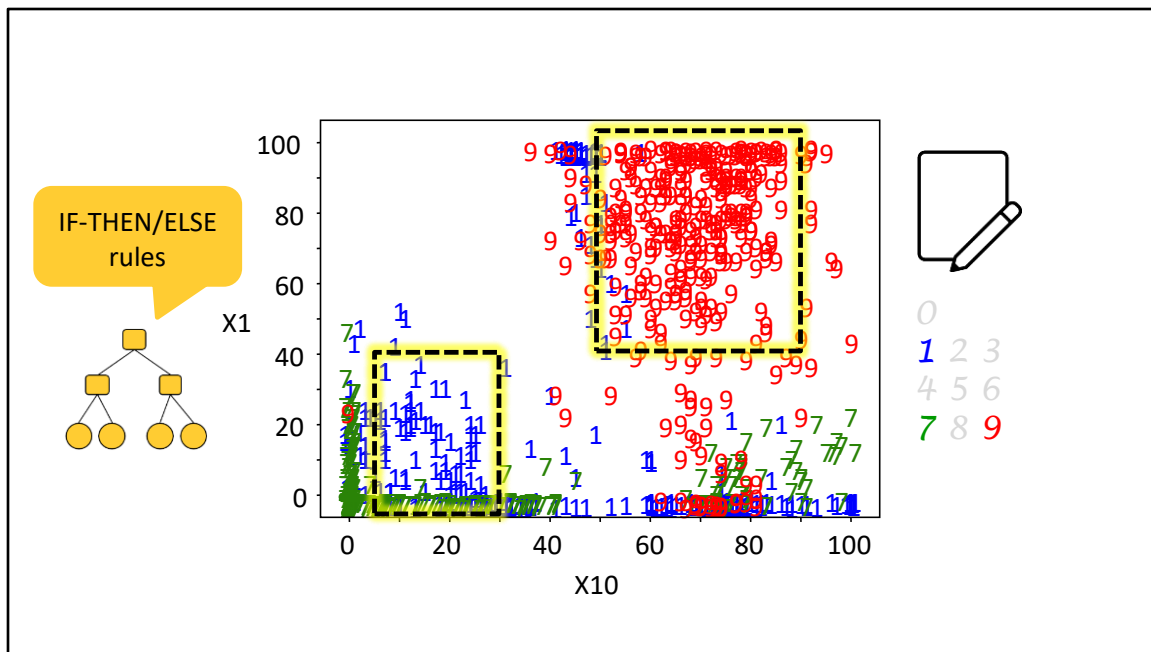
Decision Trees for Categorical Targets: Classification Trees



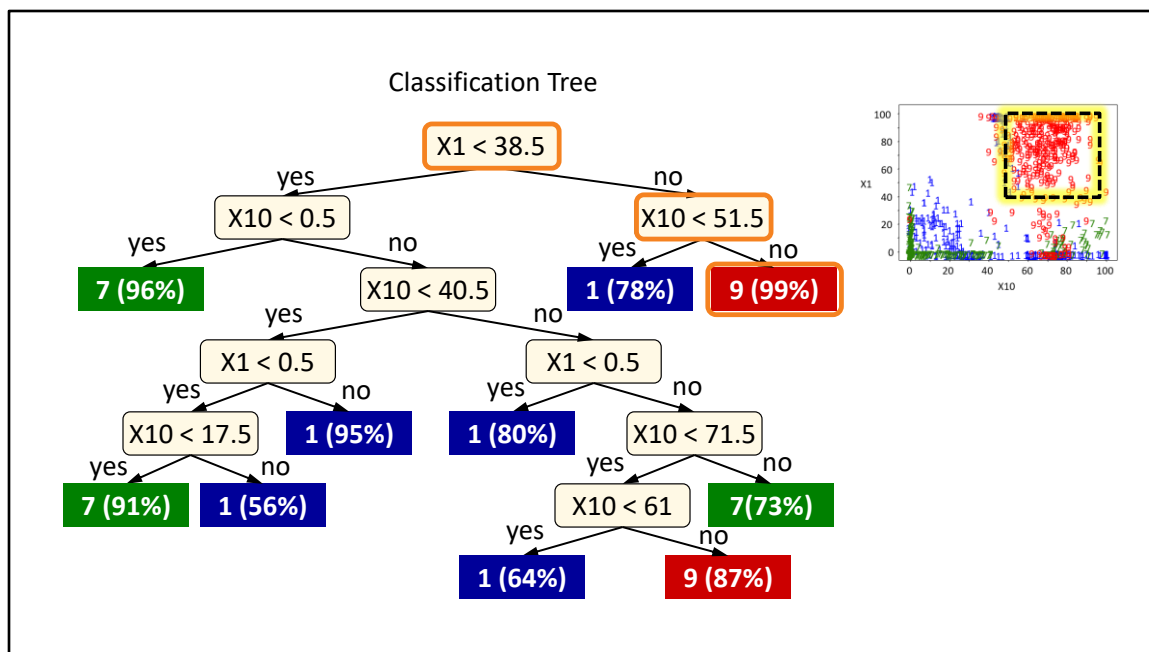
Decision trees were originally developed to be pure classifier models (in other words, to make *decisions* for a categorical target). They have since been expanded to provide estimates such as probabilities for categorical targets and numeric predictions for interval targets.



Handwriting recognition is a classic application for a classifier model. In the example shown here, the categorical target is a hand-written digit, from 0-9. The digits were written on a pressure-sensitive tablet. The input variables are measurements of the positions of the pen, and the inputs are scaled between 0 and 100. In this example, the goal is to predict whether the written digit is a 1, a 7, or a 9. The analysis data set contains a subset of cases: only the cases in which the target is one of the specified digits. The analysis data set contains two of the original 16 inputs: X1 and X10.

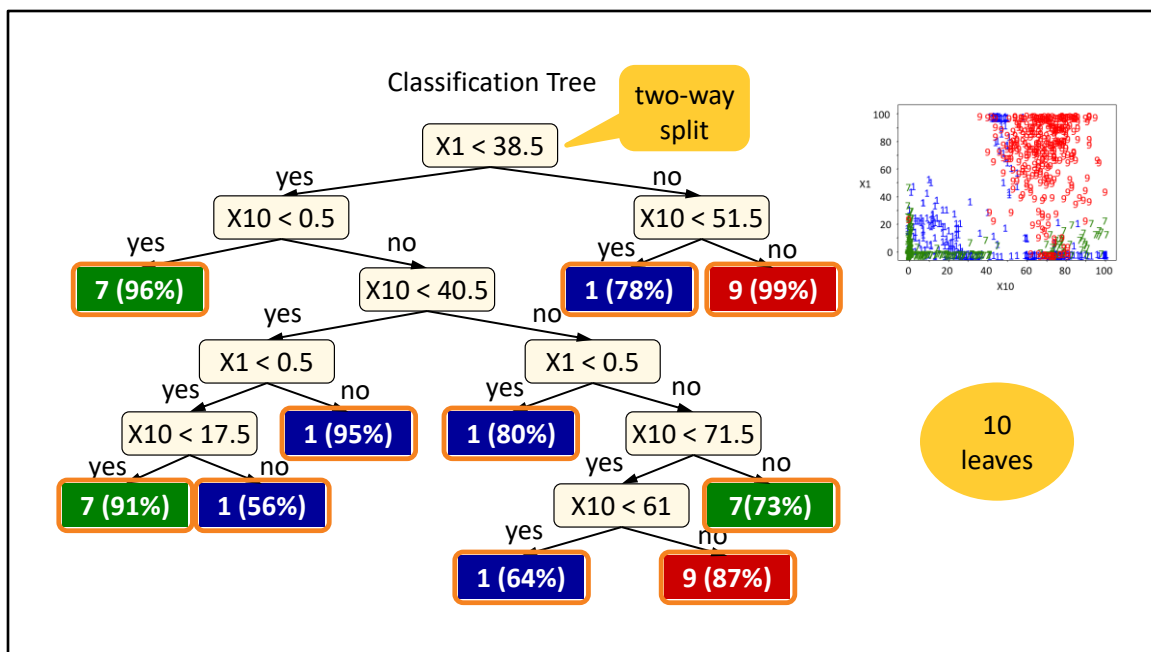


Notice that, in general, specific digits tend to group together based on specific values, or ranges, of the inputs. There are many 9s when X1 is approximately greater than 40, and X10 is between 55 and 90. There are many 1s when X1 is between 5 and 40 and X10 is between 5 and 30. A decision tree is based on if-then-else rules based on the values of the inputs. If X1 takes certain values and X10 takes certain values, then the outcome is likely to be a certain digit.



Here is a decision tree built against the subset of the handwriting data. Do you remember that group of 9s in the upper right quadrant of the data? In the tree, we can get to the leaf node for that group in two steps. Starting at the root node, if X_1 is *not* less than 38.5 *and* X_{10} is *not* less than 51.5, then there is a 99% chance that the written digit is a 9. In other words, the model found a subset of the data where 99% of the target values correspond to the written digit 9.

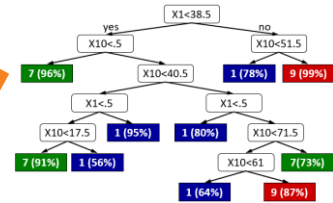
This type of decision tree is called a *classification* tree because it is built against a categorical target.



For this model, each branch of the tree is based on a two-way split, but decision trees can also have multi-way splits.

This tree provides 10 predictions (that is, there are 10 leaves). As shown, each estimate prediction represents the percentage of all data points in that leaf node that are the specified digit.

Leaf	Pr(1 x)	Pr(7 x)	Pr(9 x)	Decision
1	.03	.96	.01	7
2	.09	.91	.00	7
3	.56	.44	.00	1
4	.95	.05	.00	1
5	.80	.10	.10	1
6	.64	.09	.27	1
7	.00	.13	.87	9
8	.10	.73	.17	7
9	.78	.01	.21	1
10	.01	.00	.99	9



- This defines several multivariate step functions.
- Each function corresponds to the posterior probability of a target class.

This table shows the ten leaves of the tree and the posterior probability for each numeric digit. The posterior probability is the probability generated by a predictive model.

The last column in the table is the decision prediction. The decision for each leaf is the digit with the largest posterior probability for that leaf. Leaf 1 discovered a subset of the data where 3% of the values are 1, 96% of the values are 7, and only 1% are 9.

You can think of a classification tree as defining several multivariate step functions. Each function corresponds to the posterior probability of a target class.

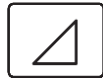
Decision Trees for Interval Targets: Regression Trees



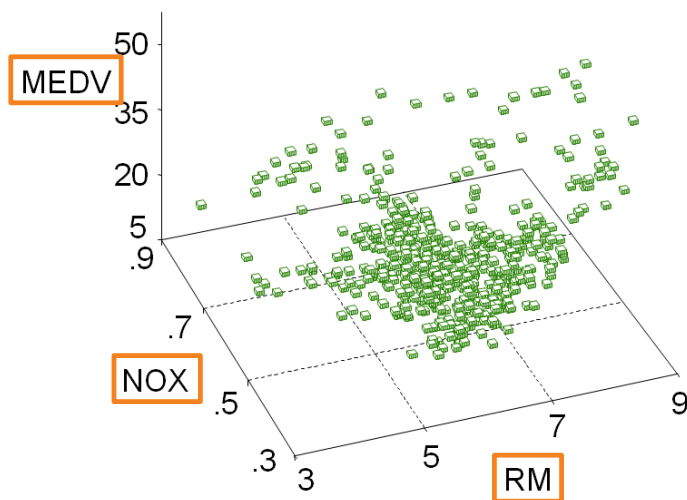
Copyright © SAS Institute Inc. All rights reserved.

Decision Trees for Interval Targets: Regression Trees

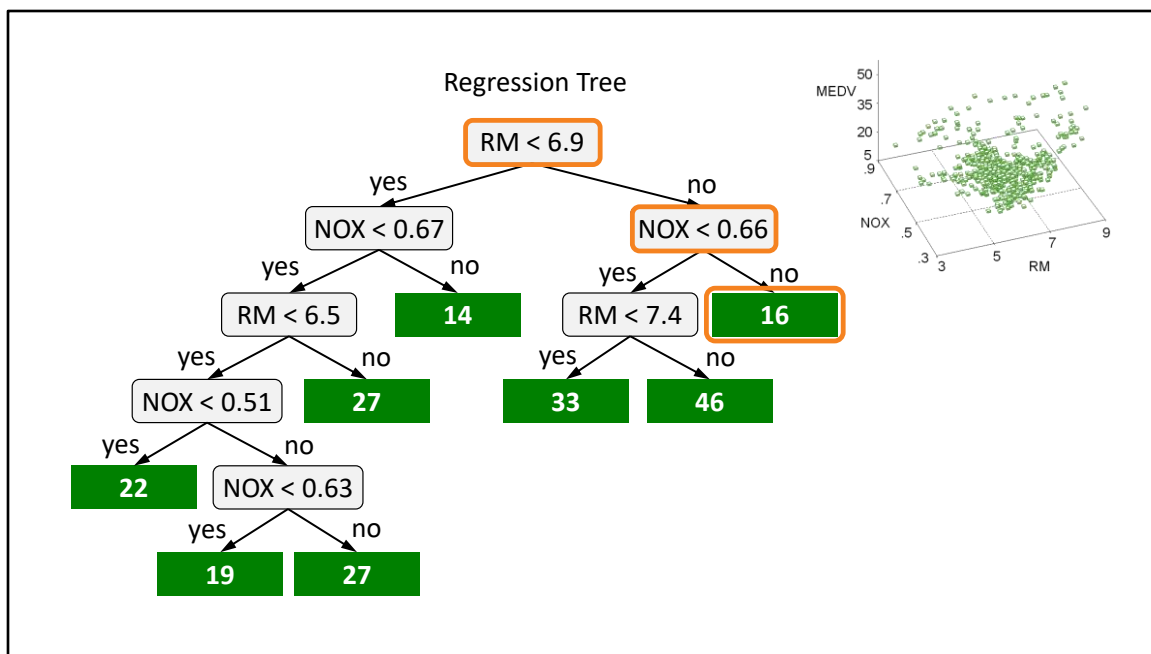
Interval Target



any numeric
value within a
certain range



Let's look at the use of a decision tree for an interval target. An interval target can have any numeric value, within a certain range, including decimal values. In the data set shown here, the target (MEDV) represents median home values in the Boston area (in units of 1,000 dollars US currency). The two input variables are RM (number of rooms) and NOX (nitrogen oxide concentration in the air).



A decision tree with an interval target is called a *regression* tree. The leaves represent the average of the median home values in that subset of the data, defined by the path that leads to the particular leaf. Just as in the classification tree shown earlier, the branches to the left indicate a "yes" response to the rule, and branches to the right indicate a "no" response.

For example, a house that does not have fewer than 6.9 rooms (that is, the house has 7 rooms or more) and does not have a nitrogen oxide concentration less than 0.66 (that is, the house has a nitrogen oxide concentration greater than or equal to 0.66) has a predicted median home value of 16,000 dollars.

This regression tree has eight leaves and thus can provide eight numeric predictions.

Leaves = Boolean Rules

Leaf	RM	NOX	Predicted MEDV
1	< 6.5	< .51	22
2	< 6.5	[.51, .63)	19
3	< 6.5	[.63, .67)	27
4	[6.5, 6.9)	< .67	27
5	< 6.9	$\geq$.67	14
6	[6.9, 7.4)	< .66	33
7	$\geq$ 7.4	< .66	46
8	$\geq$ 6.9	$\geq$.66	16

if $RM \in \{values\}$
and $NOX \in \{values\}$,
then $MEDV = value$.

Here is the regression tree for the Boston housing data, shown as a table. Unlike the table for the classification tree that you saw earlier, this table also shows the prediction rules. These rules are often referred to as English rules. Remember that the rules are based on Boolean logic (that is, they consist of a series of if-then-else statements).

For example, the rule for the first leaf is: if the number of rooms is less than 6.5 and the nitrogen oxide concentration is less than 0.51, then the predicted median home value is 22,000 dollars.

Question: Decision Tree Splits



Copyright © SAS Institute Inc. All rights reserved.

Question: Decision Tree Splits

A decision tree can have only two-way splits.

- a. True
- b. False

Answer: Decision Tree Splits

A decision tree can have only two-way splits.

- a. True
- ☒ b. False

CA: b

FB: A decision tree can have multi-way splits.

Improving the Decision Tree Model



Copyright © SAS Institute Inc. All rights reserved.

Improving the Decision Tree Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- Improve the model.
- Optimize the complexity of
- Regularize and tune the hyperparameters of the model.
- Build ensemble models.

structure

recursive
partitioning
method

After building an initial tree, the next essential discovery task is to improve the model. With decision trees, there are many settings that we can consider modifying. Some settings affect the structure of the tree, such as depth and leaf size. Other settings relate to the recursive partitioning method that is used to grow a tree, as well as any associated parameters.

Demo: Modifying the Structure Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Decision Trees



Copyright © SAS Institute Inc. All rights reserved.

Question: Decision Trees

Which of the following statements is true regarding decision trees?

- a. To predict cases, decision trees use rules that involve the values or categories of the input variables.
- b. Decision trees can handle only categorical targets.
- c. Each predictor variable can appear only in a single split in the tree.
- d. A decision tree for a categorical target can provide decisions but not estimates.

Answer: Decision Trees

Which of the following statements is true regarding decision trees?

- ☒ a. To predict cases, decision trees use rules that involve the values or categories of the input variables.
- b. Decision trees can handle only categorical targets.
- c. Each predictor variable can appear only in a single split in the tree.
- d. A decision tree for a categorical target can provide decisions but not estimates.

CA: a

FB: To predict cases, decision trees use Boolean (if-then-else) rules that involve the values or categories of the input variables.

3.3 Modifying the Model: Recursive Partitioning



Recursive Partitioning



Copyright © SAS Institute Inc. All rights reserved.

Recursive Partitioning

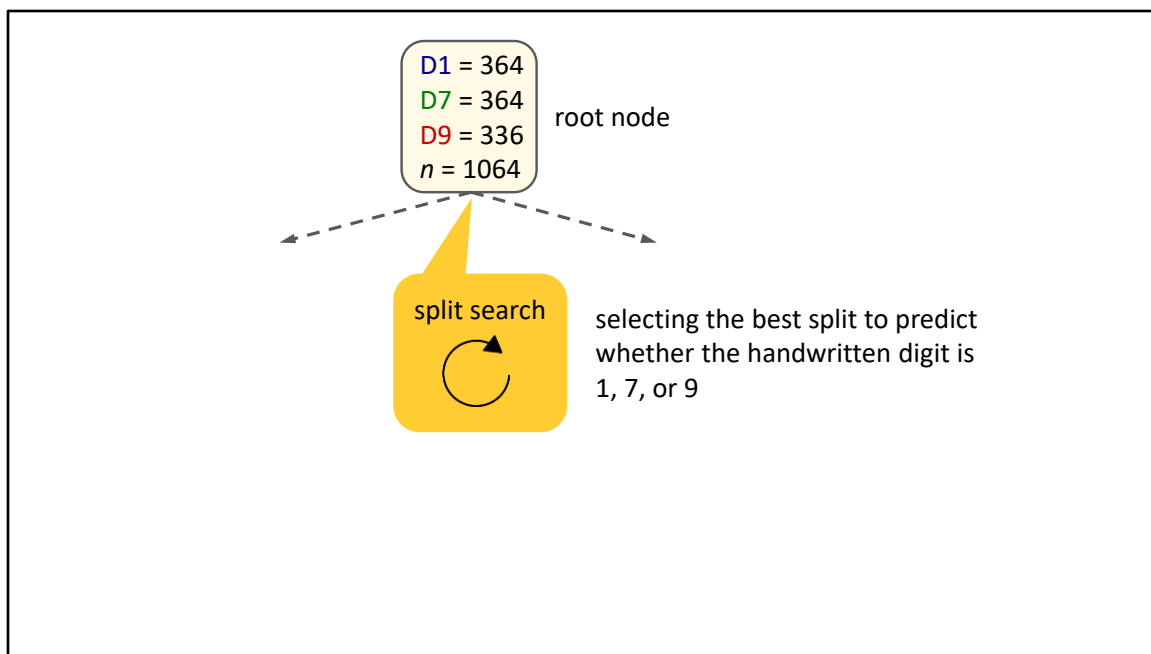


Essential Discovery Tasks

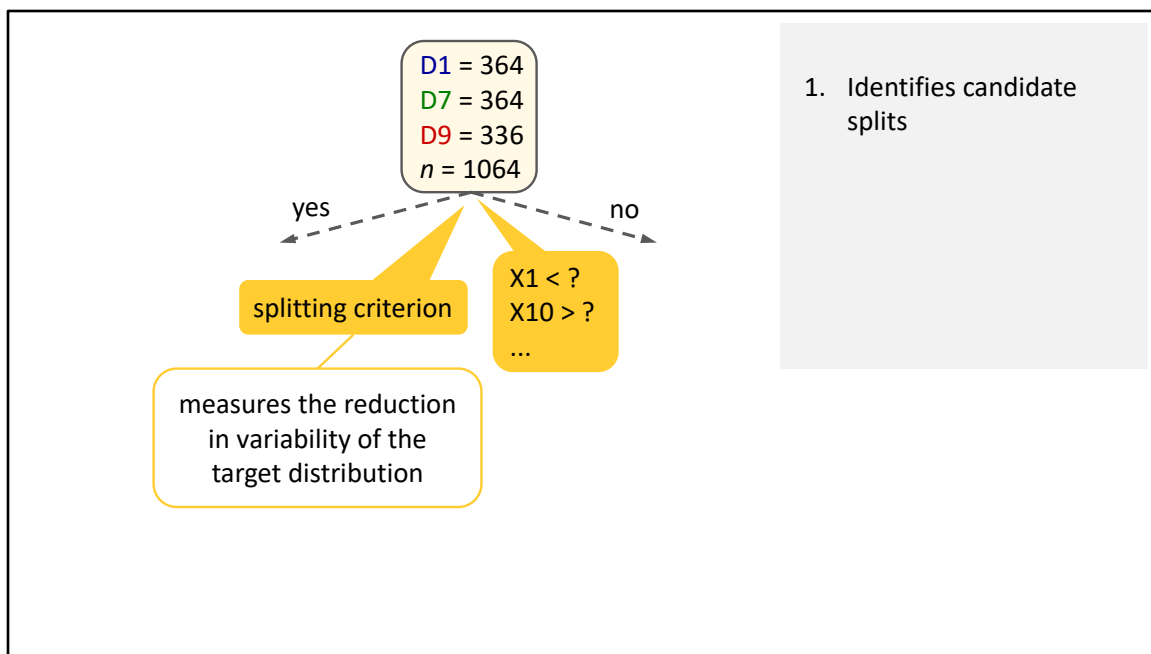
- ✓ Select an algorithm.
- Improve the model.
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.
- Build ensemble models.

recursive
partitioning method

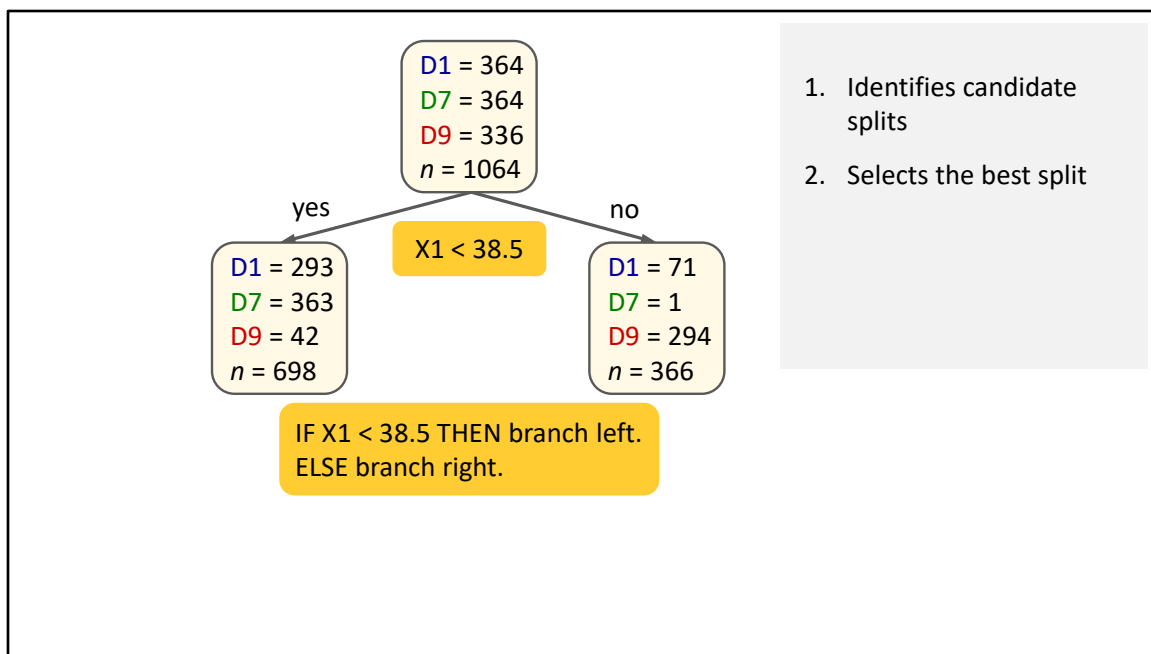
Another way to improve a decision tree model is to modify its growth by adjusting the recursive partitioning parameters that define the splits. Recursive partitioning is the standard method used to grow decision trees.



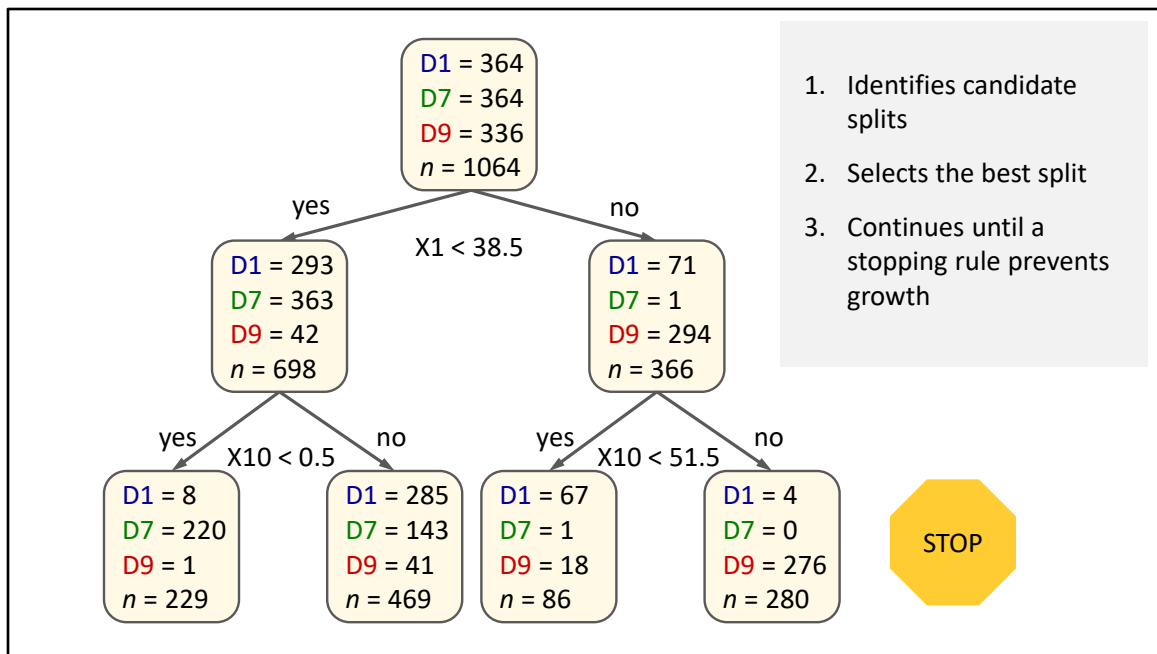
Here is the root node of a classification tree that was shown earlier. The analysis goal is to predict whether the handwritten digit is 1, 7, or 9. The root node represents the number of cases for each digit. Starting at the root node, recursive partitioning uses an iterative process to select the best split for the node. This process is called a split search.



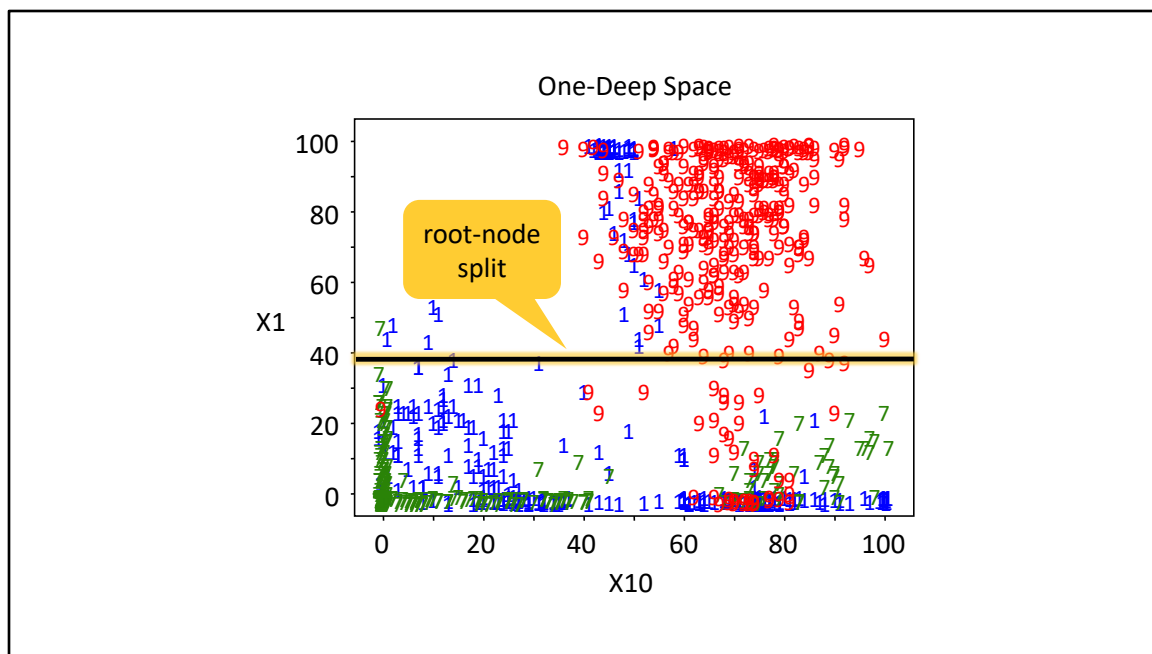
First, recursive partitioning identifies the best of all possible splits for each input variable based on the value of a splitting criterion. These are the candidate splits. The splitting criterion measures the reduction in variability of the target distribution in the child nodes. The goal is to reduce variability and thus increase purity in the child nodes.



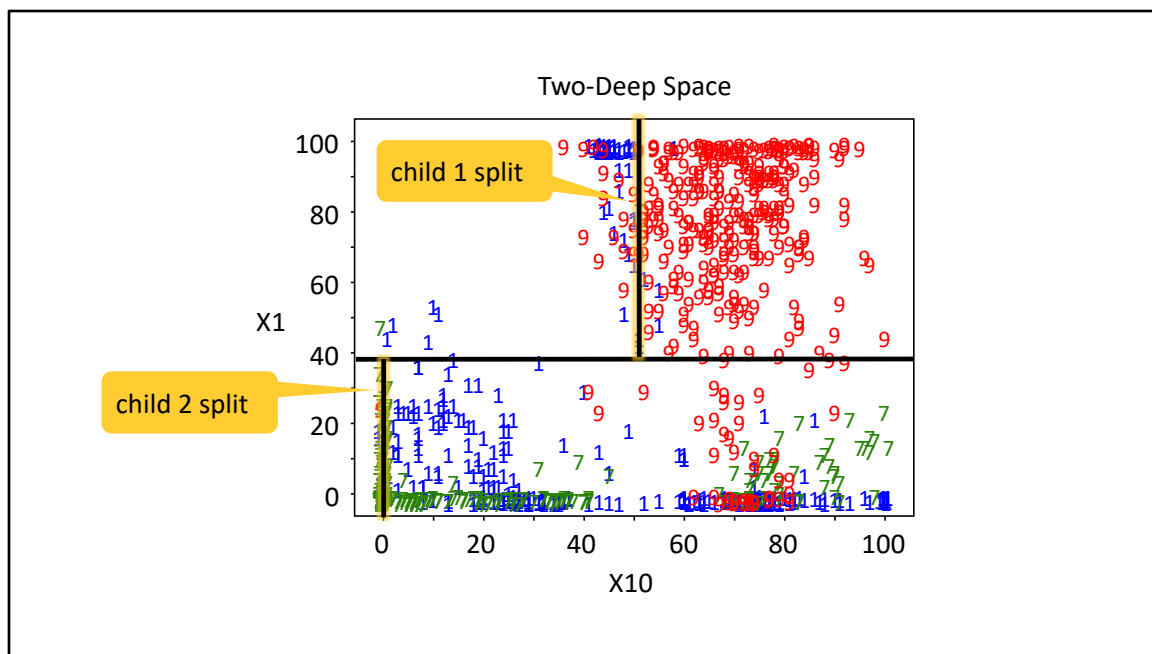
The process then selects the overall best split (that is, a split that is based on a single input). This split is expressed as an IF-THEN-ELSE rule. The cases in the root node are then partitioned according to the selected split. In this tree, the optimal split is to branch left when $X1$ is less than 38.5, and branch right when $X1$ is greater than or equal to 38.5. The 1064 observations in the root node have been partitioned such that 698 follow the rule to the left branch, and 366 follow the rule to the right branch.



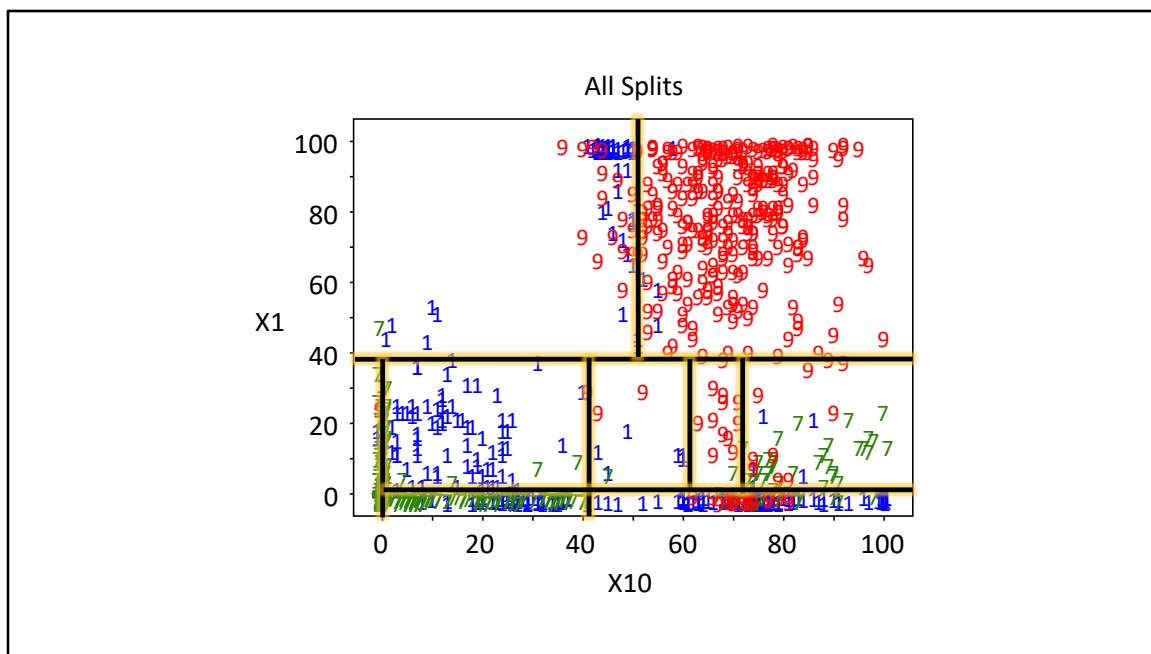
The split search is now repeated in each child node. The split selection at a node depends entirely on the cases in that local region of the input space. As the split search continues deeper into the tree, the data become more fragmented. The splitting criterion and other stopping rules determine the end of the process and the final depth of the tree.



Let's see what the split search process looks like in this plot of the training data. The root-node split corresponds to a partition of the input space where the boundary is perpendicular to one input dimension. The resulting tree has a depth of one (or is a one-deep space).



After the second iteration of the split search, the resulting tree has a depth of two. It is now a two-deep space. The resulting splits are shown in the plot.

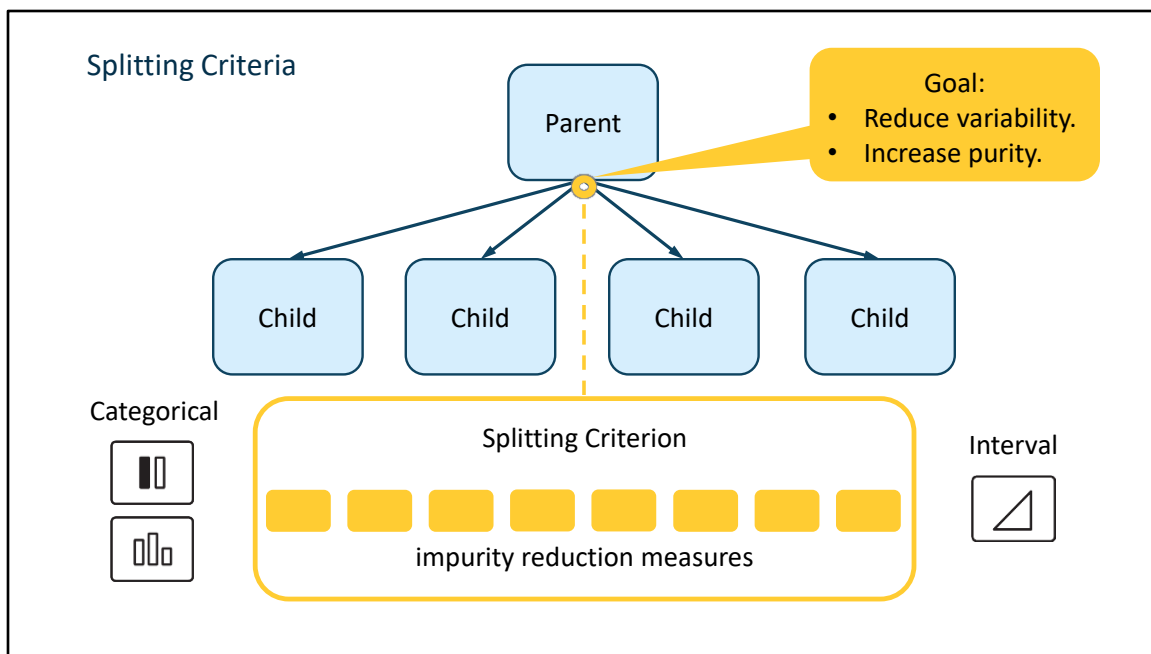


Finally, the plot shows all the splits that occurred before the recursive partitioning process stopped.

Splitting Criteria

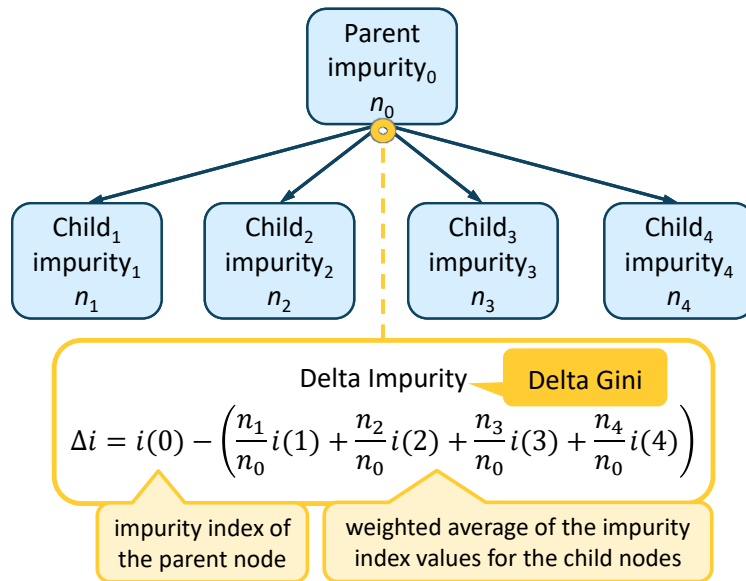


Copyright © SAS Institute Inc. All rights reserved.



The goal of splitting is always to reduce the variability of the target distribution and thus increase purity in the child nodes. A splitting criterion measures this reduction. Splitting criteria can be based on a variety of impurity reduction measures. Most of these measures apply only to categorical targets (including binary targets) or only to interval targets.

Categorical



Let's look at an example of a splitting criterion commonly used for each type of target. For categorical data, commonly-used splitting criteria measure the reduction in *node impurity* (that is, the reduction of within-node variability) induced by the split.

For these splitting criteria, the delta impurity criterion measures the difference between the impurity index of the parent node and a weighted average of the impurity index values for the child nodes. For example, the delta *Gini* splitting criterion measures the difference between the Gini index value of the parent node and a weighted average of the Gini index values of the child nodes.

Categorical



Node

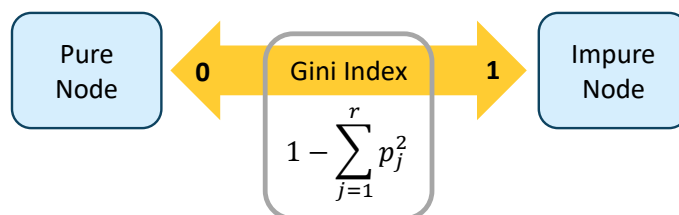
Gini Index

$$1 - \sum_{j=1}^r p_j^2$$

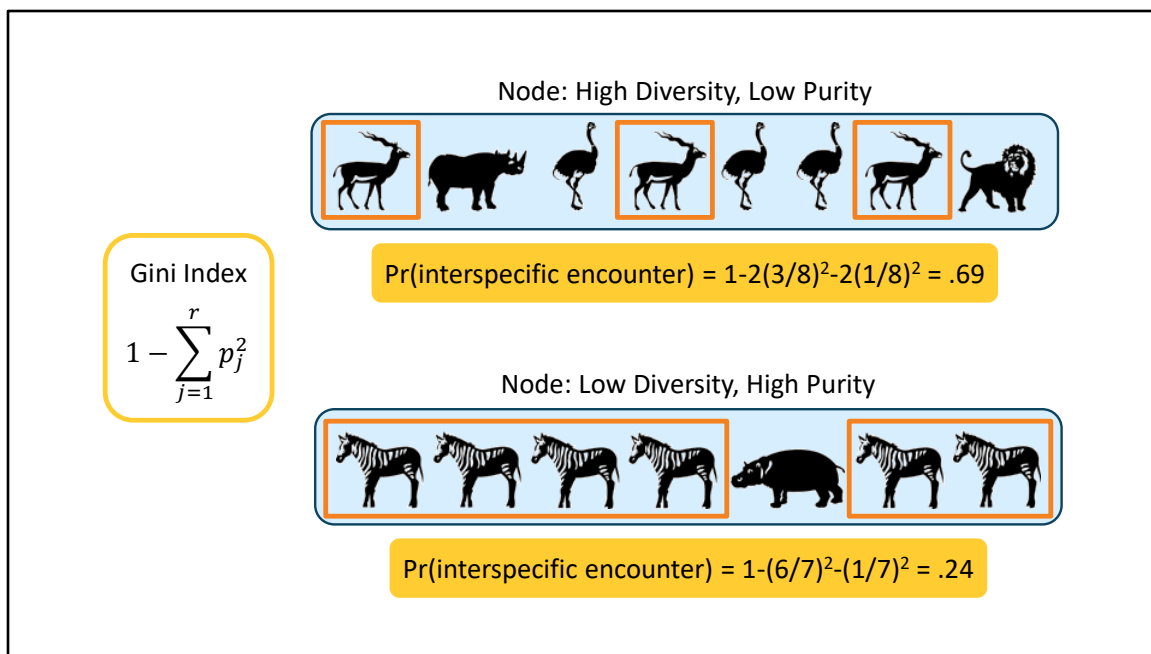
the probability that any two elements of a group, chosen at random, with replacement, are different

The Gini index equals 1 minus the sum of the probabilities for each individual target class appearing in the node. The Gini index can be interpreted as the probability that any two elements of a group, chosen at random (with replacement), are different.

Categorical

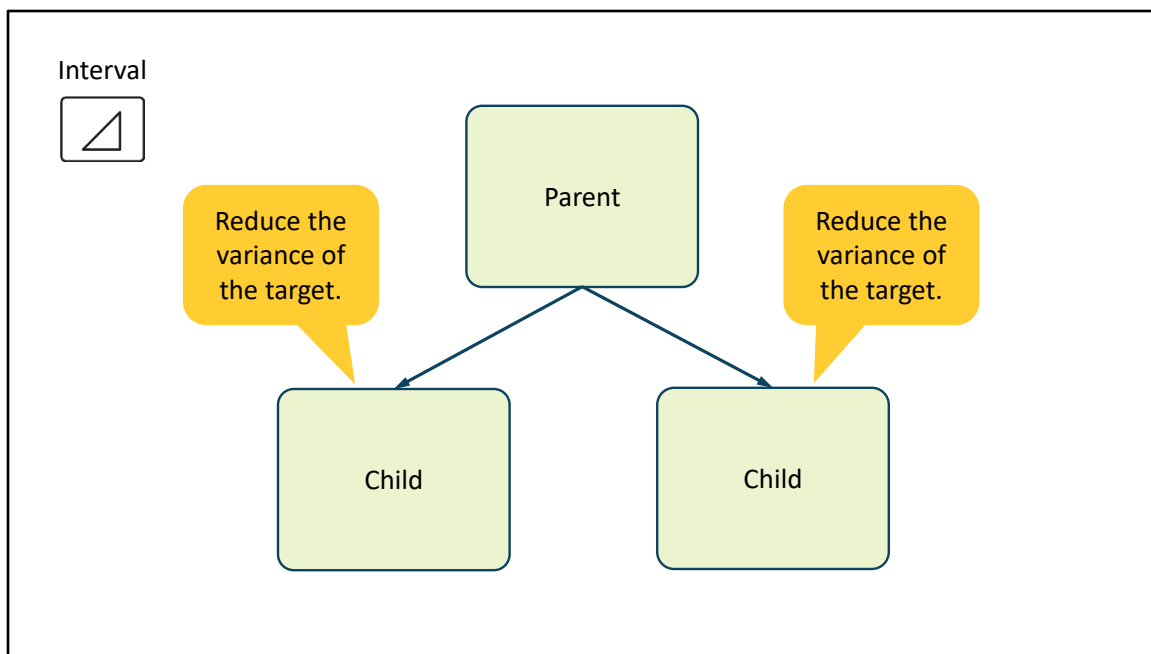


A pure node has a Gini index of 0. As the number of evenly distributed classes increases, the Gini index approaches 1.



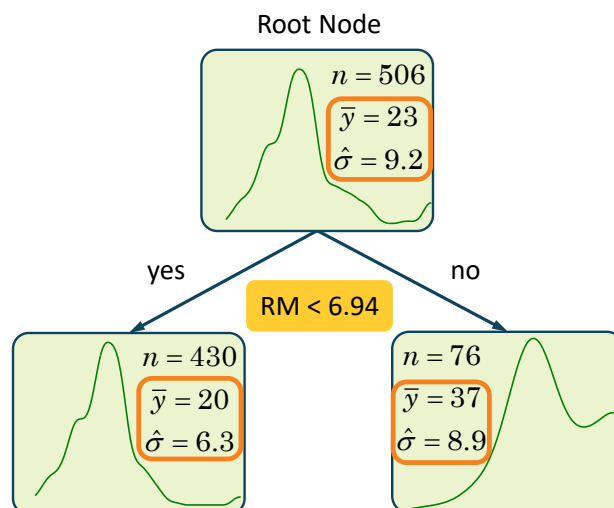
This diagram shows two groups of animals. Each group represents a node. The target classes that occur in each group are shown. In this case, the target is the species of animal. The first group has eight animals, which represent four species. In this group, the highest number of animals of any one species is three. This group has high diversity and low purity, and its Gini index value is 0.69. This means that, if we randomly select two animals from this group, there is a 69% chance that the animals will be different from each other. In other words, this is the chance of an interspecific encounter.

The second group has seven animals, which represent only two species: six zebras and one hippopotamus. This group has low diversity and high purity, and its Gini index value is 0.24. This means that, if we randomly select two animals from this group, there is only a 24% chance of an interspecific encounter.



When the target has an interval measurement level, splitting criteria can be based on reducing the variance of the target in the child nodes.

Interval



This example is based on the regression tree for the Boston housing data that was shown earlier in the lesson. The variance reduction in the first split of the tree is shown here. The root node represents all cases where the overall average median home value is 23,000 dollars with a standard deviation of 9,200 dollars. The split is made on RM (the number of rooms per house). All houses that have RM less than 6.94 have an average median home value of 20,000 dollars with a standard deviation of 6,300 dollars. All houses that do not meet this rule have an average median home value of 37,000 dollars and a standard deviation of 8,900 dollars. This makes sense because houses with fewer rooms typically have a lower value than houses with more rooms.

Notice that the variability in median home value in each child node is smaller than the variability in the parent node.

Impurity Reduction Measures for Categorical and Interval Targets

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Splitting Criteria in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Leaf Purity



Copyright © SAS Institute Inc. All rights reserved.

Question: Leaf Purity

Suppose you build a decision tree to predict whether a customer makes a purchase on the internet (yes or no). A leaf has perfect purity when it contains which of the following?

- a. all events
- b. all nonevents
- c. either all events or all nonevents
- d. an equal number of events and nonevents

Answer: Leaf Purity

Suppose you build a decision tree to predict whether a customer makes a purchase on the internet (yes or no). A leaf has perfect purity when it contains which of the following?

- a. all events
- b. all nonevents
- ☒ c. either all events or all nonevents
- d. an equal number of events and nonevents

CA: c

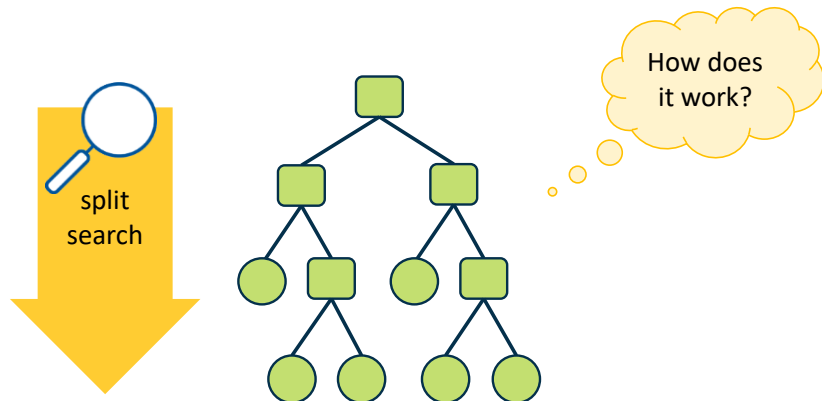
FB: Given a binary target, a leaf has perfect purity when all its cases have the same target outcome, but it does not matter whether they are events or nonevents. If multiple cases in the training data set have the same values of all the input variables but different target values, it is not possible to achieve perfect purity in the leaf that contains those cases.

Split Search

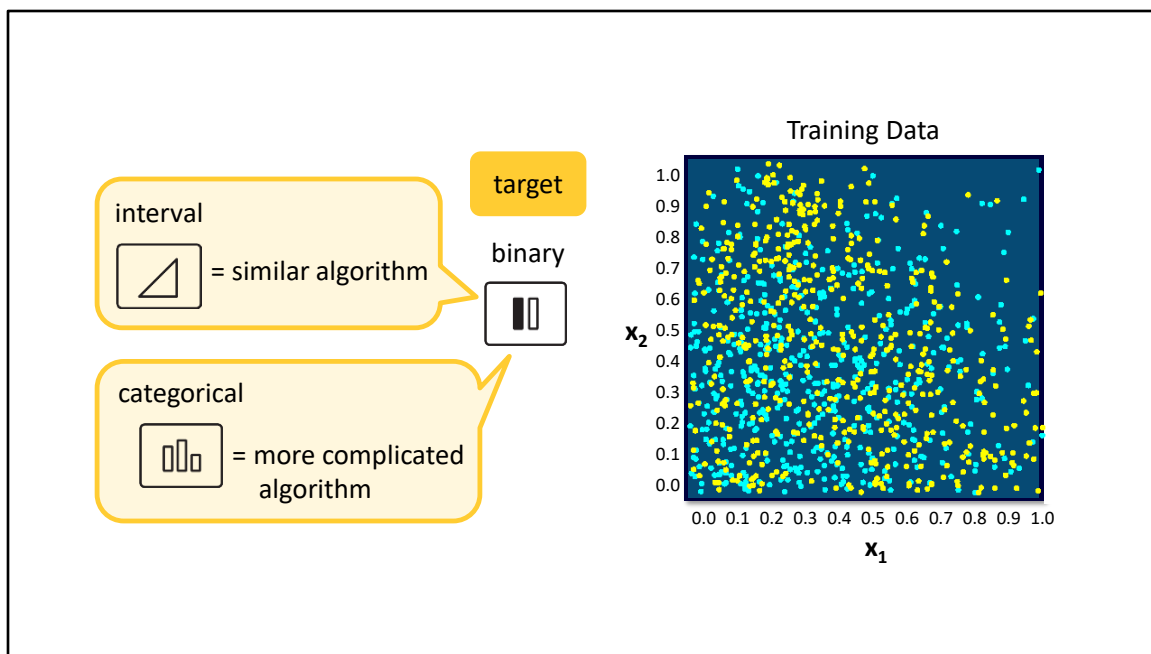


Copyright © SAS Institute Inc. All rights reserved.

Split Search

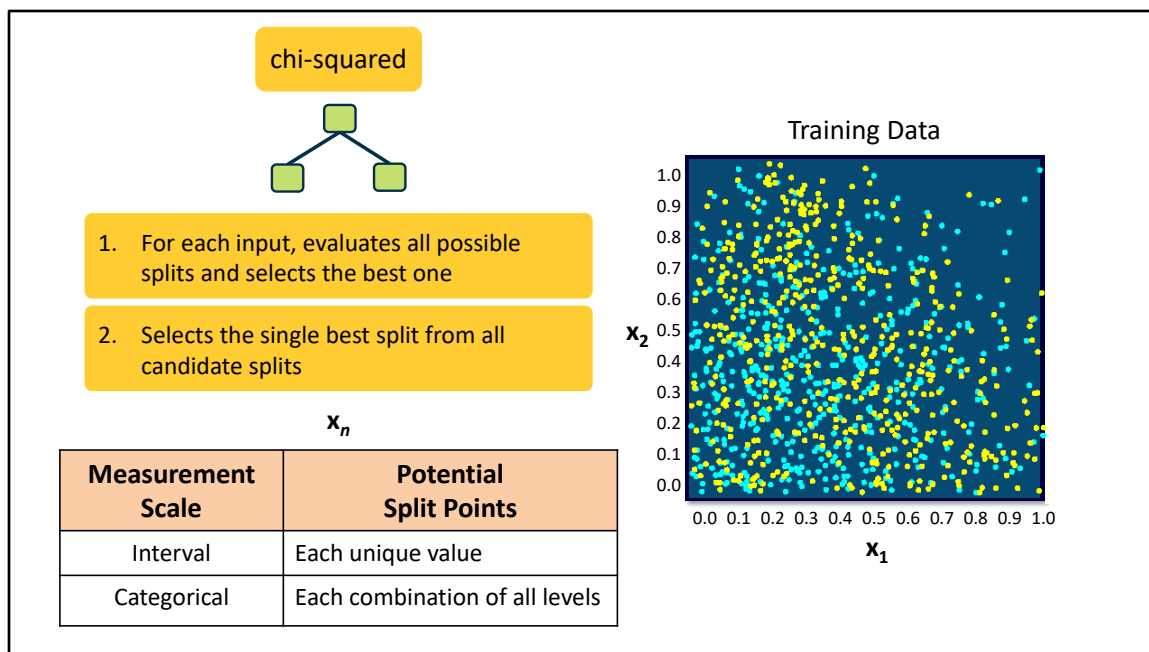


Let's look at a typical split-search process for growing a decision tree.



In this example, the training data is shown in a scatter plot. The target is binary and the two outcomes are represented as blue and yellow dots. There are two interval inputs (x_1 on the x axis, and x_2 on the y axis). The values of each input range from zero to 1. Note that the algorithm for interval targets is similar to the algorithm for binary targets.

The algorithm for categorical targets with more than two outcomes is more complicated and is beyond the scope of this course.



In this example, the split search algorithm uses chi-squared as the splitting criterion to grow a tree with 2-way splits.

For each input, the split search evaluates all possible splits and selects the best one for that input. Inputs whose splits do not meet the predefined threshold for the splitting criterion might be ignored. Then, the algorithm selects the single best split from all the candidate splits. In this example, the split search starts with the input x_1 .

An input's measurement scale determines how the algorithm selects the split point.

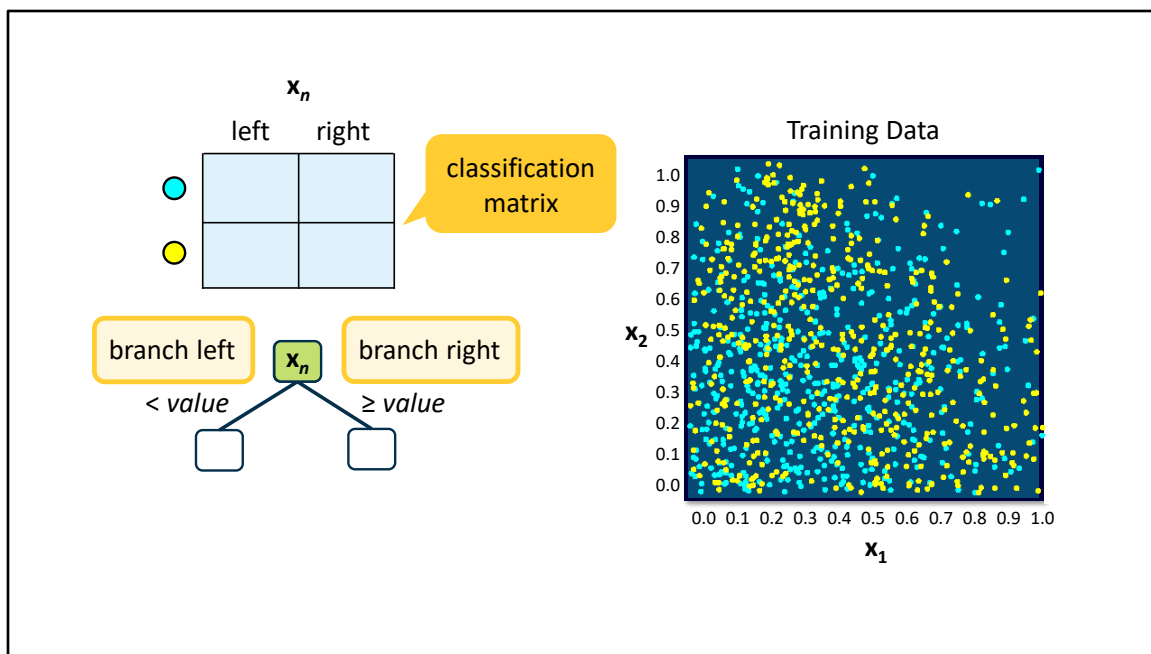
If the measurement scale of the selected input is *interval*, each unique value is a potential split point.

Interval variables with millions of values can be computationally expensive with regard to the number of split points and thus the decision tree in SAS Viya bins the numerical variables.

If the input is *categorical*, each combination of all levels is a potential split point.

If the input is ordinal, the order of the levels must be maintained when they are combined to form the split.

For now, we focus on interval inputs.

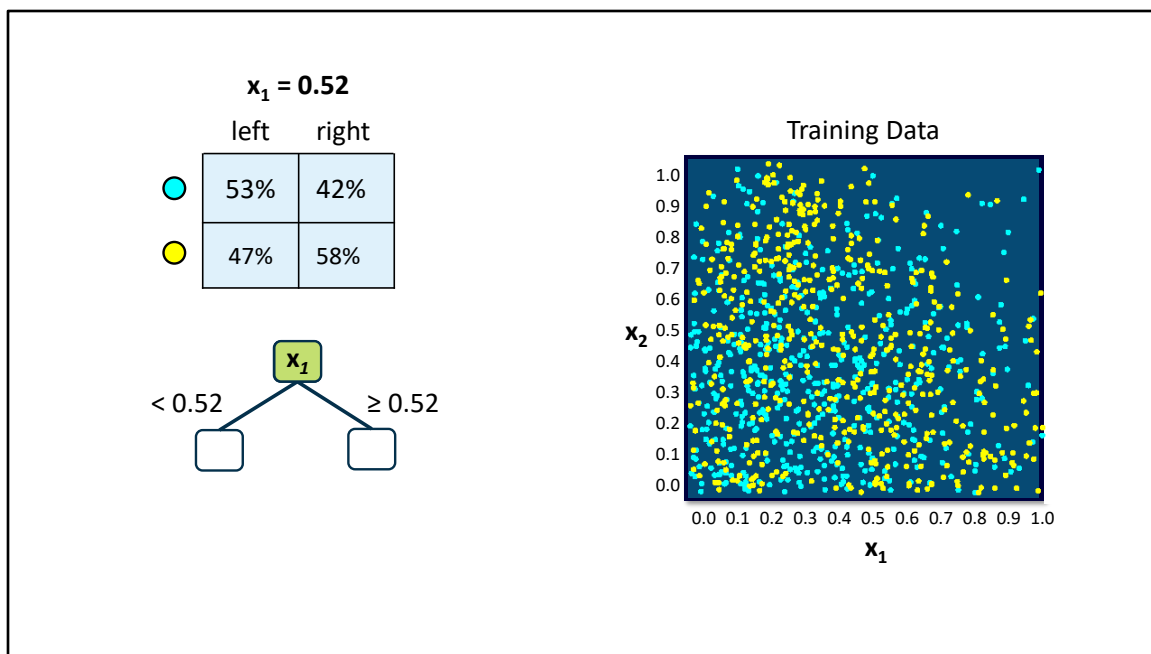


Splitting the data generates two groups, one for each branch direction.

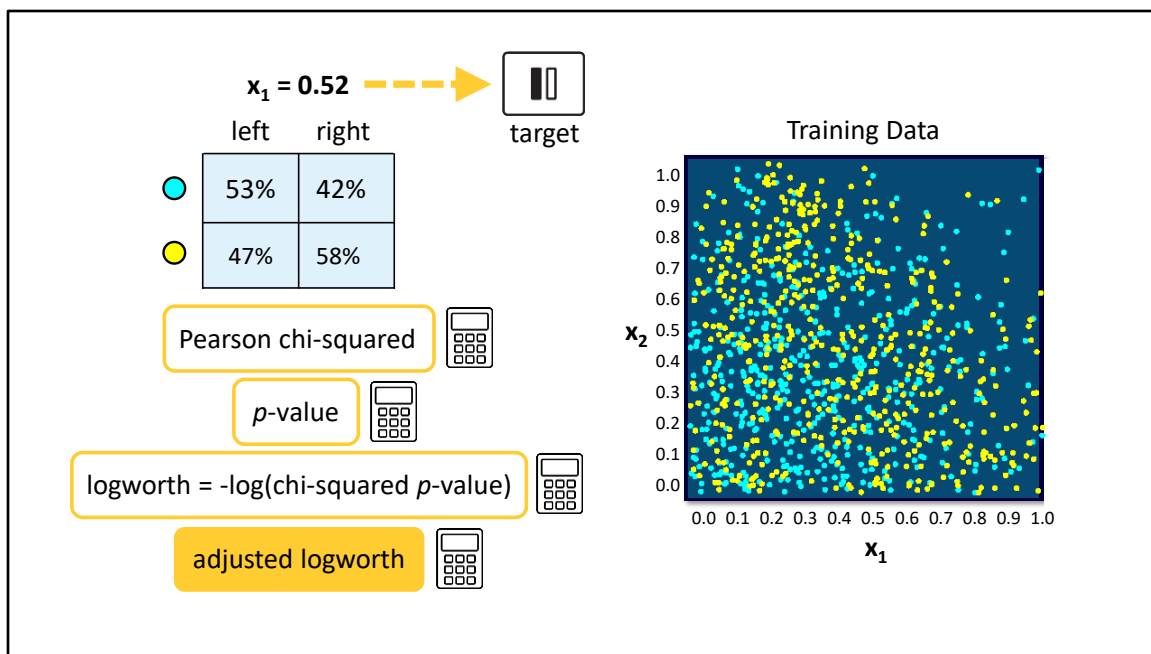
Cases with input values less than the split point are said to *branch left*.

Cases with input values greater than or equal to the split point are said to *branch right*.

The groups, combined with the target outcomes, form a two-by-two classification matrix. The columns specify the two groups defined by the split: left and right. The rows specify the target value.



Each cell contains a frequency count of the cases based on the split point and the outcome level. In the classification matrix shown here, the counts have been converted to percentages. These percentages are for one particular split based on one value of the input x_1 . The algorithm will calculate a different set of percentages for every possible split point (that is, every value of x_1).

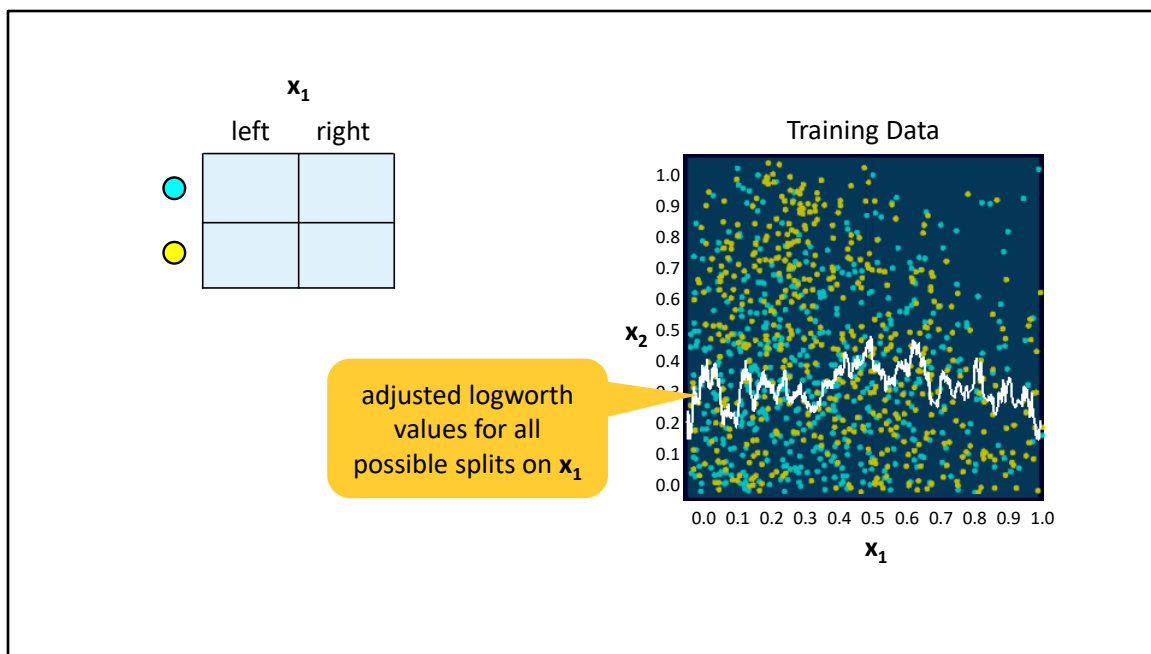


Next, for each possible split based on x_1 , the algorithm quantifies the strength of the relationship between the input and the target by using the frequencies to calculate the Pearson chi-squared statistic. Large values of the chi-squared statistic suggest that the proportion of zeros and ones in the left branch is different from the proportion in the right branch. A large difference in outcome proportions indicates a strong relationship (or dependency) and, thus, a good split.

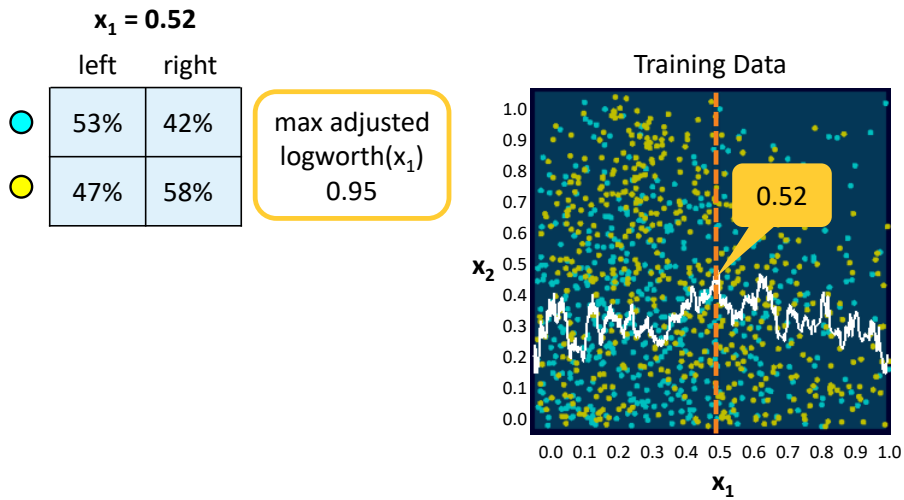
The Pearson chi-squared statistic, along with its degrees of freedom, has an associated probability value, or p -value. The p -value indicates the likelihood of obtaining the observed value of the Pearson chi-squared statistic assuming identical target proportions in each branch direction. For a large data set, this p -value can be very close to zero.

For this reason, the algorithm uses the logworth of the p -value instead to determine the quality of the split. The logworth is equal to negative log of the chi-squared p -value.

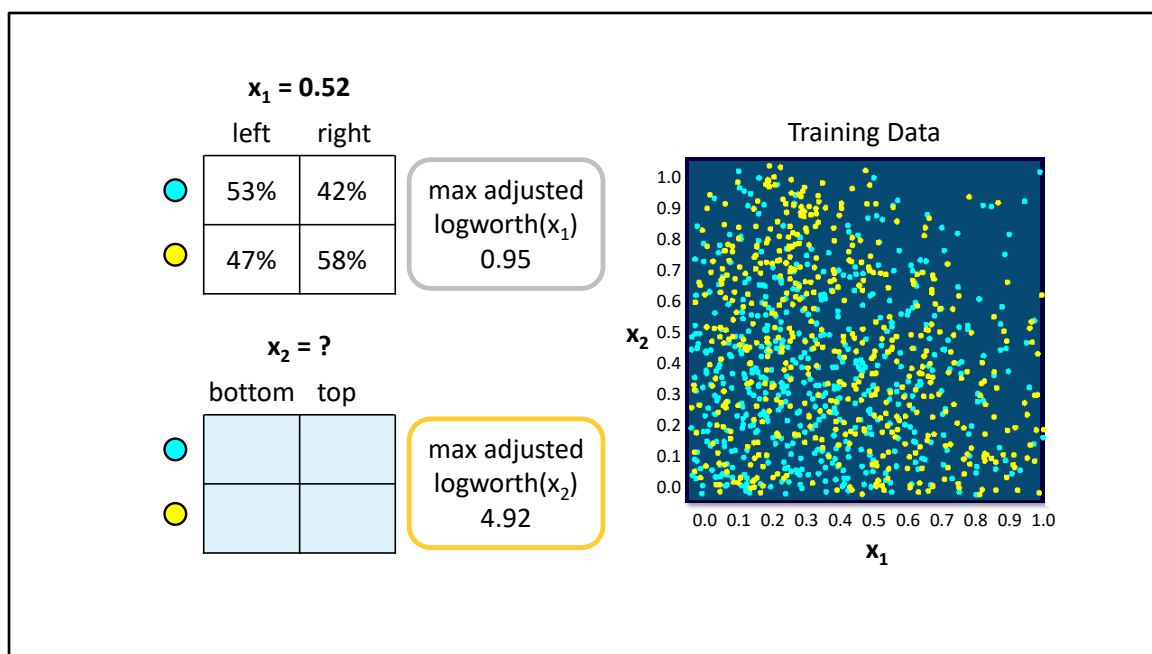
To account for issues that arise when multiple tests are performed, the algorithm adjusts the logworth values. This adjustment is similar to the Bonferroni adjustment that statisticians use to overcome the issues that arise when performing multiple tests. You learn more about the Bonferroni adjustment later.



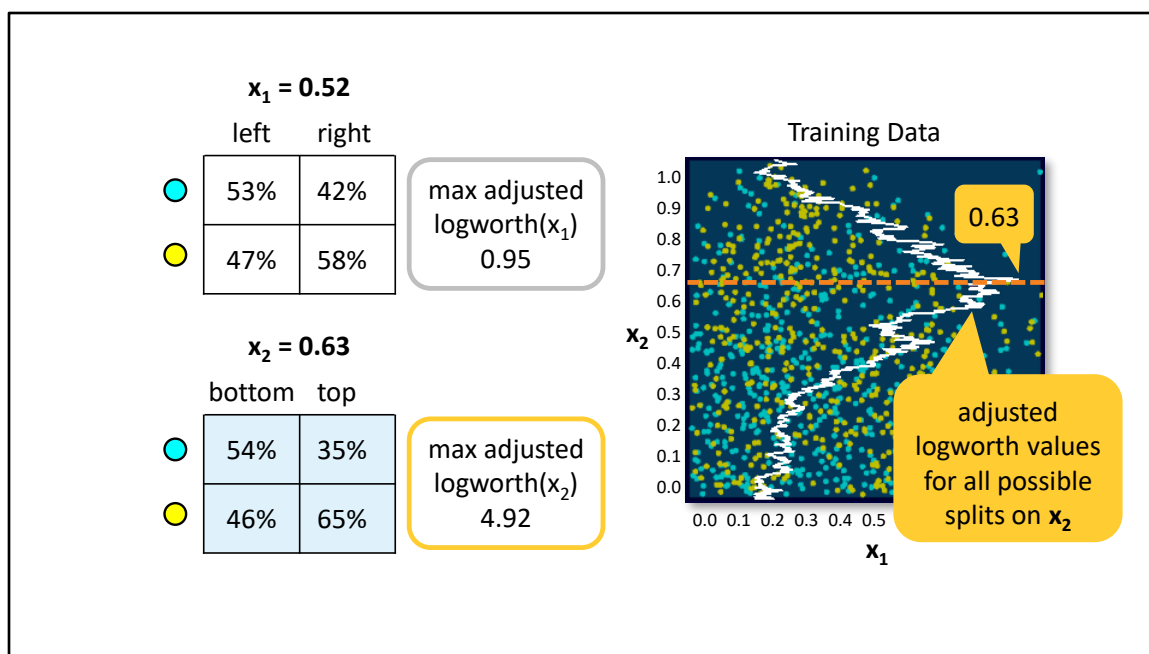
In the scatter plot of the training data, the horizontal white jagged line represents the adjusted logworth values for all possible splits on x_1 .



The best split for an input is the split that yields the maximum adjusted logworth value. Here, the vertical line in the scatter plot shows that the maximum adjusted logworth for x_1 , which is 0.95, is based on the input value 0.52.

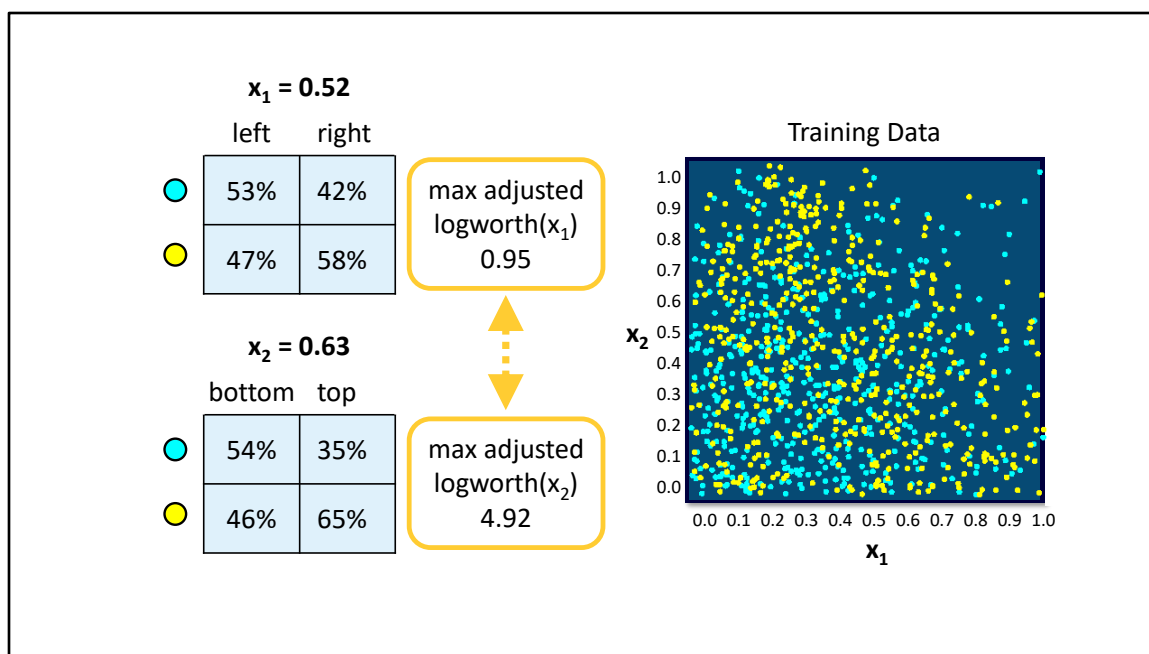


The split search process is repeated for every input in the training data. Inputs whose adjusted logworth fails to exceed the splitting criterion's threshold are excluded from consideration.

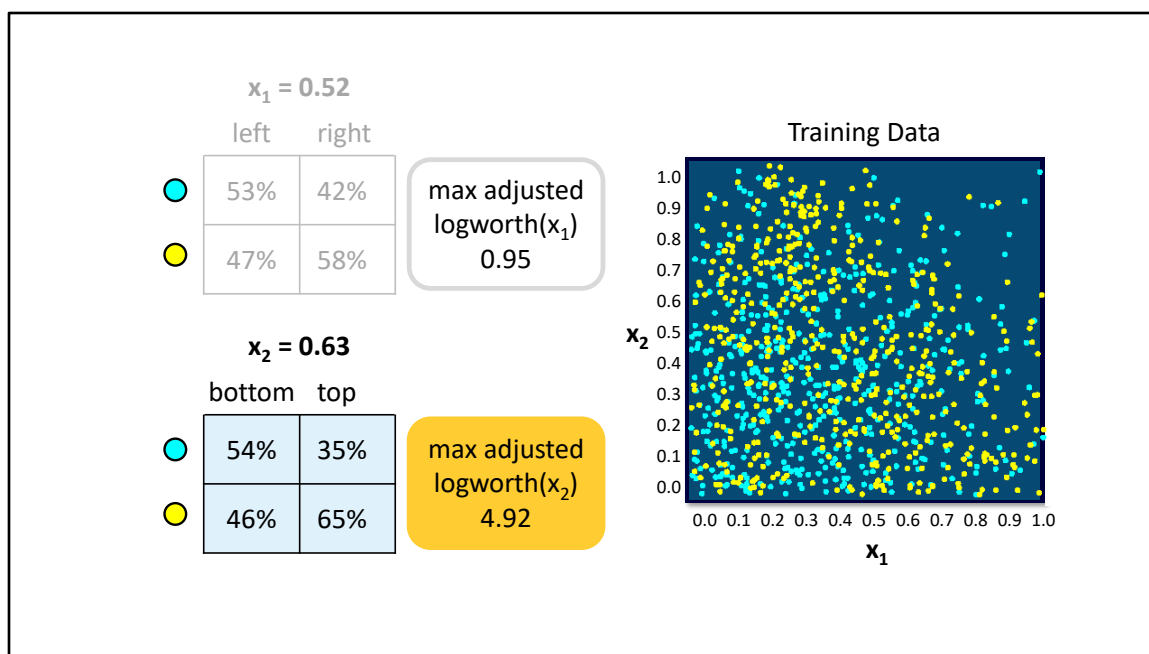


In the scatter plot for this example, the vertical white jagged line represents the adjusted logworth values for all possible splits on x_2 .

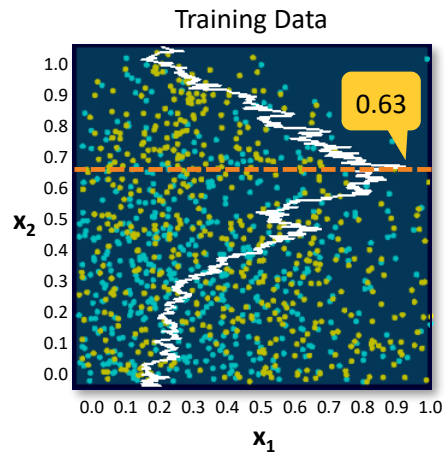
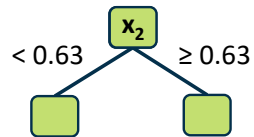
The horizontal line in the scatter plot shows that the maximum adjusted logworth for x_2 , which is 4.92, is based on the input value 0.63.



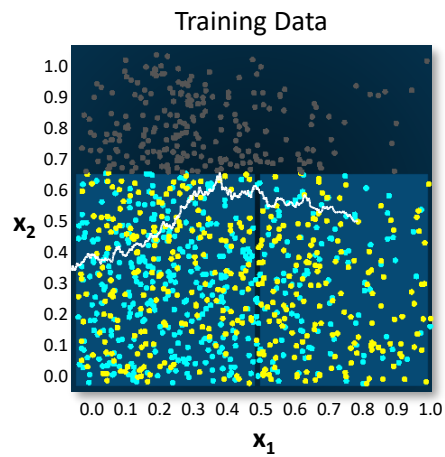
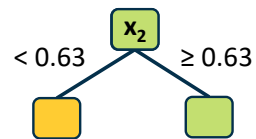
After you determine the best split for every input, the split search algorithm compares the maximum adjusted logworth values across all of the inputs. The split with the highest adjusted logworth becomes the splitting rule.



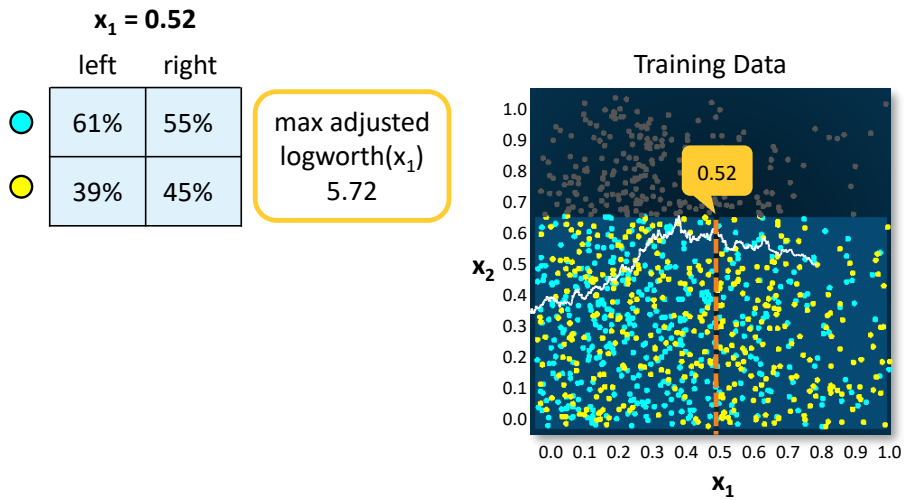
In this example, the split on x_2 has the highest maximum adjusted logworth value, so it becomes the basis for the splitting rule.



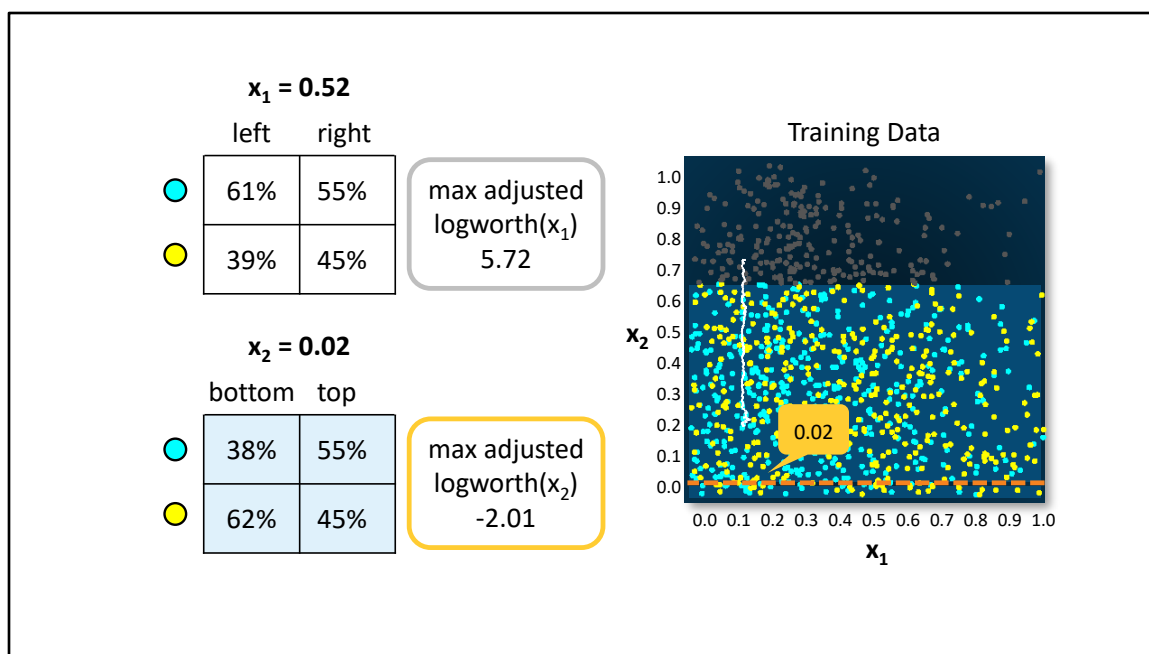
The algorithm uses the splitting rule to separate the training data into two groups. In this example, if x_2 is less than 0.63, then the case branches left. Else if x_2 is greater than or equal to 0.63, the case branches right.



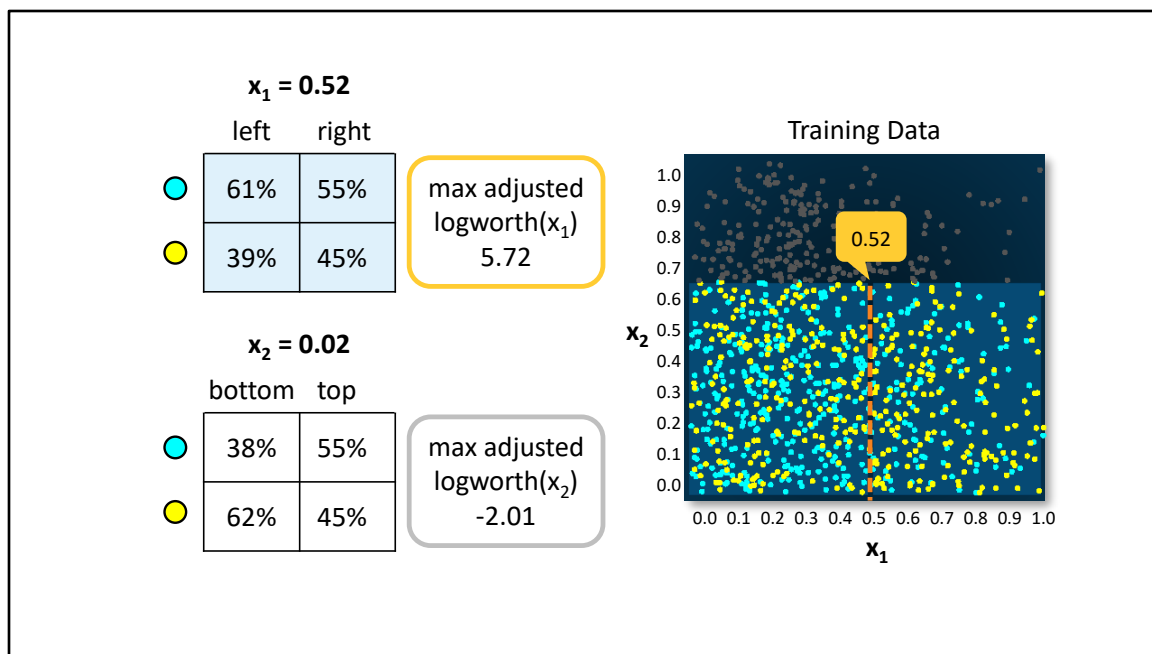
The algorithm moves on to one of the groups in the first generation, and repeats the split search process for that group.



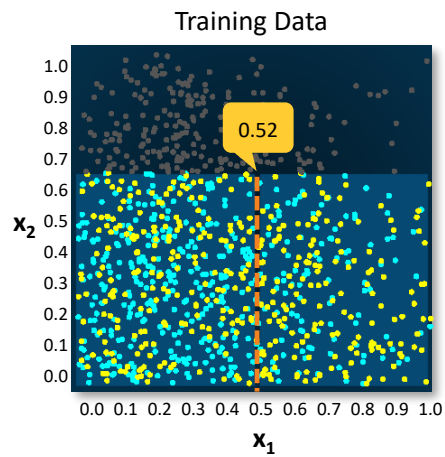
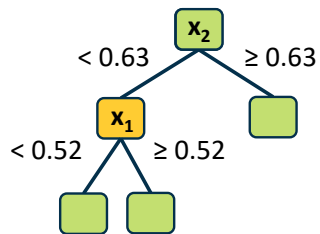
In this scatter plot, the horizontal white jagged line shows the adjusted logworth values of x_1 . The vertical line shows the value of x_1 , which is 0.52, that corresponds with the maximum adjusted logworth value 5.72. This is the optimal split point for x_1 .



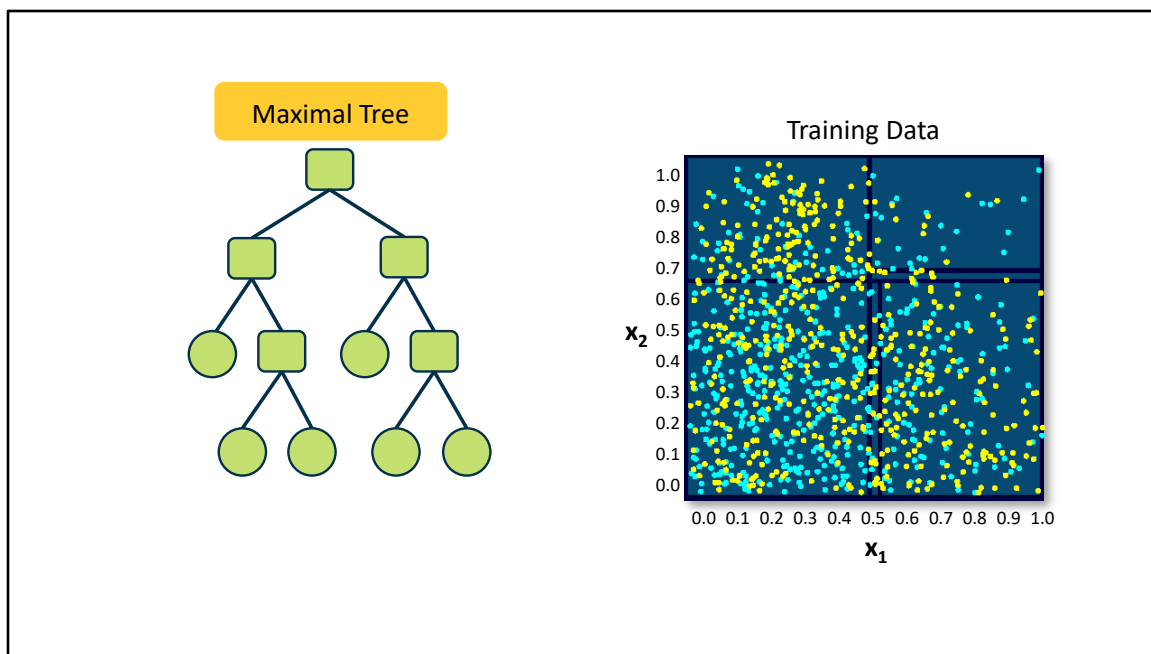
The optimal split point for x_2 is likewise determined. The maximum adjusted logworth, which is -2.01, is based on the x_2 value of 0.02. You might be surprised that the logworth of the x_2 split is negative. However, this results from several adjustments made to the logworth calculation, which you learn about later.



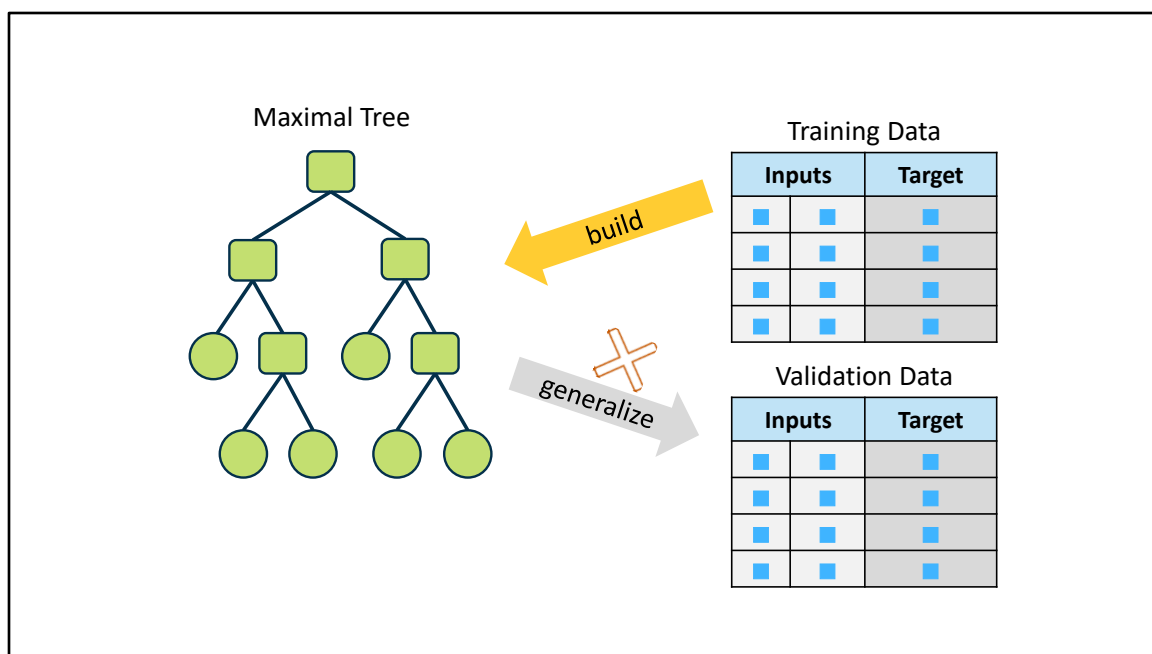
The x_1 maximum adjusted logworth, which is 5.72, is higher than the maximum adjusted logworth for x_2 , so the selected splitting rule is based on the associated value of x_1 .



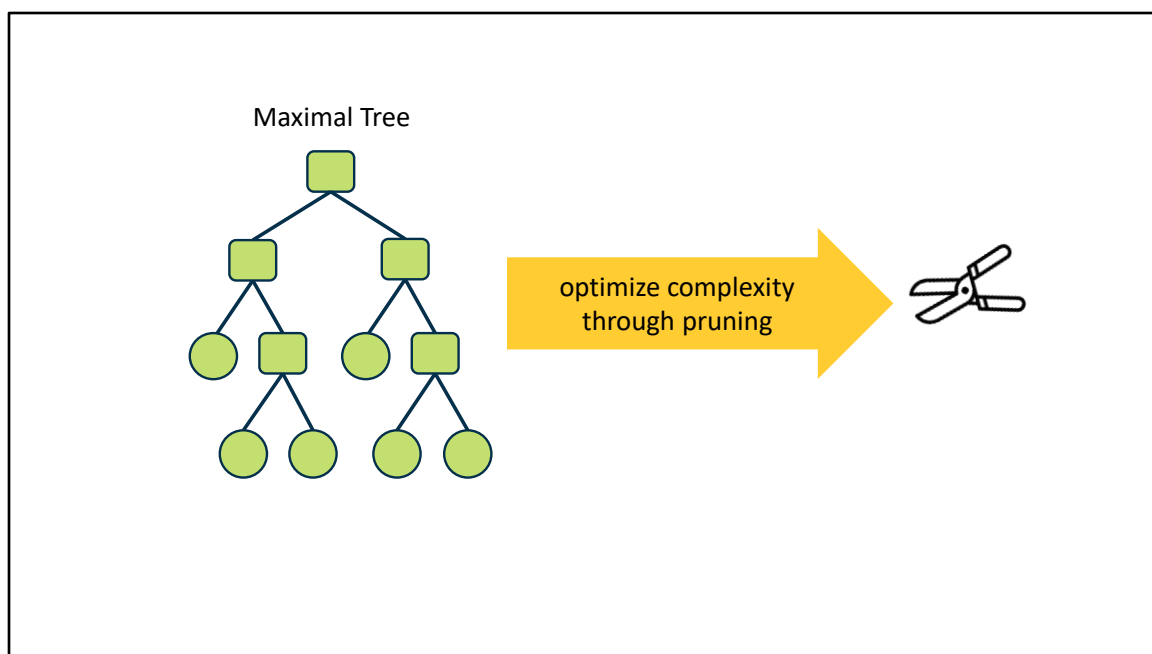
Now the algorithm uses the adjusted splitting rule to separate the data into two groups.



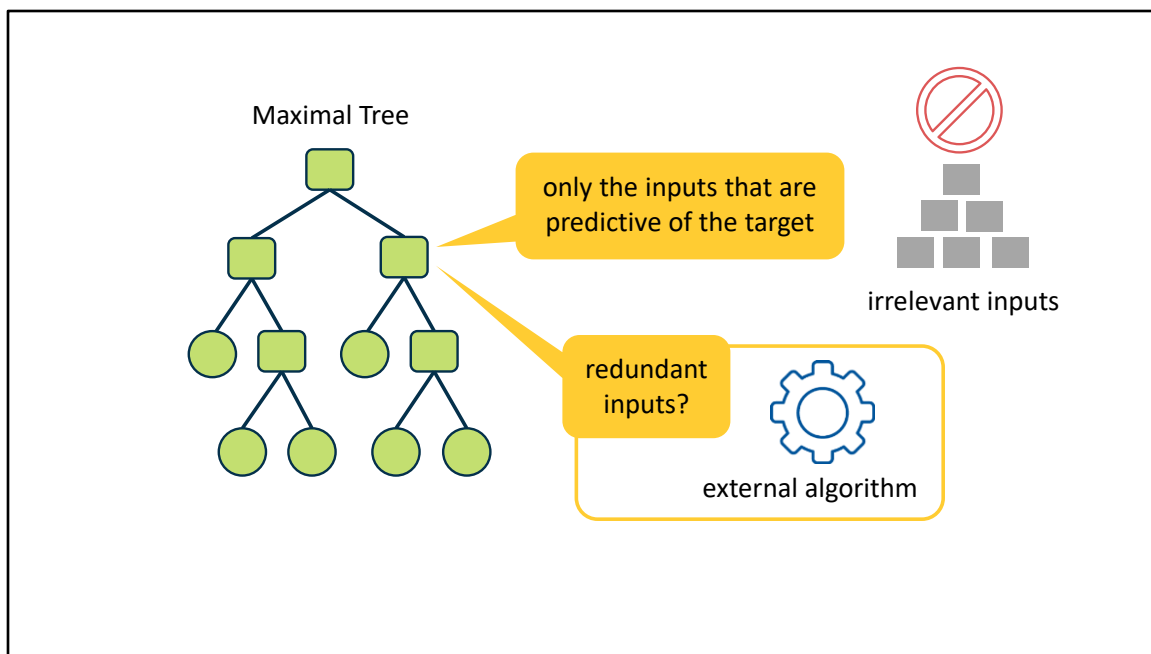
Unless another stopping rule is triggered first (such as a limit on the tree's depth), the split-search process repeats in each leaf until there are no more splits whose adjusted logworth exceeds the adjusted threshold. This stops the tree's growth. The result of the split-search process is known as the *maximal tree*.



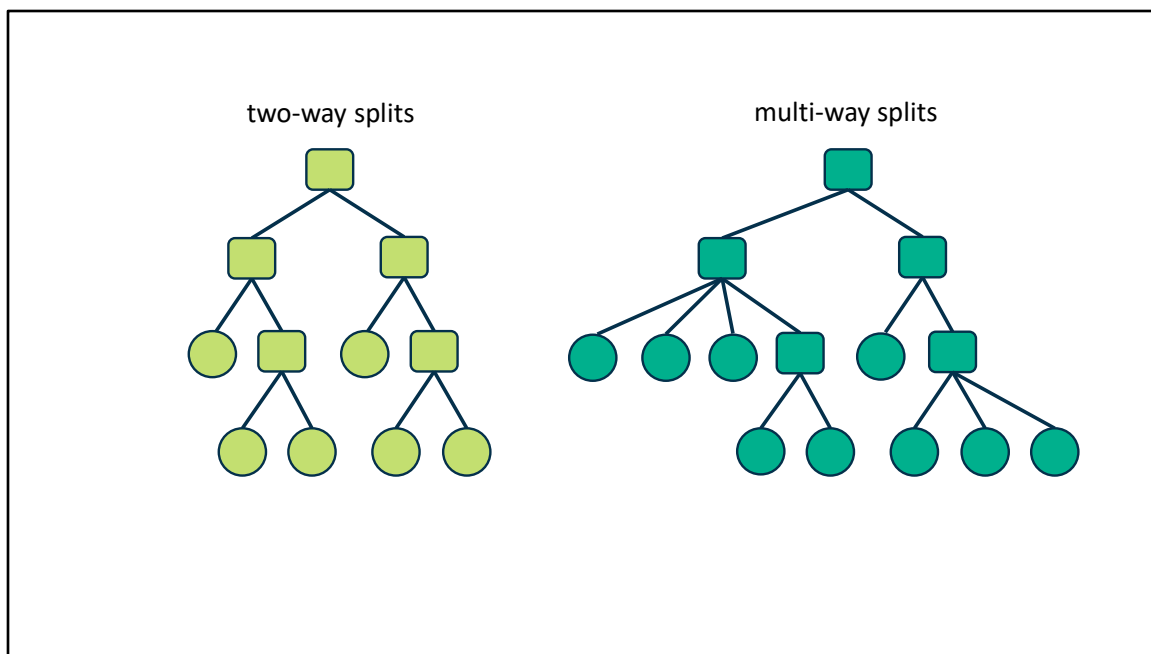
The maximal tree is built exclusively on the training data so it likely will not generalize well on the validation data.



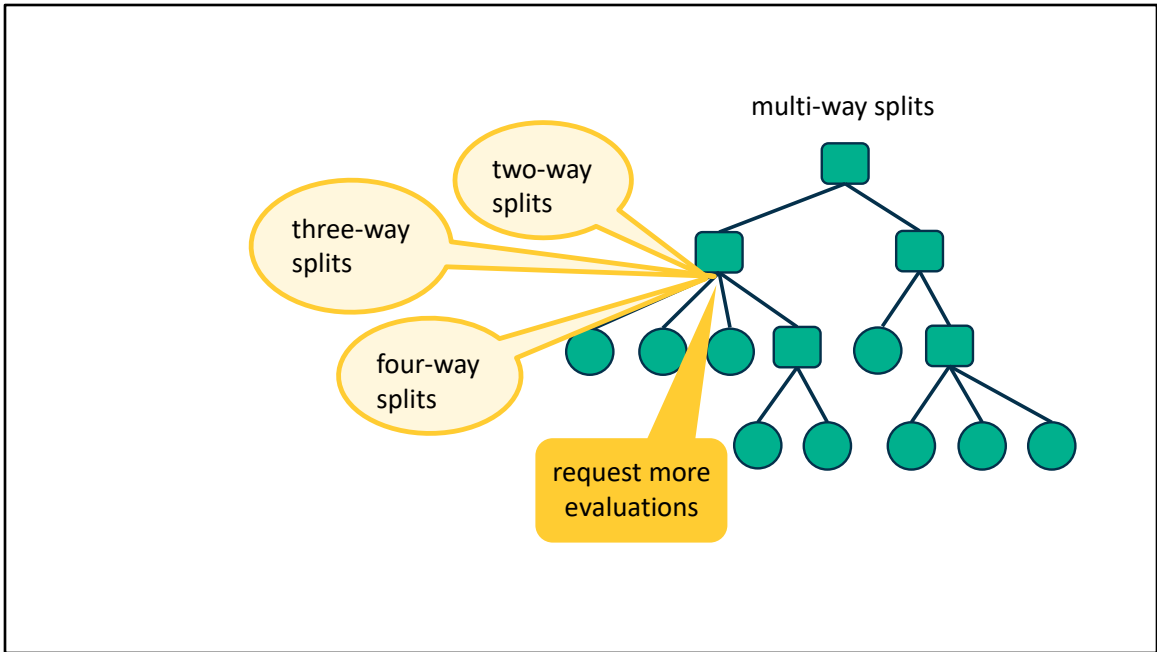
The maximal tree is the starting place for optimizing the complexity of the model, through a process called pruning. You learn about pruning later.



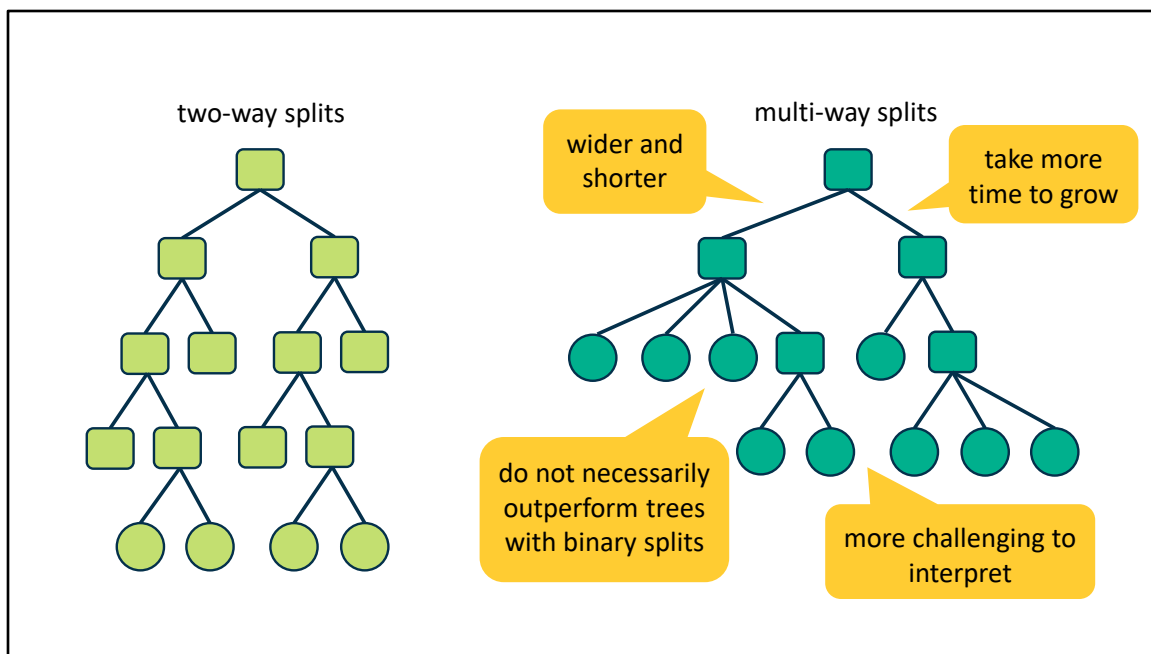
The maximal tree contains only the inputs that are predictive of the target (that is, the inputs that appear in splitting rules). This is the first point in the process of building a decision tree where irrelevant inputs are excluded. However, trees have no built-in method for ignoring redundant inputs. To manage input redundancy, you must use an algorithm that is external to the tree. For example, you can use the Variable Selection node.



Although this example shows a decision tree with two-way splits, we could have created a decision tree with multi-way splits where the values of the inputs are partitioned into disjoint ranges.



Multi-way splits request more evaluations for the candidate splits because all inputs in all n -way splits must be considered. For example, in 4-way splits, all possible candidates for two-way splits, three-way splits, and four-way splits are evaluated.



Trees with multi-way splits tend to be wider and shorter than trees with binary splits. However, research has shown that trees with multi-way splits do not necessarily outperform trees with binary splits. Trees with multi-way splits also take more time to grow, and they can be more challenging to interpret.

Question: Number of Splits



Copyright © SAS Institute Inc. All rights reserved.

Question: Number of Splits

If you build a decision tree model to predict a binary target, the rules in that decision tree are limited to two-way splits.

- a. True
- b. False

Answer: Number of Splits

If you build a decision tree model to predict a binary target, the rules in that decision tree are limited to two-way splits.

- a. True
- ☒ b. False

CA: b

FB: This statement is false. The number of splits per rule does not depend on the type of target. If the split is based on binary input (for example, whether a person owns a home), a two-way split is the only possibility. As you learn later, you can specify the maximum number of splits that Model Studio uses to build a decision tree.

Think About It



Copyright © SAS Institute Inc. All rights reserved.

Discussion

How do decision trees address the curse of dimensionality?



FB: The split search process reduces the number of inputs in the model by eliminating irrelevant inputs. Irrelevant inputs do not appear in any splitting rules in the decision tree.

Adjustments in a Split Search

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Missing Values in Decision Trees in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Modifying the Recursive Partitioning Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Surrogate Splits

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Calculating Variable Importance for Surrogate Splits

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



3.4 Modifying the Model: Pruning



Optimizing the Complexity of a Decision Tree Model



Copyright © SAS Institute Inc. All rights reserved.

Optimizing the Complexity of a Decision Tree Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
 - Optimize the complexity of the model.
 - Regularize and tune the hyperparameters of the model.
 - Build ensemble models.

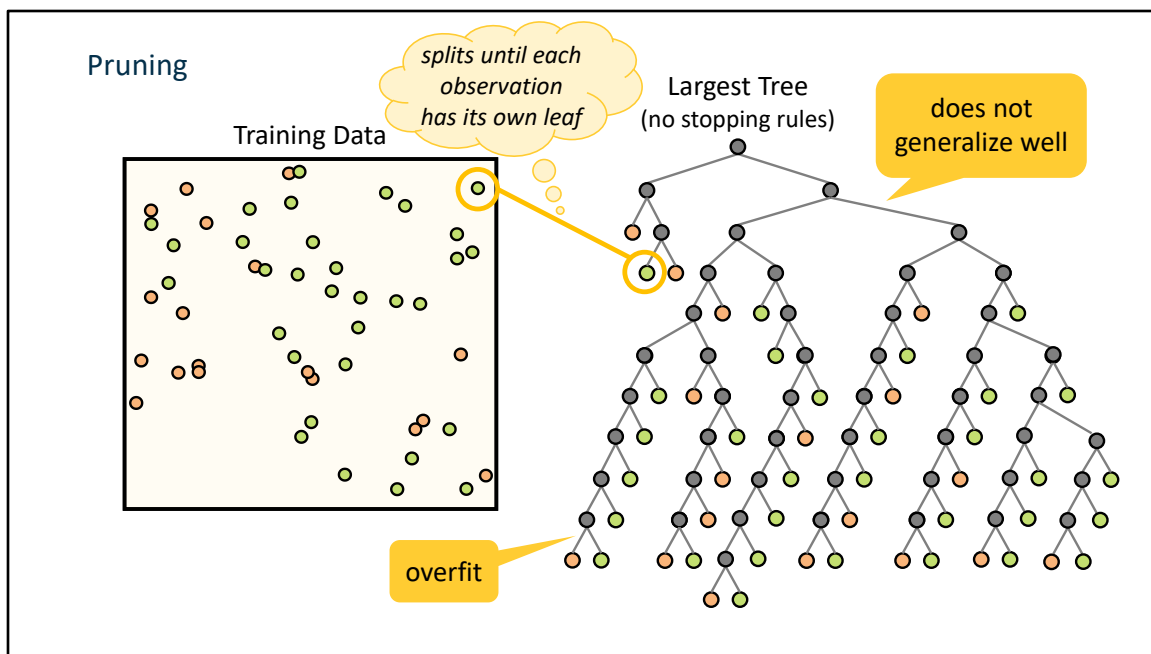
pruning

Remember that the goal of predictive modeling is to build models that can be applied to future cases so that you can make business decisions. With this goal in mind, the next task in the discovery phase for decision trees is optimizing complexity. Optimizing complexity starts with the maximal tree and then removes branches using a process called pruning. The optimal model is the model that generalizes to new data and is a subset of the maximal tree. The optimal tree has the best balance between bias and variance.

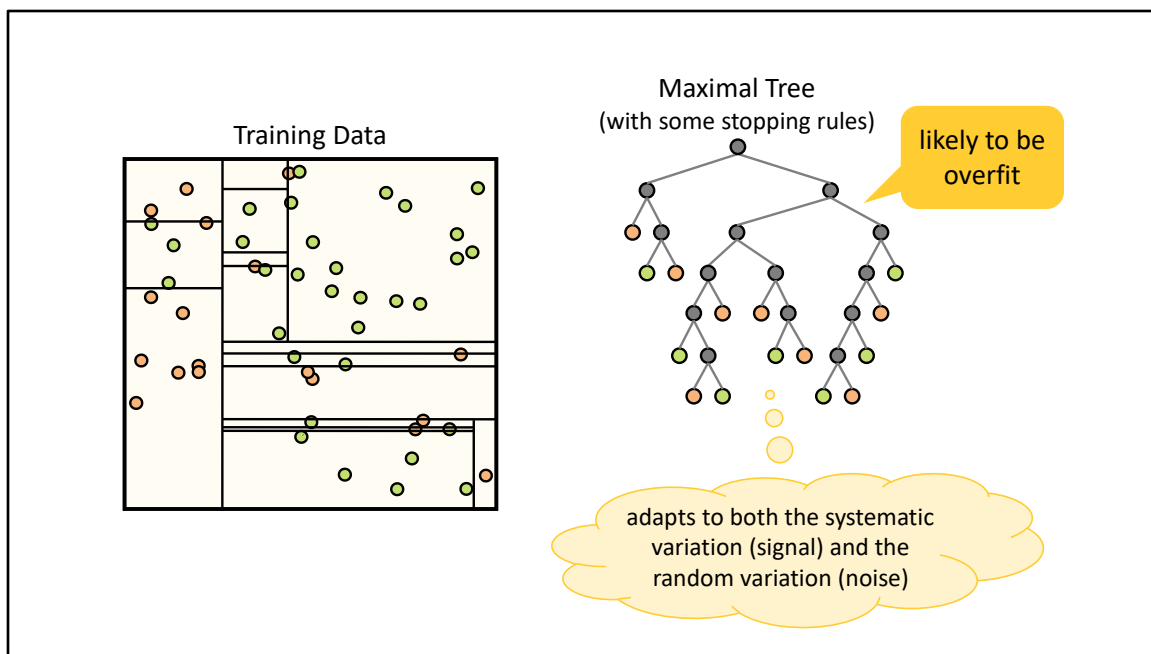
Pruning



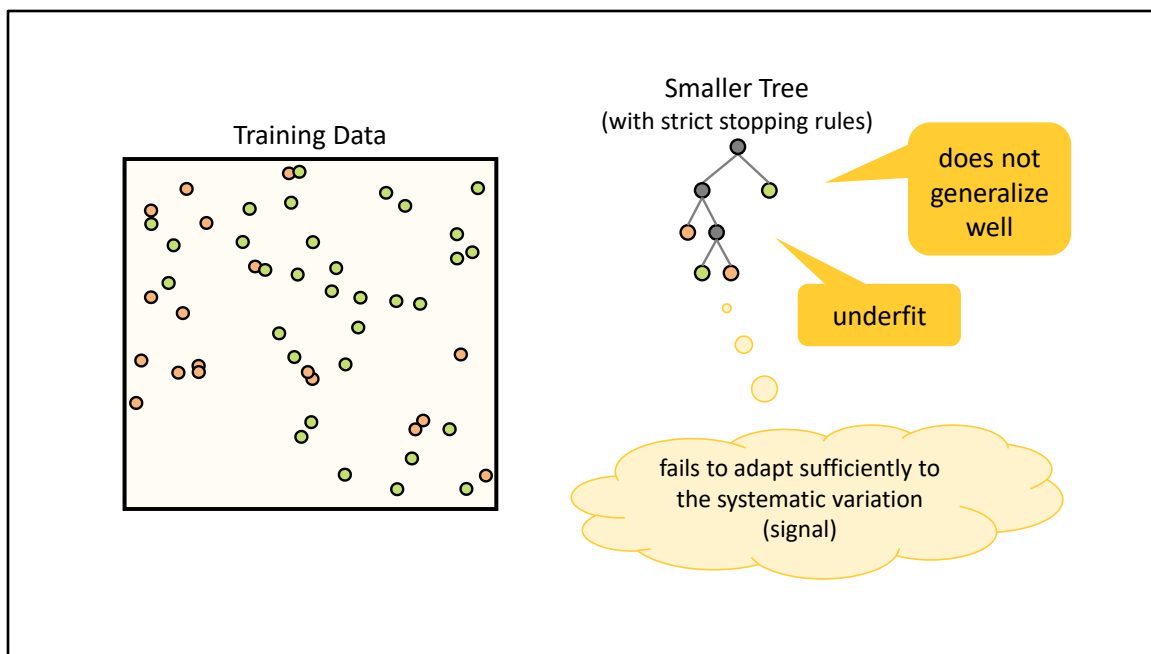
Copyright © SAS Institute Inc. All rights reserved.



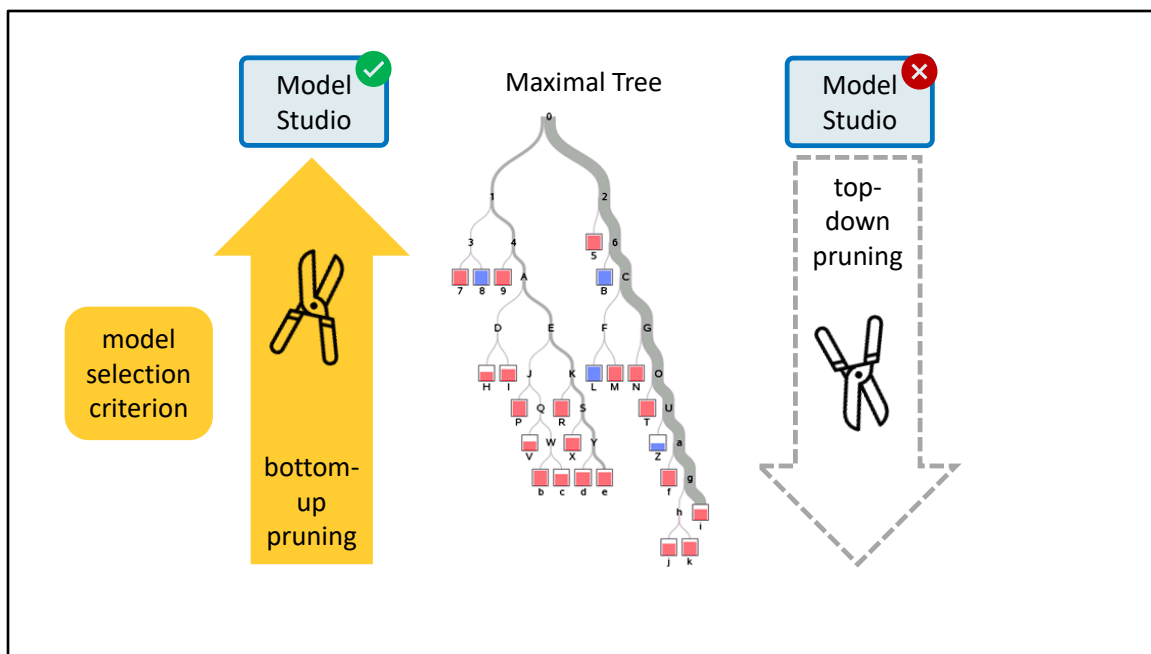
A decision tree algorithm can easily overfit the data. If recursive partitioning continues to split the data until each observation has its own leaf (that is, if there are no stopping rules), it will be 100% accurate for every observation in the training data. This tree does not generalize well to new data.



The maximal tree that results from recursive partitioning is smaller than the largest possible tree but is still likely to be overfit. The maximal tree adapts to both the systematic variation of the target (or the signal) and the random variation (or the noise).

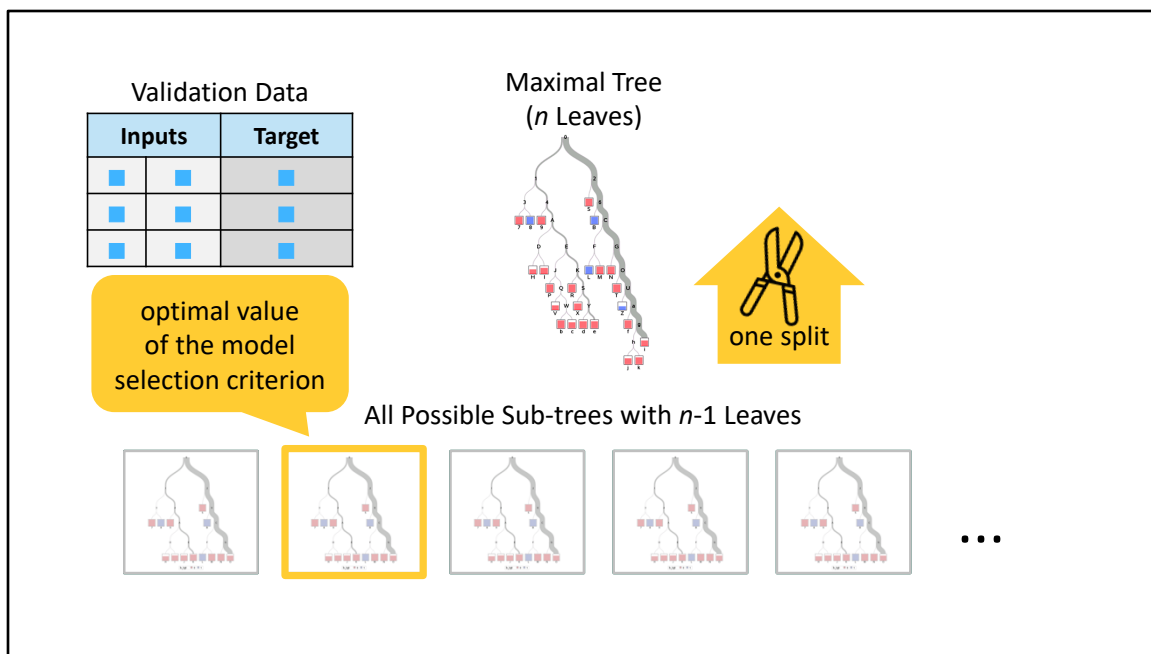


At the other extreme, a small tree with only a few branches might underfit the data. It might fail to adapt sufficiently to the signal, which usually results in poor generalization.



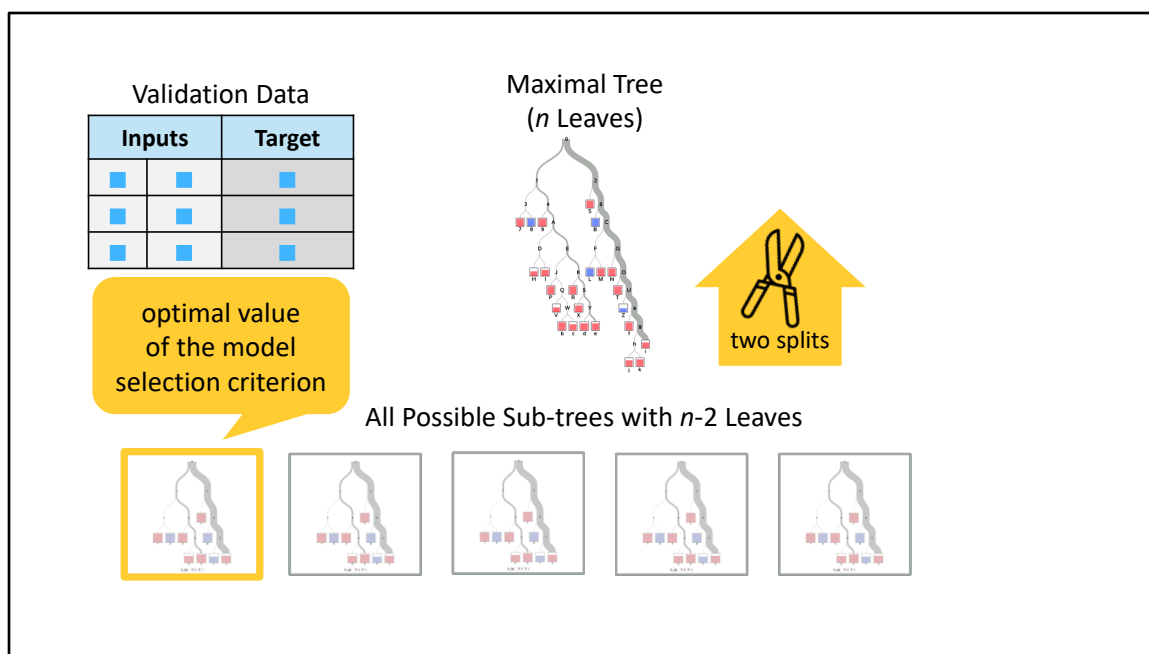
To build the optimal tree, which is neither too large nor too small, Model Studio starts with the maximal tree and uses a process called bottom-up pruning to remove branches. Bottom-up pruning is also known as post pruning or retrospective pruning. The other type of pruning, which is top-down pruning, is not available in Model Studio. Top-down pruning is usually faster but is considered less effective than bottom-up pruning.

Bottom-up pruning starts at the leaf level and removes branches in a backward fashion using a model selection criterion.



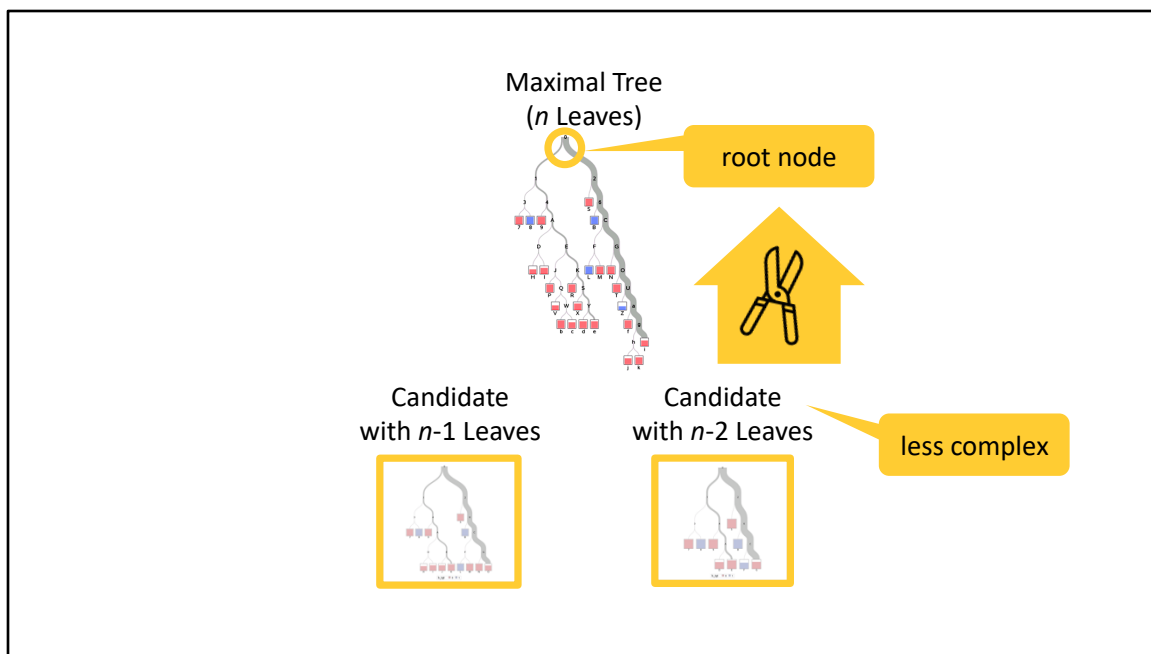
Suppose the maximal tree has n leaves. First, the pruning algorithm evaluates all possible sub-trees consisting of $n-1$ leaves. To do this, the algorithm removes (or prunes) one split.

Out of all possible sub-trees with $n-1$ leaves, the algorithm selects the sub-tree with the optimal value of the model selection criterion on the validation data. The selected sub-tree is the candidate from all sub-trees with $n-1$ leaves.



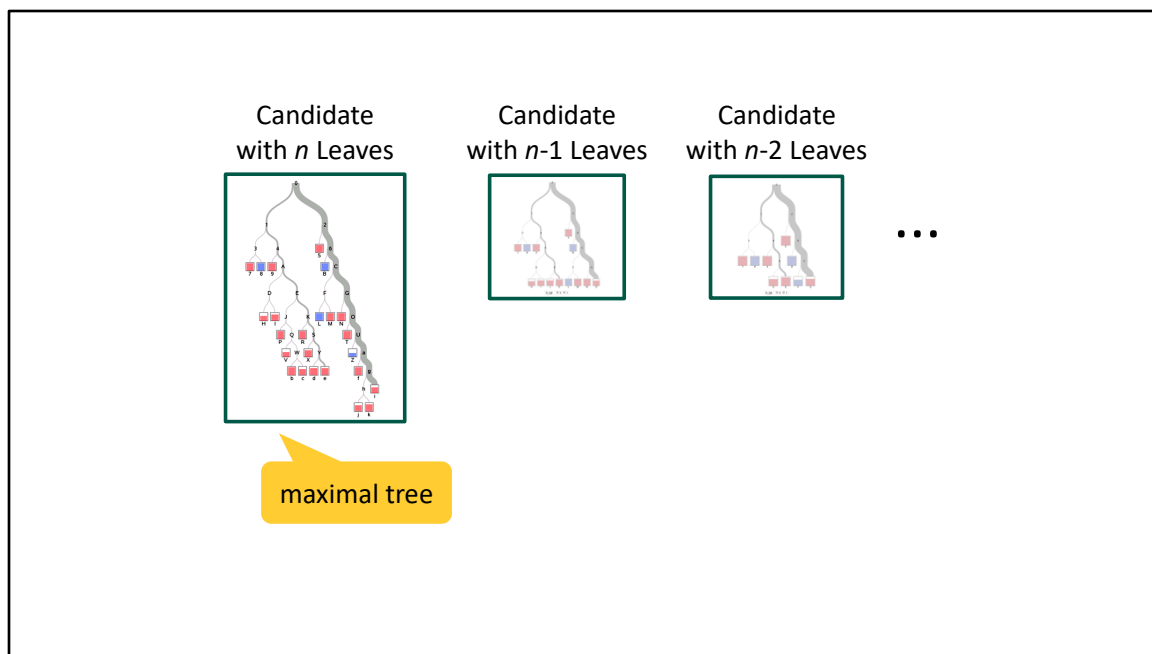
Then the pruning algorithm returns to the maximal tree.

This time, the algorithm evaluates all possible sub-trees consisting of $n-2$ leaves by pruning two splits and selecting the candidate from all sub-trees with $n-2$ leaves.

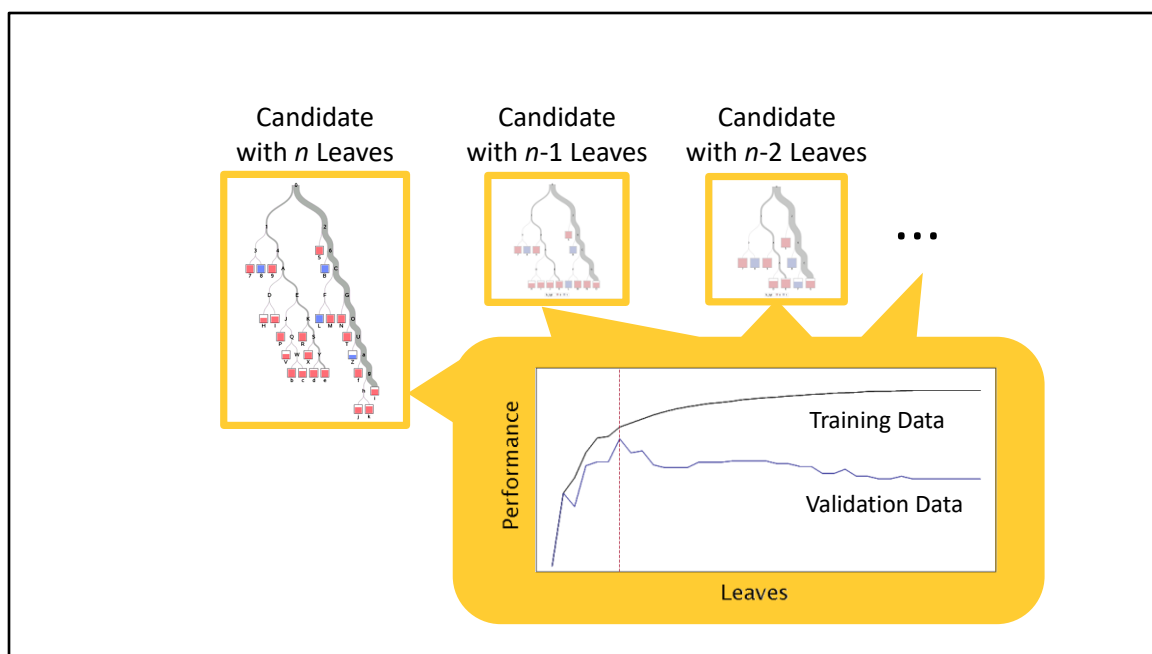


The candidate with $n-2$ leaves is less complex than the candidate with $n-1$ leaves.

The pruning algorithm continues in this way until it reaches the root node.



At this point, the algorithm has created a sequence of candidate sub-trees, ranging from the most complex (that is, the maximal tree) to the least complex (that is, the root node). The candidate sub-trees are not necessarily nested within each other.



Finally, from the sequence of candidate sub-trees, the pruning algorithm selects the sub-tree with the best performance on the model selection criterion based on the validation data. If multiple sub-trees have the same performance, the algorithm selects the least complex sub-tree from this group.

Bottom-Up Pruning Requirements

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Pruning Options in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Modifying the Pruning Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Pruning



Copyright © SAS Institute Inc. All rights reserved.

Question: Pruning

Which of the following statements is true about pruning decision trees?

- a. The goal of pruning is a tree with low bias and high variance.
- b. Pruning starts by identifying a sequence of candidate sub-trees, one for each possible number of leaves, and then selects the best of the candidates.
- c. In bottom-up pruning, the sub-tree with the best performance on training data is selected.
- d. Top-down pruning is usually more time-consuming but is considered more effective than bottom-up pruning.

Answer: Pruning

Which of the following statements is true about pruning decision trees?

- a. The goal of pruning is a tree with low bias and high variance.
- ☒ b. Pruning starts by identifying a sequence of candidate sub-trees, one for each possible number of leaves, and then selects the best of the candidates.
- c. In bottom-up pruning, the sub-tree with the best performance on training data is selected.
- d. Top-down pruning is usually more time-consuming but is considered more effective than bottom-up pruning.

CA: b

FB: Accuracy is obtained by multiplying the proportion of observations falling into each leaf by the proportion of those correctly classified in the leaf and then summing across all leaves.

Regularizing and Tuning the Hyperparameters of a Machine Learning Model



Copyright © SAS Institute Inc. All rights reserved.

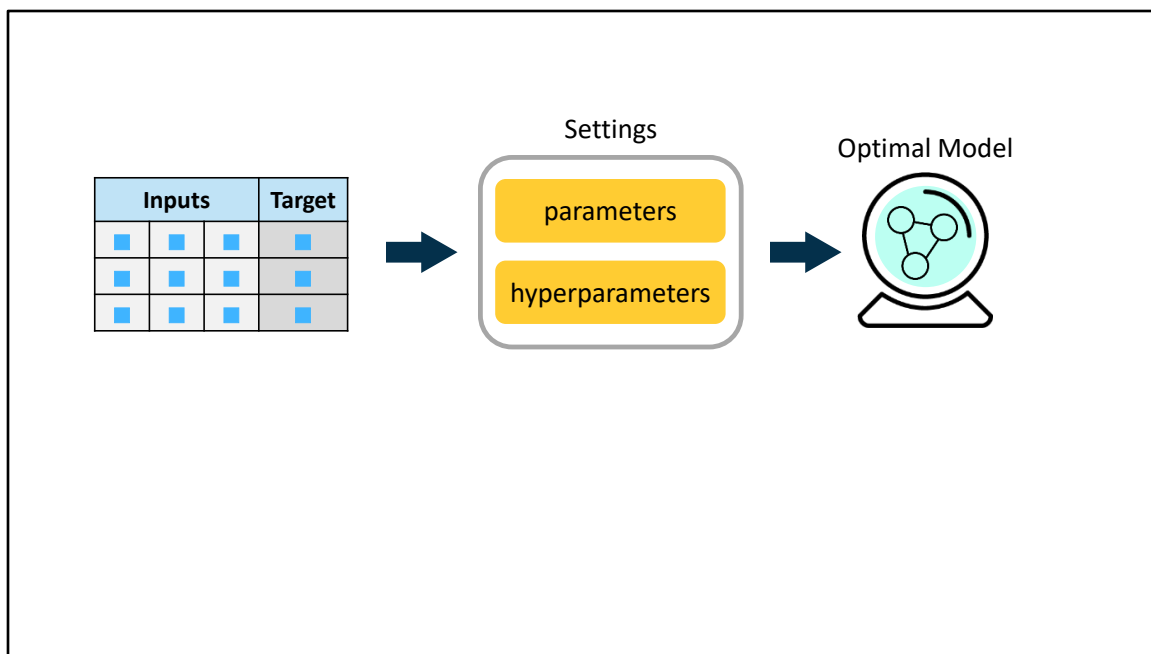
Regularizing and Tuning the Hyperparameters of a Machine Learning Model



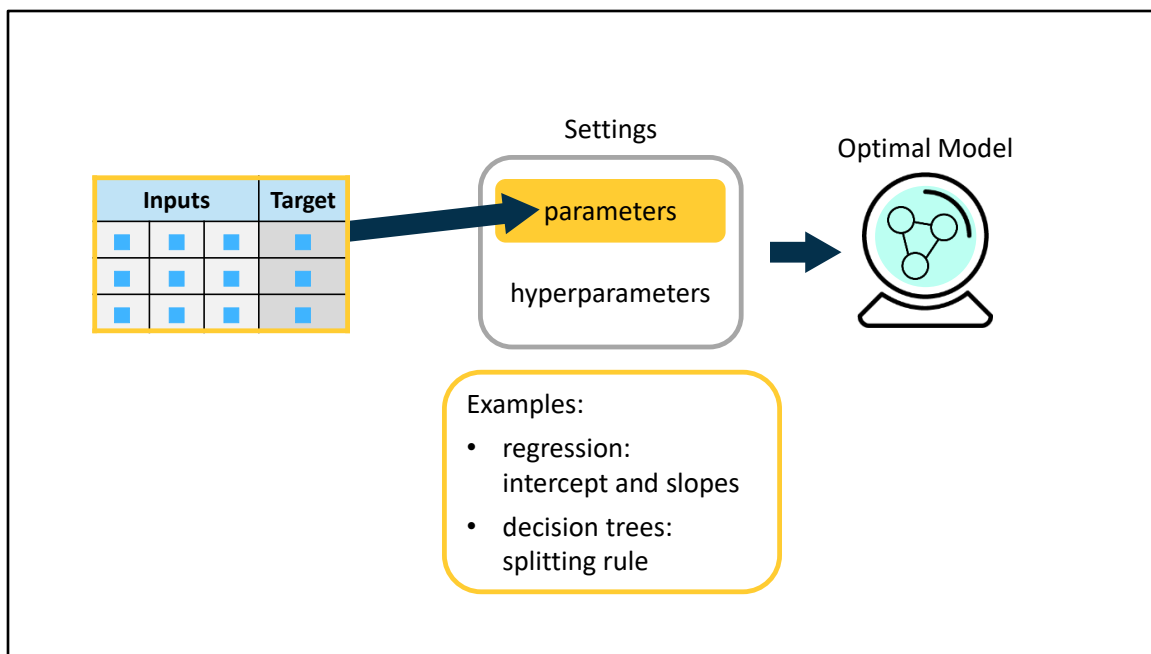
Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
- ✓ Optimize the complexity of the model.
 - Regularize and tune the hyperparameters of the model.
 - Build ensemble models.

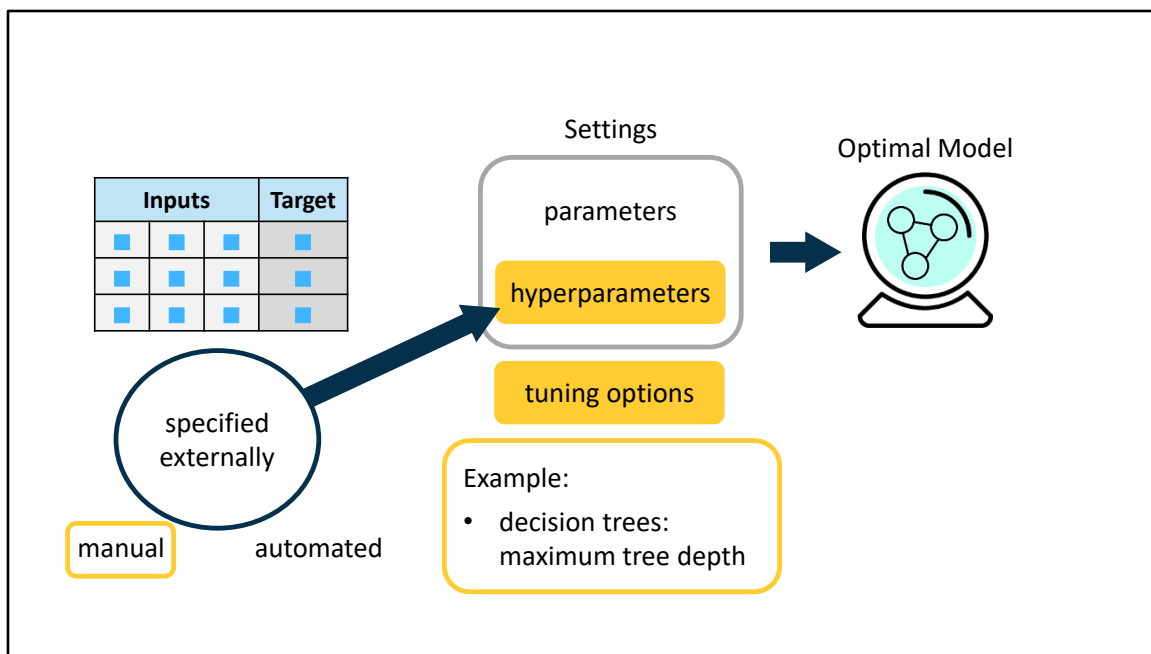
Regularizing and tuning the hyperparameters of the model is an important discovery task.



As with any predictive model, when you build a machine learning model, you want to find the model settings that result in the best model performance and generalization. Settings include both the parameters and the hyperparameters of the model. So, what is the difference?

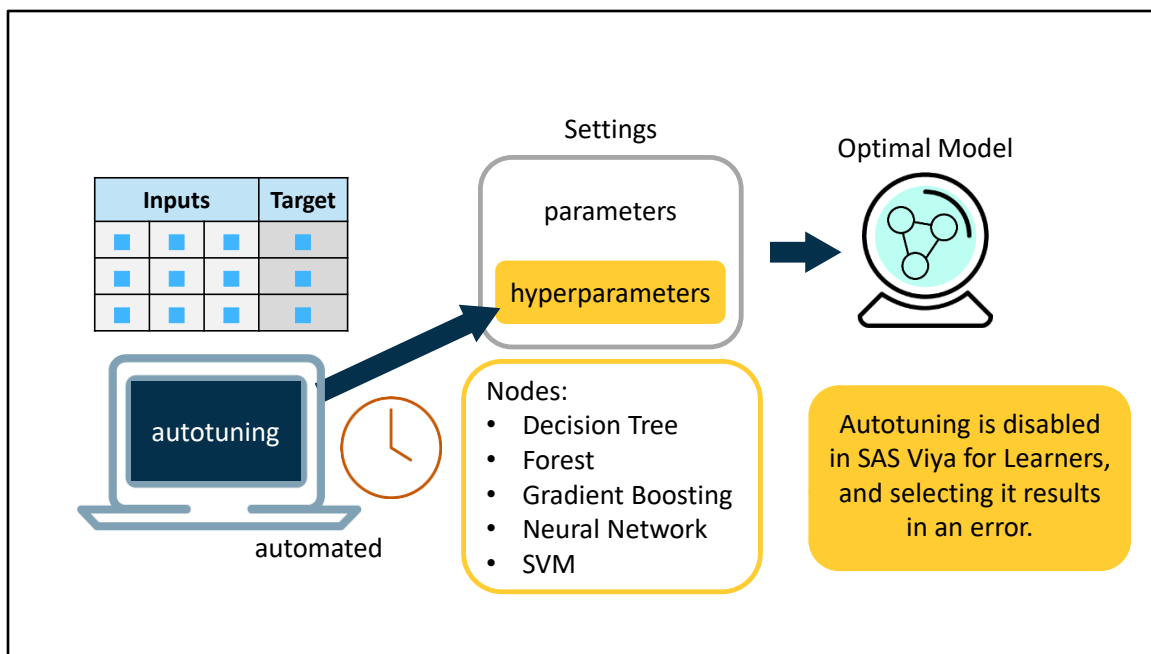


A model *parameter* is a part of the model whose value can be estimated from the data during model training. For models that can be defined by a mathematical equation that provides a prediction, a parameter is typically a variable in that equation. For example, in a regression model, the intercept and the slopes are parameters that come from the associated mathematical equation. For decision trees, a splitting rule is an example of a parameter.



A model *hyperparameter* is a variable that is used to find the optimal model but whose value cannot be estimated from the data. Instead, these values must be specified externally, either manually or through an automated process. Hyperparameters are sometimes called tuning options. For decision trees, maximum tree depth is an example of a hyperparameter.

Settings for the hyperparameters can significantly influence the resulting accuracy of the predictive models. However, there are no clear defaults that work well for different data sets. It is possible to discover the best hyperparameters manually (for example, based on experience, trial and error, and heuristics), but this is time-consuming.



As an alternative, machine learning functionality in SAS Viya provides an automated option for hyperparameter optimization, which is called autotuning. Autotuning capabilities are available for the following nodes: Decision Tree, Forest, Gradient Boosting, Neural Network, and SVM, as well as other modeling nodes that are not discussed in this course. In general, autotuning searches for the best combination of hyperparameters specific to each modeling algorithm.

When you decide whether to perform autotuning, keep in mind that autotuning can substantially increase run time.

The screenshot displays the SAS Model Studio 'Build Models' interface. The top navigation bar includes a search bar and a 'student' user profile. The main workspace shows a pipeline diagram with the following steps: Data, Reformat, Transform, Split, Tree Mining, Merge Variables, Variable Selection, Logistic Regression, Decision Tree, Gradient Boosting, Forest, Decision Tree (1), and Decision Tree (2). The 'Tree Based' model is selected. On the right, the 'Perform Autotuning' settings panel is open, showing the following configuration:

- Perform Autotuning:** Enabled (toggle switch).
- Maximum Depth:** Initial value: 10, From: 1, To: 19.
- Interval Input Bins:** Initial value: 20, From: 20, To: 200.
- Grow Criterion:** Class target: Entropy, CHAID, Information gain ratio (all checked).

To turn on autotuning in Model Studio, you select the **Perform Autotuning** option in the properties pane for the selected modeling node. You can then specify the hyperparameters that you want to tune. The available hyperparameters are specific to each model. Autotuning involves building and evaluating models based on a range of values for each of the selected hyperparameters.

There is no guarantee that autotuning will arrive at the best model. However, autotuning can provide a good starting place for you to use your knowledge and domain expertise to improve the model.

Hyperparameter Optimization Approaches

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Autotuning Options for Decision Trees in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Using the Best Hyperparameter Values for a Model

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



(Optional) Practice: Build a Decision Tree Model Using Autotuning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



3.5 Building and Modifying Ensembles of Trees



Building Ensemble Models



Copyright © SAS Institute Inc. All rights reserved.

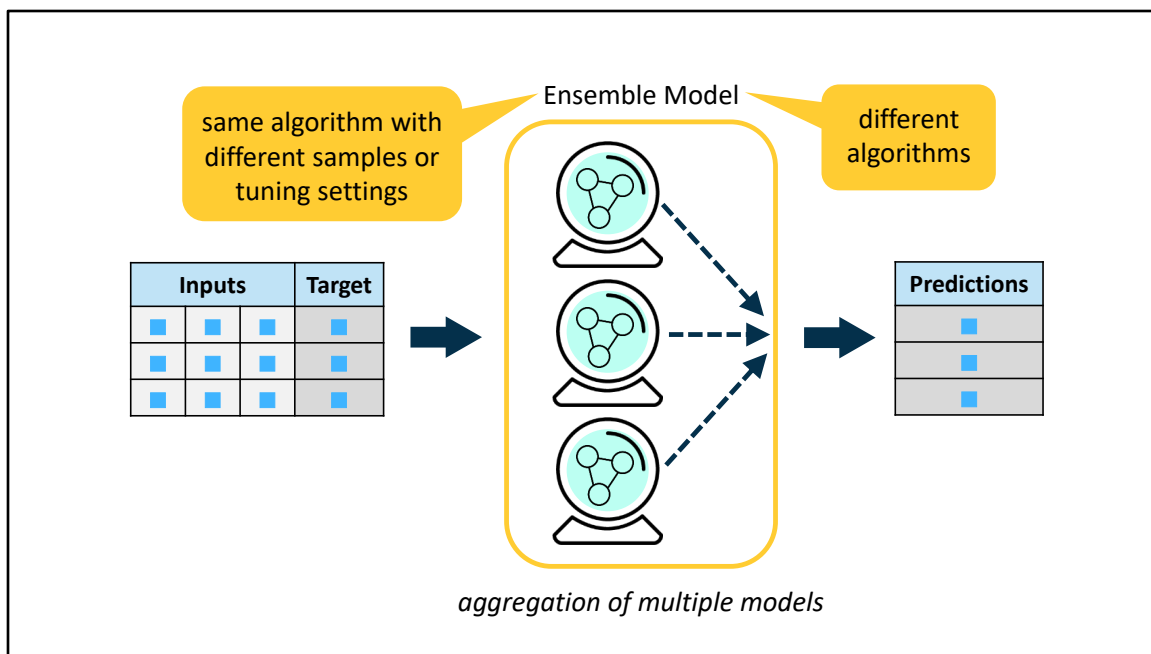
Building Ensemble Models



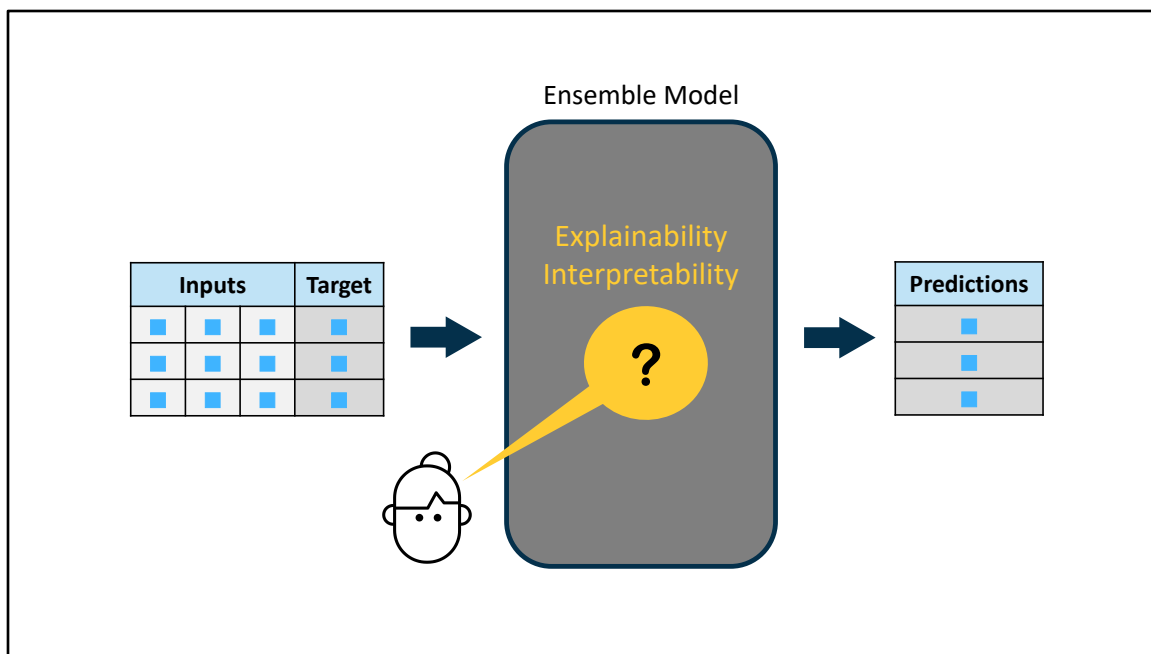
Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
- ✓ Optimize the complexity of the model.
- ✓ Regularize and tune the hyperparameters of the model.
- Build ensemble models.

You can spend a fair amount of time building, improving, optimizing, and tuning a single machine learning model for each algorithm. However, any one of these individual models might not be the most effective solution for a specific problem and data set. In this case, you can consider building an ensemble model.



An ensemble model is an aggregation of multiple models. The final prediction from the ensemble model is a combination of the predictions from the component models. These component models can be trained from different algorithms that approach the problem from different perspectives, or from the same algorithm applied to different samples or using different tuning settings, or both.



It is typically not possible to interpret an ensemble model.

The screenshot displays the SAS Model Studio 'Build Models' interface. The top navigation bar includes a search bar and a user profile 'student'. The main workspace is titled 'Demo' and shows a 'Pipeline Comparison' view. The selected pipeline is 'Tree Based', which is a sequence of steps: 'Data' (input), 'Exploration', 'Transformation', 'Imputation', and 'Test Mining'. From 'Test Mining', the pipeline branches into two parallel paths: one through 'Message Selection' to 'Logistic Regression', and another through 'Variable Selection' to 'Decision Tree', 'Gradient Boosting', and 'Random Forest'. All four models then feed into a final 'Model Comparison' step. The right-hand configuration panel is set to the 'Forest' model, showing its description and various tuning parameters: 'Number of trees' is set to 100, 'Class target voting method' is 'Probability', 'Tree-splitting Options' include 'Class target criterion' (Information gain ratio) and 'Interval target criterion' (Variance), and 'Maximum number of branches' is set to 2.

When you build an ensemble model in Model Studio, you can choose to include models based on a variety of algorithms or on a single algorithm. In this lesson, you learn to build two single-algorithm ensemble models: gradient boosting and forest.

Perturb and Combine Methods

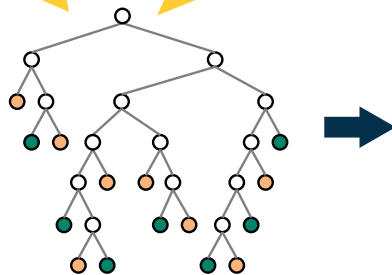


Copyright © SAS Institute Inc. All rights reserved.

Perturb and Combine Methods

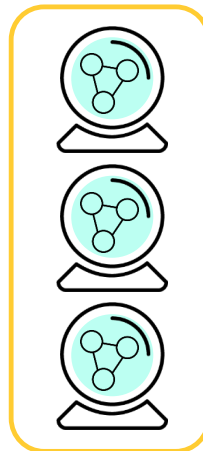
unstable structure

stable performance



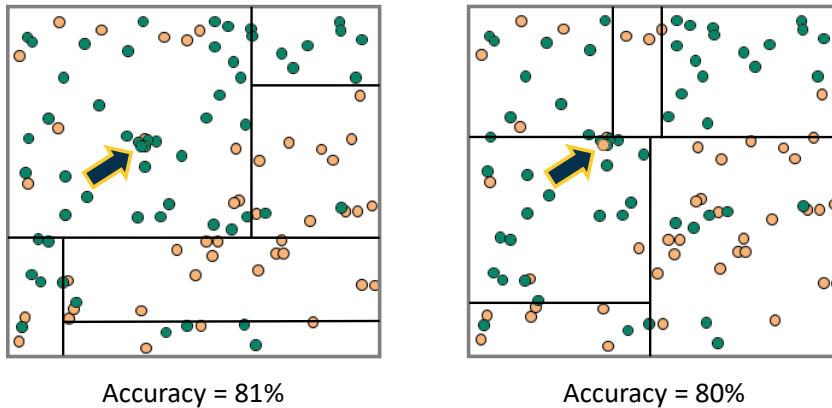
*Trees are good candidates
for ensemble models.*

Ensemble Model



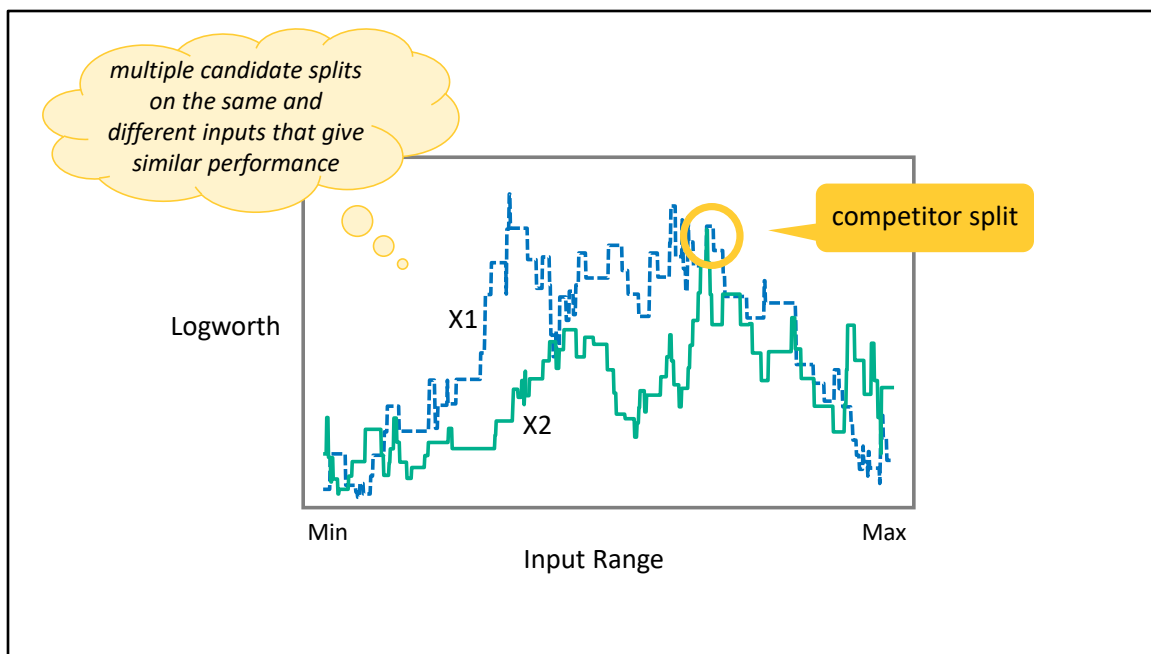
Decision tree models have a very unstable structure, so they are a good candidate for ensemble models. Even small changes in the training data can cause large changes in the tree structure and other aspects of the tree model. However, despite these changes, the overall performance of the trees usually remains stable.

Instability in a Single Tree



This example shows a scatter plot of two inputs and a binary target with the two outcomes green and gold. The plot also contains lines showing the results of recursive partitioning. This tree has an accuracy of 81%.

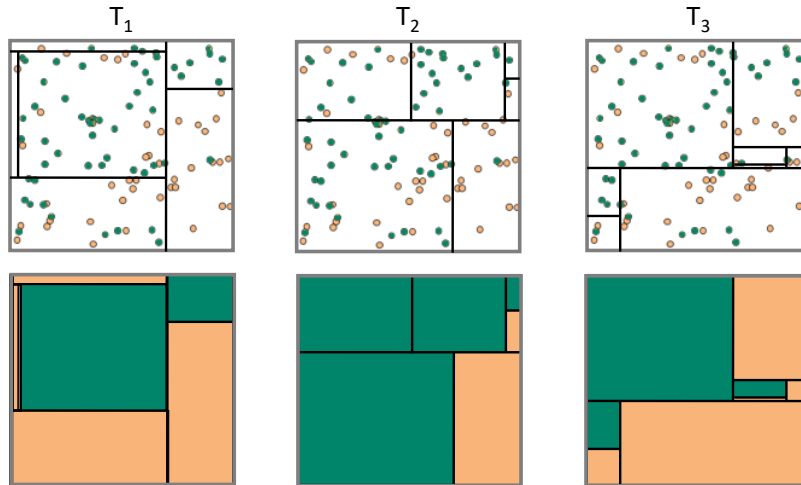
Changing the outcome of one case from green to gold results in a completely different tree, as shown by the change in the results of recursive partitioning. However, the accuracy is almost identical: 80%. So, what causes this instability?



This plot shows the logworth for all possible values of two input variables, X1 and X2, which are candidates for a single split. The input range is on the x axis and logworth is on the y axis.

The instability of trees results from the large number of univariate splits considered during recursive partitioning and the increasing fragmentation of the data. At each split point, there are typically multiple candidate splits on the same and different inputs that give similar performance. These are called competitor splits. A small change in the data can easily result in the selection of a different competitor split, which produces different subsets in the child nodes. The changes in the resulting subsets increase with each new generation of the tree.

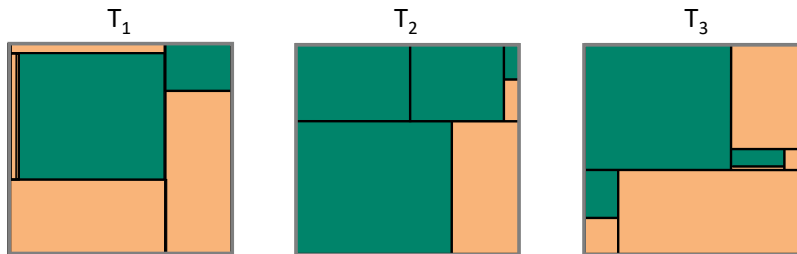
1. **Perturb:** Create different models.



The three pairs of plots shown here represent three different trees created during the perturb step. These trees are based on data shown earlier, which has a binary target (with green and gold outcomes) and two inputs. For each tree, the top plot is a scatterplot of the data with lines showing the splits. The bottom plot shows the decision predictions from the tree. Notice that the data points are identical in all three scatterplots. From this, we can infer that the perturb method in this example involves altering the construction method, such as changing the splitting criterion, instead of manipulating the data distribution.

1. **Perturb:** Create different models.

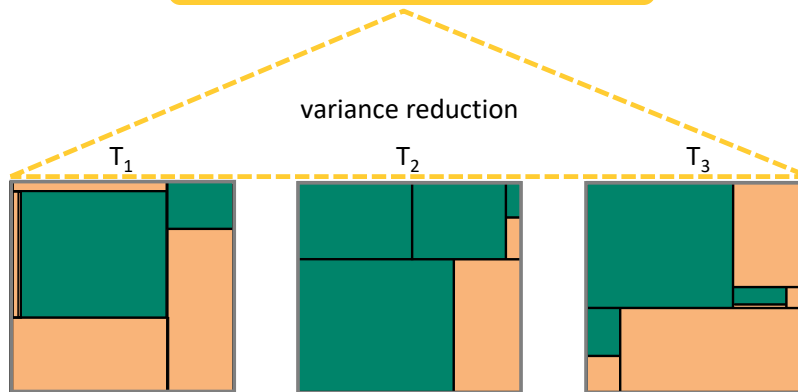
- resampling
- subsampling
- adding noise
- adaptively reweighting
- randomly choosing from the competitor splits



Commonly-used perturbation methods include resampling, subsampling, adding noise, adaptively reweighting, and randomly choosing from the competitor splits.

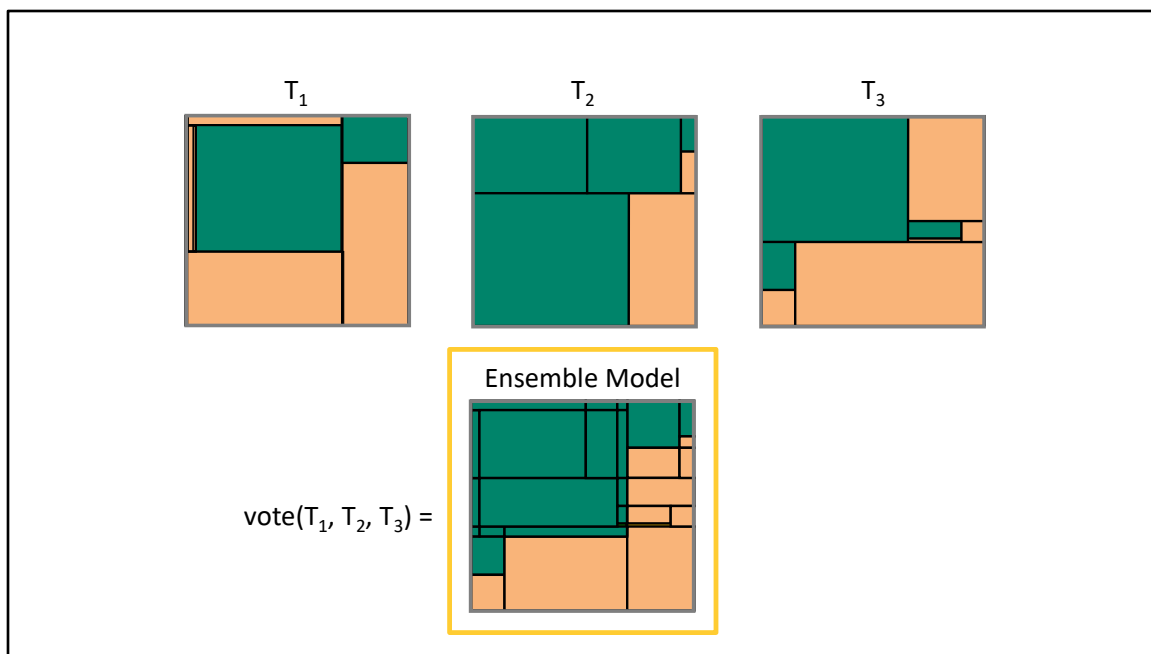
2. **Combine:** Create a single prediction.

- voting
- weighted voting
- averaging

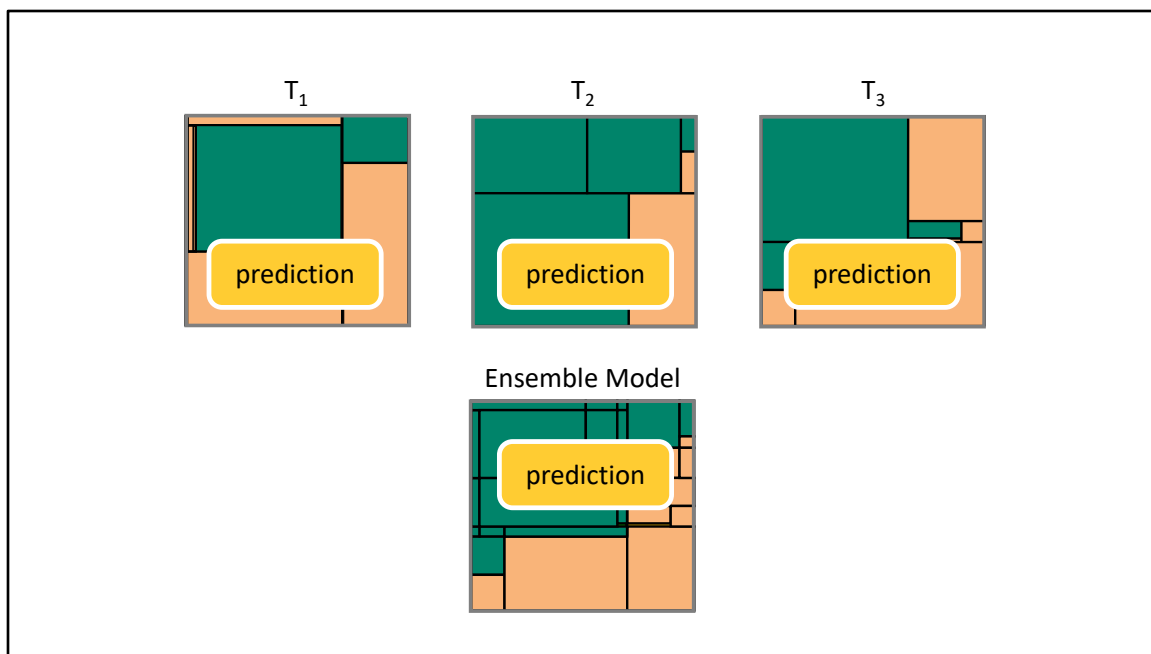


To create the ensemble model, the individual models can be combined in various ways, such as voting on the classifications, using weighted voting (where some models have more weight), and averaging the predicted values (either weighted or unweighted). The attractiveness of P & C methods is their improved performance over single models.

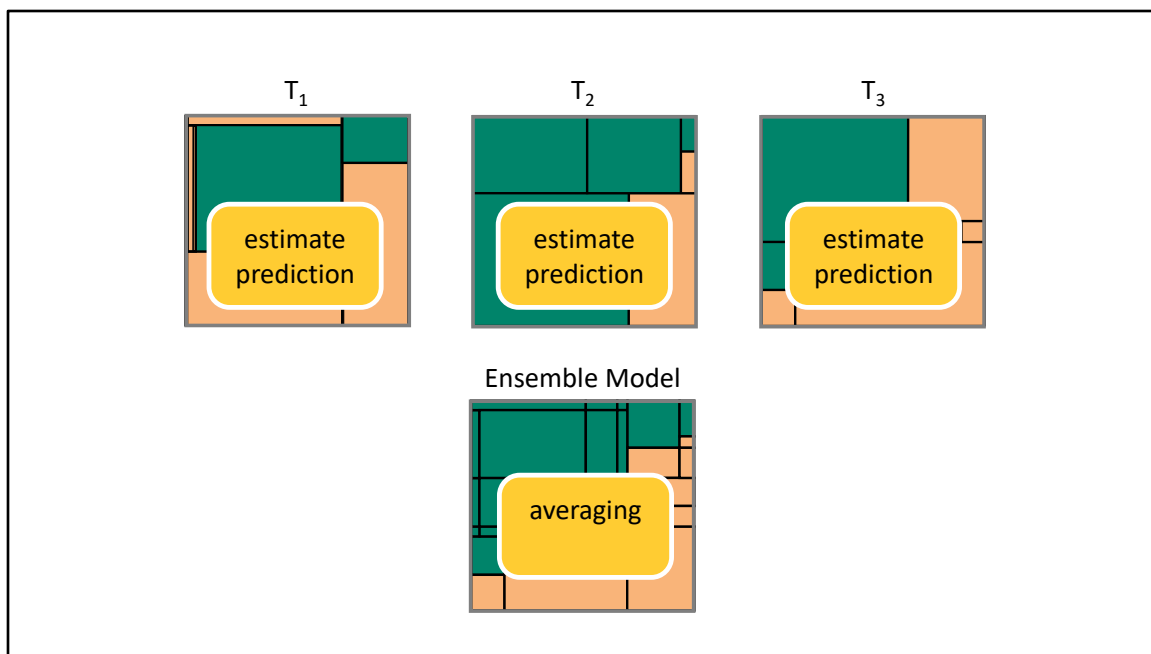
One reason why simple P & C methods give improved performance is variance reduction. If the base models have low bias and high variance, then averaging decreases the variance. In contrast, combining stable models can negatively affect performance. P & C methods create effective ensemble models for reasons beyond simple variance reduction, and these are the topic of much research.



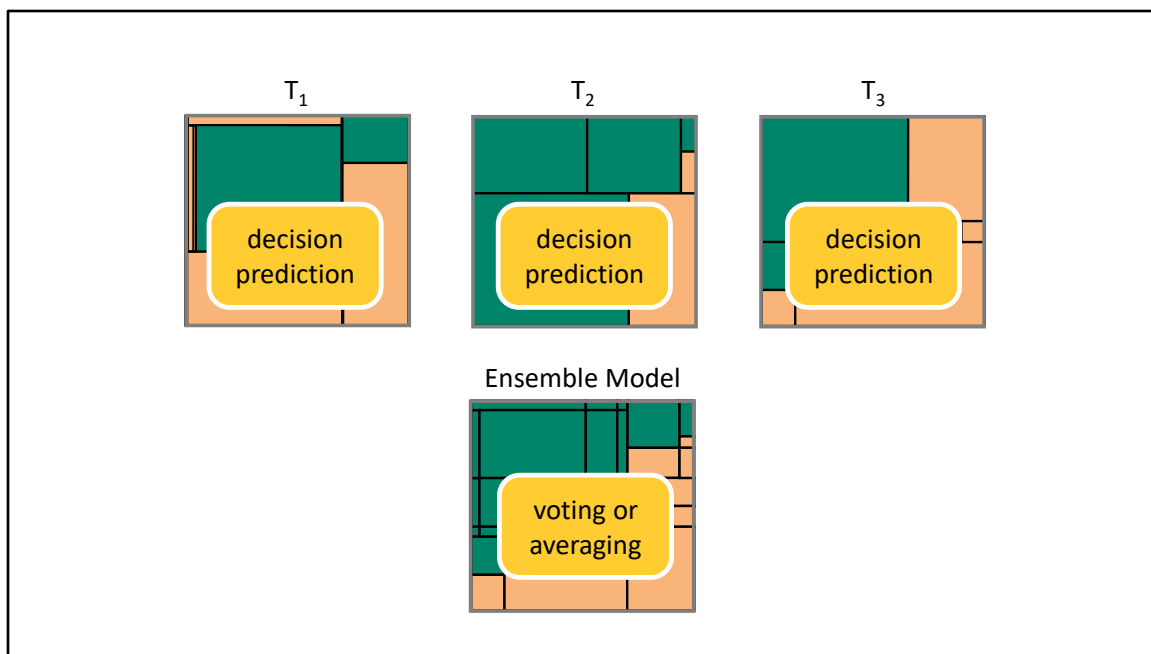
Here is a graph of the ensemble model for this example. Graphical representations show that ensembles of trees have decision boundaries of much finer resolution than would be possible with a single tree. The decision boundaries of a single tree can be viewed as step functions. The benefit of ensembling many trees together is that, by adding more steps, the steps themselves are essentially smoothed out.



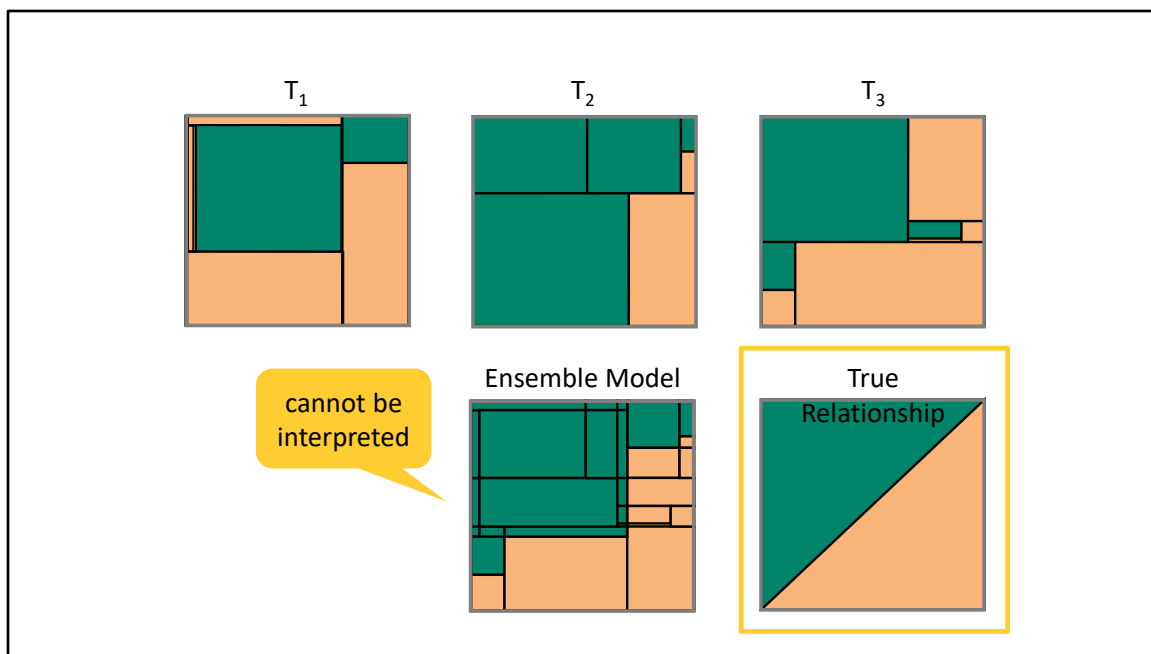
A new case is scored by obtaining a prediction from each individual tree and then combining the results.



If the predictions are estimates, the results are averaged.



If the predictions are decisions, results can be combined by voting or averaging the posterior probabilities as the basis for the final decision.



The main drawback of ensemble models is that they cannot be interpreted, unlike a single tree.

Here is a representation of the true relationship between the target and the inputs. This relationship is defined by a straight line. All cases above that line have a green outcome and all cases below the line have a gold outcome. Obviously, this is a simplified example. Real data are not likely to be linearly separable.

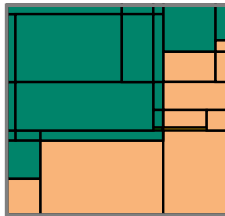
P & C Methods

bagging

boosting

...

Ensemble Model



In this course, we look at two P & C methods: bagging and boosting. Ensemble methods are a very active area of research in the fields of machine learning and statistics, so there are many additional P & C methods.

Question: Perturb and Combine Methods



Copyright © SAS Institute Inc. All rights reserved.

Question: Perturb and Combine Methods

Perturb and combine methods are used only with decision trees.

- a. True
- b. False

Answer: Perturb and Combine Methods

Perturb and combine methods are used only with decision trees.

- a. True
- ☒ b. False

CA: b

FB: Perturb and combine methods can be used with any unstable algorithm but they are mostly used with trees.

Bagging



Copyright © SAS Institute Inc. All rights reserved.

Bagging

P & C Methods

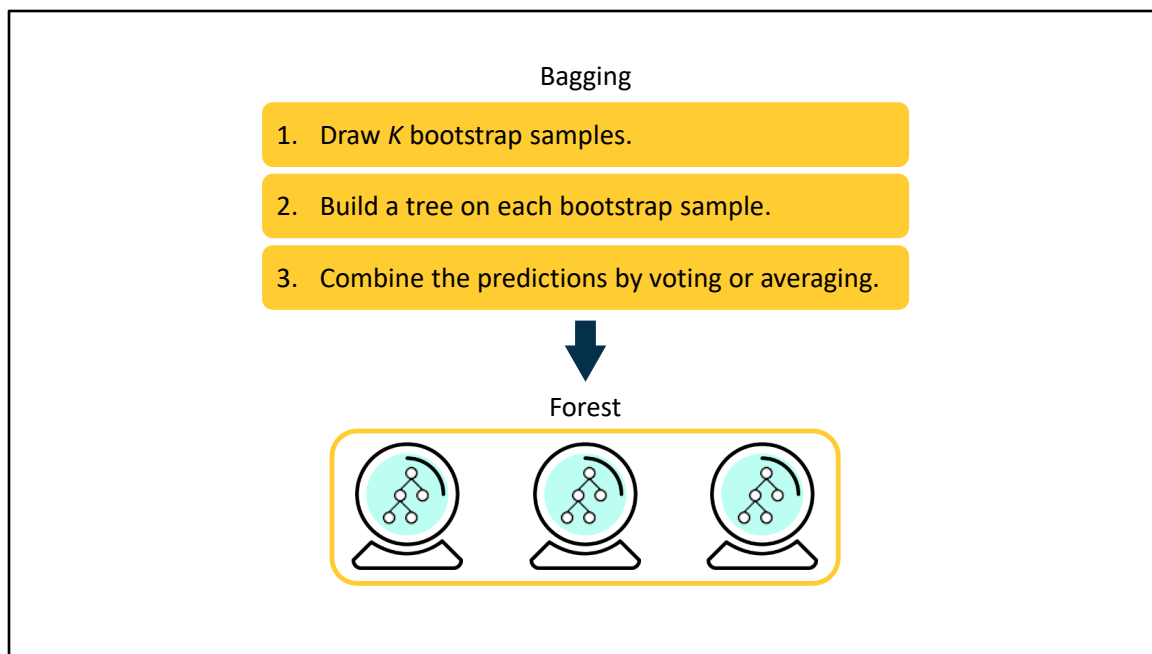
bagging

(bootstrap aggregation)

boosting

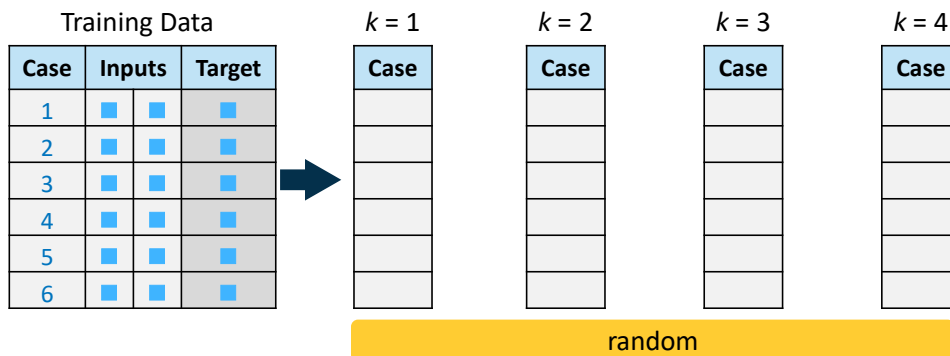
...

Bagging (or *bootstrap aggregation*) is the original perturb and combine method. It was developed by Leo Breiman in 1996.



Let's look at the three steps of the original bagging algorithm. As you'll learn later, Model Studio uses a variation of this algorithm to construct a forest. The three steps are as follows: Step 1. Draw k bootstrap samples. Step 2: Build a tree on each bootstrap sample. And step 3: Combine the predictions by voting or averaging.

1. Draw K bootstrap samples.



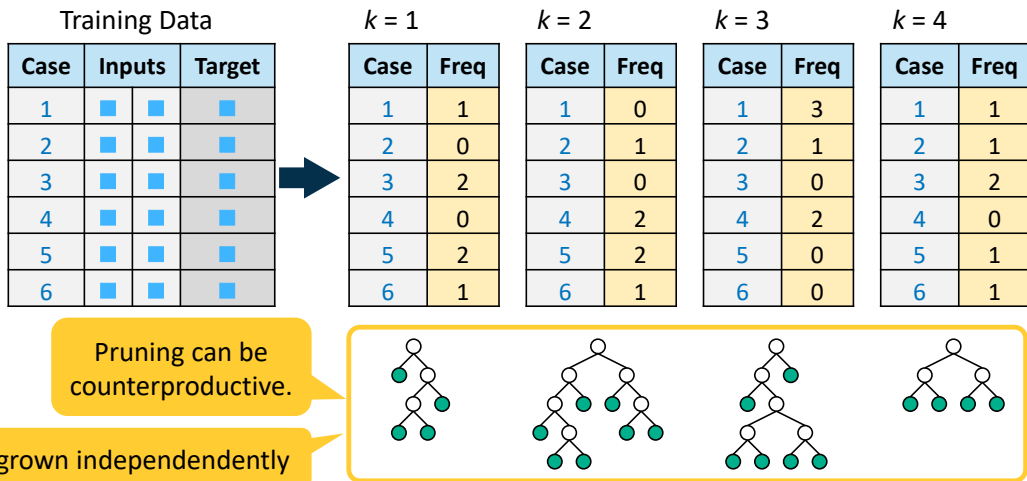
Let's look at a simple example. From a training data set that has six cases, the first step is to draw four bootstrap samples. Each sample has six cases. A *bootstrap sample* is a random sample from a training data set.

1. Draw K bootstrap samples.

Training Data					$k = 1$		$k = 2$		$k = 3$		$k = 4$	
Case	Inputs		Target		Case	Freq	Case	Freq	Case	Freq	Case	Freq
1	■	■	■	➡	1	1	1	0	1	3	1	1
2	■	■	■		2	0	2	1	2	1	2	1
3	■	■	■		3	2	3	0	3	0	3	2
4	■	■	■		4	0	4	2	4	2	4	0
5	■	■	■		5	2	5	2	5	0	5	1
6	■	■	■		6	1	6	1	6	0	6	1
					random with replacement							

For bagging, a bootstrap sample is drawn with replacement. This means that some of the cases might be left out of the sample, and some cases might be represented more than once. For example, in the first bootstrap sample, cases 3 and 5 appear twice, cases 1 and 6 appear once, and cases 2 and 4 do not appear at all.

2. Build a tree on each bootstrap sample.



The second step is to build a tree on each bootstrap sample. When you create an ensemble of bagged trees, large trees with low bias and high variance are ideal, so pruning can be counterproductive. In the bagging method, trees are grown independently of each other, so they can be built in parallel in a distributed computing environment like SAS Viya. This makes bagging a relatively fast method compared to other P&C methods.

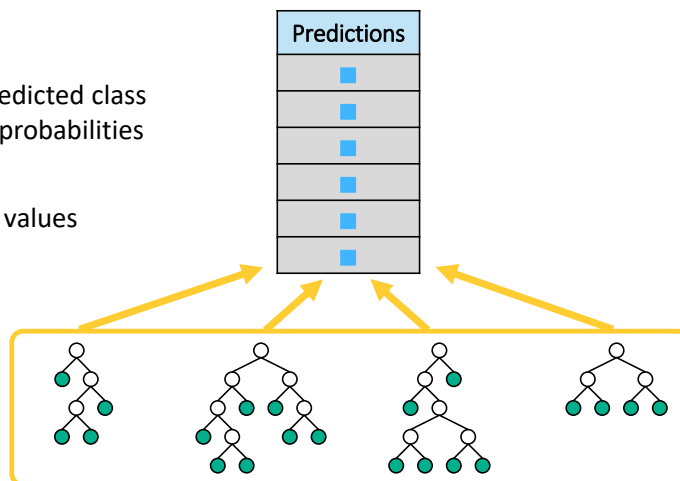
3. Combine the predictions by voting or averaging.

Classification:

- plurality vote of the predicted class
- mean of the posterior probabilities

Interval:

- mean of the predicted values



The third step is to combine the predictions by voting or averaging. For classification problems, you take the plurality vote of the predicted class or the mean of the posterior probabilities. Averaging the posterior probabilities often gives slightly better performance than voting. If you have an interval target, you take a mean of the predicted values.

Boosting



Copyright © SAS Institute Inc. All rights reserved.

Boosting

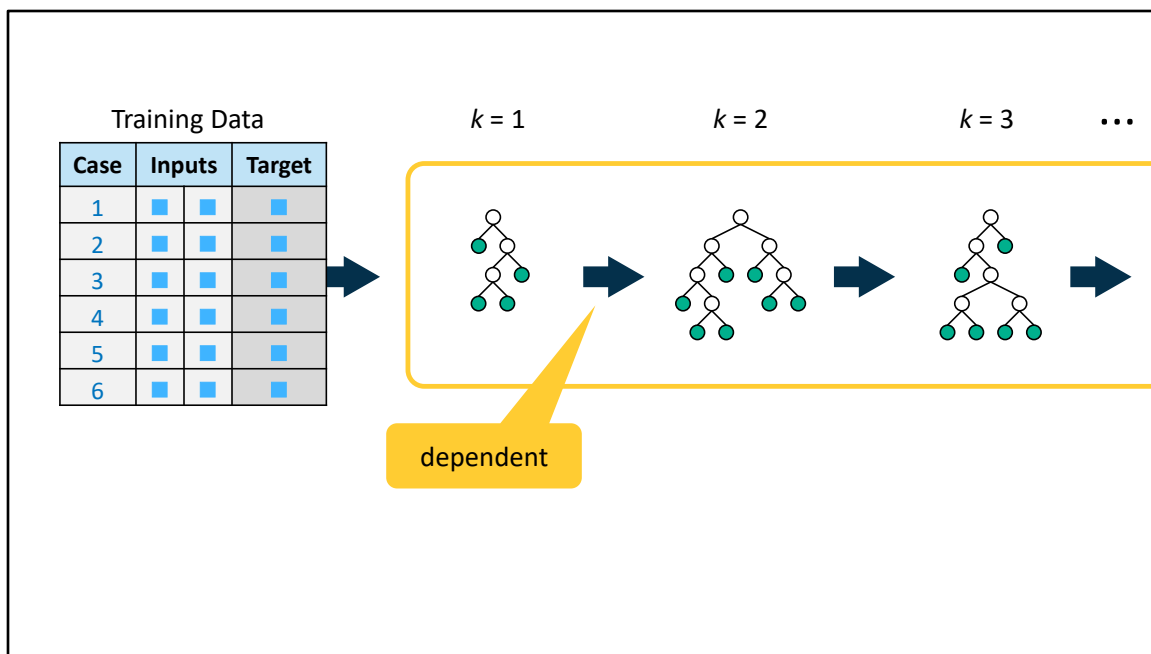
P & C Methods

bagging

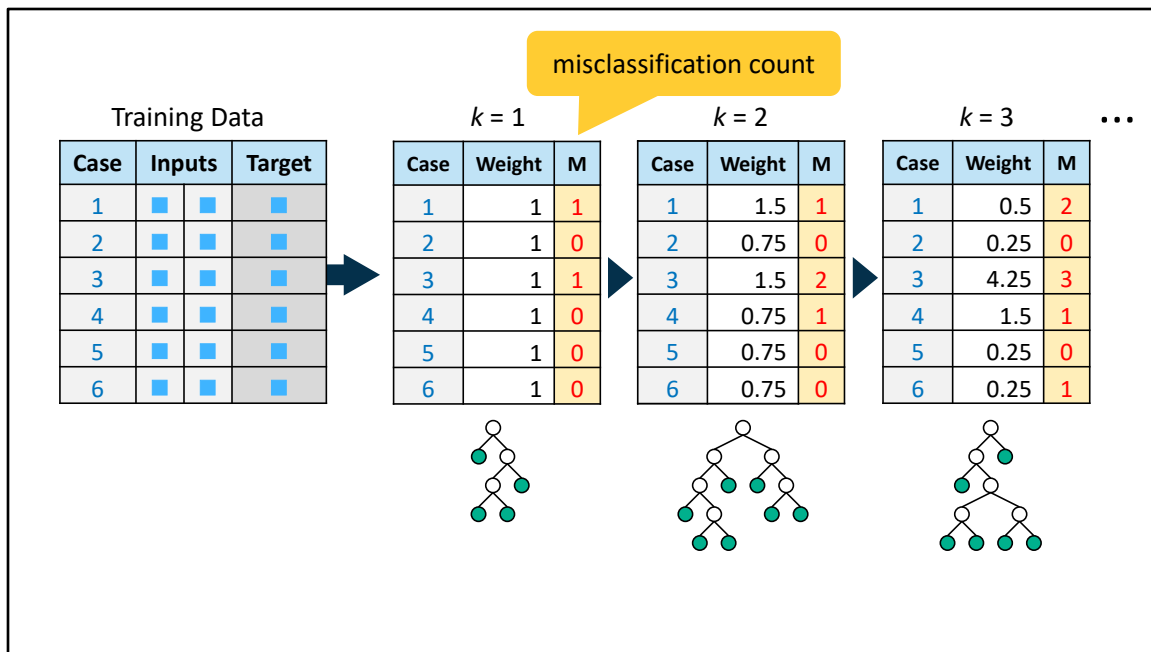
boosting

...

Boosting is a perturb and combine method for creating tree-based ensemble models for categorical targets.

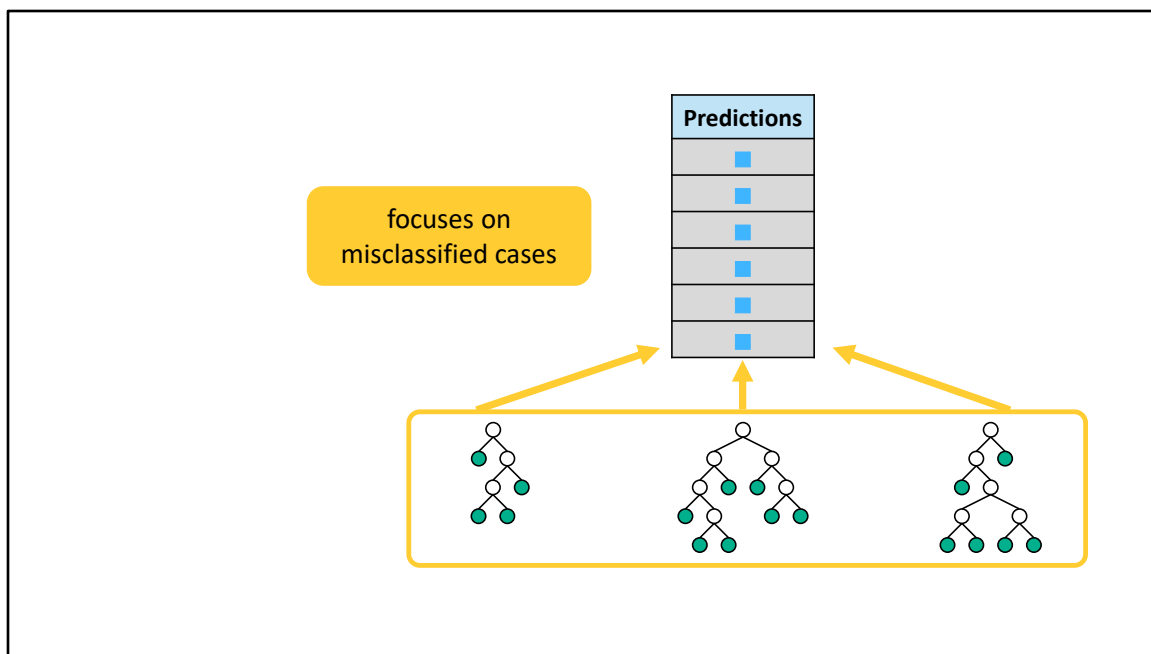


Unlike the bagging method, each tree that boosting creates is dependent on the tree from the previous iteration.



Across iterations, the algorithm keeps a cumulative count of misclassifications. Each case is then weighted based on whether its misclassification count increases in the current iteration. If the misclassification count increases, the weight increases. If the misclassification count remains the same, the weight decreases. The weights influence the likelihood that the case is selected in the sample for the next iteration.

The boosting process continues in this fashion for a predetermined number of iterations. The number of iterations equals the number of trees in the series.



The main advantage of boosting is that the algorithm focuses on the misclassified cases. This improves the performance of the model.

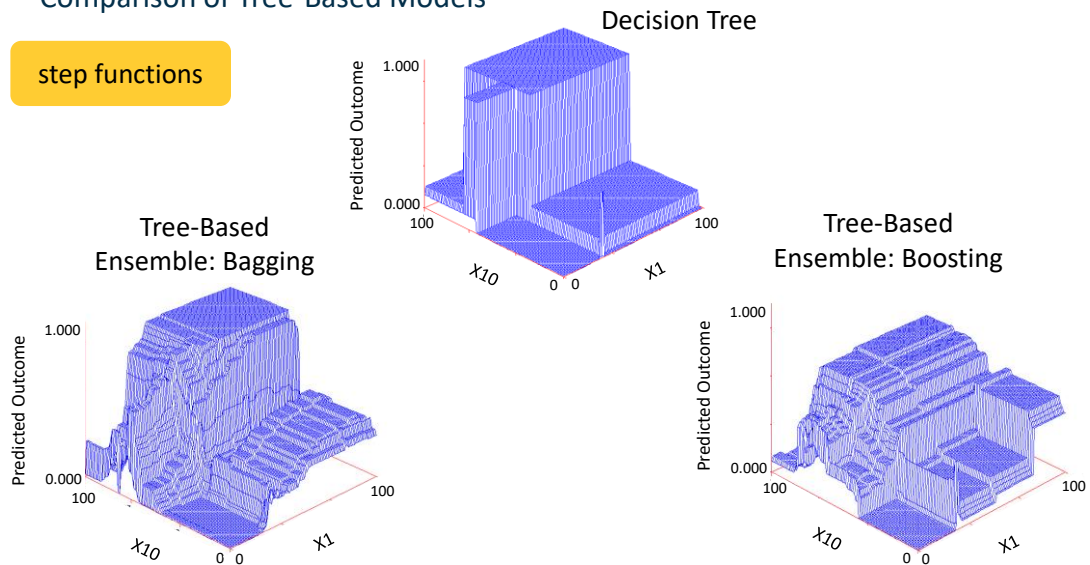
Comparison of Tree-Based Models



Copyright © SAS Institute Inc. All rights reserved.

Comparison of Tree-Based Models

step functions



Let's compare the accuracy of three tree-based models: a single decision tree, a tree-based ensemble model based on bagging, and a tree-based ensemble model based on boosting. The three plots shown here are based on a data set with two inputs.

A decision tree model is created through recursive partitioning, so all tree-based models use step functions to split the input space.

The plot for the single decision tree shows that there are few steps and those steps are large. Therefore, relatively few steps are used to classify the target, which can decrease the accuracy.

The plots for the two ensemble models have more steps and those steps are smaller. Because more steps are used to classify the target, accuracy can be increased compared to a single tree.

Notice that boosting smooths the prediction surface more than bagging because boosting emphasizes the misclassified cases during the training.

Gradient Boosting Models

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Building a Gradient Boosting Model

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Autotuning Options for Gradient Boosting in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



(Optional) Practice: Build a Gradient Boosting Model Using Autotuning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Tree-Based Models



Copyright © SAS Institute Inc. All rights reserved.

Question: Tree-Based Models

Which of the following statements is true regarding tree-based models?

- a. Small changes in the training data can cause large changes in the structure of a tree.
- b. Ensemble models are used only with decision trees.
- c. In the boosting algorithm, cases that are correctly classified are given more weight in subsequent models.
- d. In the bagging algorithm, the training data are resampled without replacement.

Answer: Tree-Based Models

Which of the following statements is true regarding tree-based models?

- ☒ a. Small changes in the training data can cause large changes in the structure of a tree.
- b. Ensemble models are used only with decision trees.
- c. In the boosting algorithm, cases that are correctly classified are given more weight in subsequent models.
- d. In the bagging algorithm, the training data are resampled without replacement.

CA: a

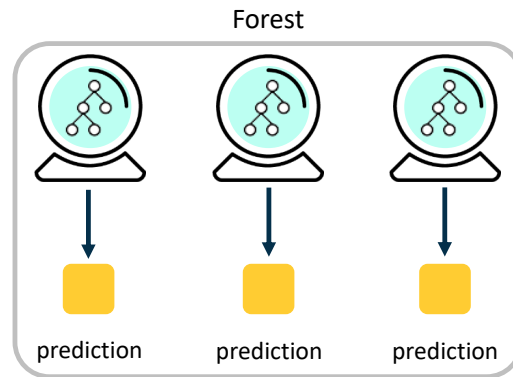
FB: Even small changes in the training data can cause large changes in the tree structure. However, despite these changes, the overall performance of trees usually remains stable.

Forest Models

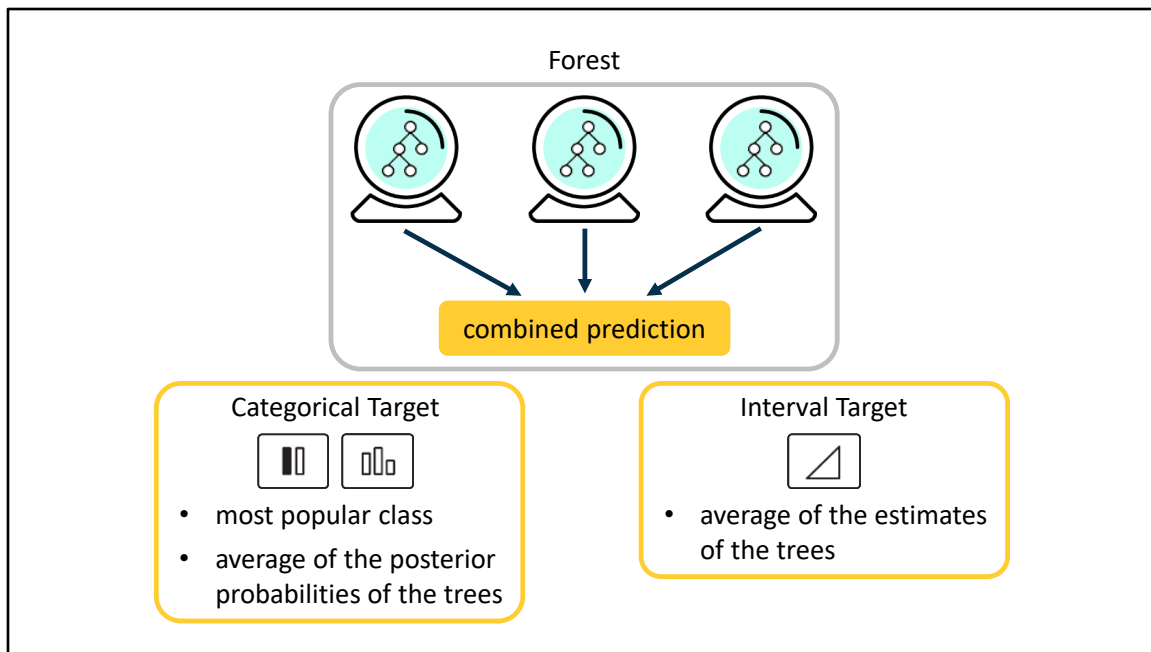


Copyright © SAS Institute Inc. All rights reserved.

Forest Models



A forest is an ensemble of decision trees, each one able to predict its own response to a set of input variables.



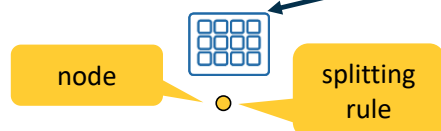
The results from the individual trees are combined to provide the final prediction. For a categorical target, the forest model's prediction is either the most popular class (as determined by a vote) or the average of the posterior probabilities of the individual trees. For an interval target, the forest model's prediction is the average of the estimates from the individual decision trees.

1. Select a sample of cases, with replacement.

2. Select a sample of input variables from all available inputs.

Training Data

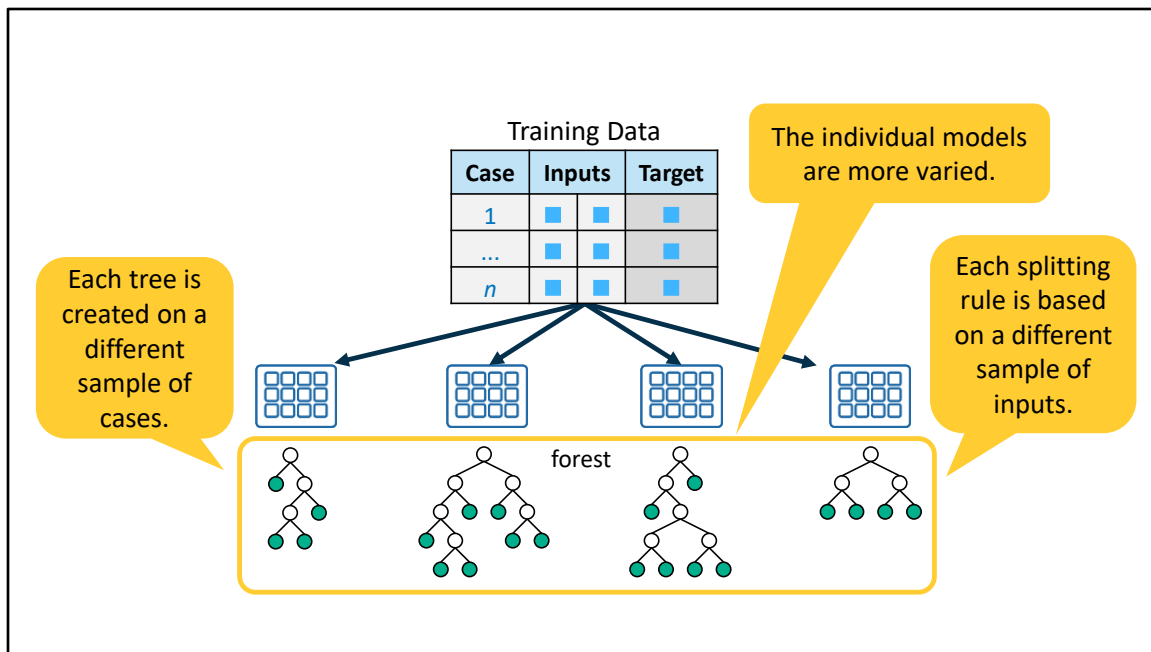
Case	Inputs		Target
1	■	■	■
...	■	■	■
n	■	■	■



The input that has the ***strongest association with the target*** is used in the splitting rule for that node.

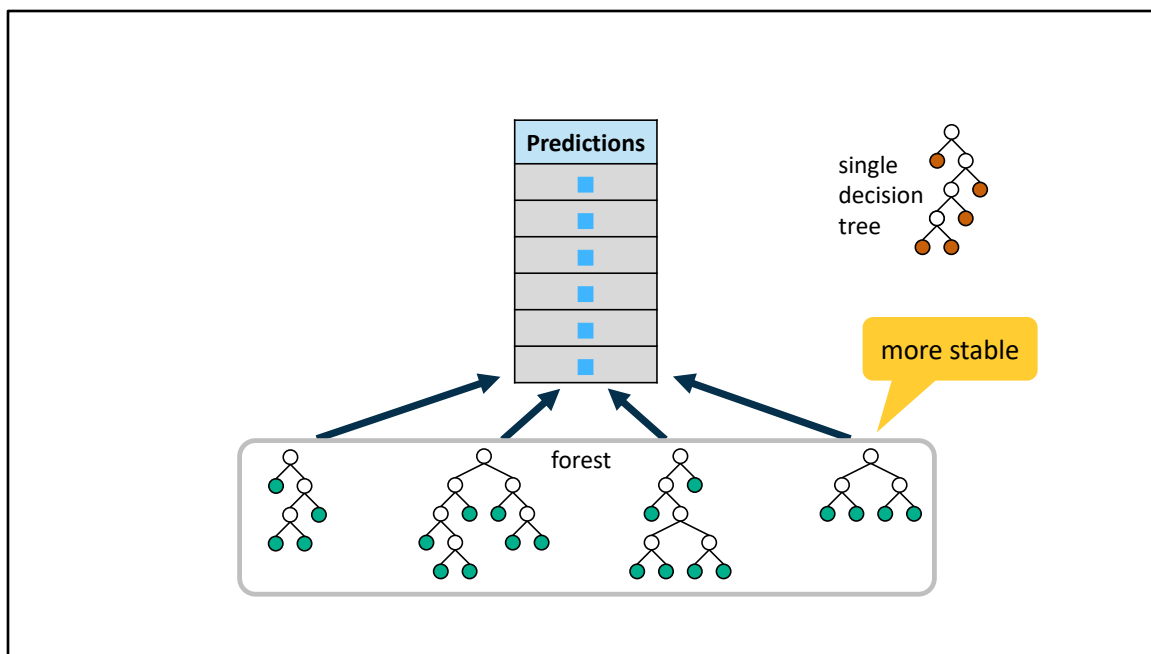
In Model Studio, the forest algorithm uses the following process to build each tree: The algorithm selects a sample of cases, with replacement, from the original training data. Then, for each node, the algorithm selects a sample of input variables from all available inputs.

From this sample, the input that has the strongest association with the target is used in the splitting rule for that node. Therefore, the method of selecting the input variable for a splitting rule is different for a forest than it is for the split-search process used to build an individual tree.

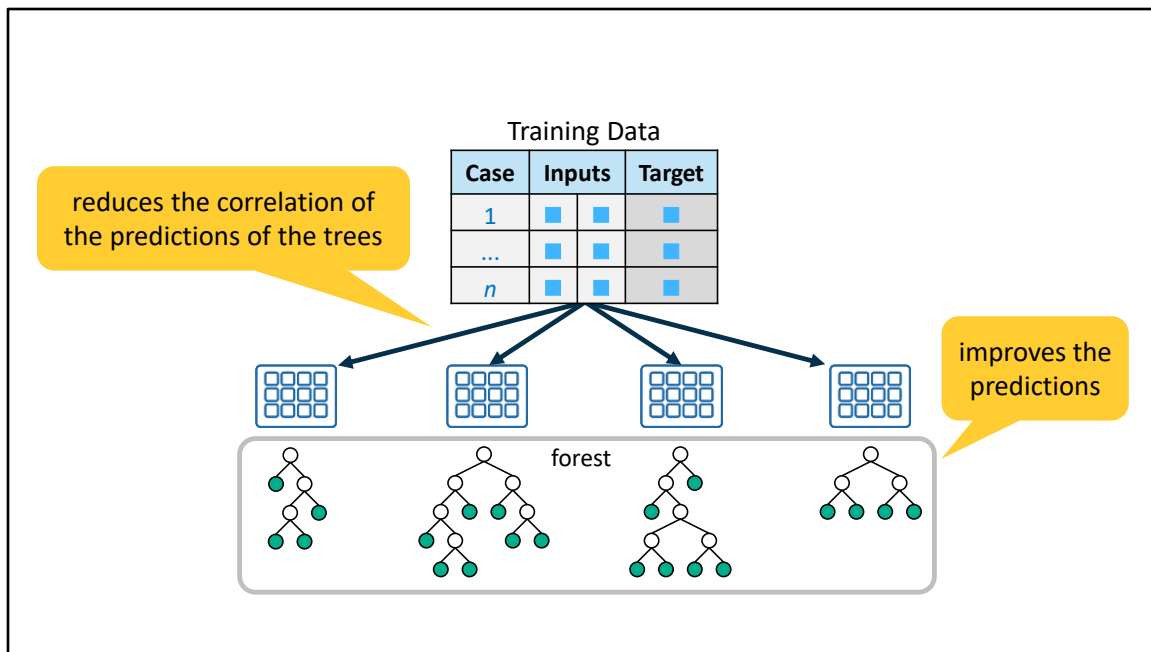


As you can see, the trees in a forest created in Model Studio differ from each other in two ways.

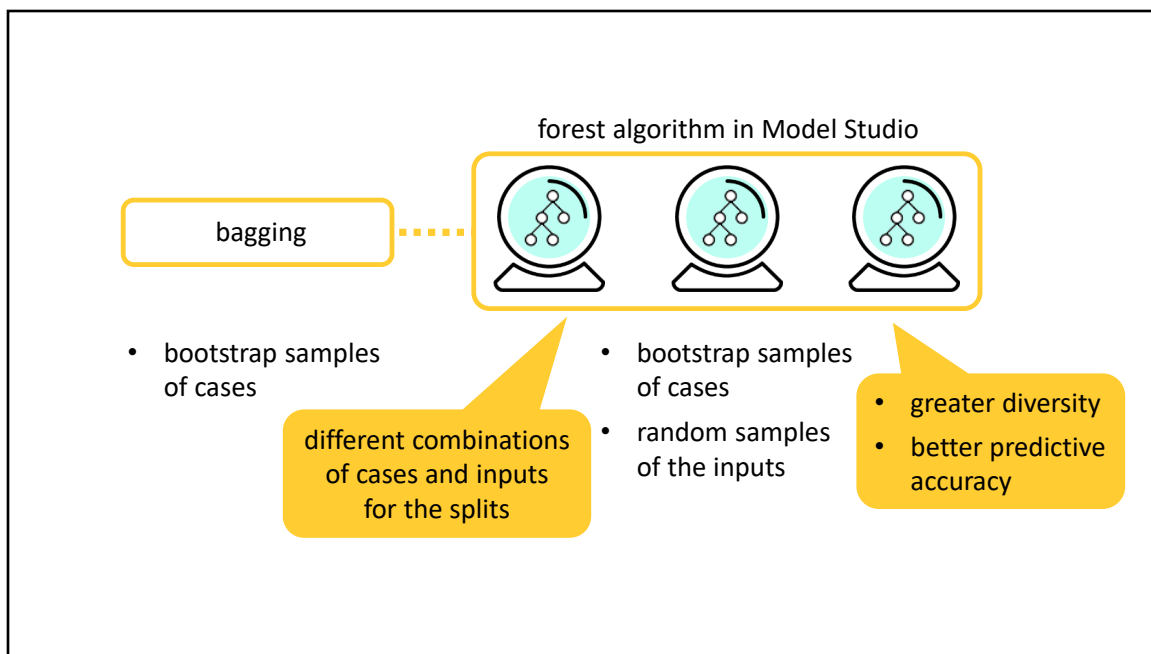
Each tree is created on a different sample of the cases, and each splitting rule is based on a different sample of the inputs. This process ensures that the individual models in the ensemble are more varied.



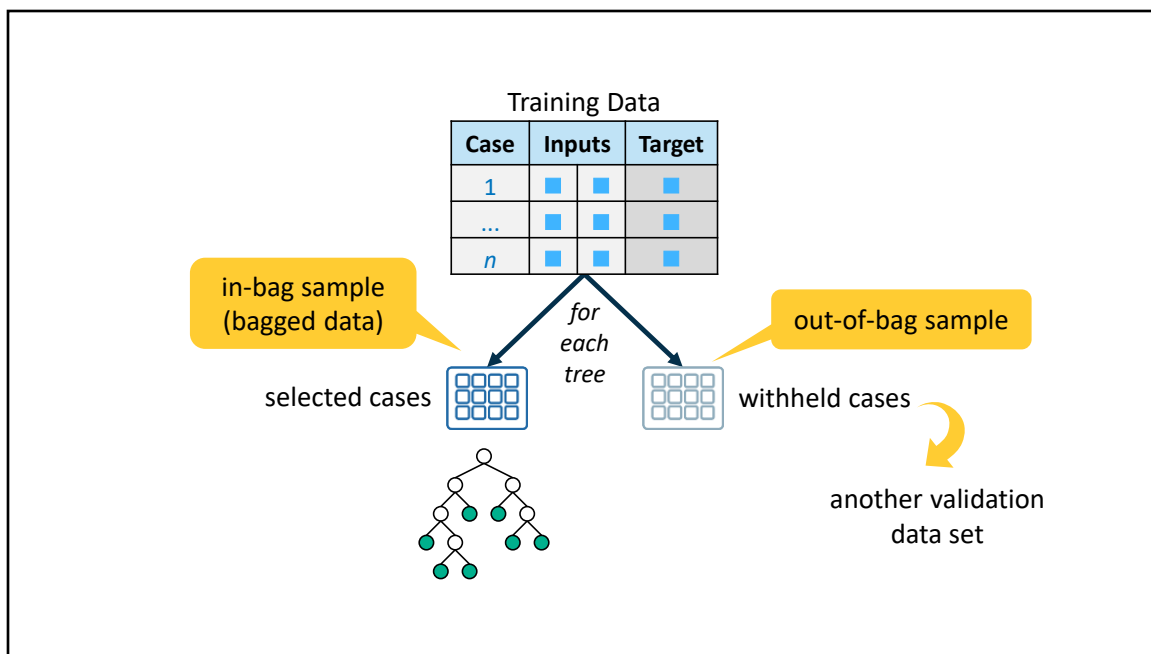
The process that the forest algorithm uses to build the individual trees and then combine the predictions results in a more stable model than a single tree.



Training each tree with different data reduces the correlation of the predictions of the trees. This, in turn, is likely to improve the predictions of the forest as compared to the naïve method of using the same data to build all the trees in a forest.



Although the forest algorithm in Model Studio is based on bagging, the sampling method is not entirely the same. Like bagging, Model Studio's forest algorithm takes bootstrap samples of the cases. However, this forest algorithm also takes random samples of the inputs. Therefore, the trees in the forest use different combinations of cases and inputs to determine the splits. This additional perturbation (beyond bagging) leads to greater diversity in the trees, and often better predictive accuracy than with bagging.



Each sample of the original training data that is selected to train a specific decision tree is called *bagged data* or *in-bag sample*. For each tree in the forest, the data that are withheld from training are called an *out-of-bag sample*. Model assessment measures (such as misclassification rates and average squared error) and iteration plots are constructed on both the entire training data set and the out-of-bag sample. So, you can think of the out-of-bag sample as another validation data set.

Demo: Building a Forest Model

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Gradient Boosting versus Forest Models



Copyright © SAS Institute Inc. All rights reserved.

Question: Gradient Boosting versus Forest Models

Which of the following is a difference between gradient boosting and forest models in Model Studio?

- a. Forests are based on bagging. Gradient boosting models are based on boosting.
- b. The gradient boosting method samples the inputs at each iteration. The forest method does not.
- c. In a forest, each tree's target is the residual from the previous tree. In a gradient boosting model, the trees are independent of each other.

Answer: Gradient Boosting versus Forest Models

Which of the following is a difference between gradient boosting and forest models in Model Studio?

- a. Forests are based on bagging. Gradient boosting models are based on boosting.
- b. The gradient boosting method samples the inputs at each iteration. The forest method does not.
- c. In a forest, each tree's target is the residual from the previous tree. In a gradient boosting model, the trees are independent of each other.

CA: a

FB: Forests and gradient boosting models are based on different P&C methods.

Autotuning Options for Forests in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



(Optional) Practice: Build a Forest Model Using Autotuning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



3.6 Lesson Conclusion



Lesson Quiz

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Lesson 4: Neural Networks



4.0 Lesson Overview



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



4.1 Building a Default Neural Network Model



Beyond Traditional Regression: Neural Networks



Copyright © SAS Institute Inc. All rights reserved.

Beyond Traditional Regression: Neural Networks

Neural Network



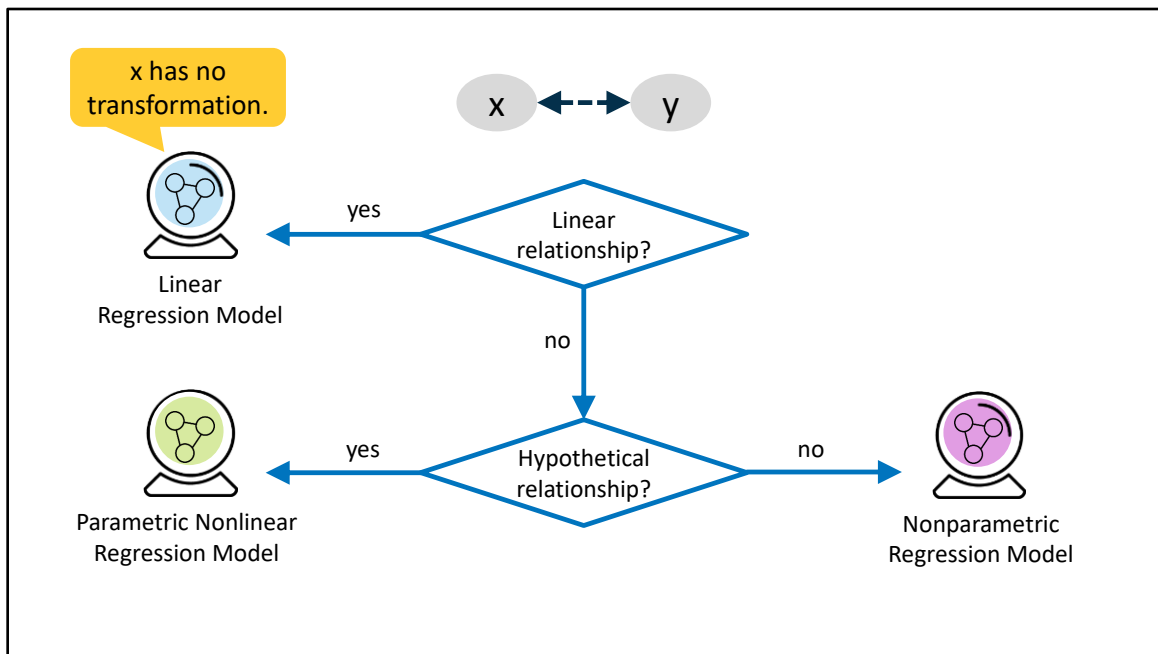
Traditional
Nonlinear Model



advantages

disadvantages

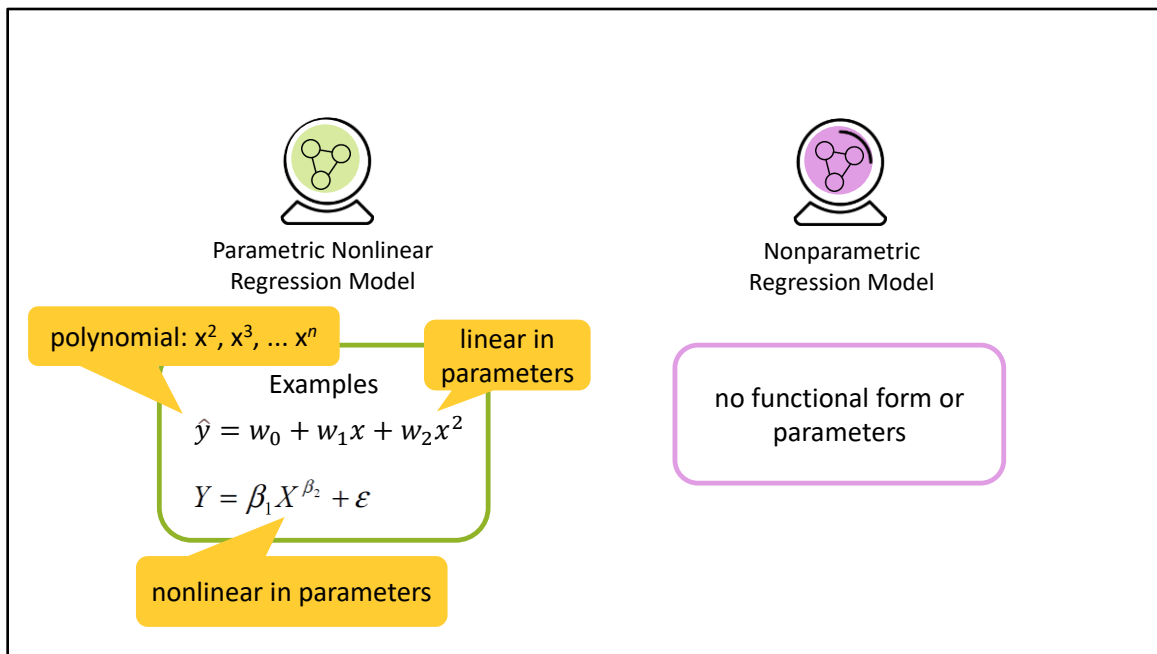
To understand the flexibility of neural networks, let's first look at the advantages and disadvantages of more traditional nonlinear models.



When an input variable x has a linear relationship with a target y , x appears in the model with no transformation.

If the relationship is nonlinear, but you can specify a hypothetical relationship between x and y , you can build a parametric nonlinear regression model.

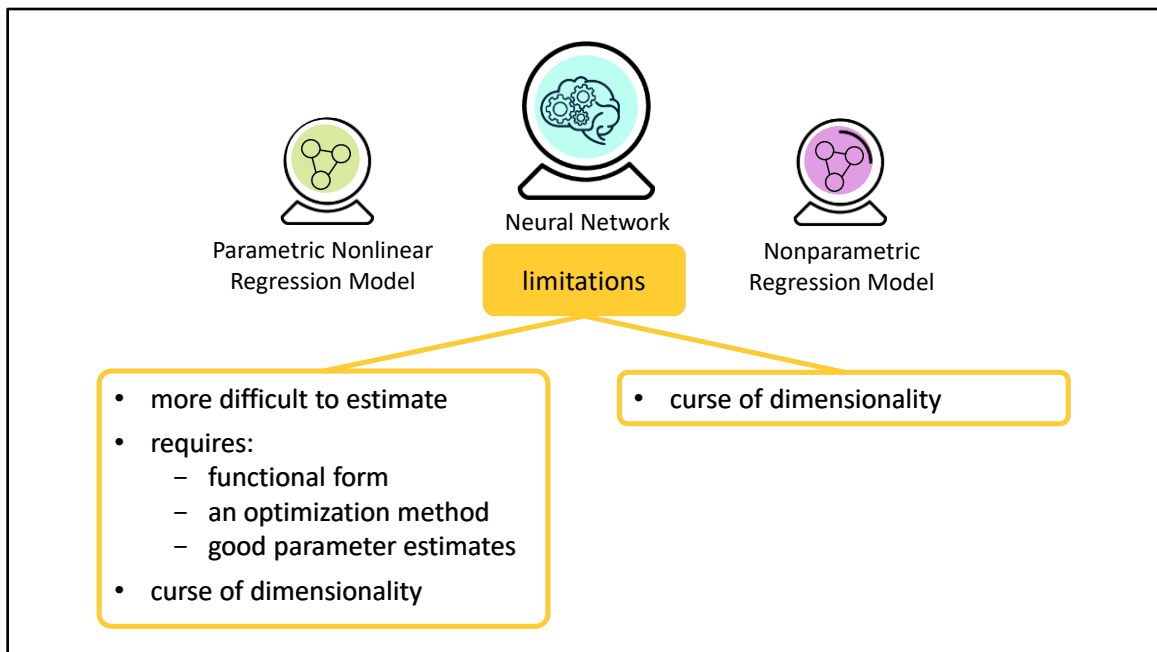
When it is not practical to specify the hypothetical relationship, you can build a nonparametric regression model.



A parametric nonlinear model requires you to specify the hypothetical relationship in a functional form. One approach is to add a transformation of x to the functional form. A popular choice is a polynomial. For a single input, x , polynomial degrees are x -squared (or quadratic), x -cubed (or cubed), and so on. For example, the model with a quadratic term that is shown here can model nonlinear relationships, but it is actually linear in parameters.

Below that is an example of a model that is nonlinear in parameters.

A nonparametric regression model has no functional form and, therefore, no parameters.

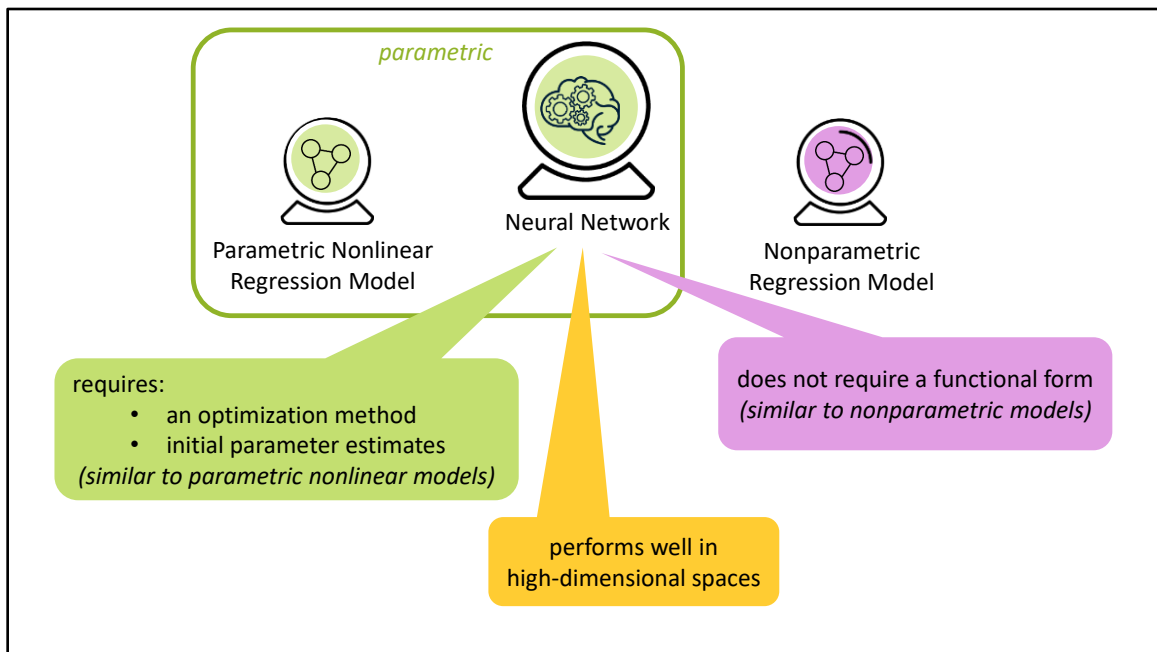


Traditional nonlinear regression models, which are parametric, and nonparametric regression models have several limitations.

Nonlinear regression models are more difficult to estimate than linear models. In addition to specifying the functional form, it is necessary to use an optimization method to efficiently guide the parameter search process. It is essential for the optimization process to start with good parameter estimates. Starting at a bad location in the parameter space results in an inferior model.

Traditional nonlinear modeling techniques become vastly more difficult as the number of inputs increases. Remember that this is called the curse of dimensionality. It is uncommon to see parametric nonlinear regression models with more than a few inputs, because deriving a suitable functional form becomes increasingly difficult as the number of inputs increases. Higher-dimension input spaces are also a challenge for nonparametric regression models.

Neural networks were developed to overcome these challenges.



Although neural networks are parametric nonlinear models, they are similar to nonparametric models in one way: neural networks do not require the functional form to be specified. This enables you to construct models when the relationship between the inputs and outputs is unknown.

Like other parametric nonlinear models, however, neural networks do require the use of an optimization process with initial parameter estimates.

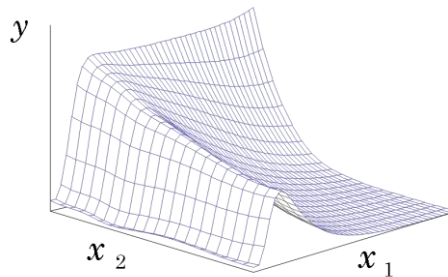
Also, unlike nonlinear parametric regression models and nonparametric regression models, neural networks generally perform well in sparse, high-dimensional spaces.



Neural Network

flexibility

Universal Approximator



A major benefit of neural networks is their unlimited flexibility.

A neural network is a universal approximator, which means that it can model any input-output relationship, no matter how complex. For example, this graph depicts the relationship between two inputs, x_1 and x_2 , and a target y . This complex relationship would be difficult to estimate accurately using traditional regression methods. However, a neural network can do the job.

Overcoming the Limitations of Neural Networks



Copyright © SAS Institute Inc. All rights reserved.

Overcoming the Limitations of Neural Networks

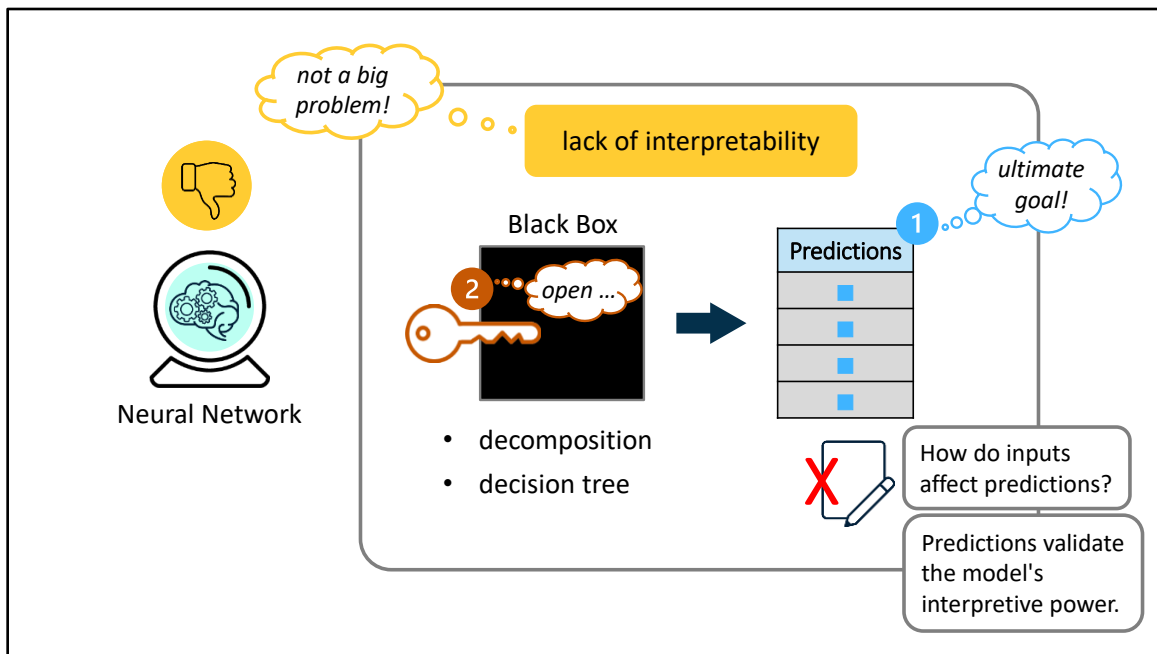


Neural Network

lack of interpretability

need for a strong signal

Neural networks overcome the main limitations of traditional regression methods, but they have a few limitations of their own: lack of interpretability and the need for a strong signal in the data.



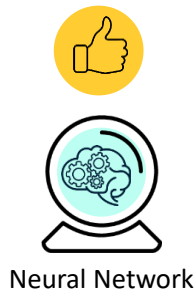
The lack of interpretability is the basis for the well-known *black box* objection to neural networks. Suggesting mystery, the term black box is often used simply to disparage neural networks by implying that they can never be interpreted.

However, the lack of interpretability is not as big a problem as it might appear. First, for many modeling tasks, pure prediction is the goal. Understanding how the inputs affect the prediction is of secondary importance. In other applications, the opposite is true: predictive power is a consideration only to the extent that it validates the interpretive power of the model. Neural networks are most appropriate for pure prediction tasks.

Second, there are methods that enable you to open the black box, at least partially (that is, to gain insight into how the neural network makes predictions).

One such method, called decomposition, approximates the parameters in a neural network using a set of IF-THEN rules.

A related approach is to use a decision tree to interpret the neural network's predictions.



lack of interpretability

Black Box

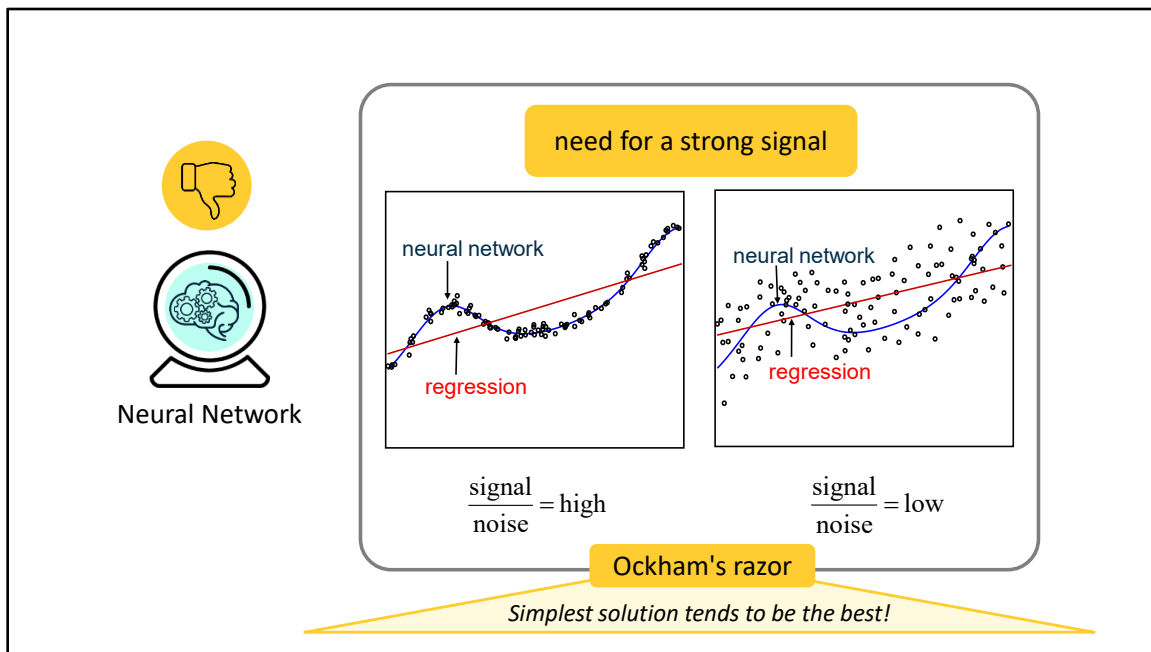


Model Interpretability

- Variable importance
- PD plots
- ICE plots
- LIME
- Hyper Shapley values

model neutral

To help you better understand your model, SAS Viya provides model interpretability techniques such as Variable Importance, PD plots, ICE plots, LIME, and Hyper Shapley values. These techniques are model agnostic, which means that they are applicable to any model that is generated by a supervised learning node. You can learn more about model interpretability later in the course.



Depending on the data, neural networks might not outperform simpler and easier to implement models like regression. In particular, neural networks require a high signal-to-noise ratio in the data. In the graph on the left, the neural network model fits the data well, as compared to the linear regression model. The data have a strong pattern (or signal) relative to the amount of variation (or noise) around the pattern. The signal-to-noise ratio is high.

The graph on the right tells a different story: the signal-to-noise ratio is low. The regression and neural network models offer a comparable fit to the data.

In a situation like this, Ockham's razor implies that there is no advantage to using the more complex neural network. What is Ockham's razor? It is a principle that has been around for centuries, and it states that the simplest solution tends to be the best.

No model is always best.



Neural Network



one of several tools in your analytics toolbox

In fact, no single type of model is always best, so think of neural networks as one of several tools in your analytics toolbox.

Question: Universal Approximators



Copyright © SAS Institute Inc. All rights reserved.

Question: Universal Approximators

Neural networks are universal approximators. This means that neural networks can do which of the following?

- a. model any input-output relationship, no matter how complex
- b. outperform any other type of model, no matter how strong the signal
- c. clearly indicate how the inputs affect the prediction, no matter the number of inputs

Answer: Universal Approximators

Neural networks are universal approximators. This means that neural networks can do which of the following?

- a. model any input-output relationship, no matter how complex
- b. outperform any other type of model, no matter how strong the signal
- c. clearly indicate how the inputs affect the prediction, no matter the number of inputs

CA: a

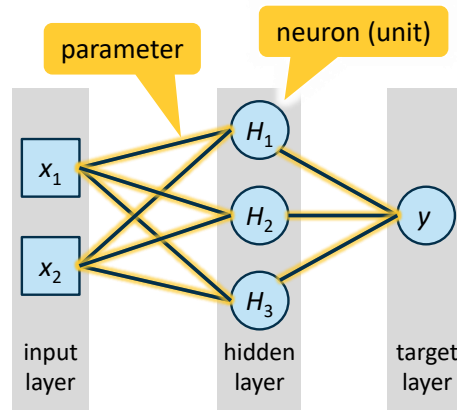
FB: Neural networks are called universal approximators because they can model any input-output relationship, no matter how complex.

Basics of Neural Networks

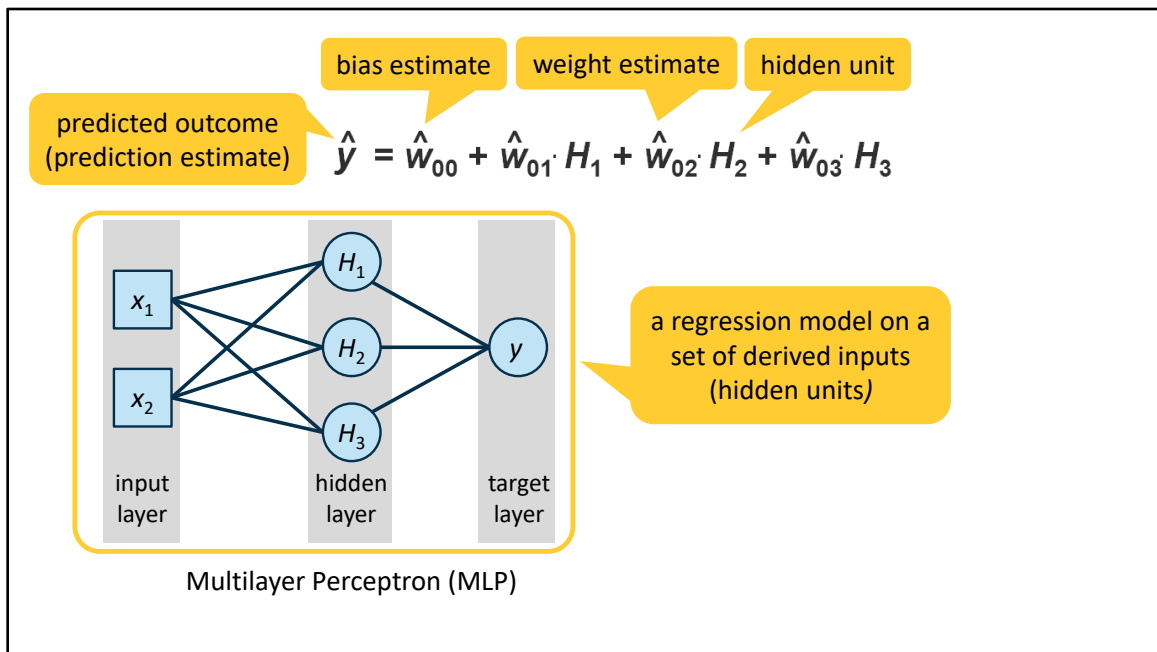


Copyright © SAS Institute Inc. All rights reserved.

Basics of Neural Networks



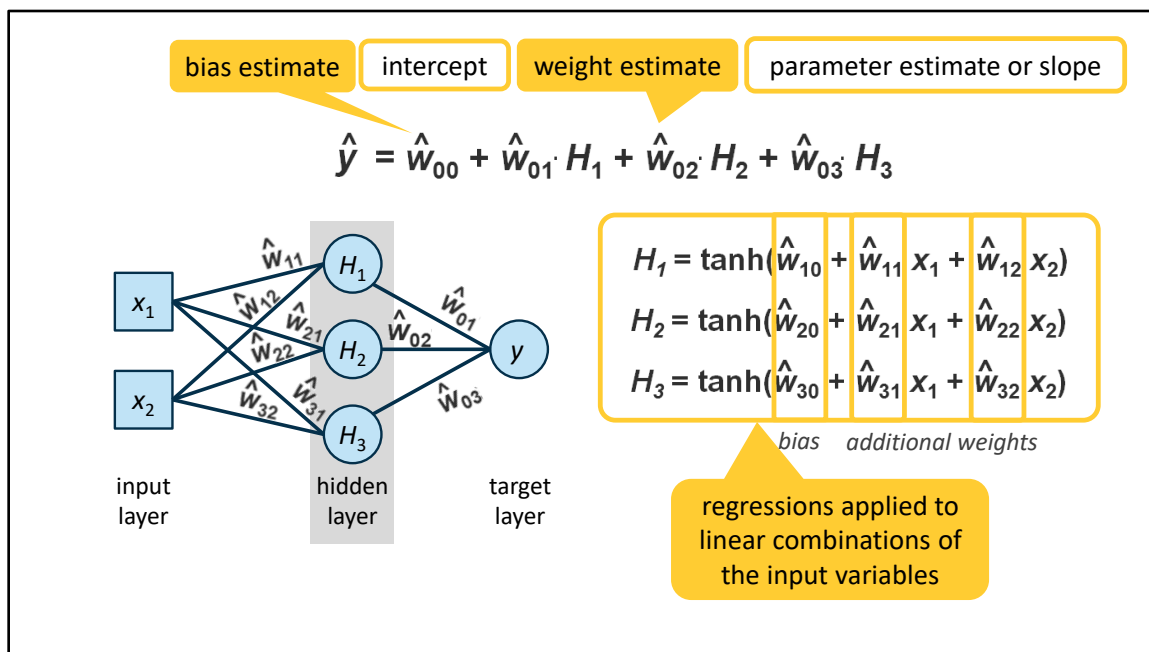
A neural network is structured in layers that contain neurons (or units). Lines represent the parameters that connect the neurons between layers.



This diagram shows a basic multilayer perceptron (or MLP), which is a common type of neural network. A basic MLP has three layers: an input layer, a hidden layer, and a target (or output) layer. In this particular MLP, the input layer has two inputs (x_1 and x_2) and the target layer has one target (y). The hidden layer has three hidden units (H_1 through H_3). So, what are hidden units?

A neural network can be thought of as a regression model on a set of derived inputs, called *hidden units*. The functional form (or the prediction formula) for this MLP is shown at the top.

In this example, the predicted outcome equals the sum of a bias estimate and three terms that are products of a weight estimate and a hidden unit.



Bias estimate is the neural network term for an intercept.

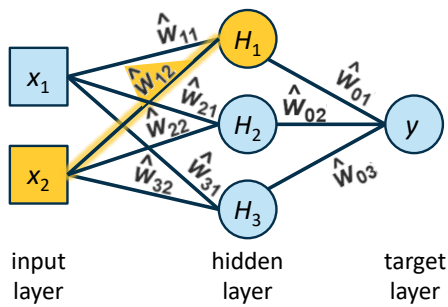
Weight estimate is the neural network term for a parameter estimate or slope.

When there is a single hidden layer, the hidden units can be thought of as regressions applied to linear combinations of the input variables. In this example, an equation for each of the three hidden units is shown on the right.

Each linear combination of the input variables contains additional weights and a bias term.

Remember that the lines in the neural network diagram represent the weights. The weights for the linear combinations of the input variables and the weights from the hidden units are now labeled in the diagram.

$$\hat{y} = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$



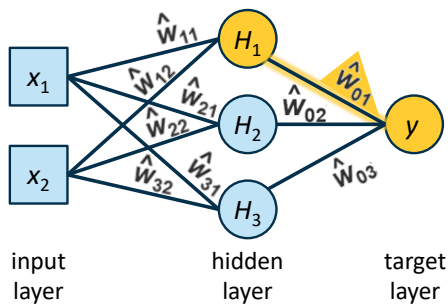
$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

For example, the line connecting hidden unit H_1 to input x_2 represents the weight $\hat{w}_{12}$.

$$\hat{y} = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$



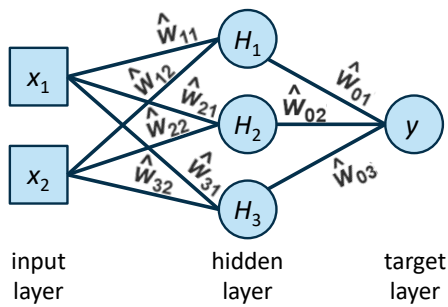
$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

And the line connecting the target to the hidden unit H_1 represents the weight w -hat sub zero one.

$$\hat{y} = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$



$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

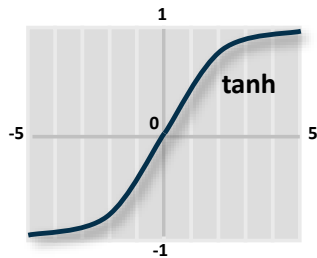
$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

bias terms

In each equation, the first w -hat term is the bias term, which is not represented in the diagram. The second subscript on a bias term is always zero.

$$\hat{y} = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$



$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

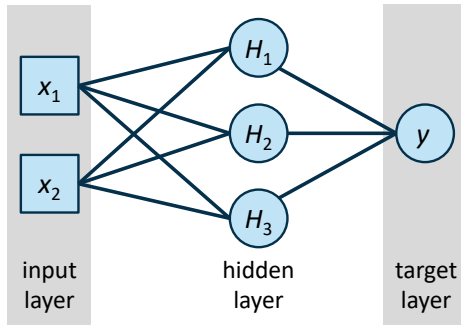
$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

activation function

The hidden unit regressions include an activation function. This activation function is a mathematical transformation that is applied to the input layer. In the equations shown here, the activation function is the hyperbolic tangent, which is the default in Model Studio. The hyperbolic tangent is a shift and rescaling of the logistic function.

can approximate virtually any association

$$\hat{y} = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$



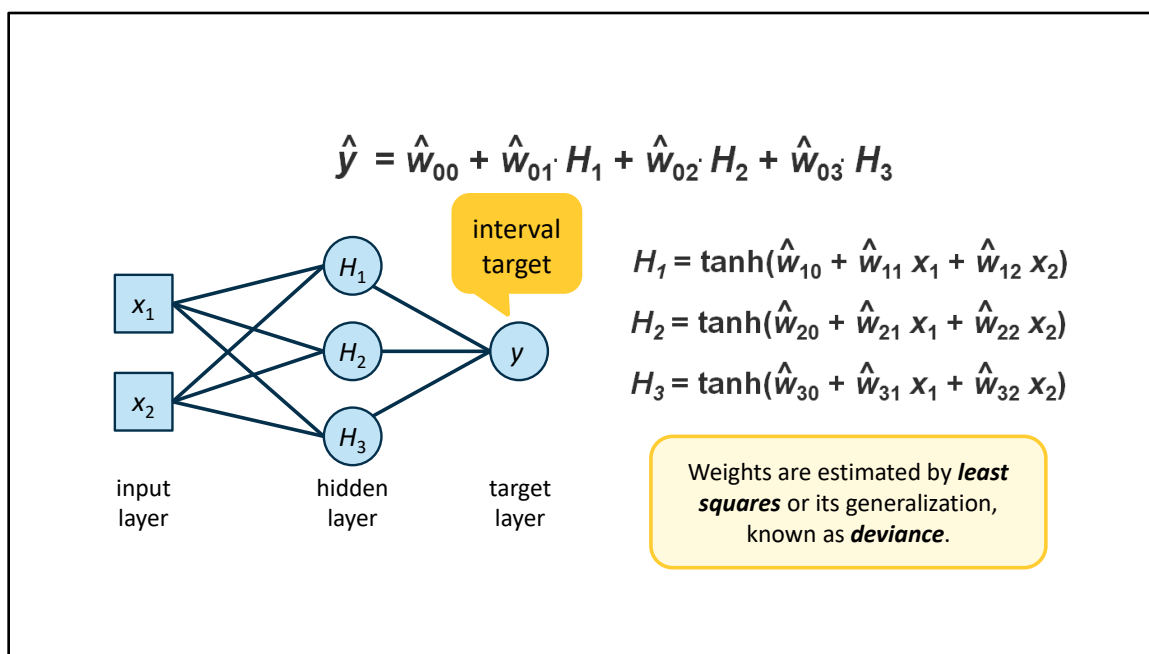
$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

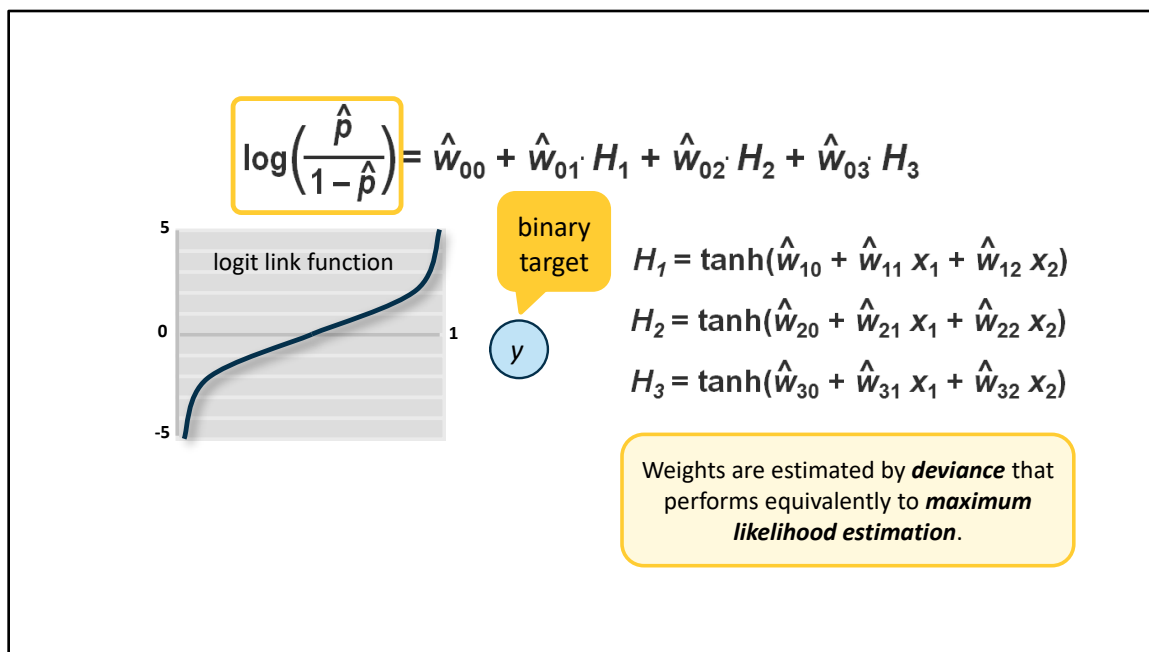
$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

specify the correct
number of hidden units

What makes neural networks interesting is their ability to approximate virtually any association between the inputs and the target. You simply specify the correct number of hidden units and find reasonable values for the weights. Specifying the correct number of hidden units involves some trial and error.



For interval targets, least squares estimation is usually used to estimate the weights. For certain distributions of interval targets, a generalization of least squares estimation, known as deviance estimation, is used instead.



When the target variable is binary, the main neural network regression equation uses the same logit link function as a logistic regression does. For binary targets, the weight estimation process uses deviance estimation. When the distribution is in the exponential family (as is the case with a binary target), deviance estimation performs equivalently to maximum likelihood estimation.

Question: Parameter Estimates



Copyright © SAS Institute Inc. All rights reserved.

Question: Parameter Estimates

Which of the following terms refers to a parameter estimate or slope that is associated with an input in a neural network?

- a. activation function
- b. bias estimate
- c. weight estimate
- d. hidden unit

Answer: Parameter Estimates

Which of the following terms refers to a parameter estimate or slope that is associated with an input in a neural network?

- a. activation function
- b. bias estimate
- ☒ c. weight estimate
- d. hidden unit

CA: c

FB: **Weight estimate** is the neural network term for a parameter estimate or slope.

Question: Intercept Estimate



Copyright © SAS Institute Inc. All rights reserved.

Question: Intercept Estimate

Which of the following terms refers to the intercept estimate in a neural network?

- a. activation function
- b. bias estimate
- c. weight estimate
- d. hidden unit

Answer: Intercept Estimate

Which of the following terms refers to the intercept estimate in a neural network?

- a. activation function
- ☒ b. bias estimate
- c. weight estimate
- d. hidden unit

CA: b

feedback: **Bias estimate** is the neural network term for an intercept.

Estimating Weights and Making Predictions



Copyright © SAS Institute Inc. All rights reserved.

Estimating Weights and Making Predictions

produces a ranking or logit score

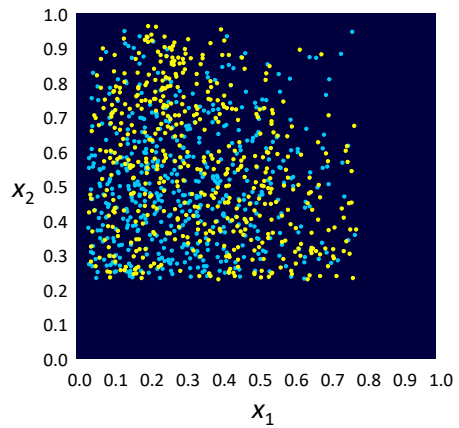
$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

binary
target

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$



To show how a neural network estimates weights and makes predictions, let's take another look at the two-color prediction problem. In this example, remember that the goal is to predict the target color based on the location in the plot. As with regressions, a neural network's predictions can be decisions, rankings, or estimates. In this example, the target is binary, so the estimated prediction formula produces a ranking or logit score.

1. Standardize (scale) the input variables.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

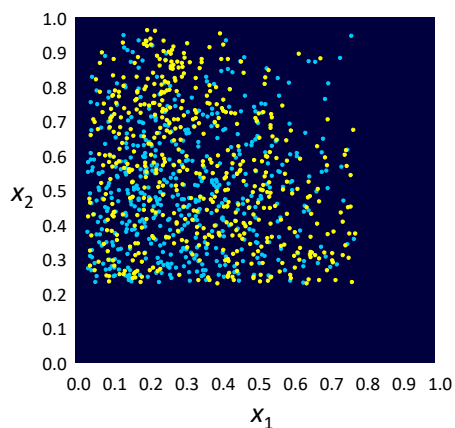
$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

Midrange

- midrange: 0
- minimum: -1
- maximum: 1

default in
Model Studio



In Model Studio, the Neural Network node starts by standardizing all the input variables. This is also known as scaling the inputs. The default standardization method, which is midrange, scales the variables so that they have a midrange of zero, a minimum of -1, and a maximum of 1. The midrange is the mean of the smallest value and the largest value.

Other available methods are Z-score standardization and no standardization.

2. Find the weight estimates.

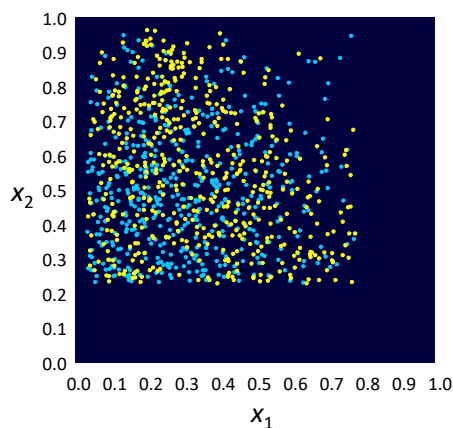
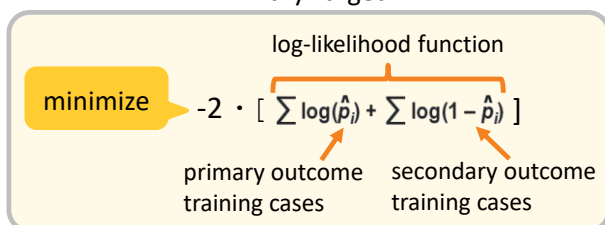
$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

Binary Target



The primary task is to find the weight estimates that result in accurate predictions. Compared to a regression model, a neural network has a higher number of values to be estimated and the relationship among the weights is more complex. For a binary target, the weight estimation process is driven by an attempt to minimize the negative of two times the log-likelihood function. The log-likelihood function is the sum of the log of the primary outcome training cases plus the sum of the log of the secondary outcome training cases.

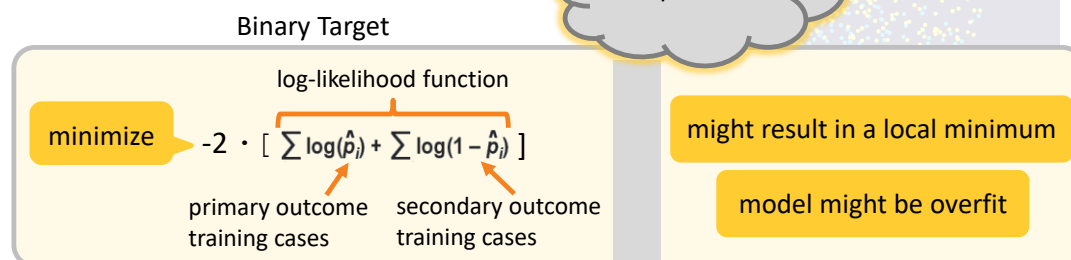
2. Find the weight estimates.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$



Unfortunately, with a neural network model, this minimization process has two complications.

First, the minimization process might result in a local minimum, which is a point that is smaller than the surrounding points, instead of the global minimum (which is the smallest point over all).

Second, neural networks tend to create overfit models. A technique called early stopping usually overcomes these difficulties and produces a reasonable model. Early stopping can also be called stopped training. You learn about early stopping later in this lesson.

2. Find the weight estimates.

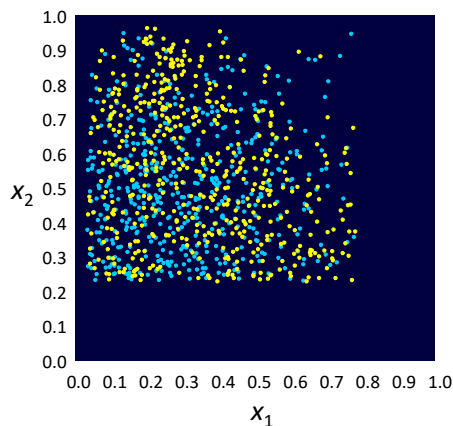
$$\text{logit}(\hat{p}) = -0.5 + -2.6 H_1 + -1.9 H_2 + -0.63 H_3$$

$$H_1 = \tanh(-1.8 + 0.25 x_1 + -1.8 x_2)$$

$$H_2 = \tanh(2.7 + 2.7 x_1 + -5.3 x_2)$$

$$H_3 = \tanh(-5.0 + 8.1 x_1 + 4.3 x_2)$$

target-inputs associations can be complex



Is the relationship between X_2 and the target positive or negative?

The weight estimates have now been inserted into the prediction formula. Keep in mind that, even in a simple neural network model, the associations between the inputs and the target can be complex, and the model can be difficult to interpret.

In this example, can we determine whether the relationship between X_2 and the target is positive or negative? Clearly, we cannot. Notice that two of the weights estimated for X_2 are negative but one is positive.

3. Obtain a prediction.

$$\text{logit}(\hat{p}) = -0.5 + -2.6 H_1 + -1.9 H_2 + -0.63 H_3$$

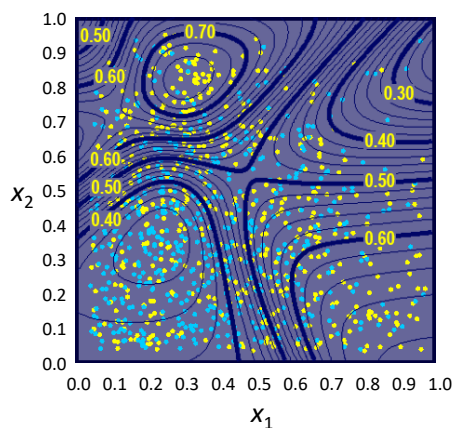
$$H_1 = \tanh(-1.8 + 0.25 x_1 + -1.8 x_2)$$

$$H_2 = \tanh(2.7 + 2.7 x_1 + -5.3 x_2)$$

$$H_3 = \tanh(-5.0 + 8.1 x_1 + 4.3 x_2)$$

Logistic Function

$$\hat{p} = \frac{1}{1 + e^{-\text{logit}(\hat{p})}}$$



After the prediction formula is established, obtaining a prediction is strictly a matter of inserting the input measurements into the hidden units. For example, to obtain a probability estimate, given values of x_1 and x_2 , the estimated prediction formula provides a logit, which is then inserted into the logistic function.

Standardization Methods

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Learning Process



Copyright © SAS Institute Inc. All rights reserved.

Learning Process

numerical
optimization

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

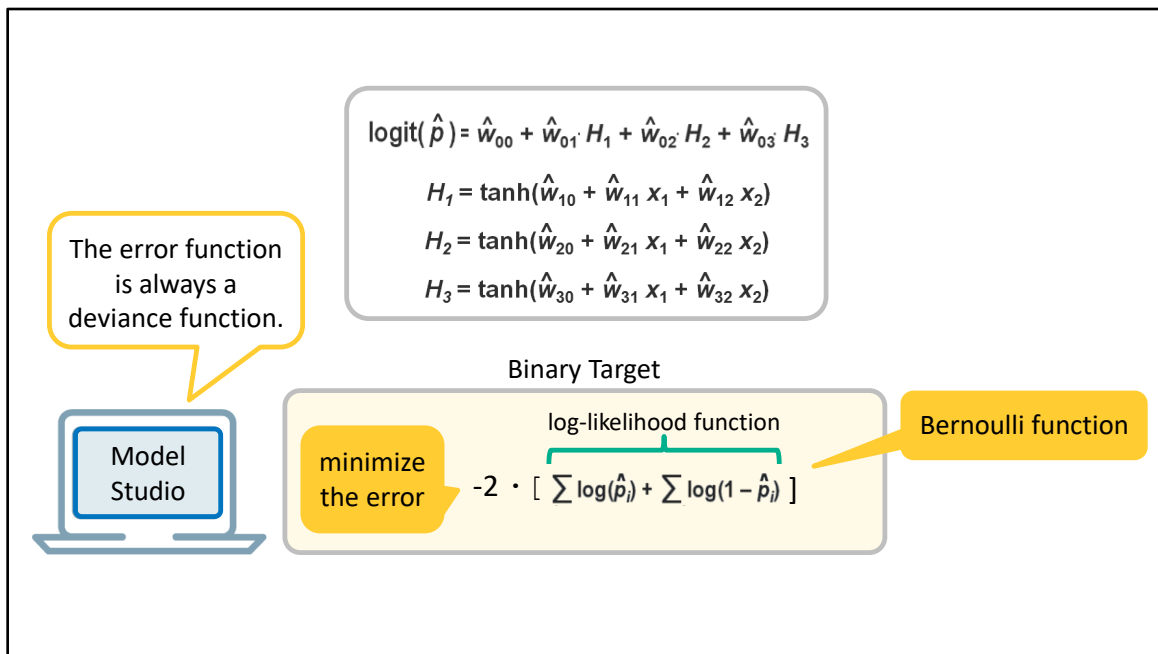
$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$



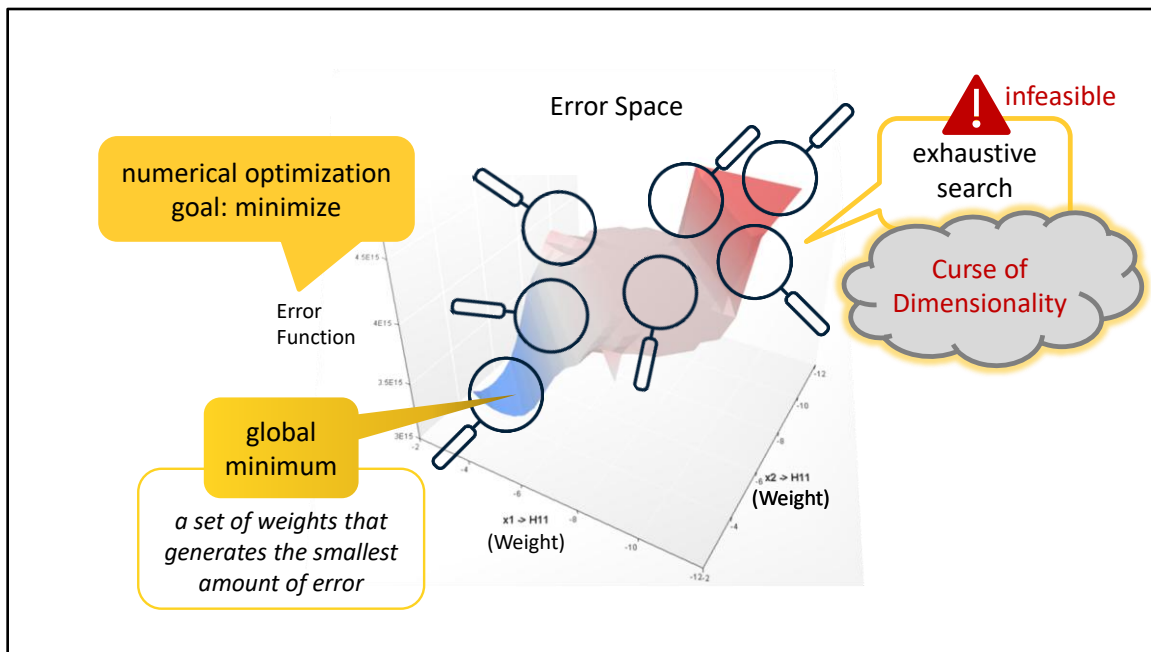
learning

using an optimization
method to estimate the
weights that minimize
the error function

A neural network uses a numerical optimization method that you specify to estimate the weights that minimize the error function. This process is called learning.



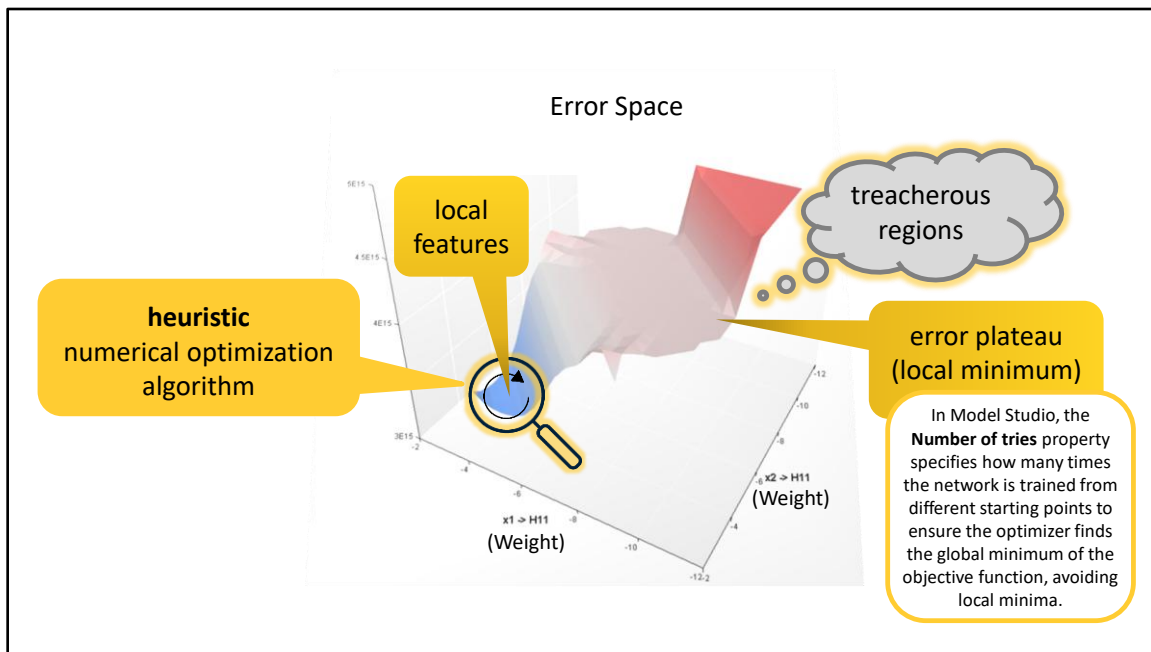
In this example, shown earlier, weight estimates for a binary target are produced by minimizing the error, which is -2 times the log-likelihood. This particular error function is known as the Bernoulli function. In Model Studio, the error function is always a deviance function.



A three-dimensional plot of an error space is shown here. The error function and the weights in the model define the surface of the error space. The error space can also be called an error surface.

Remember that the goal of numerical optimization is to minimize the error function. This is the same as finding the lowest point in the error space, which is called a global minimum. Formally, a *global minimum* is a set of weights that generates the smallest amount of error.

A simple strategy to ensure that a global minimum has been found is an exhaustive search of the error space. Unfortunately, the curse of dimensionality quickly makes this method infeasible.



It is more efficient to use a heuristic numerical optimization algorithm to search for the optimal weight estimates. A heuristic is a "shortcut" that is known to work well in most situations. Numerical optimization algorithms use **local** features of the error surface to decide how to proceed.

However, using local features makes these algorithms vulnerable to getting stuck in treacherous regions of the error surface that result in little improvement. These regions can be referred to as *error plateaus* or local minima.

The learning process stops when an error plateau is reached, as any further movements yield very little improvement in the error.

Convergence Criteria

- The specified error function stops improving.
- The gradient has no slope.
- The magnitude of the weights stops changing substantially.



The learning process stops when the numerical optimization algorithm determines that it has learned the data well enough (that is, within a certain margin of error).

The decision to stop is based on meeting at least one of three convergence criteria:

The specified error function stops improving.

The gradient has no slope (in other words, the rate of change of the error function with respect to the weights is zero).

Or, the magnitude of the weights stops changing substantially.

Iterative Updating in Numerical Optimization

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Numerical Optimization Methods in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Deviance Measures in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Optimization Methods



Copyright © SAS Institute Inc. All rights reserved.

Question: Optimization Methods

Optimization methods are used to efficiently search the complex landscape of the error surface to find an error maximum.

- a. True
- b. False

Answer: Optimization Methods

Optimization methods are used to efficiently search the complex landscape of the error surface to find an error maximum.

- a. True
- ☒ b. False

CA: b

FB: Optimization methods are used to efficiently search the complex landscape of the error surface to find an error **minimum**.

Essential Discovery Tasks for Neural Networks



Copyright © SAS Institute Inc. All rights reserved.

Essential Discovery Tasks for Neural Networks



Essential Discovery Tasks

- ✓ Select an algorithm.
- Improve the model.
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.

The essential discovery tasks shown here for neural networks are the same as for decision trees, with one exception: building ensemble models is not listed. Ensembles are not considered for neural network models as often as they are for tree-based models.

As usual, after selecting the algorithm, we build a neural network using the default settings.

Demo: Building a Neural Network Using the Default Settings

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



4.2 Modifying the Model: Network Architecture



Improving the Neural Network Model



Copyright © SAS Institute Inc. All rights reserved.

Improving the Neural Network Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- Improve the model. architecture
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.

After building the neural network using the default settings, we move on to the next discovery task: improving the model. At this point, it is common to examine the neural network architecture. By modifying the architecture, we might be able to increase model flexibility and improve model performance.

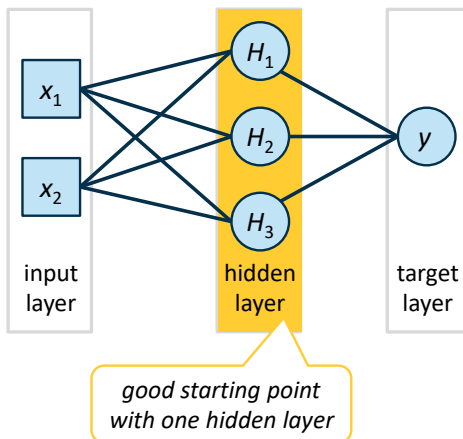
Neural Network Architectures



Copyright © SAS Institute Inc. All rights reserved.

Neural Network Architectures

Multilayer Perceptron



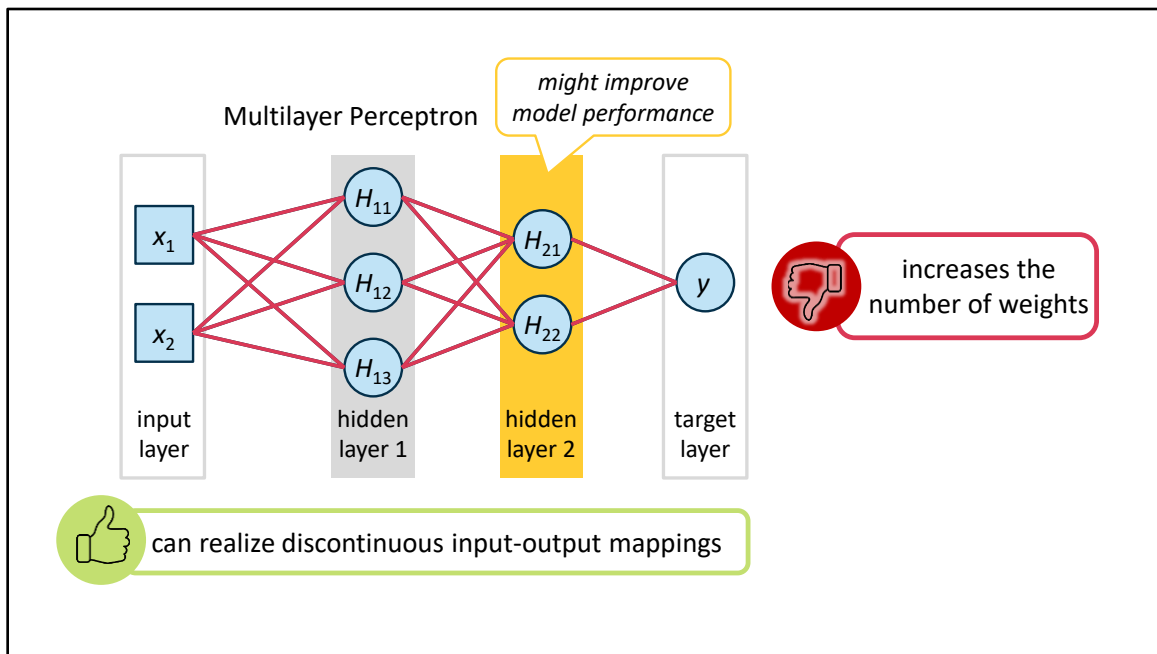
*might require a more flexible,
more complex architecture*

+ layers

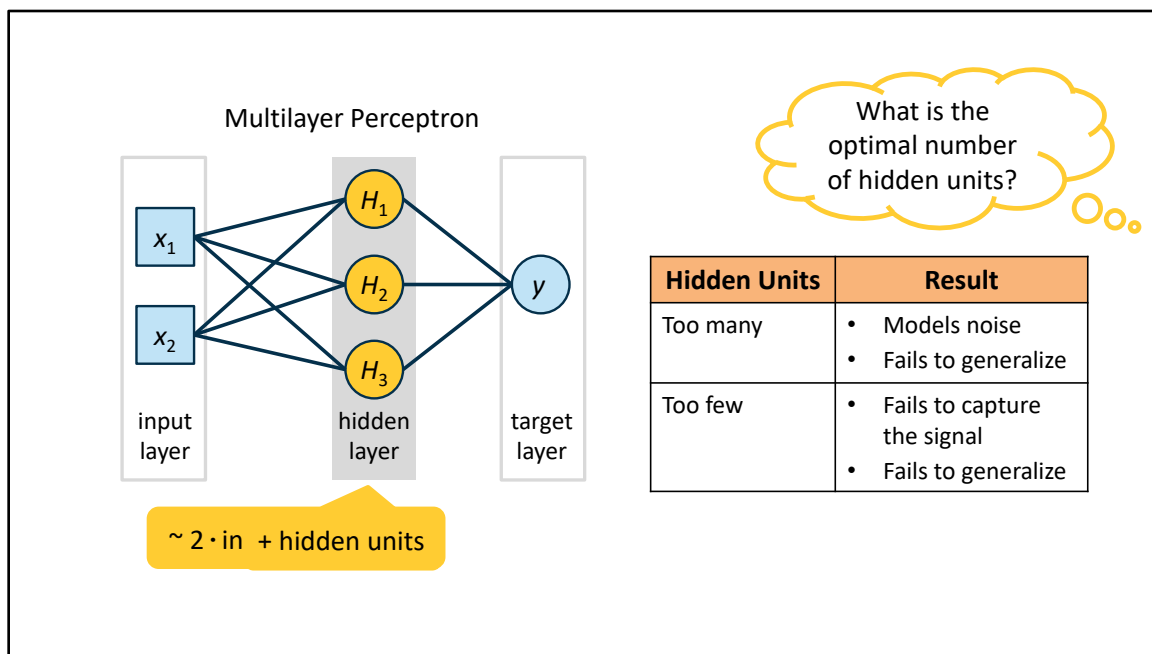
+ neurons in each layer

different connection types

You have already seen one neural network architecture: a basic MLP with one hidden layer. This architecture is a good starting point. However, some modeling situations require a more flexible, more complex neural network architecture. Neural networks can have additional layers, additional neurons in each layer, and different types of connections. In many situations, a basic MLP with one hidden layer is sufficient.

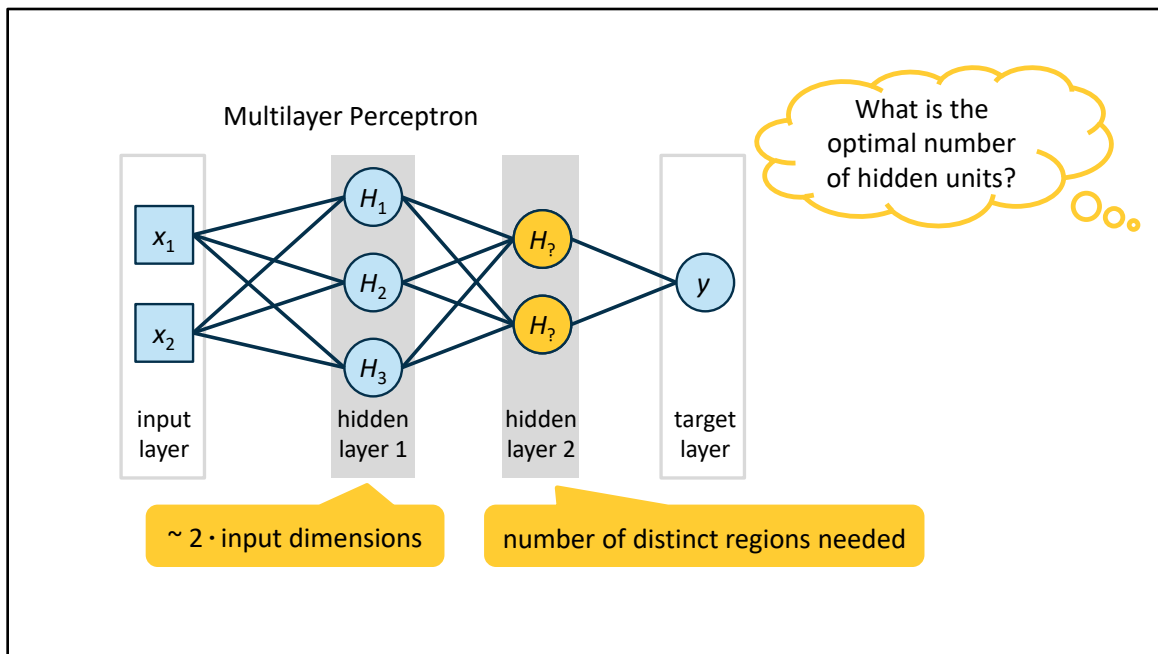


However, a second layer of hidden units can improve model performance. Specifically, adding a second hidden layer enables the MLP to realize discontinuous input-output mappings. On the other hand, adding a second hidden layer also significantly increases the number of weights.

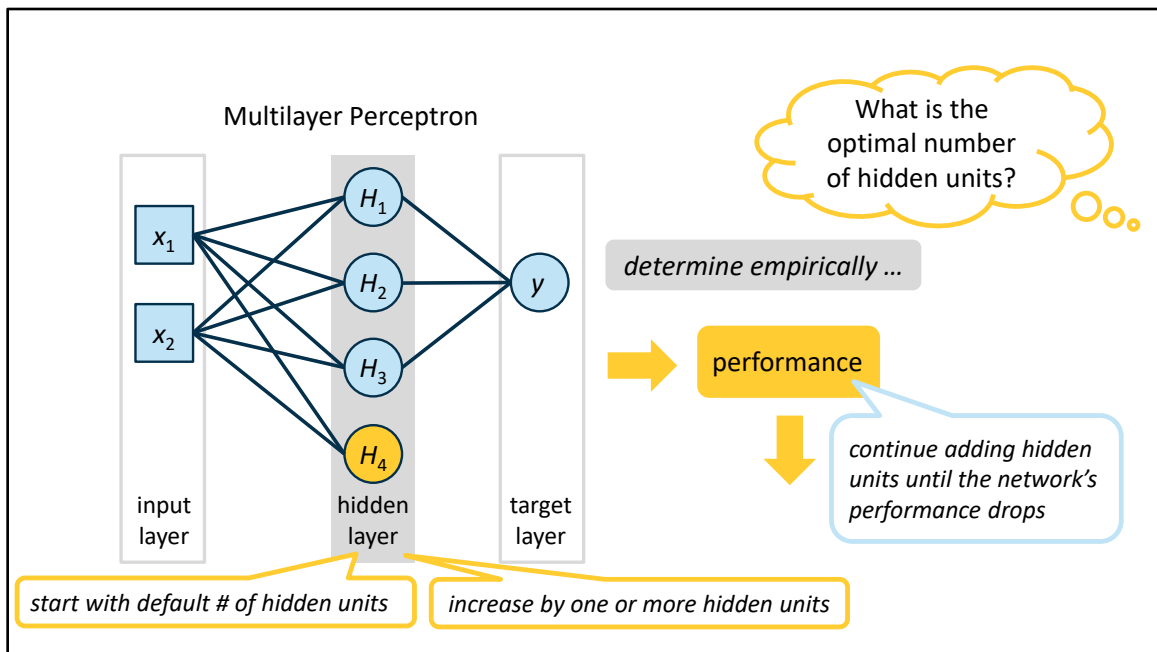


Adding hidden units can improve model performance. However, determining the optimal number of hidden units is more difficult than determining the optimal number of hidden layers. If the network has too many hidden units, it will model random variation, or noise, as well as the desired pattern, or signal. This means that the model will fail to generalize. Conversely, having too few hidden units will fail to adequately capture the underlying signal. Again, the model will fail to generalize.

The optimal number of hidden units is problem specific. Fortunately, there are guidelines to help you determine the number of hidden units. Research suggests that the number of units in the first hidden layer should be about twice the number of input dimensions.



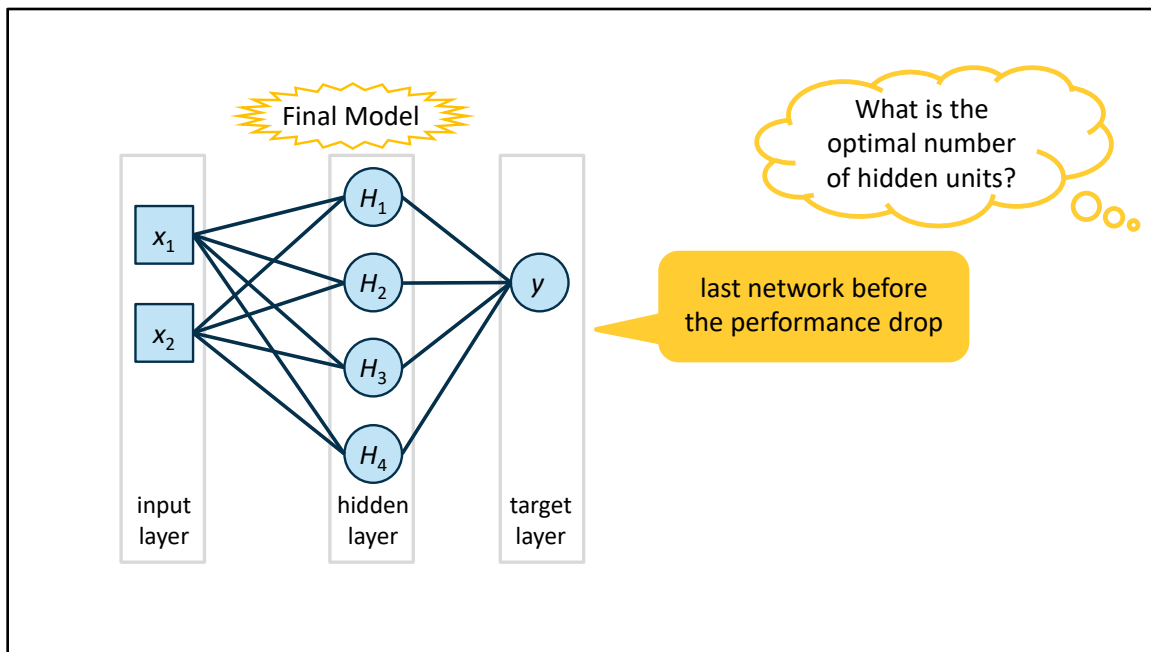
If a second hidden layer is required, then the number of hidden units in the second layer should reflect the number of distinct regions needed.



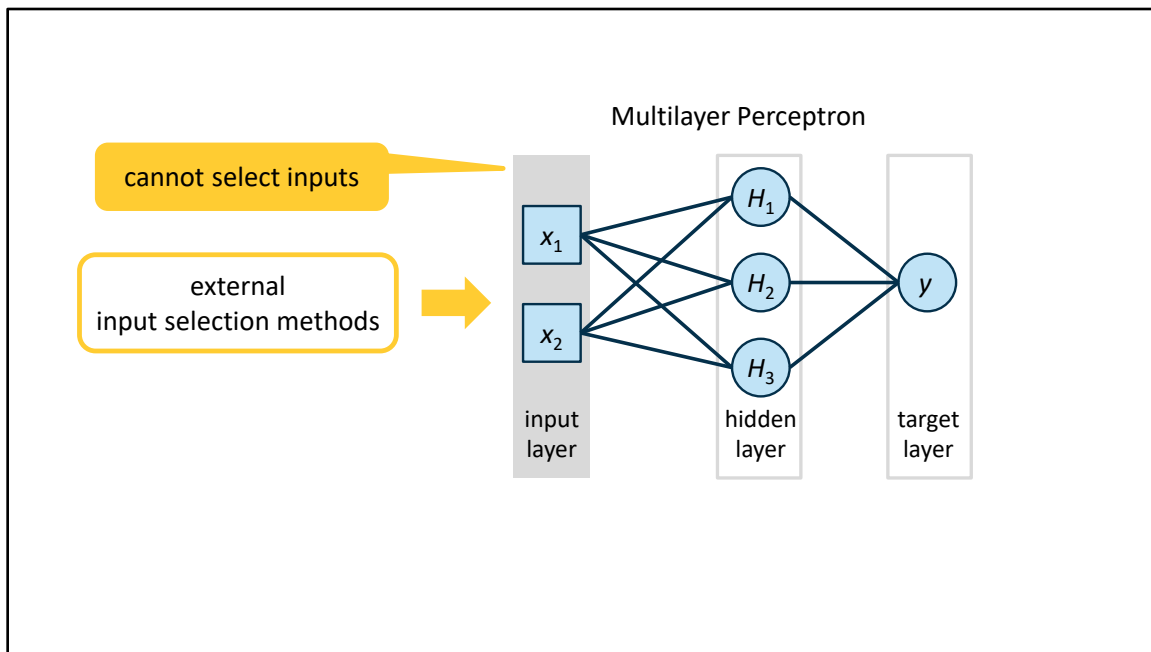
It might be best to determine the appropriate number of hidden units empirically, in the following way:

Start with a default neural network and measure its performance on some appropriate metric.

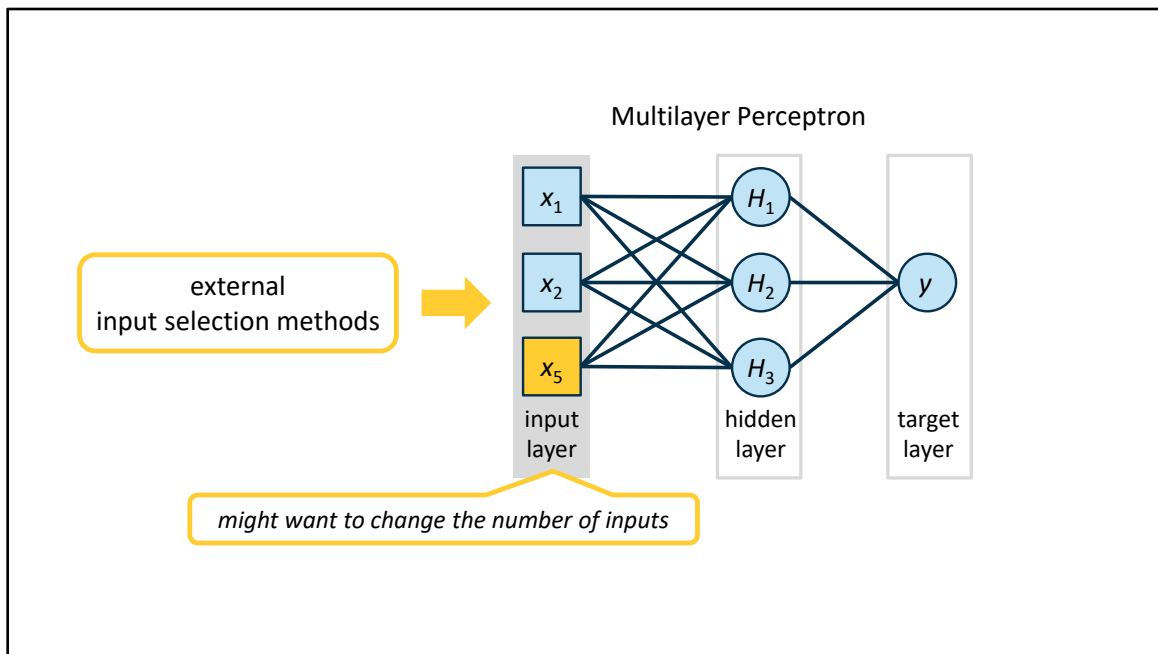
Then increase the number of hidden units by one and observe the impact on the network's fit. Continue adding hidden units until the network's performance drops.



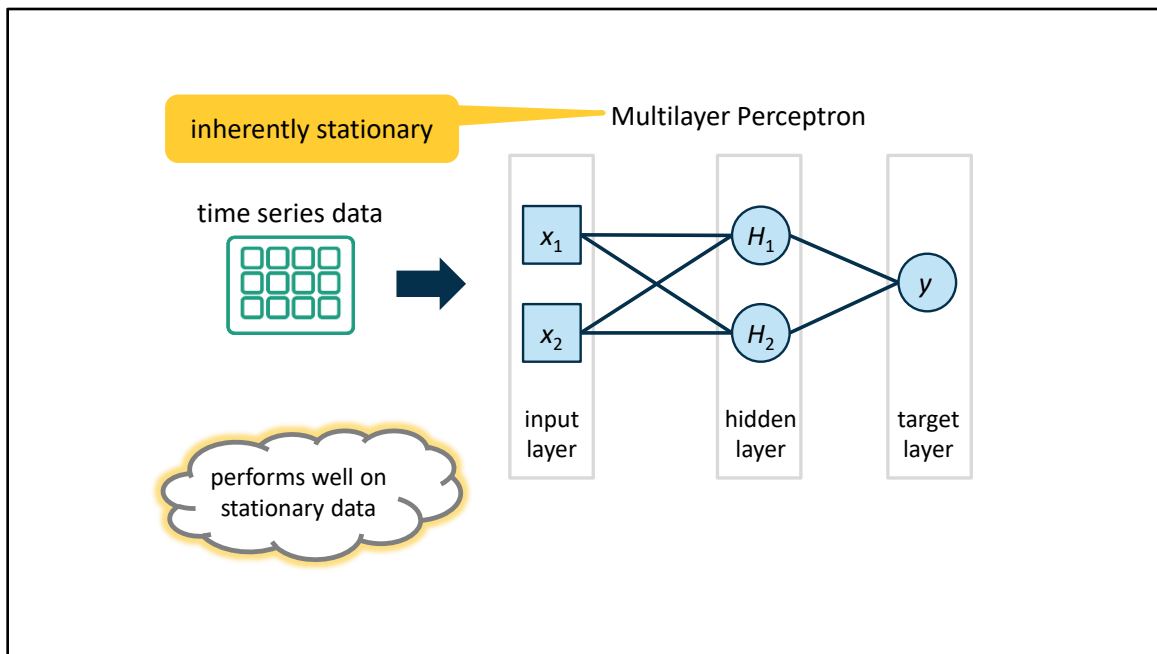
As the final model, select the last network generated before the drop in performance. This final model has the optimal number of hidden units.



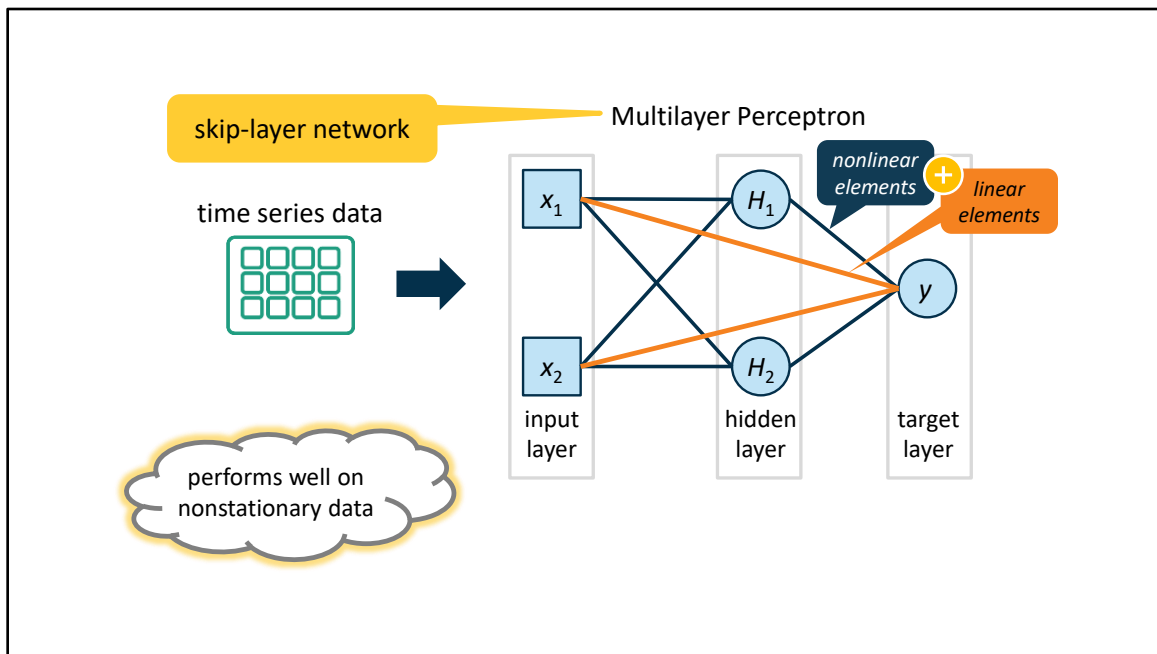
Unlike some other types of models, such as tree-based models, a neural network cannot select inputs. It is a good idea to try several external input selection methods to find the best combination of inputs for your model.



As a result, you might want to change the number of inputs in the input layer.



An MLP is inherently stationary. This means that it will tend to perform poorly when applied to nonstationary data (that is, data that has elements that change over time). An example of nonstationary data (also known as time series data), is monthly electricity usage per household.



When working with time series data, it can be helpful to add direct connections from the input layer to the target layer, bypassing the hidden layer or layers. The result is a special type of MLP known as a skip-layer network. This combines linear elements (that is, the direct connections) and nonlinear elements (that is, connections from the hidden layer) into a single neural network. An MLP is already a universal approximator, so there is often little to be gained by adding direct connections when you are not working with time series data.

In the diagram shown here, the skip-layer perceptron has two inputs and one hidden layer with two hidden units. However, a skip-layer perceptron can have a different number of inputs, hidden layers, and hidden units. The direct connections between the inputs and the target define a skip-layer perceptron.

Stationary versus Nonstationary Data

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Calculating the Number of Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Deep Learning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Deep Learning Models



Copyright © SAS Institute Inc. All rights reserved.

Question: Deep Learning Models

Which of the following techniques do deep learning models use to overcome the computational challenges associated with multiple hidden layers?

- a. direct connections between the input layer and the target layer
- b. fast moving gradient-based optimizations
- c. a first hidden layer in which the number of hidden units is twice the number of inputs in the input layer
- d. standardized inputs

Answer: Deep Learning Models

Which of the following techniques do deep learning models use to overcome the computational challenges associated with multiple hidden layers?

- a. direct connections between the input layer and the target layer
- ☒ b. fast moving gradient-based optimizations
- c. a first hidden layer in which the number of hidden units is twice the number of inputs in the input layer
- d. standardized inputs

CA: b

FB: Deep learning models use fast moving gradient-based optimizations, such as Stochastic Gradient Descent, for this purpose.

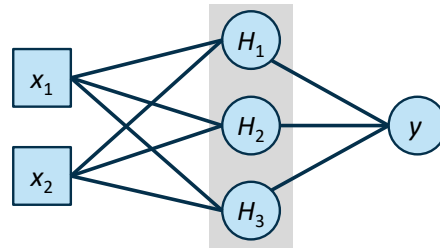
Activation Functions



Copyright © SAS Institute Inc. All rights reserved.

Activation Functions

ability to model nonlinear relationships



hidden unit
activation function

Remember that the strength of neural networks is their ability to model nonlinear relationships. The hidden unit activation function is the key contributor to the model's ability to model nonlinearities.

Many hidden unit activation functions are members of the sigmoid family.

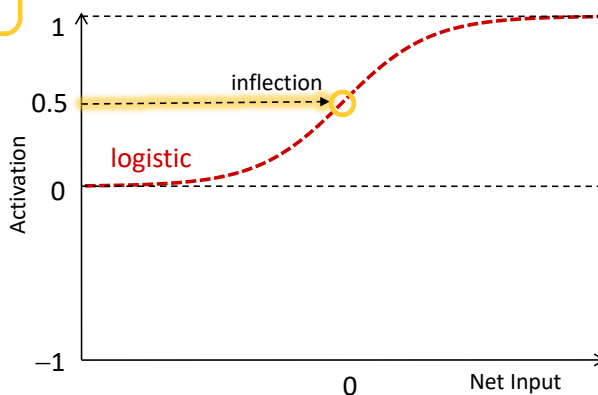
$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

hidden unit
activation function



In each hidden unit, after the weighted inputs and the bias are combined, the hidden unit's net input is passed through an activation function. Many of the available hidden unit activation functions are members of the sigmoid family.

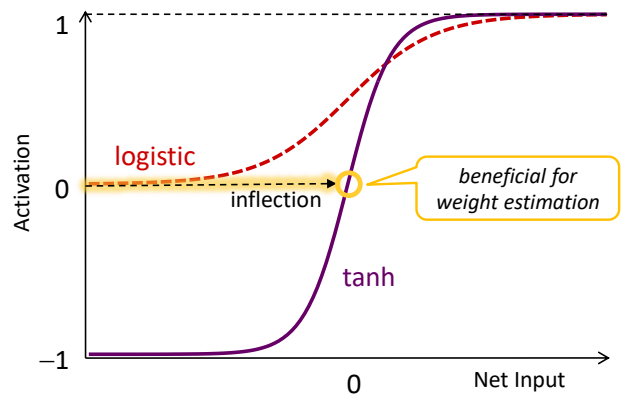
A well-known member of the sigmoid family is the logistic function. The logistic activation function constrains its output to the range of 0 to 1 with the inflection point at 0.5.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

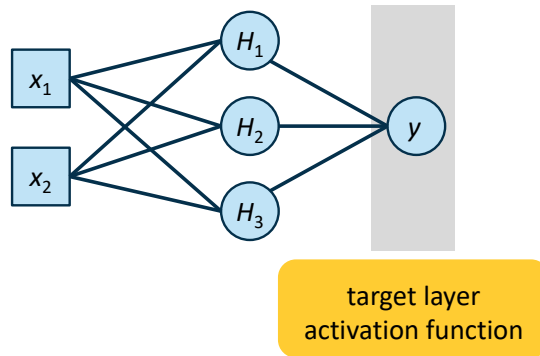
$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$



Although the logistic activation function was used as the hidden unit activation function in early neural network research, many other sigmoidal activation functions exist. In Model Studio, the Neural Network node uses the hyperbolic tangent (or tanh) as the default activation function. The hyperbolic tangent ranges from -1 to 1. This means that the inflection point is at 0 rather than at 0.5, as it is in the logistic sigmoid. This offers a small advantage during weight estimation.



Activation functions can also be used in the target layer to model outcome distributions for different types of targets.

binary target

logistic function

$$\log\left(\frac{\hat{p}}{1-\hat{p}}\right) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

In this example, the target is binary. The target layer link function (also known as the target layer activation function) is the logistic function, which is typically used with a binary target. The logistic function is the inverse of the logistic activation function. The logistic activation function constrains its output to the range of 0 to 1, so it is an ideal candidate for generating probability estimates when used in the target layer.

Hidden Layer Activation Functions in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Target Layer Activation Functions and Error Functions in Model Studio

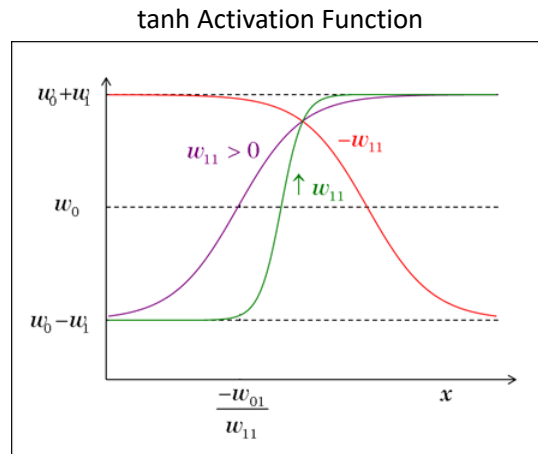
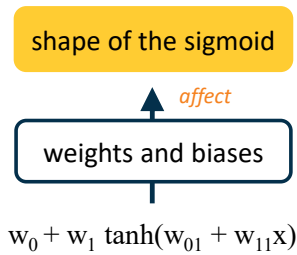
To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Shaping the Sigmoid

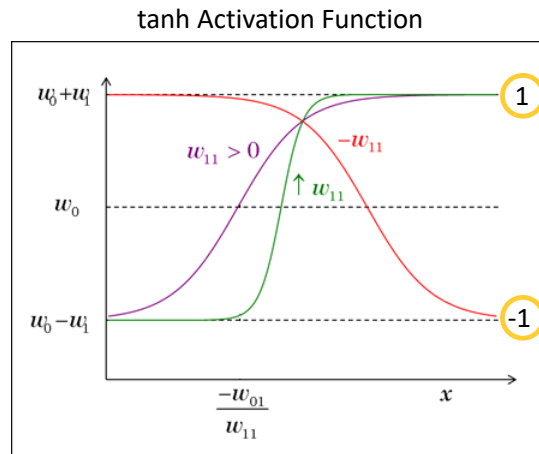
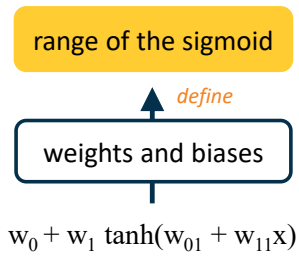


Copyright © SAS Institute Inc. All rights reserved.

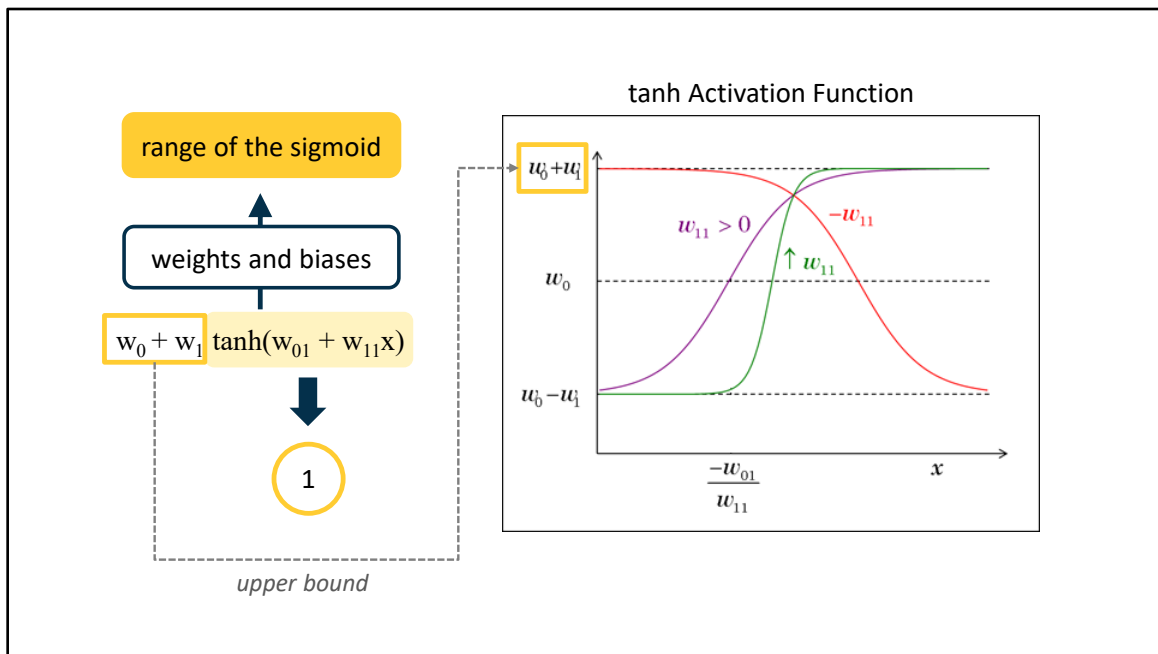


In a neural network, the weights and biases affect the shape of the activation function's sigmoid.

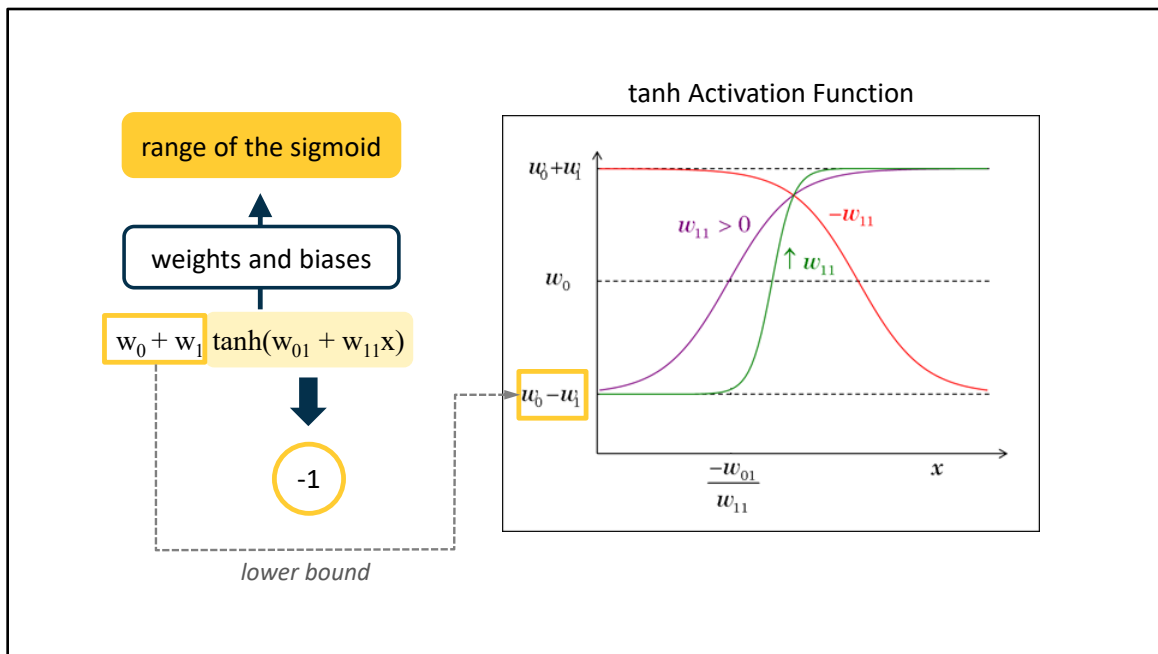
For example, this plot shows sigmoids for the tanh activation function in a very basic MLP. This MLP has a single input (x) and one hidden unit (represented as tanh of the quantity $w_{01} + w_{11}x$).



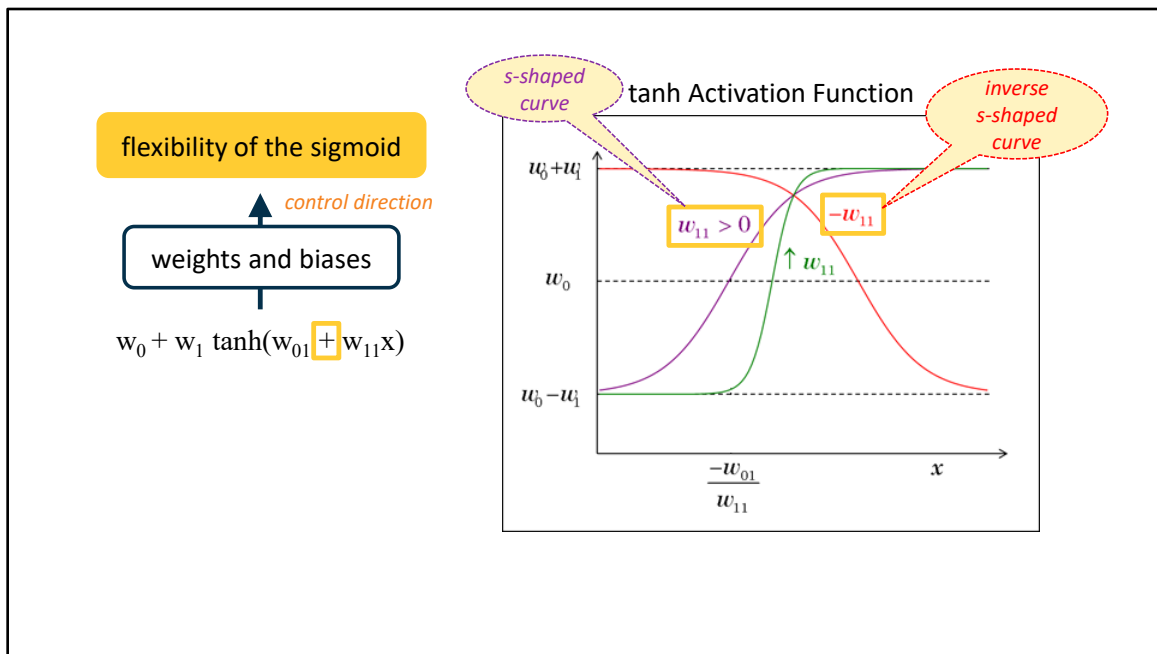
The weights and biases define the *range* of the sigmoid. Remember that the range of the tanh function is 1 to -1.



When tanh returns the value 1, the upper bound of its activation range is given by the output unit's bias plus its weight. In this example, the upper bound equals $w_0 + w_1$.



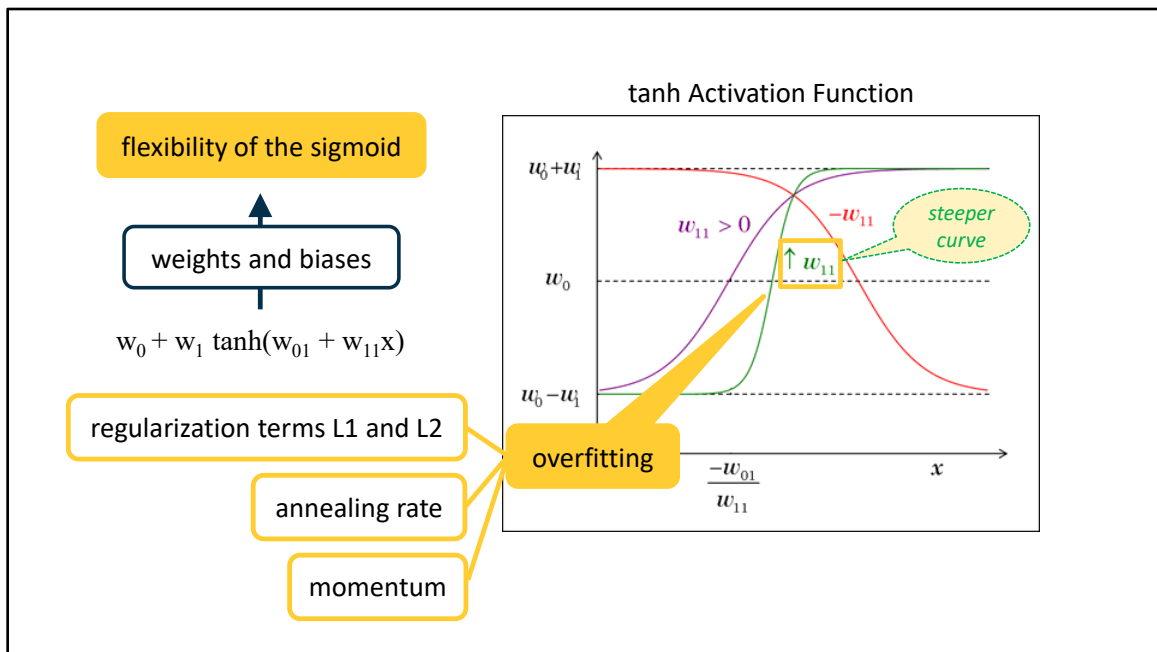
Conversely, the minimum activation value of the hyperbolic tangent function is -1, which means that the lower bound of the hyperbolic tangent's activation range is the output unit's bias minus its weight. In this example, the lower bound is $w_0 - w_1$.



The weights and biases also give the sigmoid surfaces their *flexibility*. The sign of the weight associated with the input x controls the direction of the sigmoid.

Positive weight values produce the familiar s-shaped curve (shown here by the purple line labeled $w_1 > 0$).

Negative weight values flip the sigmoid horizontally (shown by the red line labeled $-w_1$).



The larger the absolute value of the weight, the steeper the curve (shown by the green line labeled by an up arrow followed by w_{11}). Steep sigmoids are often thought to be responsible for overfitting. Overfitting is controlled by the regularization terms L1 and L2, the annealing rate, and the momentum, which you learn about later.

Demo: Modifying the Neural Network Architecture

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



4.3 Modifying the Model: Network Learning and Optimization



Optimizing the Complexity of a Neural Network Model



Copyright © SAS Institute Inc. All rights reserved.

Optimizing the Complexity of a Neural Network Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
 - Optimize the complexity of the model.
 - Regularize and tune the hyperparameters of the model.

Now that we have changed the model architecture, in an attempt to improve the model, we're ready to optimize the complexity of the model.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

optimizing complexity

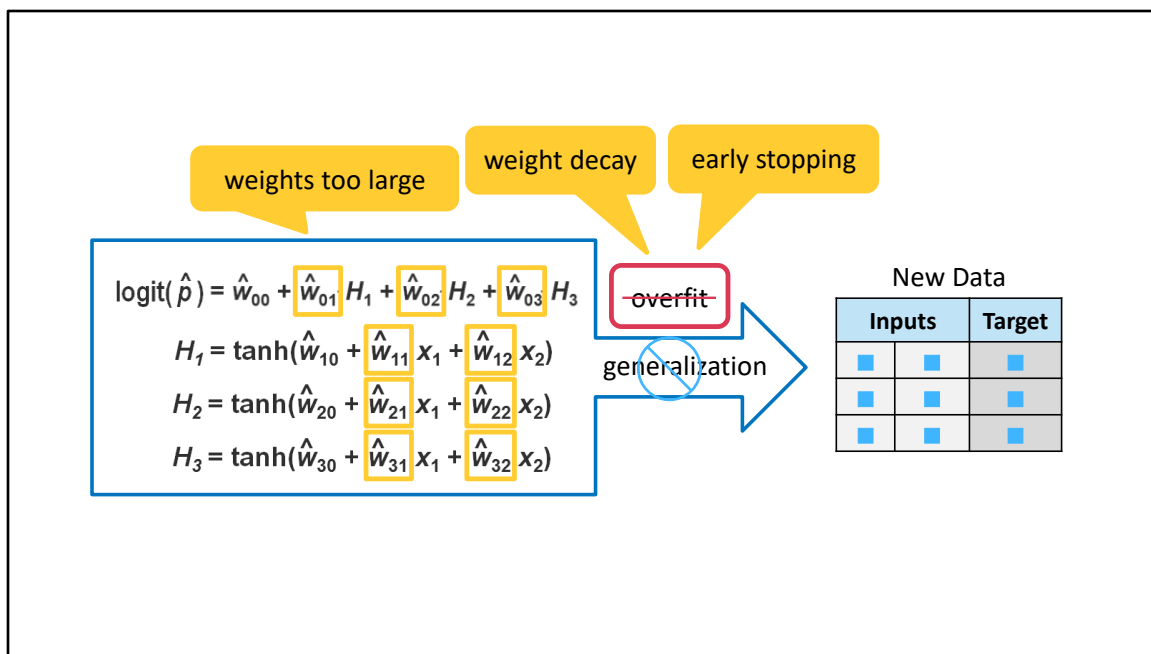


adding more terms
or rules



controlling the magnitude
of the weights

For neural networks, optimizing complexity does not involve adding more terms or more rules to the model. Instead, optimizing the complexity of a neural network involves controlling the magnitude of the weights.



If the weights grow too large, the model will be overfit and will not generalize well.

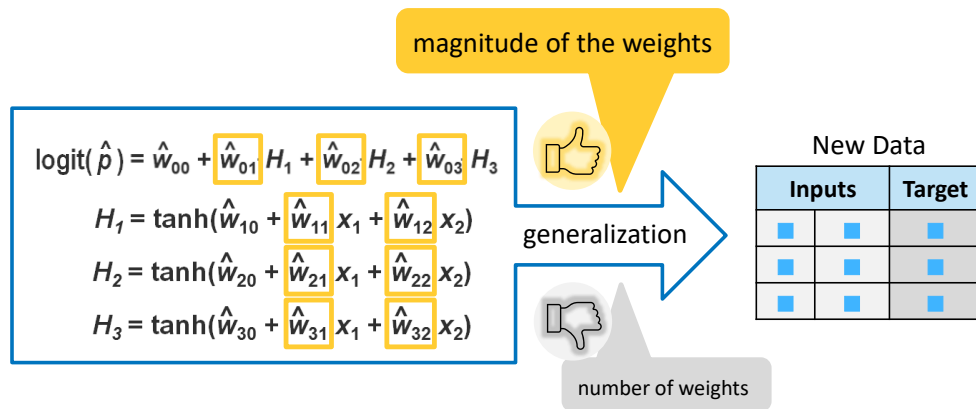
The two main methods of avoiding overfitting are weight decay and early stopping. These methods are often used together.

Weight Decay

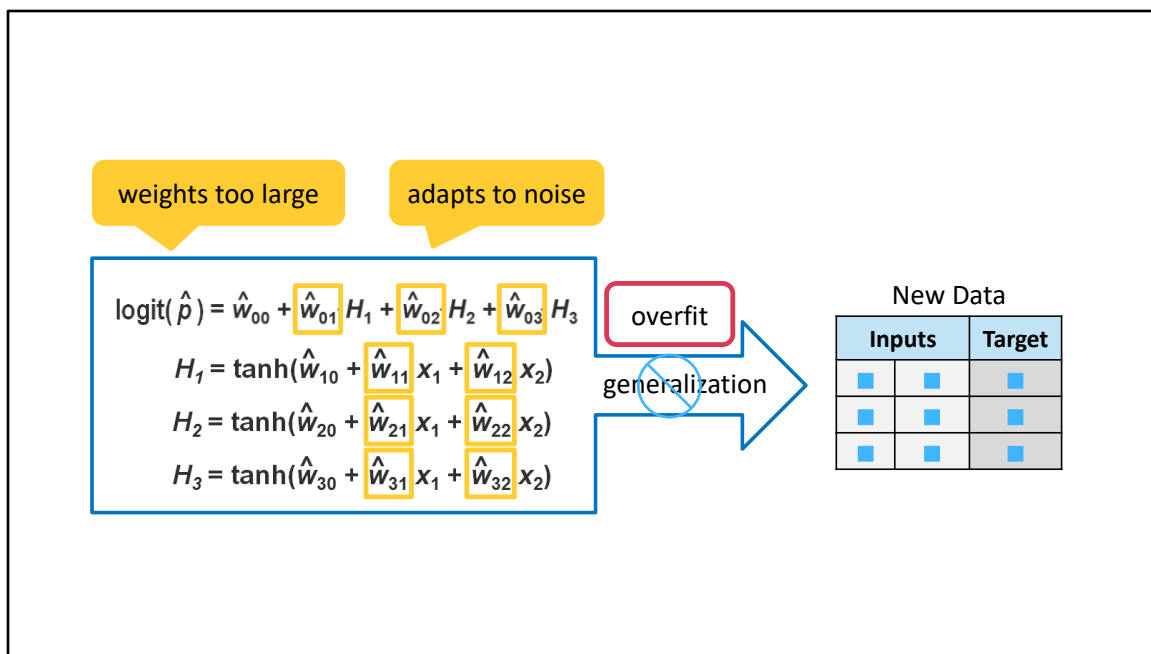


Copyright © SAS Institute Inc. All rights reserved.

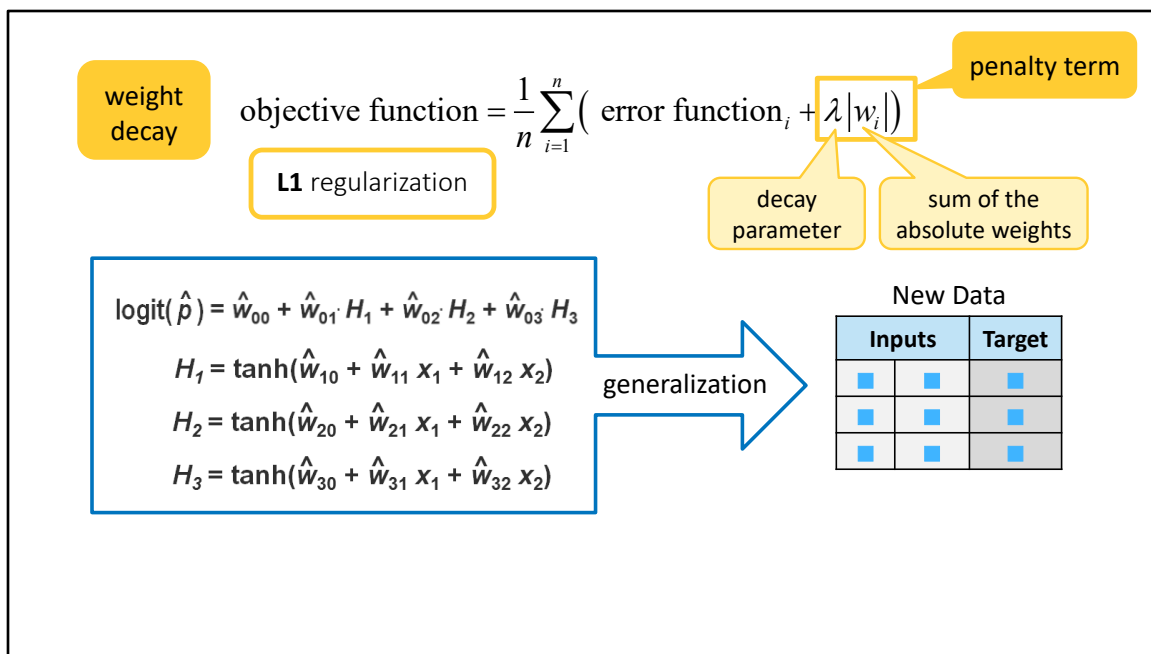
Weight Decay



Model generalization depends more on the magnitude of the weights than on the number of weights.

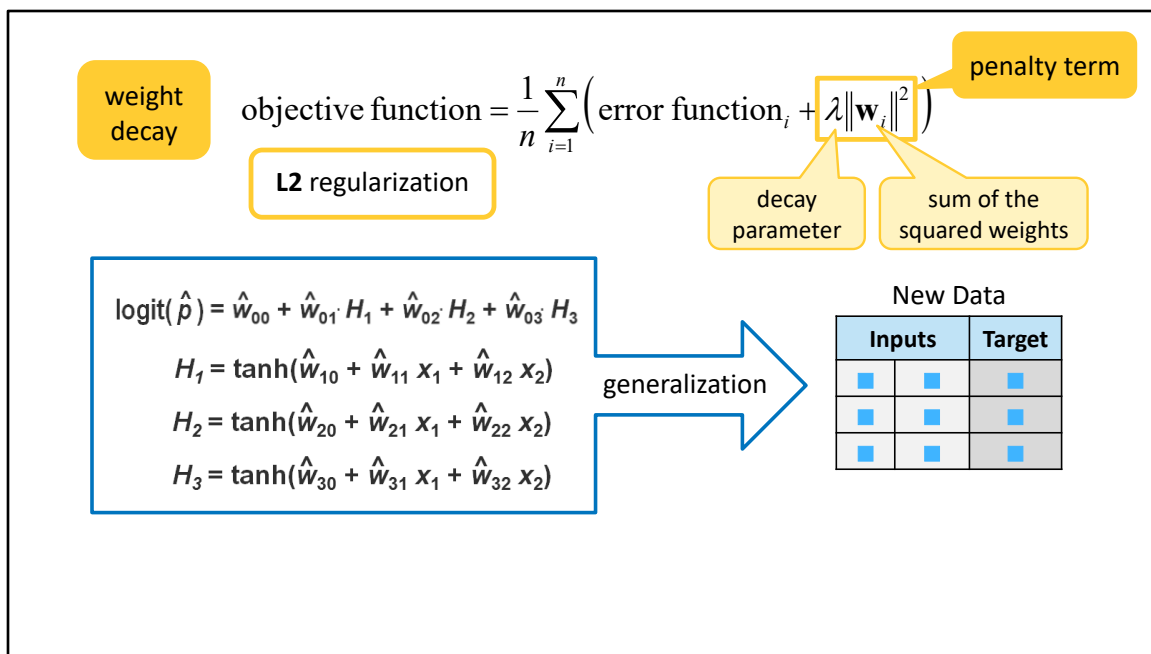


Very large magnitude weights tend to generate an irregular fit to the data because the model adapts to noise (that is, random variation). In other words, large weights are responsible for overfitting.



To avoid overfitting, we use a weight decay method with one or more regularizations. Regularizations are methods to help keep the sigmoids from becoming too steep. L1 regularization shrinks the weights toward zero.

To prevent the weights from growing too large, the weight decay method penalizes large weights by adding a term at the end of the objective function. This penalty term is the product of lambda (the decay parameter) and the sum of the absolute weights. The decay parameter controls the relative importance of the penalty term. Lambda commonly ranges from zero to one, although Model Studio does allow for values larger than 1.



L2 regularization is similar to L1 regularization in that both methods penalize the objective function for large network weights.

Instead of summing the absolute weights in the penalty term, L2 uses the sum of the squared weights. Here the penalty term is the product of lamda (the decay parameter) and the sum of the squared weights.

weight
decay

$$\text{objective function} = \frac{1}{n} \sum_{i=1}^n \left(\text{error function}_i + \lambda \|\mathbf{w}_i\|^2 \right)$$

penalty term
too large

The advantages outweigh the risks.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

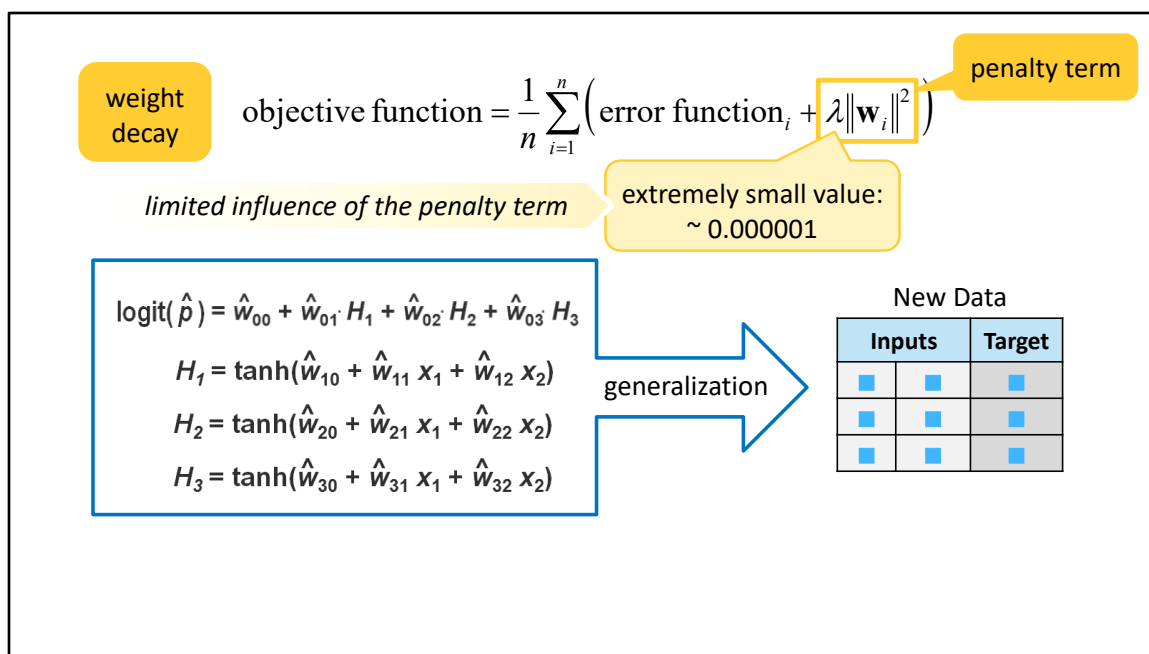
underfit

generalization

New Data

Inputs		Target
■	■	■
■	■	■
■	■	■

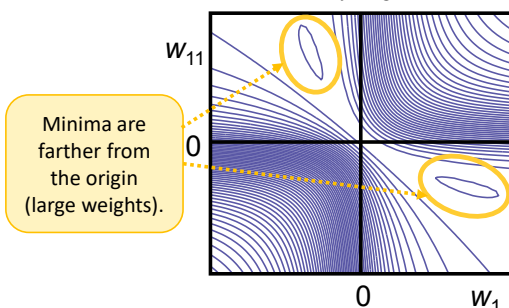
Specifying too large a penalty term risks the model under-fitting the data. However, the advantages of weight decay far outweigh its risks.



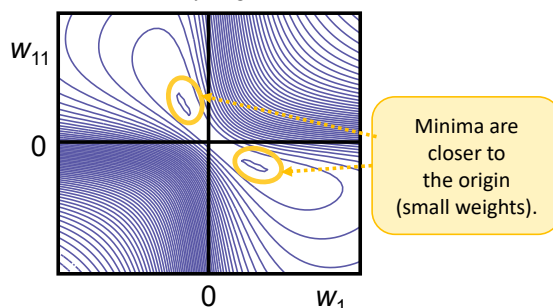
In addition, the influence of the penalty term is usually limited by selecting an extremely small value for the decay parameter. In many published studies, the magnitude of the decay parameter (lambda) is on the order of 0.000001.

$$\text{objective function} = \frac{1}{n} \sum_{i=1}^n \left(\text{error function}_i + \lambda \|\mathbf{w}_i\|^2 \right)$$

Objective Function
without Weight Decay
 $\lambda = 0$



Objective Function
with Weight Decay
 $\lambda > 0$



The graph on the left shows the objective function without weight decay. The graph on the right shows the objective function with weight decay.

In both graphs, the horizontal axis shows the magnitude of the weight w_1 , and the vertical axis shows the magnitude of the weight w_{11} .

In the graph on the left, with no weight decay, the minima are farther from the origin, so the weights used to find the minima are large.

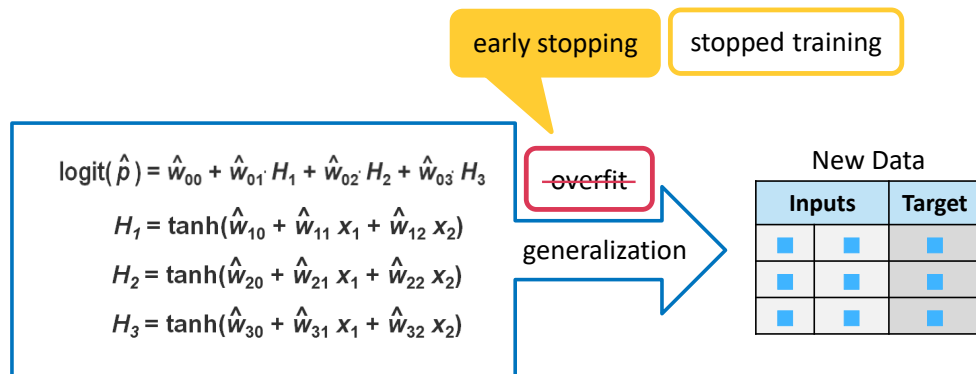
In the graph on the right, the minima are constrained closer to the origin, so the weights used to find the minima are smaller (thus, less likely to overfit).

Early Stopping



Copyright © SAS Institute Inc. All rights reserved.

Early Stopping



Early stopping, also called stopped training, is another method of avoiding overfitting by preventing the weights from growing too large.

Single model is estimated while building a neural network with fixed architecture.

$$\text{logit}(\hat{p}) = \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

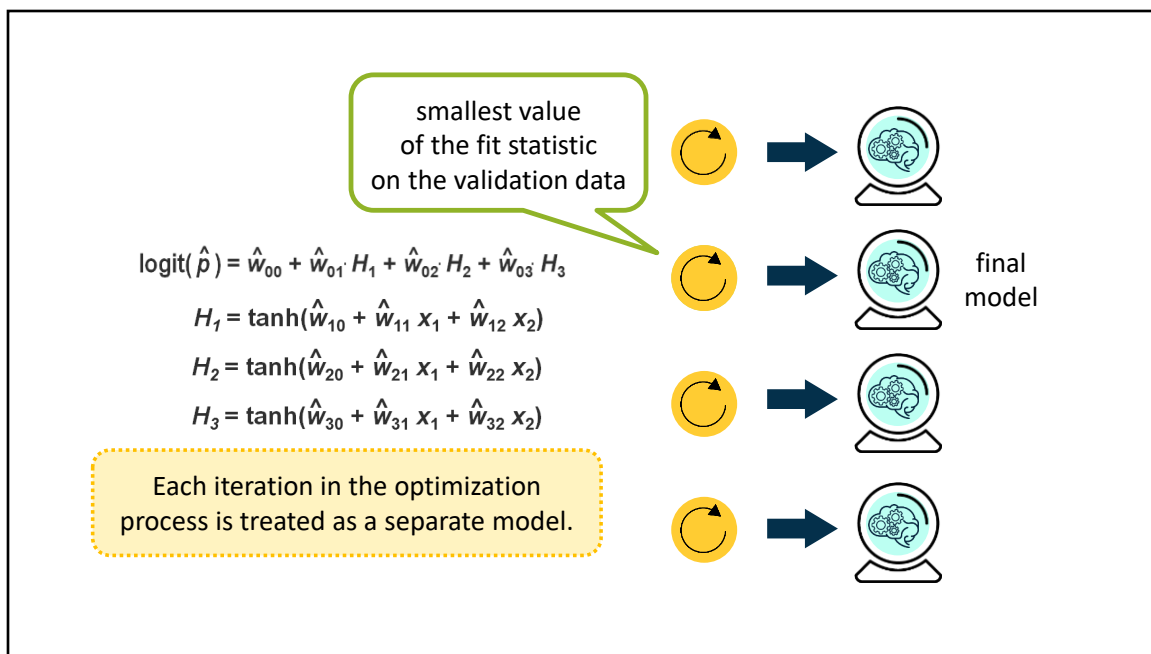
$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

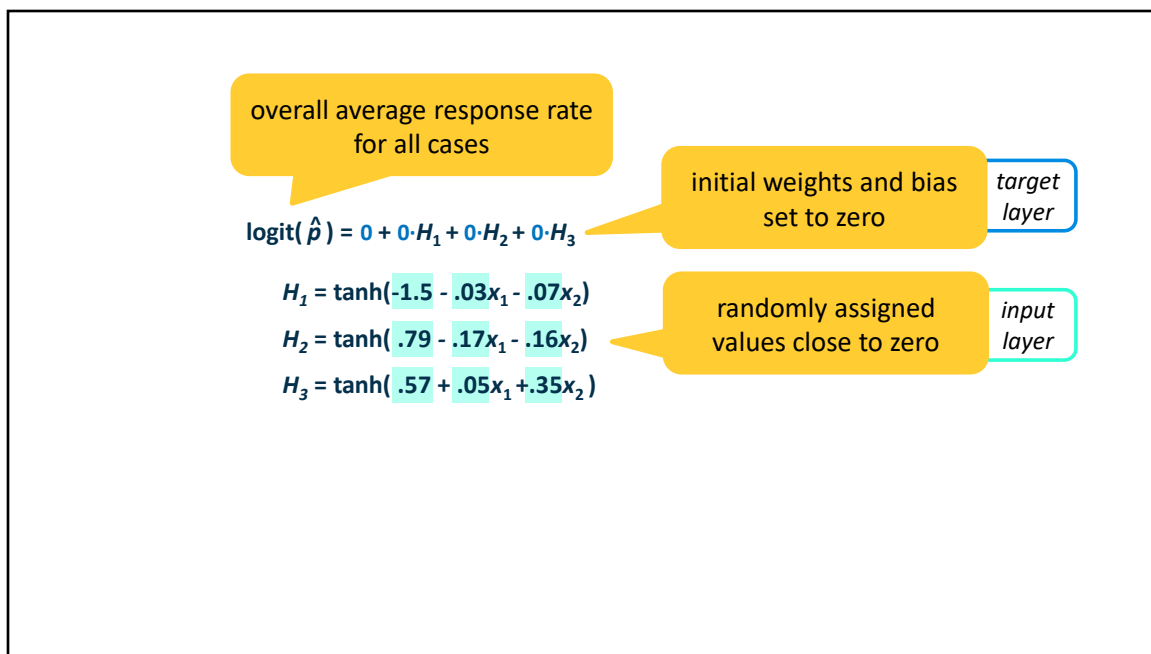


The weight magnitude changes complexity.

When you build a neural network, a single model is estimated, and the magnitude of the weight coefficients allows the complexity of that model to change.



SAS Viya treats each iteration in the optimization process as a separate model. The iteration with the smallest value of the selected fit statistic on the validation data is chosen as the final model.



At the beginning of the optimization process, the neural network model is set up to predict the overall average response rate for all cases. To do so, all weights and the bias in the target layer are set to zero, as shown in the equation. Next, the weights and the biases of the input layer are randomly assigned values close to zero (some positive and some negative).

updating the weight estimates such that ...

$$\text{objective function} = \frac{1}{n} \sum_{i=1}^n \left(\text{error function}_i + \lambda \|\mathbf{w}_i\|^2 \right)$$

$$\text{logit}(\hat{p}) = 0 + 0 \cdot H_1 + 0 \cdot H_2 + 0 \cdot H_3$$

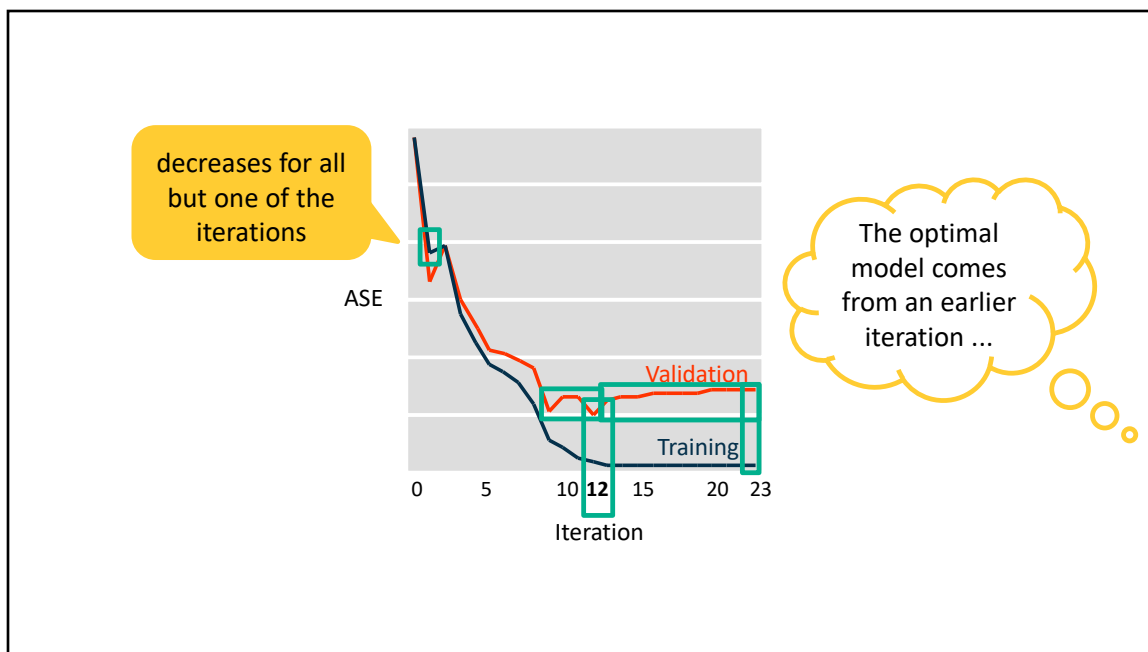
$$H_1 = \tanh(-1.5 - .03x_1 - .07x_2)$$

$$H_2 = \tanh(.79 - .17x_1 - .16x_2)$$

$$H_3 = \tanh(.57 + .05x_1 + .35x_2)$$

decreases the value

Training proceeds by updating the weight estimates in a manner that decreases the value of the objective function.



Question: Model Generalization



Copyright © SAS Institute Inc. All rights reserved.

Question: Model Generalization

For neural networks, model generalization depends more on the number of weights than on the magnitude of the weights.

- a. True
- b. False

Answer: Model Generalization

For neural networks, model generalization depends more on the number of weights than on the magnitude of the weights.

- a. True
- ☒ b. False

[CA: b]

FB: Model generalization depends more on the magnitude of the weights than on the number of weights. In fact, large weights are responsible for overfitting.

Question: Early Stopping



Copyright © SAS Institute Inc. All rights reserved.

Question: Early Stopping

When early stopping is used, where does the optimal model come from?

- a. one of the iterations that occur earlier than the final training iteration
- b. the iteration immediately before the final training iteration
- c. the final training iteration

Answer: Early Stopping

When early stopping is used, where does the optimal model come from?

- a. one of the iterations that occur earlier than the final training iteration
- b. the iteration immediately before the final training iteration
- c. the final training iteration

[CA: a]

FB: The philosophy of early stopping says that the optimal model comes from an earlier iteration where the objective function on the validation data has a smaller value than it does for the final iteration. The optimal model does not necessarily need to come from the iteration immediately before the final iteration.

Regularizing and Tuning the Hyperparameters of a Neural Network Model



Copyright © SAS Institute Inc. All rights reserved.

Regularizing and Tuning the Hyperparameters of a Neural Network Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
- ✓ Optimize the complexity of the model.
 - Regularize and tune the hyperparameters of the model.

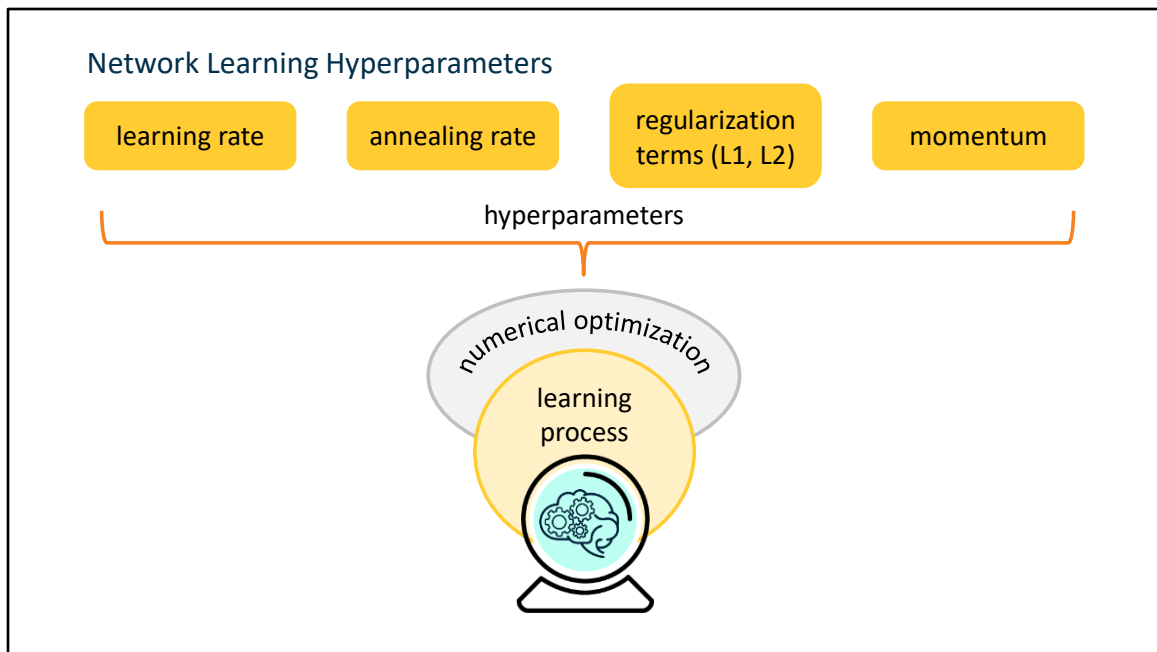
Autotuning is disabled in SAS Viya for Learners, and selecting it results in an error.

We are now at the last discovery task: Regularize and tune the hyperparameters of the model. One of the most difficult aspects of neural network modeling is finding the model hyperparameters that result in the best model. These hyperparameters are associated with the number of hidden layers, the number of hidden units, the activation function, the target function, and so on. You can tune the hyperparameters manually or you can use Model Studio's autotuning feature.

Network Learning Hyperparameters

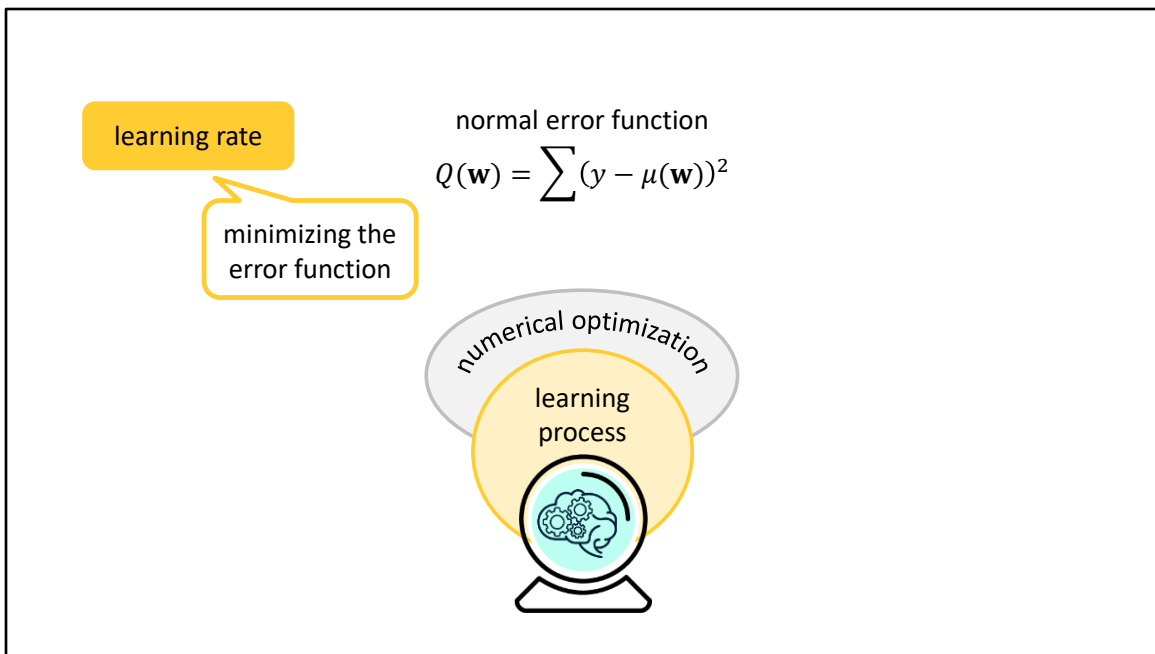


Copyright © SAS Institute Inc. All rights reserved.



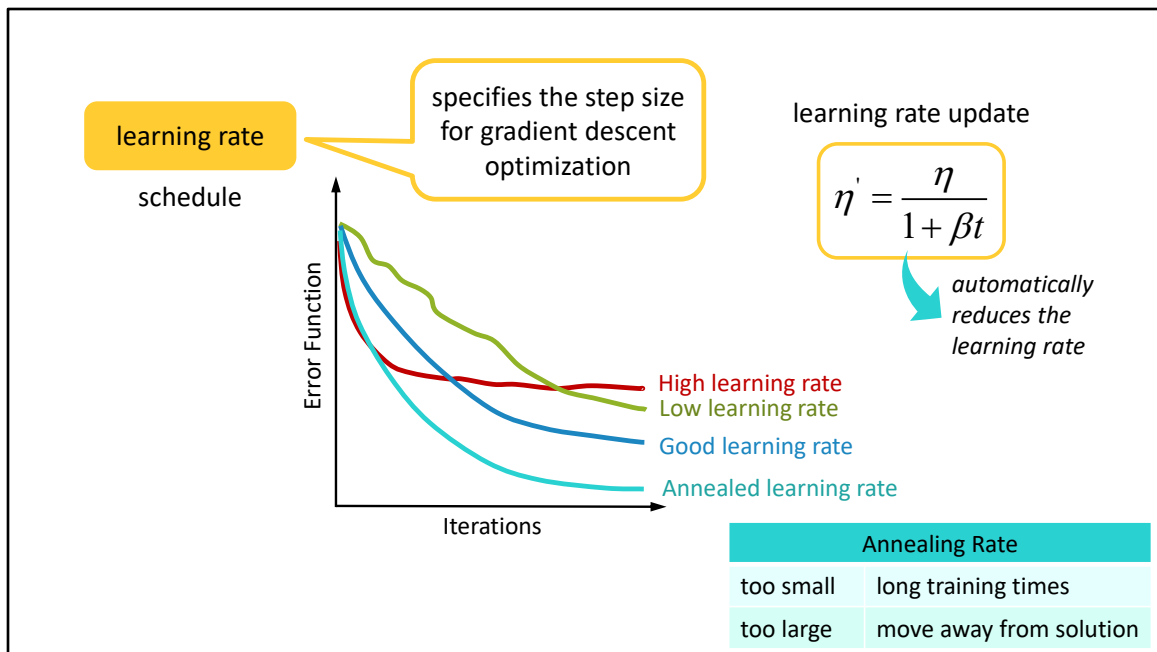
The learning process is critical during the building of a neural network. Remember that a neural network uses a numerical optimization method that you specify to estimate the weights that minimize the error function. This process is called learning.

A subset of important hyperparameters for neural networks directly affects the numerical optimization and thus the learning process. Therefore, these options affect the final performance of a neural network model. These hyperparameters include the learning rate, the annealing rate, the regularization terms L1 and L2, and the momentum.



The learning rate is defined in the context of optimization, and it is related to minimizing the error function (also known as the deviance function). For example, the default deviance measure that is used to fit an interval target is a normal distribution. This error function is the familiar ordinary least squares equation, shown here.

While minimizing the error function, we use for example, gradient descent or other variants of weight decay. In weight decay, the model parameters (that is, weights and biases in the network) are updated in a way that decreases the error function.



The learning rate is a training parameter that controls the size of weight and bias changes in learning the algorithm. Setting the learning rate too high yields great progress in decreasing the error function in the first few iterations, but at the risk of diverging later in the training process. On the other hand, setting the learning rate too small can result in very long training times. A "good" learning rate is a balance between the two and is considered to be problem specific.

A learning rate schedule is one approach used to optimize the learning process. The underlying idea is to train the model for a few iterations using a large learning rate. Then the learning rate is adjusted to a lower value, and the model is trained for a few more iterations. This process is repeated until convergence.

A learning rate schedule uses an annealing rate. Annealing is a way to automatically reduce the learning rate as stochastic gradient descent progresses, causing smaller steps as stochastic gradient descent approaches a solution. Effectively, it replaces the learning rate parameter, η , with η' , as shown in the equation. Here, t is the number of iterations that SGD has performed and is the annealing parameter.

Setting the annealing rate too small results in very long training times. Setting it too large can result in movement away from the desired solution (that is, divergence).

Important Hyperparameters for Neural Networks: Summary

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Modifying the Learning and Optimization Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Autotuning Options for Neural Networks in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



(Optional) Practice: Build a Neural Network Model Using Autotuning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



4.4 Lesson Conclusion



Lesson Quiz

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Lesson 5: Support Vector Machines



5.0 Lesson Overview



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



5.1 Building a Default Support Vector Machine Model



Support Vector Machines as Classifier Models



Copyright © SAS Institute Inc. All rights reserved.

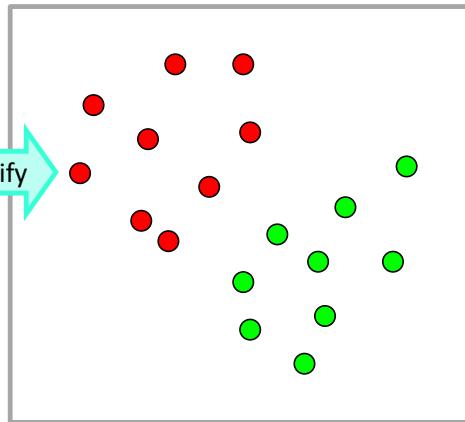
Support Vector Machines as Classifier Models

Support Vector
Machine



Classify

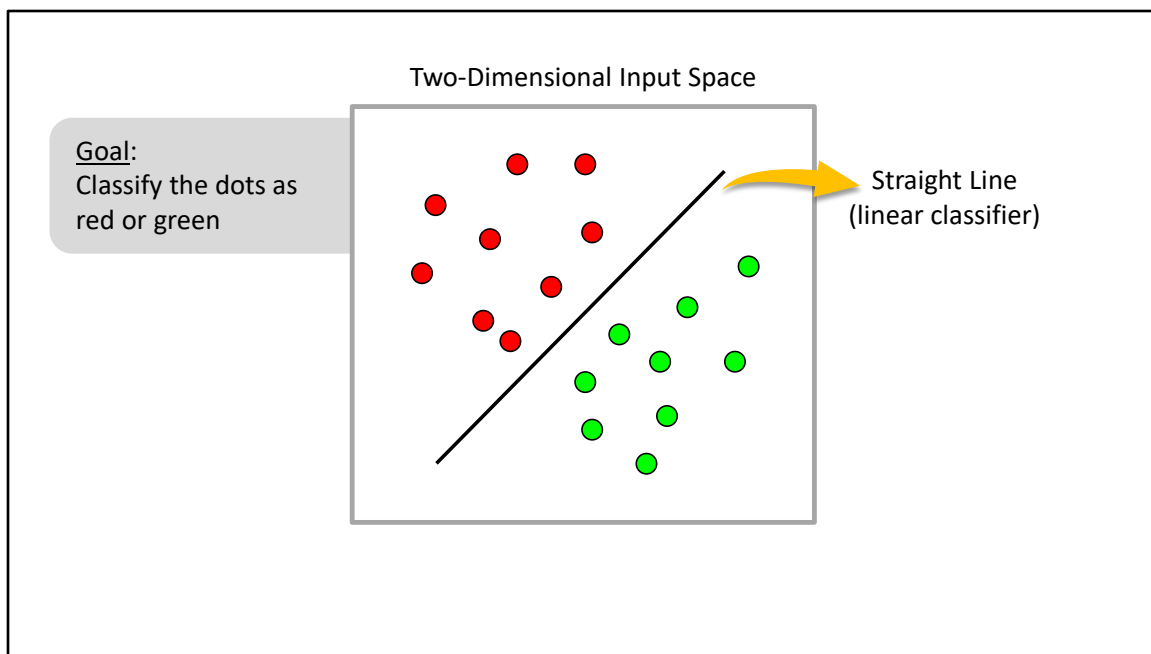
- decision
- rank
- estimate



Like decision trees, support vector machines were originally developed simply to classify outcomes. In other words, the model makes decision predictions instead of ranks or estimates.

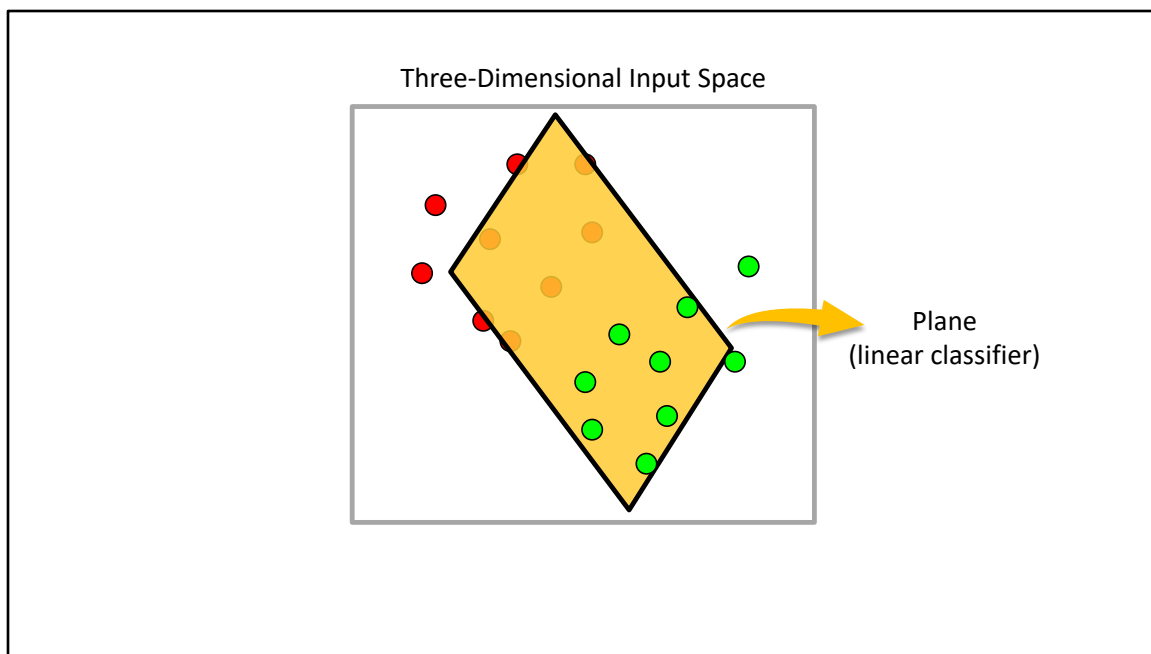
In the basic example shown here, a support vector machine separates the outcomes of a binary target into two classes, red and green.

More recently, support vector machine models have expanded in many ways. Now they can be used for regression tasks to produce estimates. For classification, support vector machines can now provide ranks and estimates. The SVM node in Model Studio fits a support vector machine via interior-point, active-set, or coordinate descent optimization for a binary or interval target. Binary target is the primary focus in this course.

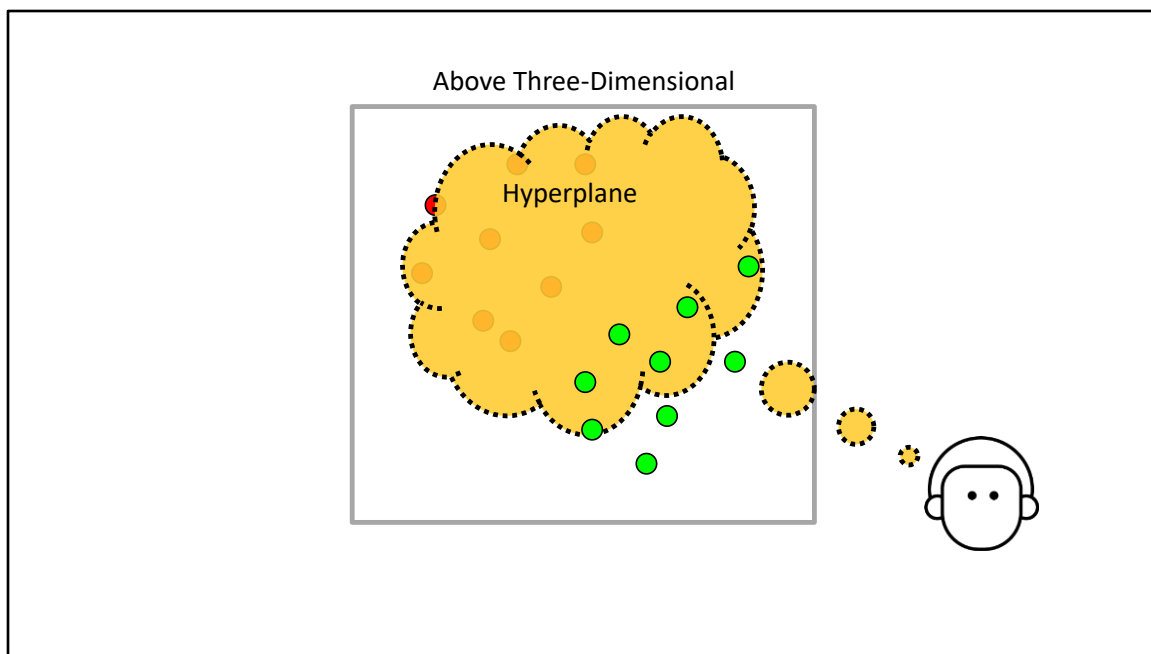


To learn how support vector machines work, we'll focus on this simple, red-versus-green decision case. In this example, the training data set has two inputs, so this is a two-dimensional input space. The goal is to classify the dots as red (that is, the dots in the upper left area) or green (that is, the dots in the lower right area).

The binary outcome is linearly separable, which means that it is possible to draw a line that perfectly separates the target outcomes into two classes (that is, with no errors). This line is the support vector machine. It can also be called a classifier, a classifier model, or a classification rule.

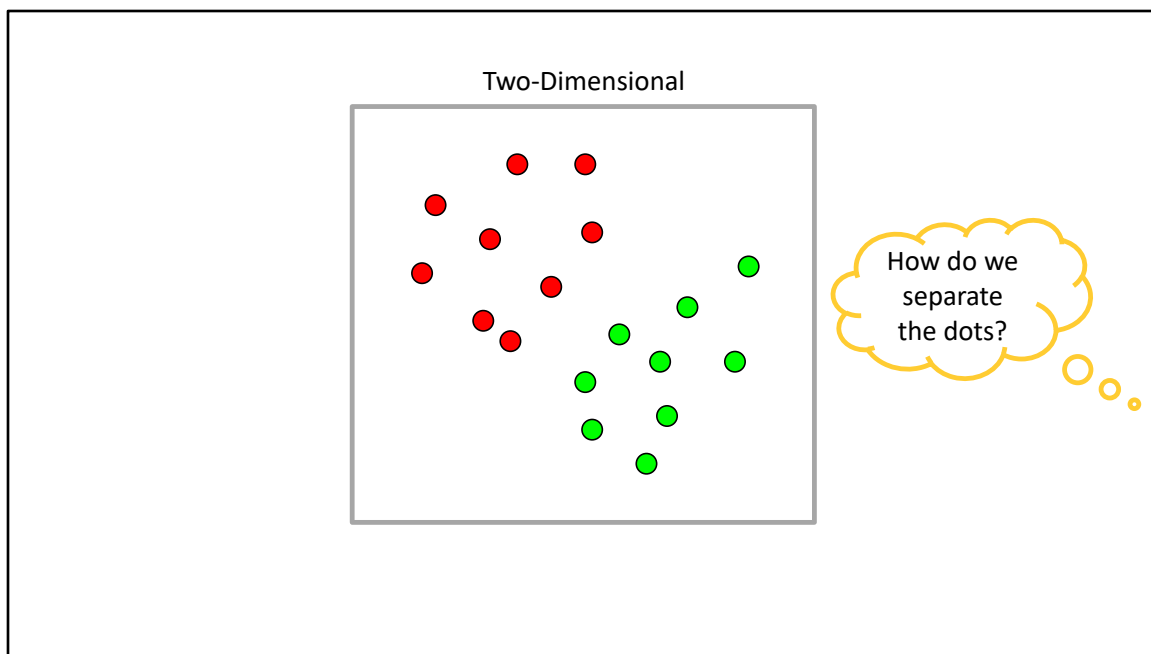


In a three-dimensional input space (that is, when there are three input variables), the classifier is a plane instead of a line. Notice that this diagram shows how the classifier changes but, for simplicity, the third input variable is not shown.



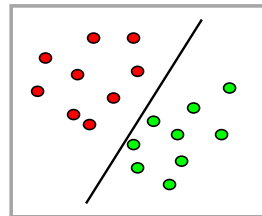
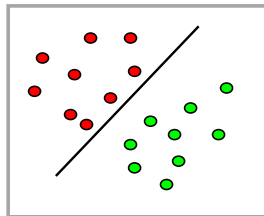
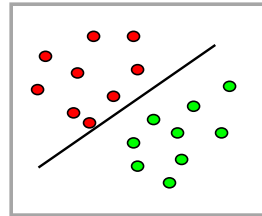
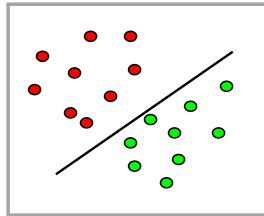
With more than three variables, the classifier is a hyperplane. The diagram represents this concept, but the classifier and additional inputs are not shown.

The techniques covered in this lesson apply to problems of two or more dimensions, so keep in mind that the term hyperplane is sometimes used generally.



Returning to our two-dimensional example, how do we separate the red and green dots?

infinitely
many lines
that perfectly
separate the
target
outcomes ...



Is there
a best
line?

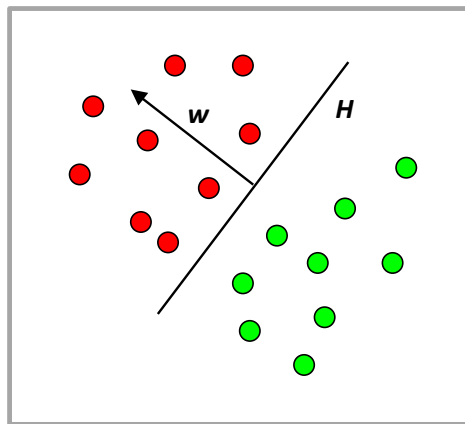
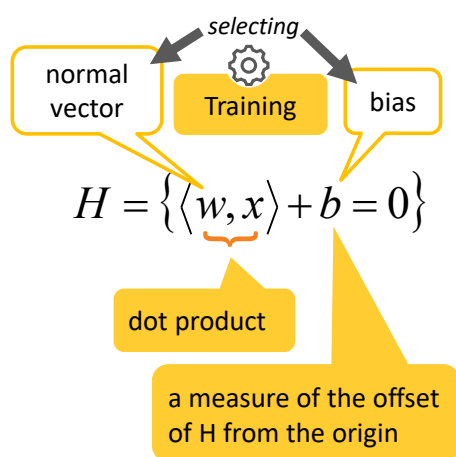
Clearly a line would work. But which line? If the dots are linearly separable, there are infinitely many lines that separate the outcome with no errors. Of the infinitely many solutions, is there a best line?

Mathematical Definition of a Support Vector Machine



Copyright © SAS Institute Inc. All rights reserved.

Mathematical Definition of a Support Vector Machine

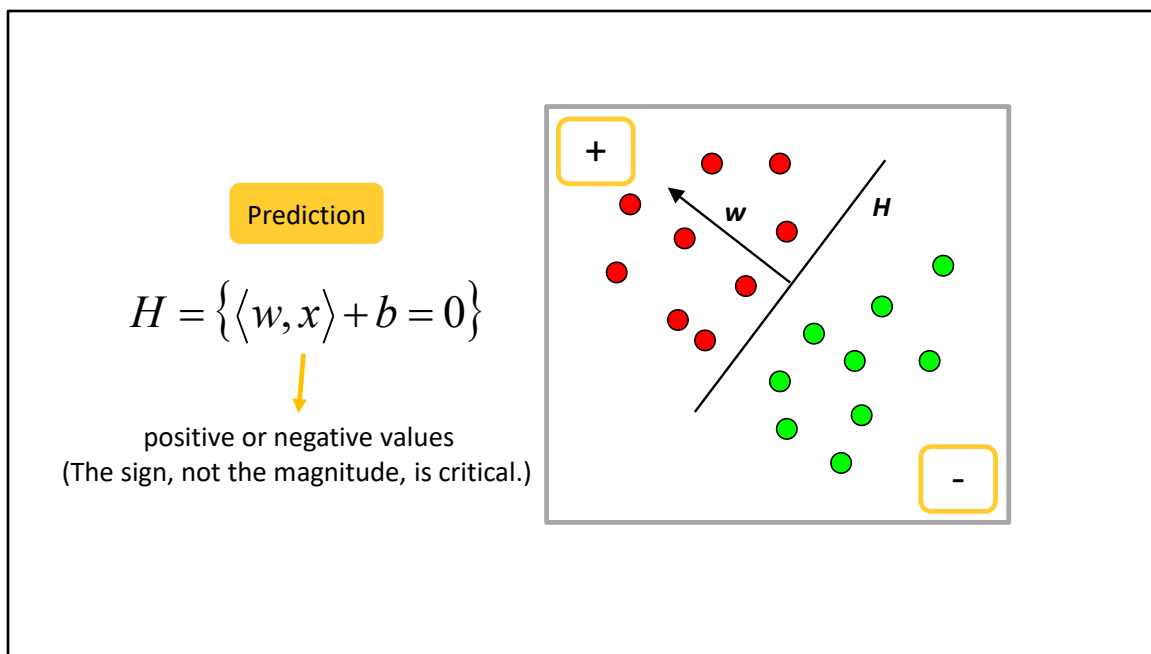


Finding the best solution to a linear classification problem is a mathematical problem, so let's look at the mathematical definition of a support vector machine model. The classifier model (that is, the separating line) is symbolically denoted uppercase H . H has two elements: a normal vector w and a bias term b . The vector w is perpendicular to H . In a two-dimensional input space, where H is a line, w affects the slope of the line. Imagine w as a handle that rotates and affects the flatness or steepness of the line.

The bias term b is a measure of the offset of the separating line from the origin.

H is defined as the dot product between the vector w and the vector of inputs x , plus b . When a point falls exactly on the line, the value of H is 0. A dot product is one way to multiply vectors. A dot product returns a scalar, or single number, as the answer.

To train a support vector machine model, we select w and b so that the line separates the values of the target. (This also applies to a higher-dimension example, which has a hyperplane instead of a line.)



After training, how does the model make predictions for new observations? The formula for H returns predictions as positive or negative values, where the sign, not the magnitude, is the critical aspect. Points on one side of the line (the points in the direction of the normal vector) return a positive value. Points on the other side of the line return a negative value.

Dot Products

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.

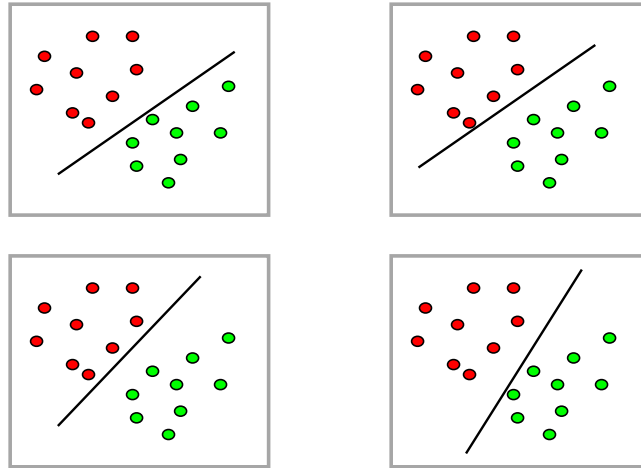


Maximum-Margin Hyperplane and Support Vectors

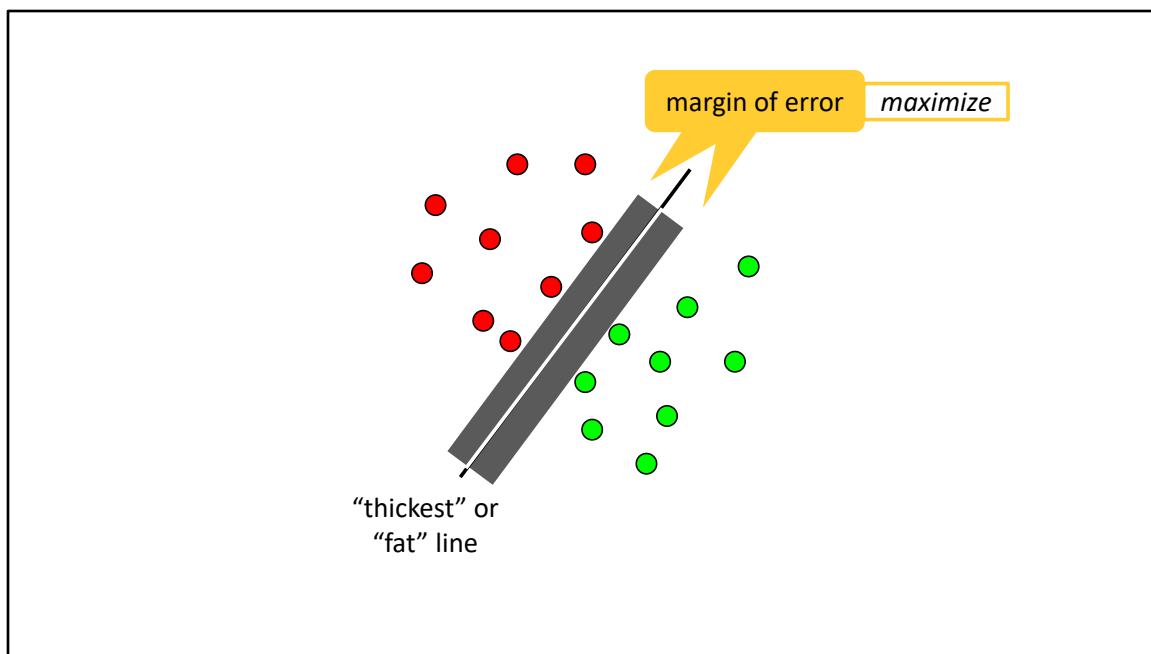


Copyright © SAS Institute Inc. All rights reserved.

Maximum-Margin Hyperplane and Support Vectors



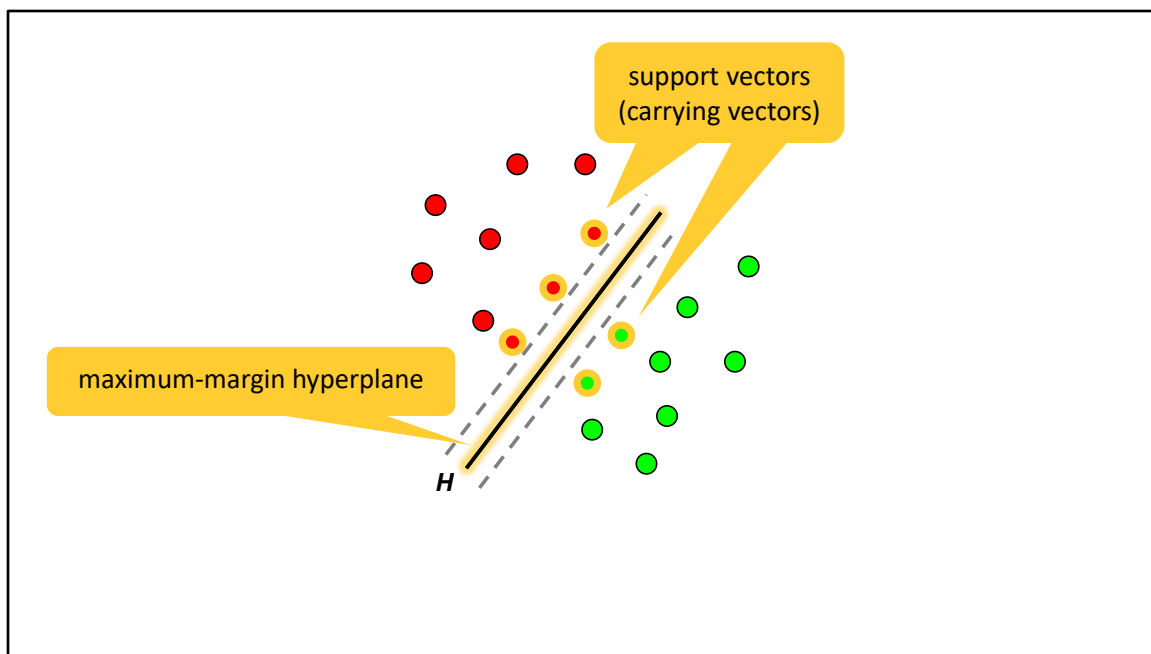
To identify the best classifier for linearly separable data, a good approach is to find the maximum-margin hyperplane.



Let's see how this works. We want to maximize the margin of error on either side of the separator.

In our two-dimensional example, visualize the thickest line that touches the innermost red values of the target and the innermost green values of the target. The thickest (or fat) line is a good starting place for the "best" line because it results in the largest margin of error on the positive and negative sides.

The best solution is the exact center, or median, of the fat line.



Taking the exact center of the fat line produces a unique solution known as the maximum-margin hyperplane. This solution, which is represented as H in the diagram, has the greatest margin of error on either side.

Support vectors, also known as the carrying vectors, are the points in the data that are closest to the hyperplane. These points, and only these points, determine the exact location of H . In this example, there are five support vectors. Using support vectors avoids the curse of dimensionality because all other points are not relevant to the solution.

According to the curse of dimensionality, the more input variables you have, the more data points that you need to fill the space—that is, to determine the true relationship between the inputs and the target.

Question: SVM and the Curse of Dimensionality



Copyright © SAS Institute Inc. All rights reserved.

Question: SVM and the Curse of Dimensionality

To construct a support vector machine, only the observations closest to the separating hyperplane are used. This avoids the curse of dimensionality.

- a. True
- b. False

TAG_Movie_Num:

ecpml4105_1_q1

CA: true

Using only the observations closest to the separating hyperplane avoids the curse of dimensionality by limiting the number of data points in the solution.

Answer: SVM and the Curse of Dimensionality

To construct a support vector machine, only the observations closest to the separating hyperplane are used. This avoids the curse of dimensionality.

- ☒ a. True
- ☐ b. False

CA: true

Using only the observations closest to the separating hyperplane avoids the curse of dimensionality by limiting the number of data points in the solution.

Essential Discovery Tasks for Support Vector Machines



Copyright © SAS Institute Inc. All rights reserved.

Essential Discovery Tasks for Support Vector Machines



Essential Discovery Tasks

- ✓ Select an algorithm.
- Improve the model.
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.

The essential discovery tasks for support vector machines are the same as for neural networks. Let's build a support vector machine model.

Demo: Building a Support Vector Machine Using the Default Settings

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



5.2 Modifying the Model: Methods of Solution



Improving the Support Vector Machine Model



Copyright © SAS Institute Inc. All rights reserved.

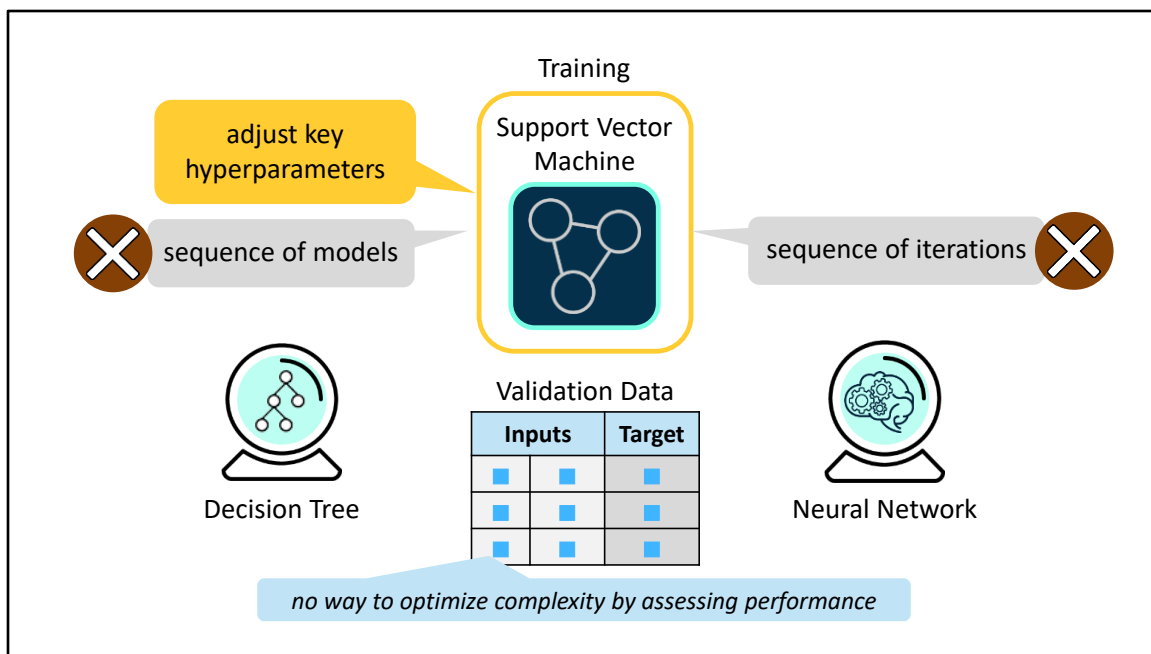
Improving the Support V Machine Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- Improve the model.
- Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.

Now that you have built a support vector machine model using the default settings, you are ready to refine it. For support vector machines, there is no clear distinction between the next two discovery tasks: improving the model and optimizing complexity.



Training a support vector machine does not involve a sequence of models, as with decision trees, or a sequence of iterations, as with neural networks. This means that, for support vector machines, there is no way to optimize complexity by assessing performance on validation data across multiple models or iterations.

Instead, you can adjust a few key hyperparameters and then look at the performance of the support vector machine model on validation data.

increase the
model's flexibility

Critical Options in Model Studio

Penalty

a term that accounts for misclassification errors in model optimization

Kernel

a mathematical function that operates as a dot product on transformed data in a higher dimension

Tolerance

a value that balances the number of support vectors and model accuracy

In this lesson, you learn to increase the flexibility of a support vector machine model by modifying the settings of three critical options in Model Studio: penalty, kernel, and tolerance.

The penalty is a term that accounts for misclassification errors in model optimization.

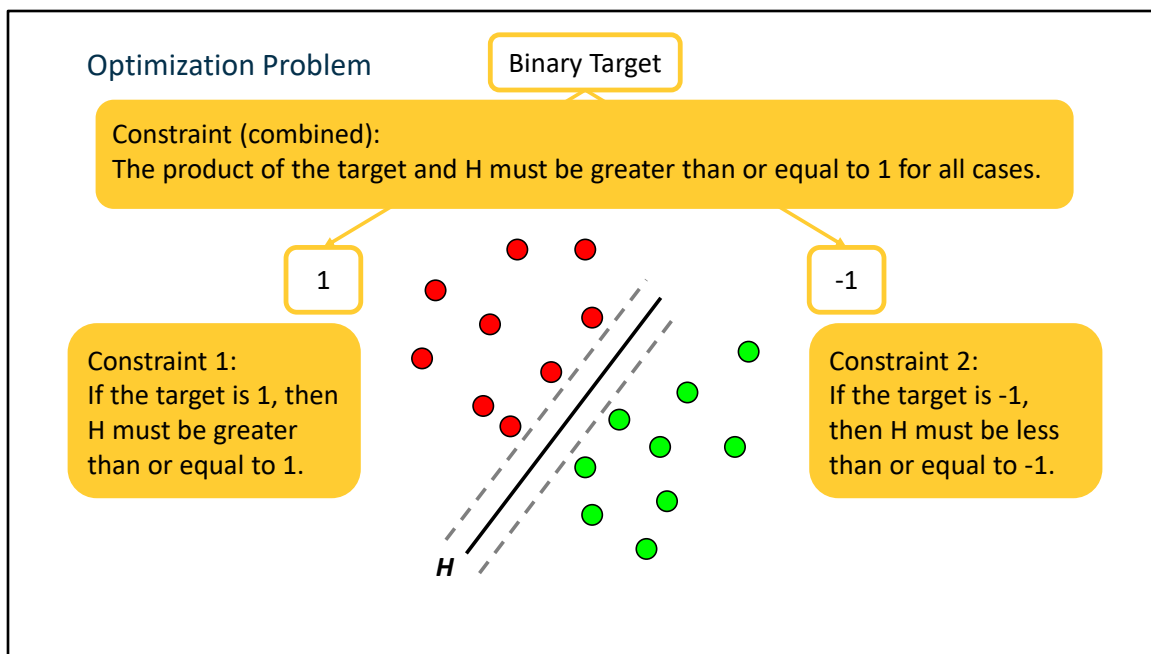
The kernel is a mathematical function that operates as a dot product on transformed data in a higher dimension.

The tolerance value balances the number of support vectors and model accuracy.

Optimization Problem



Copyright © SAS Institute Inc. All rights reserved.



To find the best solution to a classification problem, we want to optimize the location of the hyperplane. So, solving for the support vector machine is actually an optimization problem with two constraints.

For mathematical convenience, we define the binary target as either +1 or -1. You learn why a bit later.

The first constraint is that if the target is +1, then the solution for H must be greater than or equal to +1, because it must fall on the correct side of the hyperplane. The second constraint is that if the target is -1, then H must be less than or equal to -1 in order to fall on the correct side of the hyperplane. Because the binary target is defined as +1 or -1, we can combine the two constraints into a single constraint.

The product of the target value and H must be greater than or equal to +1 for all cases. Why? Because we are multiplying either +1 and a value greater than or equal to +1, or -1 and a value less than or equal to -1. In both cases, the product is a positive value.

Constraints for Optimization

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Accounting for Errors with Nonlinearly Separable Data



Copyright © SAS Institute Inc. All rights reserved.

Accounting for Errors with Nonlinearly Separable Data

to account for errors use ...

penalty = $C * \text{distance to the hyperplane}$

error weight

soft-margin hyperplane

does make some
classification errors

margin
size



correct
classifications

Trade-Off

small C	larger margin, more misclassified points
large C	smaller margin, fewer misclassified points

H

Our example has now been modified so that the data are no longer linearly separable. When the data are not linearly separable, one solution is a support vector machine known as a soft-margin hyperplane. A soft-margin hyperplane does make some classification errors. It is the analyst's job to decide whether the performance on validation data is acceptable. In a two-dimensional input space, *soft margin* means that a line can separate most of the points but some errors occur.

We account for errors by using a penalty term in the optimization process. This penalty is the product of two quantities: an error weight, which is a regularization parameter often denoted by C , and the distance between a point in error and the hyperplane.

In hard margin SVM, the margin constraints are "hard", and not allowed to be violated at all. You can vary C in order to balance the goals of maximum margin separation, and correctness of the training set classification. You can achieve this trade-off for various values of the parameter C . As C increases we care less about the size of the margin, and more about keeping the error distances small.

Number of Support Vectors

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Lagrange Approach for Estimation

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Modifying the Methods of Solution Parameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Nonlinearly Separable Data



Copyright © SAS Institute Inc. All rights reserved.

Question: Nonlinearly Separable Data

When the data are not linearly separable, the process of optimizing the location of the hyperplane must account for classification errors.

- a. True
- b. False

Answer: Nonlinearly Separable Data

When the data are not linearly separable, the process of optimizing the location of the hyperplane must account for classification errors.

- ☒ a. True
- ☐ b. False

CA: a

When the data are not linearly separable, the hyperplane will misclassify some data points. In this situation, the process of optimizing the location of the hyperplane must account for these classification errors.

5.3 Modifying the Model: Kernel Function



Optimizing the Complexity of the Support Vector Machine Model



Copyright © SAS Institute Inc. All rights reserved.

Optimizing the Complexity of the Support Vector Machine Model



Essential Discovery Tasks

- ✓ Select an algorithm.
 - Improve the model.
 - Optimize the complexity of the model.
 - Regularize and tune the hyperparameters of the model.
- penalty term
- kernel function

In the previous demonstration, we attempted to refine the model by changing the penalty term. Remember that the penalty term accounts for misclassification errors that inevitably occur when the data are not linearly separable.

Suppose we decide that our model's performance is not yet satisfactory. We want to go beyond a soft-margin classifier.

Next, we apply a kernel function to improve classification.

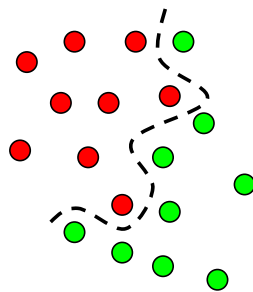
Feature Space Approach for Nonlinearly Separable Data



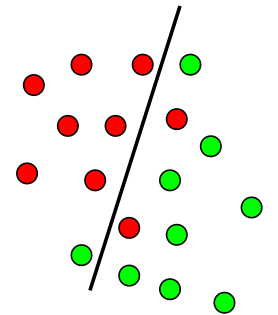
Copyright © SAS Institute Inc. All rights reserved.

Feature Space Approach for Nonlinearly Separable Data

Two-Dimensional
Input Space



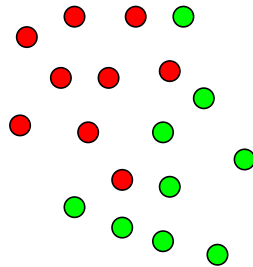
Soft-Margin Classifier



high misclassification rate!

Here is a new, more realistic version of the two-dimensional, red-versus-green example. In this example, even a soft-margin classifier would result in too many errors.

Two-Dimensional Input Space



Feature Space Approach

1. Transform the data to a higher dimension (the feature space).
For example:

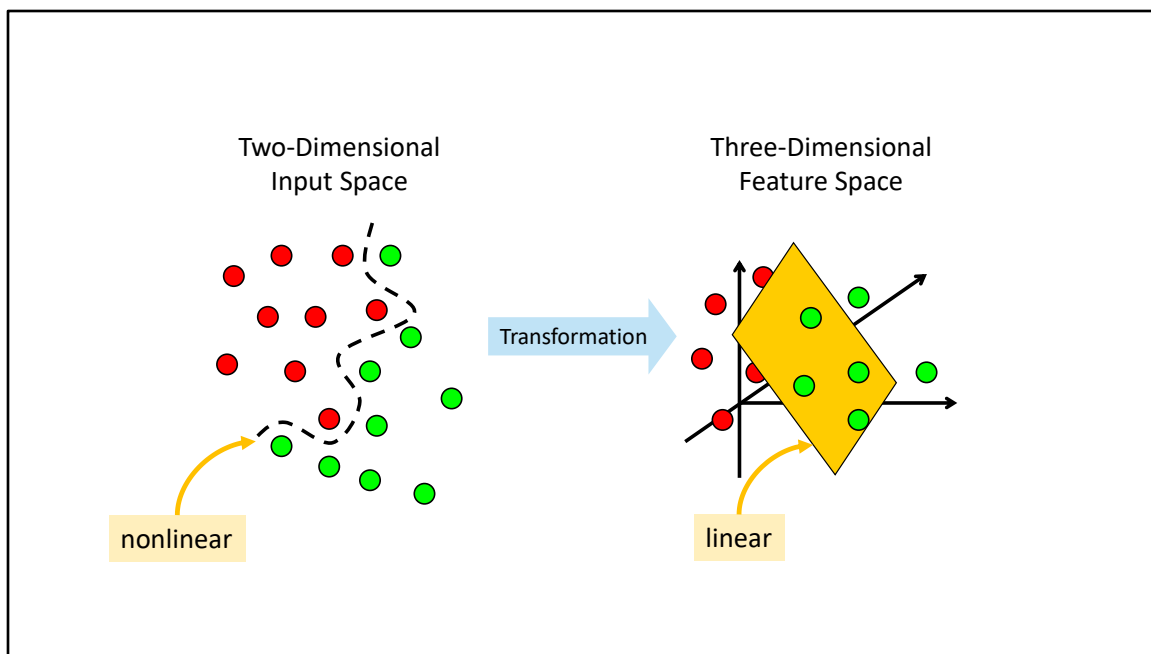
Input space	Feature space
1-D: X	2-D: X, X^2

2. Find the classifier (the maximum-margin hyperplane) in the higher dimension.

Let's look at a different approach to optimizing the complexity of the model: the feature space approach. This approach is viable for most real-world cases.

First, we transform the data – that is, we project the data to a higher dimension using a nonlinear transformation. This higher dimension is called the *feature space*. In a very simple case, with one input, we can transform the input by squaring.

Then we can use two inputs (both the original input and its squared form) to find the classifier (that is, the maximum-margin hyperplane) in this higher dimension.



In the red-versus-green example, we can transform the data into a three-dimensional feature space, where a plane separates the red and green dots.

Kernel Trick



Copyright © SAS Institute Inc. All rights reserved.

Kernel Trick

mathematical challenge!

dot product

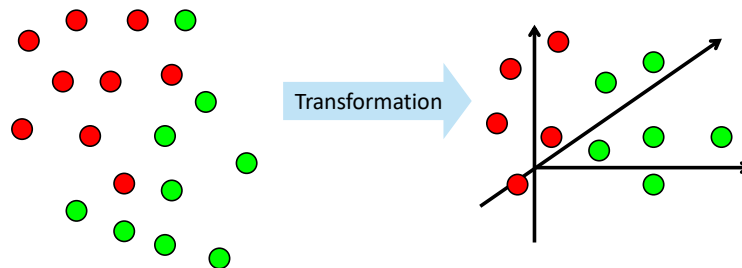
$$\langle x_i, x_j \rangle$$

$$\langle \Phi(x_i), \Phi(x_j) \rangle$$

difficult to calculate

Two-Dimensional
Input Space

Three-Dimensional
Feature Space



Optimizing the complexity of the model in a higher-dimensional feature space presents a mathematical challenge. Dot products are required to solve for H and to score new observations. If the data are transformed to a higher-dimensional feature space, then a dot product is required on the transformed data. A dot product on transformed data has a different form and is difficult to calculate.

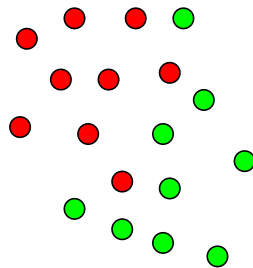
math
trick!

Kernel Function

$$K(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$$

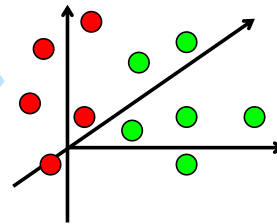
- exists in the original input space
- avoids calculating dot products on transformed data

Two-Dimensional
Input Space



Transformation

Three-Dimensional
Feature Space



To avoid having to calculate dot products on transformed data, we can use a math trick called a kernel function instead. The kernel function exists in the original input space, but it is equivalent to a dot product on the transformed data in the higher-dimensional feature space. We do not need to know the transformation itself! We call this a trick because we do not need to know exactly what the feature space looks like. It is enough to specify the kernel function as a measure of similarity. The geometric interpretation remains the same because the solution is still a hyperplane.

Kernel Function

$$K(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$$

Linear

$$k(x_1, x_2) = \langle x_1, x_2 \rangle$$

Polynomial

$$k(x_1, x_2) = (\langle x_1, x_2 \rangle + 1)^p$$

Sigmoid

$$k(x_1, x_2) = \tanh(p_1 \langle x_1, x_2 \rangle + p_2)$$

Radial Basis Function (RBF)

$$k(x_1, x_2) = \exp\left(-\frac{\|x_1 - x_2\|^2}{2\sigma^2}\right)$$

where x_1 and x_2 are two vectors and $\langle, \rangle$ is the inner product

As you'll see in the next demonstration, Model Studio provides two kernel functions, linear and polynomial. Technically, a linear kernel does not apply a transformation to the data. A polynomial kernel raises the data to a power.

Now there are two additional kernels Radial basis function and Sigmoid are available. Sigmoid kernel is equivalent to an input layer activation function used in a multilayer perceptron neural network. RBF is used to perform transformation when there is no prior knowledge about data.

SVM Properties Related to Kernels and Training Algorithms

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Increasing the Flexibility of the Support Vector Machine

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Kernel Functions



Copyright © SAS Institute Inc. All rights reserved.

Question: Kernel Functions

Which of the following statements is true about kernel functions?

- a. A kernel function must be a polynomial.
- b. A kernel function is used to account for errors during the optimization process.
- c. A kernel function operates as a dot product in a higher dimension, but it is applied to the raw data.

Answer: Kernel Functions

Which of the following statements is true about kernel functions?

- a. A kernel function must be a polynomial.
- b. A kernel function is used to account for errors during the optimization process.
- ☒ c. A kernel function operates as a dot product in a higher dimension, but it is applied to the raw data.

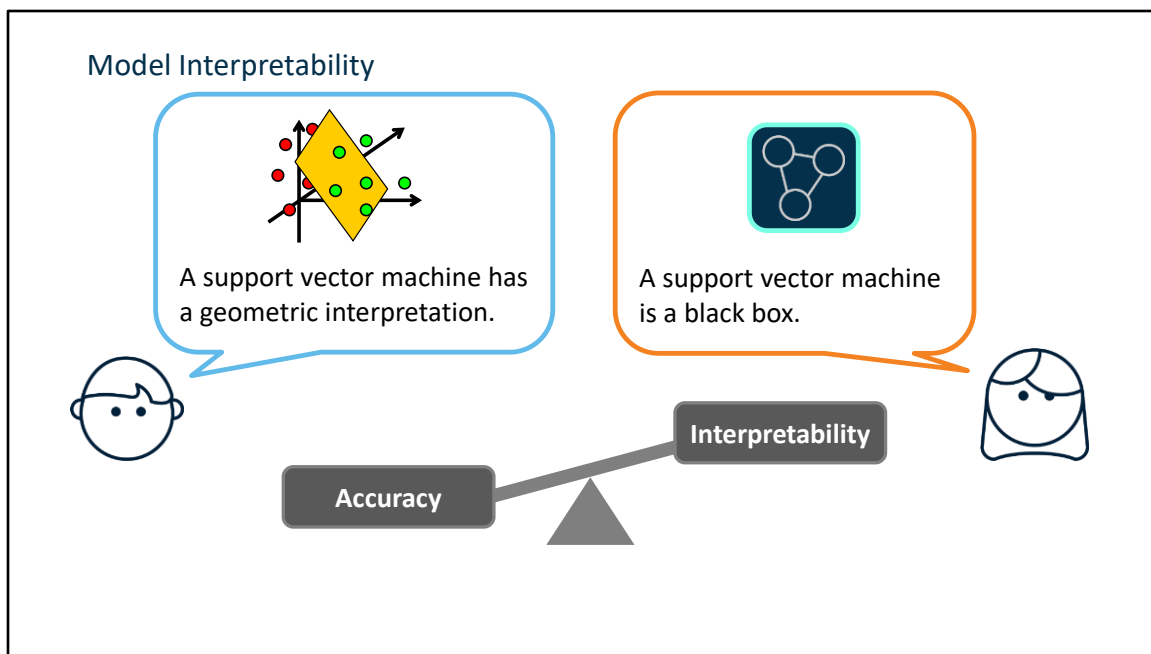
CA: c

A kernel function operates as a dot product in a higher dimension (that is, in a feature space), but it is applied to the raw data.

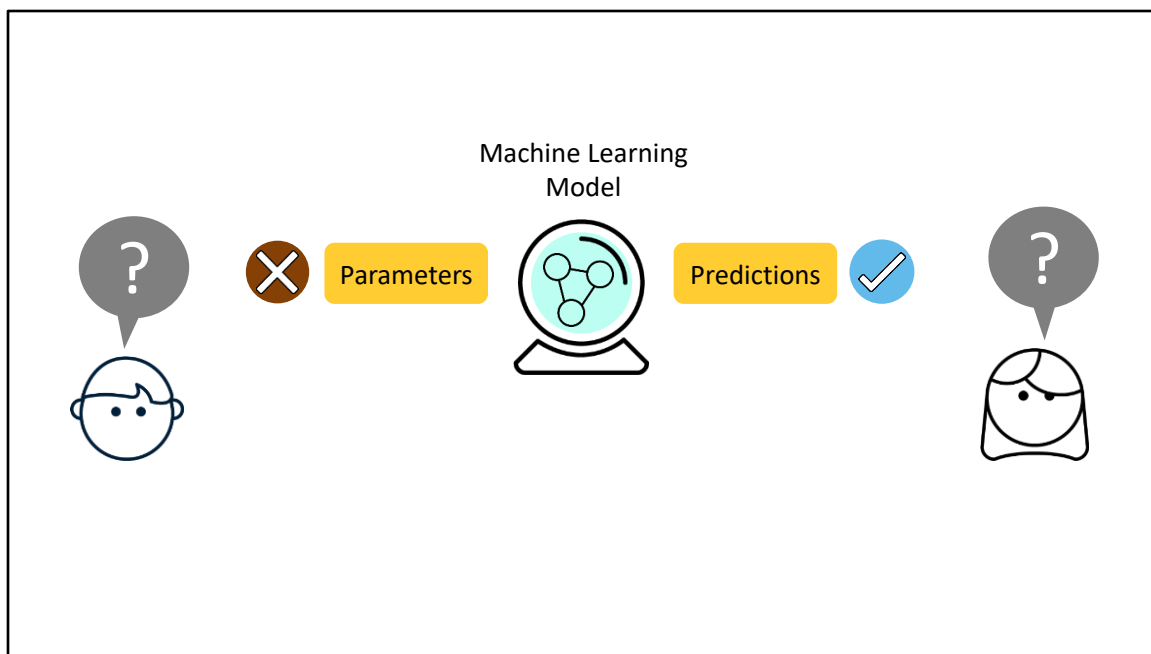
Model Interpretability



Copyright © SAS Institute Inc. All rights reserved.



In theory, support vector machines have a geometric interpretation because they are hyperplanes. However, in practice, many analysts consider support vector machine models to be black boxes, much like neural networks. Thus, there is often a trade-off between model accuracy and model interpretability.



Machine learning algorithms are good at generating accurate and generalizable predictive models, but it is often nearly impossible to understand how the algorithms arrived at those predictions or even what the behavior or trend of the model is. We cannot gain much insight into support vector machines, and some other machine learning models, by analyzing the model parameters. However, we can learn a great deal by analyzing the predictions from these models. In the next demonstration, you learn how to use Model Studio's model interpretability feature to interpret your results.

Model Interpretability Plots

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Adding Model Interpretability

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Model Interpretability



Copyright © SAS Institute Inc. All rights reserved.

Question: Model Interpretability

Which of the following machine learning models is the easiest to interpret?

- a. decision tree
- b. neural network
- c. support vector machine

Question: Model Interpretability

Which of the following machine learning models is the easiest to interpret?

- ☒ a. decision tree
- b. neural network
- c. support vector machine

CA: a.

Decision trees are highly interpretable because they are based on English rules, which are rules that use Boolean logic. However, Model Studio's interpretability feature is available for all machine learning models, including decision trees. You might want to use this feature, for example, when you are comparing decision trees to other types of models.

Regularizing and Tuning the Hyperparameters of the Support Vector Machine Model



Copyright © SAS Institute Inc. All rights reserved.

Regularizing and Tuning the Hyperparameters of the Support Vector Machine Model



Essential Discovery Tasks

- ✓ Select an algorithm.
- ✓ Improve the model.
- ✓ Optimize the complexity of the model.
- Regularize and tune the hyperparameters of the model.

Autotuning is disabled in SAS Viya for Learners, and selecting it results in an error.

An important discovery task is to regularize and tune the hyperparameters of the model. As with other types of models, we can automate the process of discovering the optimal settings for the support vector machine model. Next, you learn about the hyperparameters for support vector machines that you can autotune in Model Studio.

Autotuning Options for Support Vector Machines in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



(Optional) Practice: Build a Support Vector Machine Model Using Autotuning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



5.4 Lesson Conclusion



Lesson Quiz

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Lesson 6: Model Assessment and Deployment



6.0 Lesson Overview



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



6.1 Model Assessment and Comparison



Essential Deployment Tasks



Copyright © SAS Institute Inc. All rights reserved.

Essential Deployment Tasks



Essential Deployment Tasks

- Assess models.
- Compare models.
- Score the champion model.
- Assess model bias.
- Monitor model performance over time.
- Update the model as needed.

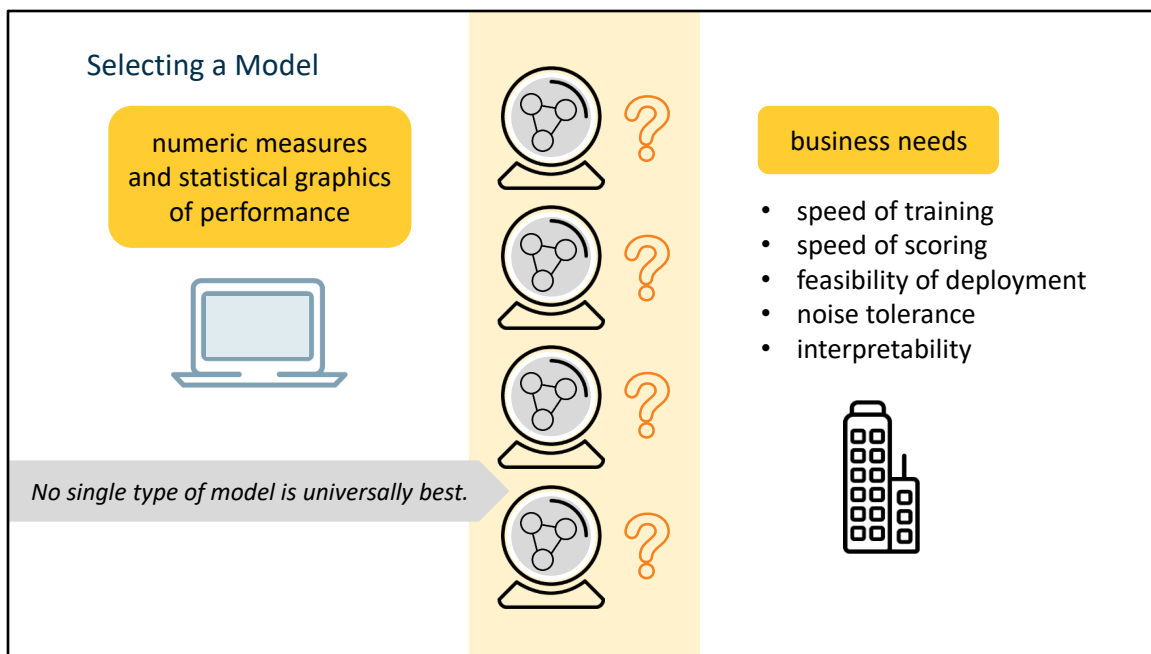
We are now at the Deployment phase of the analytics life cycle. The essential tasks for deployment start with assessing the individual models. Next, we compare the candidate models and select the one that best represents the association between the inputs and the target. This model is typically called the champion model.

Later in this lesson, you learn how to put the model into production (a process called scoring). Then you learn about assessing model bias, monitoring the model's performance over time and updating the model as needed.

Selecting a Model



Copyright © SAS Institute Inc. All rights reserved.



When selecting a champion model, it's important to remember that no single type of model is always the best (that is, in all situations and over time).

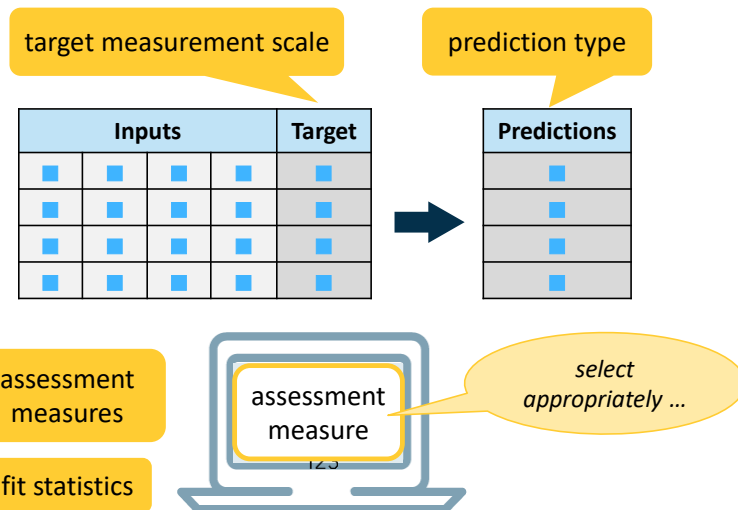
You select a model primarily based on numeric measures and statistical graphics of performance. However, you must also consider the needs of the business situation. Common business factors include the speed of training the model (for example, if the model will often need to be rebuilt), the speed of scoring, the feasibility of deployment, the noise tolerance (that is, the strength of the signal in the data), and interpretability (that is, how easy it is to explain the model's predictions).

Numeric Measures of Model Performance



Copyright © SAS Institute Inc. All rights reserved.

Numeric Measures of Model Performance by Prediction Type



Numeric measures used to evaluate model performance are called assessment measures or fit statistics.

To compare models, you select an appropriate assessment measure based on two factors: the target measurement scale and the prediction type.

An assessment measure that is appropriate for a binary target might not make sense for an interval target. Similarly, an assessment measure that is suitable for decision predictions cannot be used for estimate predictions.

Binary Target

Prediction Type	Assessment Measures
Decision	• Accuracy • Misclassification • Profit/loss
Ranking	• C-statistic • Gini coefficient
Estimate	• Average square error • Root mean square error • Schwarz criterion (SC) • Akaike's information criterion (AIC)

Shown here are some common assessment measures for a binary target, given different prediction types. For decision predictions, you might choose from accuracy, misclassification, or profit or loss. For rankings, you might choose from the C-statistic (which is the area under the ROC curve) or the Gini coefficient. And for estimates, you might choose from the average square error, root mean square error, Schwarz criterion, or Akaike's information criterion.

You learn more about assessment measures later.

Numeric Measures of Model Performance by Prediction Type

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Confusion Matrix for Decision Predictions



Copyright © SAS Institute Inc. All rights reserved.

Confusion Matrix for Decision Predictions

		Predicted Outcome		
		Primary	Secondary	
Actual Outcome	Primary	TP	FN	TP + FN
	Secondary	FP	TN	FP + TN

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

$$\text{Specificity} = \frac{TN}{TN + FP}$$

The confusion matrix is a common format for calculating and displaying assessment measures of model performance for decision predictions on a binary target. This matrix is a crosstabulation of the actual and predicted outcomes, based on a decision rule. A simple decision rule allocates cases to the target class with the greatest posterior (that is, predicted) probability. For binary targets, this corresponds to a 50% cutoff on the posterior probability.

A confusion matrix displays four counts: true positives, true negatives, false positives, and false negatives. A true positive (or TP) is a case known to be a primary outcome and also predicted as a primary outcome. A true negative (or TN) is a known secondary case predicted as a secondary case. A false positive (or FP) is a case that is predicted as a primary outcome but is actually a known secondary outcome. And a false negative (or FN) case is a case that is predicted as a secondary outcome but known to be a primary outcome.

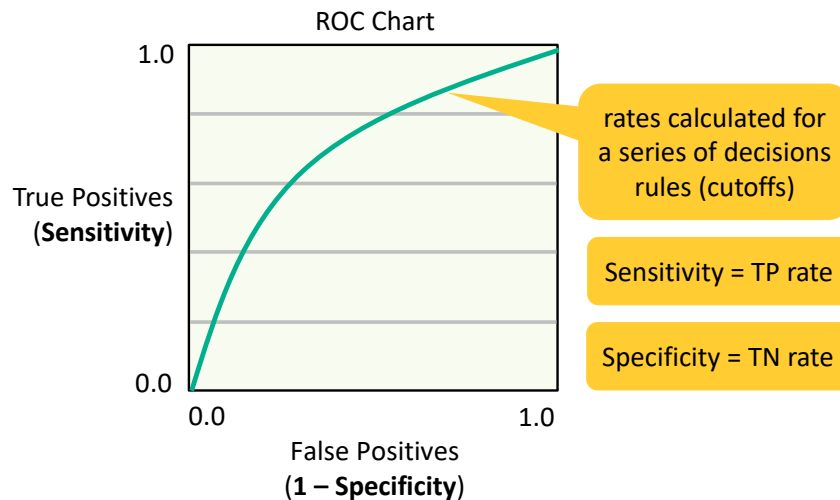
True and false positives and true and false negatives are used to calculate various assessment measures, such as sensitivity and specificity. Sensitivity, the true positive rate, is the number of true positive decisions divided by the total number of known primary cases. Specificity, the true negative rate, is the number of true negative decisions divided by the total number of known secondary cases. These measures are the basis for the ROC chart, which you learn about next.

ROC Charts and the C-Statistic



Copyright © SAS Institute Inc. All rights reserved.

ROC Charts and the C-Statistic

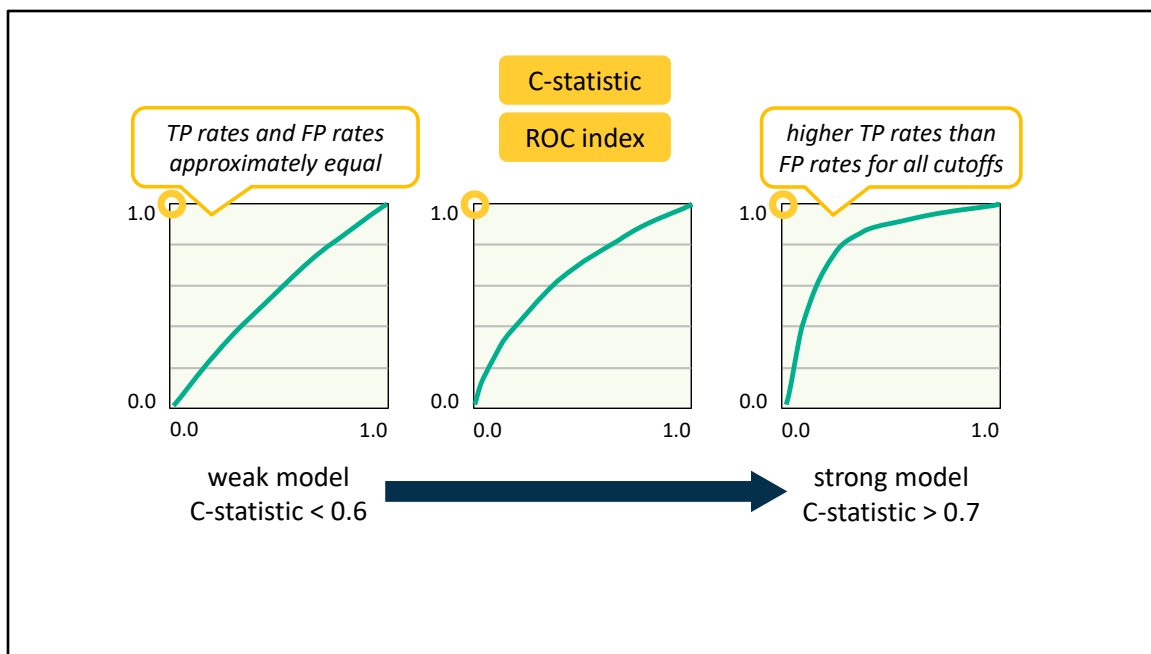


The ROC chart is a commonly used graphical representation of model performance for a binary target. ROC stands for receiver operating characteristic. ROC charts are based on measures of sensitivity and specificity. The sensitivity of a model is the true positive rate. The specificity of a model is the true negative rate.

In the ROC chart, the Y axis represents the cumulative rate of true positives (that is, **sensitivity**). The X axis represents the cumulative rate of false positives, which is **1 - Specificity**. The rate on each axis ranges from 0 to 1, so the plot is contained within a unit square. The rates are calculated for a series of decision rules (in other words, cutoffs) between 0% and 100%. However, the cutoffs are not shown in the chart.

The ROC chart plots the relationship between the change in the true positive rate and the change in the false positive rate. The closer the true positive rate is to 1 and the closer the false positive rate is to zero, the better the model. In other words, the higher the value of sensitivity is and the lower the value of 1-specificity is, the better the model.

What does this look like in the ROC chart?

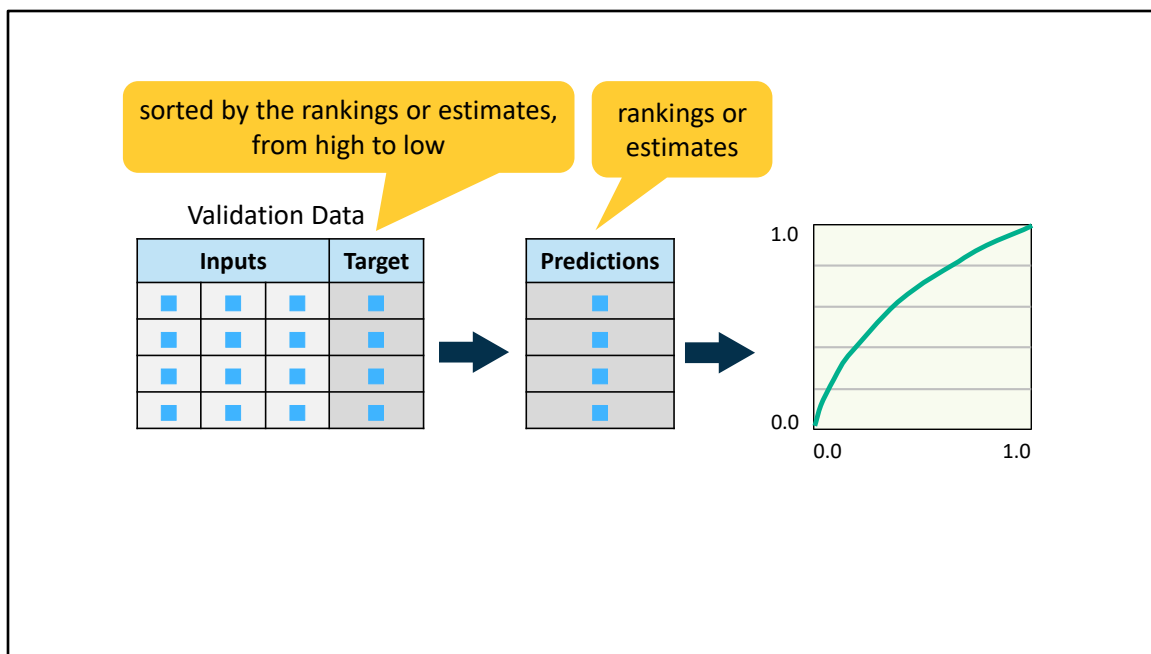


The closer the ROC curve is to the upper left corner of the chart, the better the classification accuracy of the model.

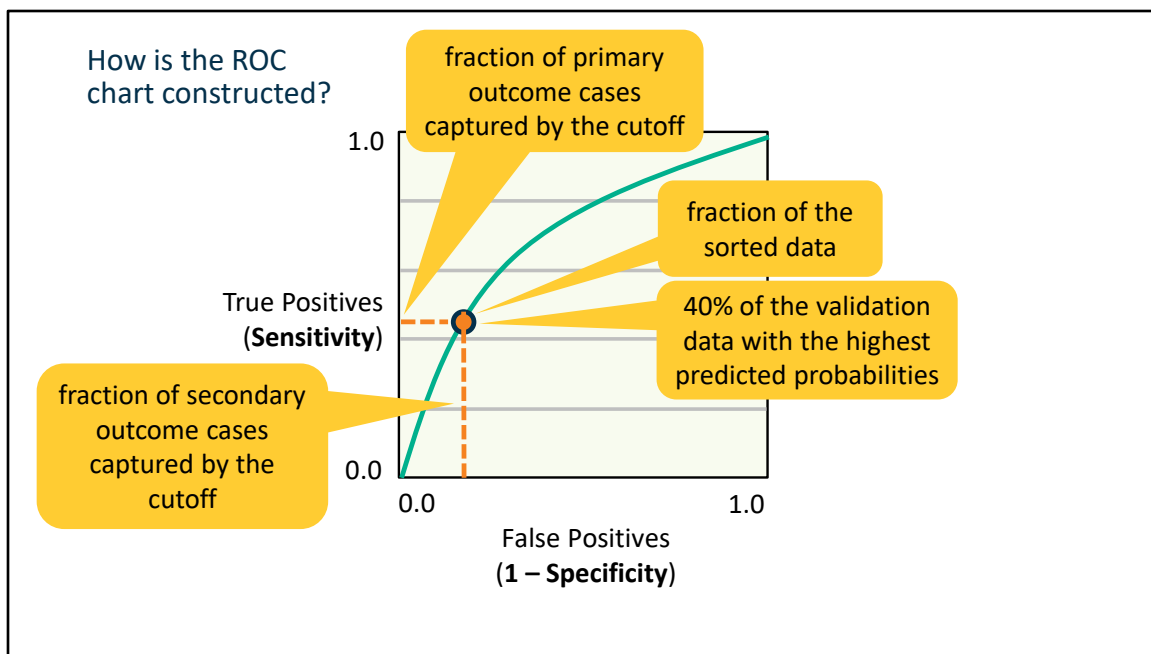
The area under the ROC curve is a measure also known as the C-statistic or the ROC index. The larger the area under the curve, the higher the C-statistic and the better the model's classification accuracy.

Models whose true positive rates and false positive rates are approximately equal are weak models. As the chart on the left shows, a weak model has a curve that is close to the diagonal between the lower left corner and the upper right corner. This weak model has a C-statistic less than 0.6.

Models that have much higher true positive rates than false positive rates for all cutoffs are strong models, as shown in the chart on the right. Notice that the curve approaches the upper left corner. This strong model has a C-statistic greater than 0.7. The area under the ROC curve summarizes the tradeoff between the primary versus secondary cases that are captured.



It is easier to interpret ROC charts when you understand how they are constructed. To create an ROC chart, predictions are generated for a set of validation data. These predictions must be rankings or estimates. Then the validation data are sorted by the ranks or estimates, from high to low. This creates a list of cases, sorted in order of importance.



Each point on the ROC chart corresponds to a specific fraction of the sorted data.

The sample point on this ROC chart corresponds to the selection of 40% of the validation data with the highest predicted probabilities. The selection value of this point cannot be seen from the chart.

The vertical, or y, coordinate of the point indicates the fraction of primary outcome cases captured by the cutoff (here, approximately 45%).

The horizontal, or x, coordinate of the point indicates the fraction of secondary outcome cases captured by the cutoff (here, approximately 25%).

The ROC chart represents the combination of similar calculations for all selection fractions.

Question: C-Statistic



Copyright © SAS Institute Inc. All rights reserved.

Question: C-Statistic

To use the C-statistic to assess model performance, you need to know the cutoff.

- a. True
- b. False

Answer: C-Statistic

To use the C-statistic to assess model performance, you need to know the cutoff.

- a. True
- ☒ b. False

CA: b

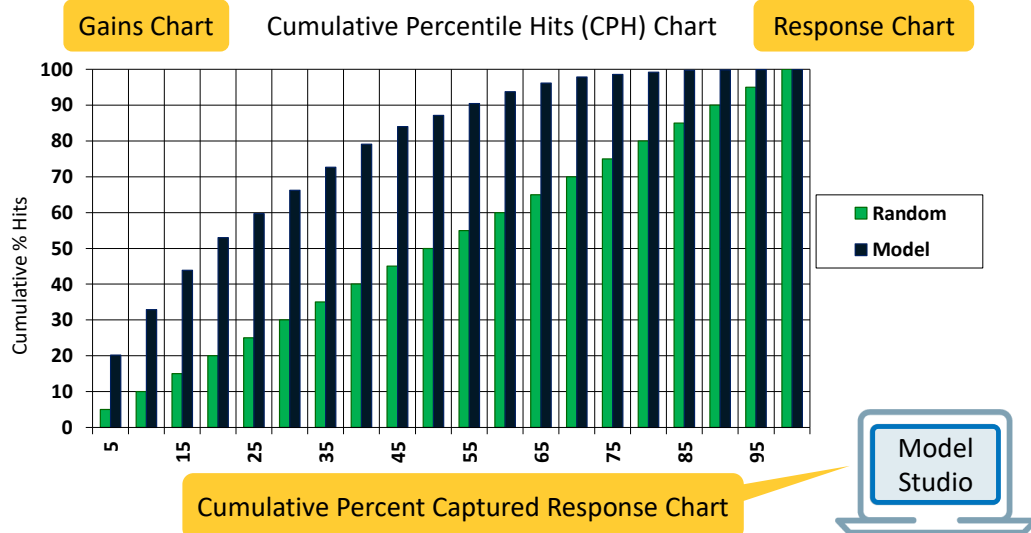
You do not need to know the cutoff to use the C-statistic. The cutoff is used in the calculation of the C-statistic.

Charts Based on Response Rate: CPH and Lift



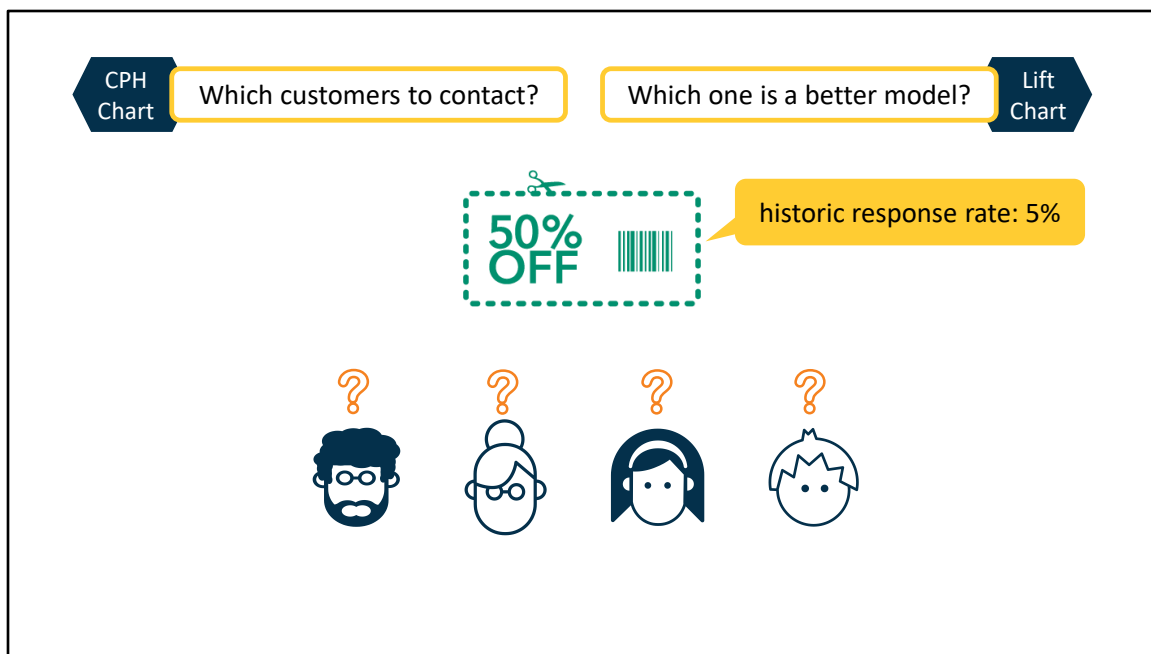
Copyright © SAS Institute Inc. All rights reserved.

Charts Based on Response Rate: CPH and Lift



The cumulative percentile hits chart (or CPH chart) illustrates the advantage of using a predictive model to make business decisions as compared to not using a model (that is, having a baseline or random model). The CPH chart is also known by other names, such as gains chart or response chart.

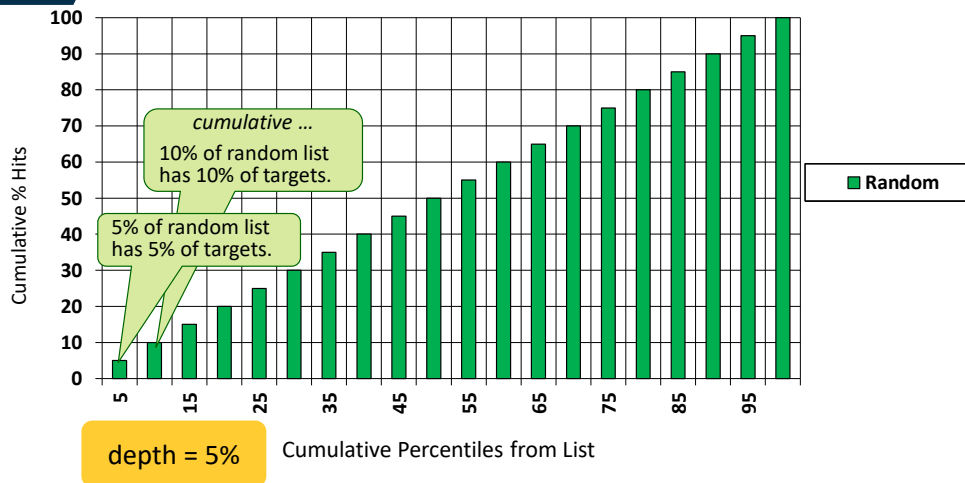
In Model Studio, this chart is called the Cumulative Percent Captured Response chart.



Suppose we want to choose which customers to contact for a direct marketing campaign. The historic response rate is 5%.

CPH
Chart

Which customers to contact?



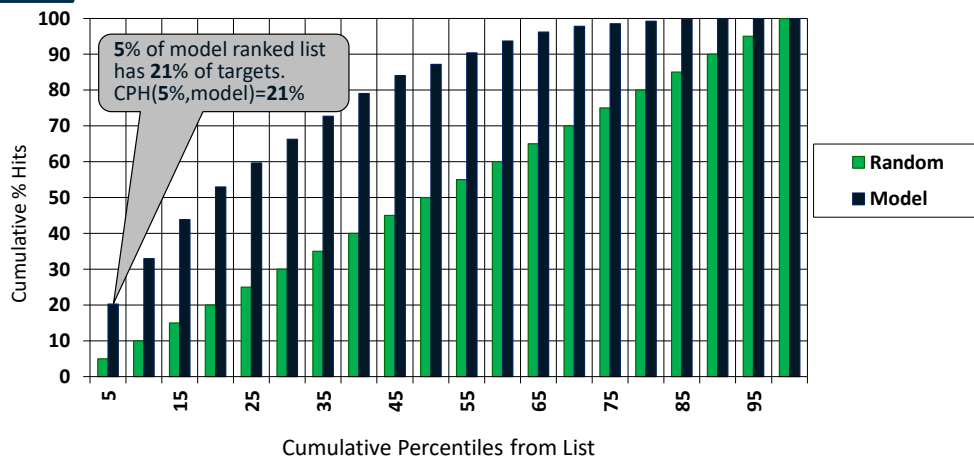
This preliminary chart shows the cumulative percentile hits when we do not use a model. The Y axis represents the cumulative percentile hits. The X axis represents the cumulative percentiles from the list of cases (in this example, customers). This particular chart has 20 percentiles (that is, 20 bars).

The first bar shows that 5% of the event cases are captured in the top 5% of customers. The selected fraction of cases (in this example, 5%) is known as the depth.

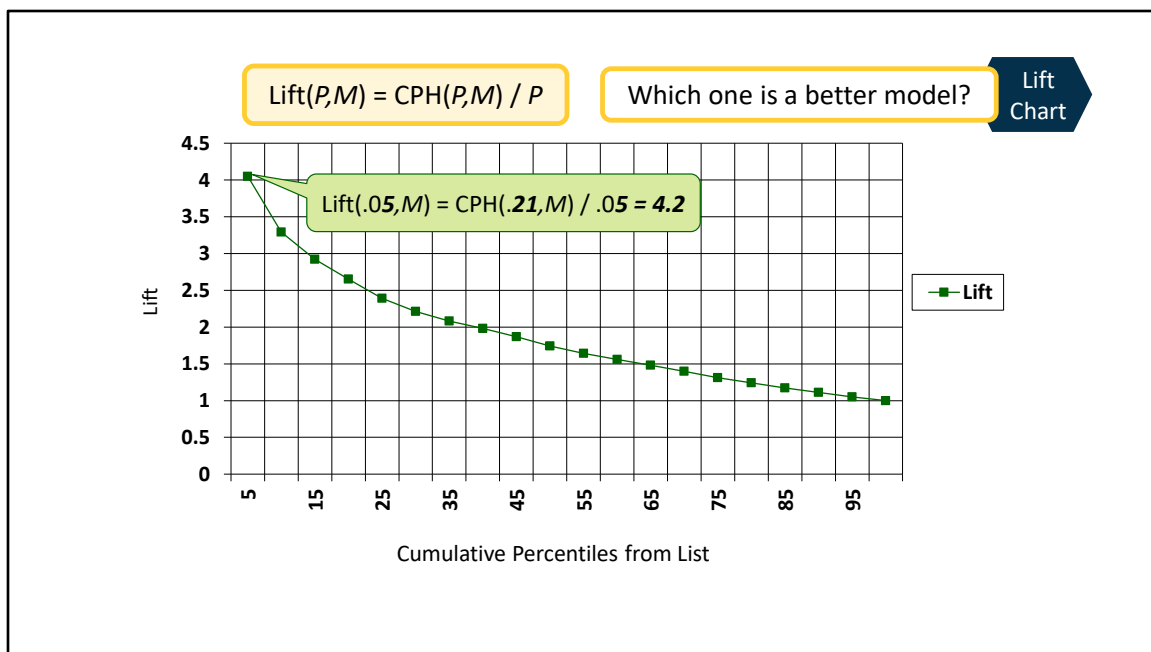
Because the graph is cumulative, the second bar shows that 10% of the event cases are captured in the top 10% of customers.

CPH Chart

Which customers to contact?



The cumulative percentile hits based on a predictive model are now added to the chart. For the top 5% of customers ranked on their posterior probabilities, the model found that 21% responded. For the top 10% of customers (again, ranked by their posterior probabilities), the model found that 32% responded, when calculated cumulatively. For all fractions of customers, the chart shows that the model captures more responders than not using a model.

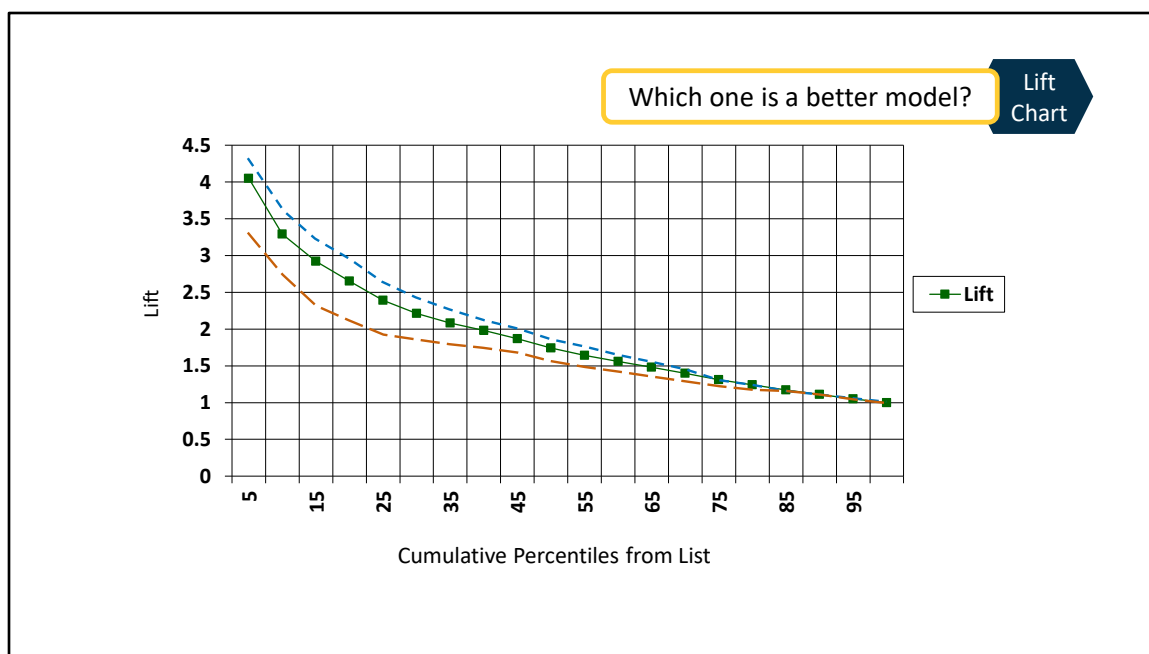


A cumulative lift chart is a graphical tool that illustrates the advantage of using a predictive model as compared to not using a model. A cumulative lift chart is often called a lift chart, as you will hear in this course. Cumulative lift is a ratio of response rates.

The response rate in the numerator equals the cumulative percentile hits for a given percentile (P) from a given model (M). The response rate in the denominator equals the cumulative percentile hits for the same percentile given no model.

In the customer response example, the CPH for the top 5% of customers, given the model, was 21%. Given no model, the response rate for the top 5% of customers (from a randomly ordered list) is 5%.

Thus, the lift at 5% is 21% divided by 5%, which is 4.2. This indicates that, for the top 5% of customers, the model captured 4.2 times as many responders, compared to not using a model.



Here is the lift chart for the customer response example. The Y axis represents the lift and the X axis represents the cumulative percentiles from the list.

A lift chart can contain lines for multiple models, which is helpful for selecting the best model in a specific business scenario. The model with the higher lift curve is generally better for model deployment. However, the best model can vary depending on the percentile. For the customer response example, if you expect to contact 20% of your customers, you want to choose the model that performs best on this percentile.

Ways of Comparing Models in Model Studio



Copyright © SAS Institute Inc. All rights reserved.

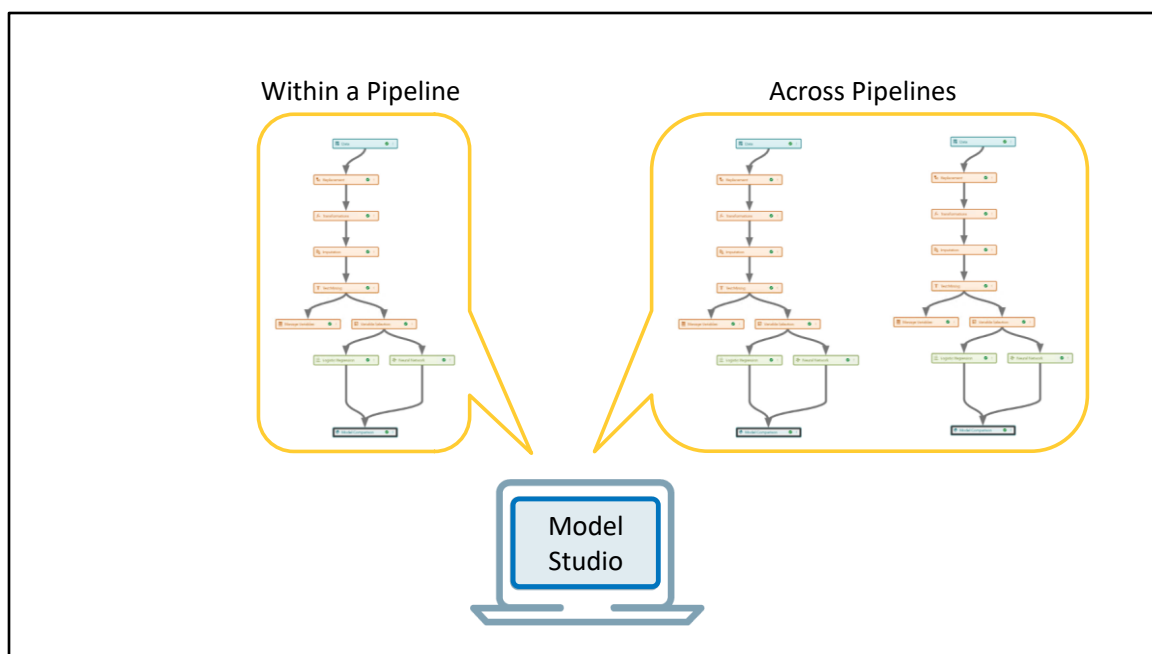
Ways of Comparing Models in Model Studio



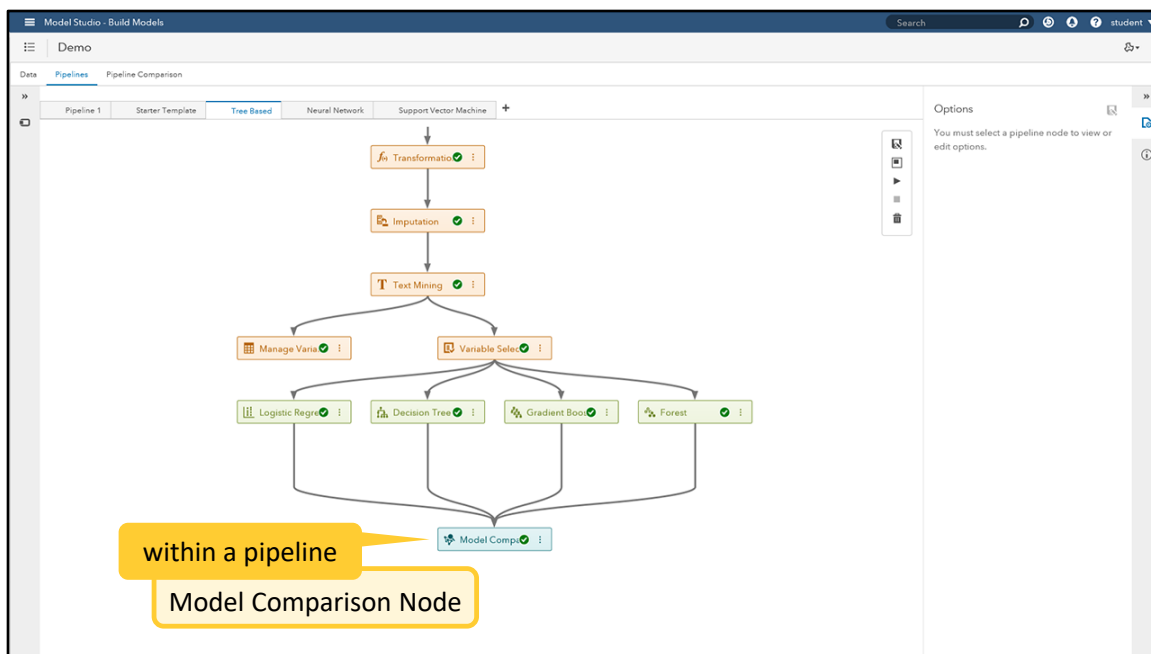
Essential Deployment Tasks

- ✓ Assess models.
- Compare models.
- Score the champion model.
- Assess model bias.
- Monitor model performance over time.
- Update the model as needed.

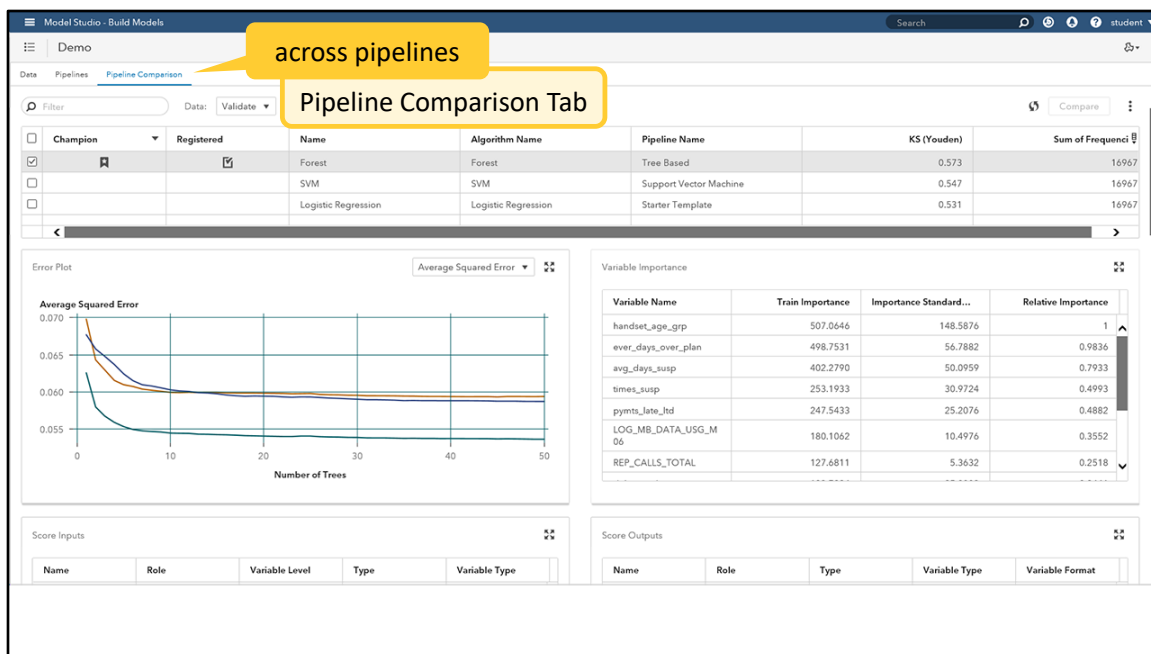
After assessing the individual models, it's time to compare them and select the champion model.



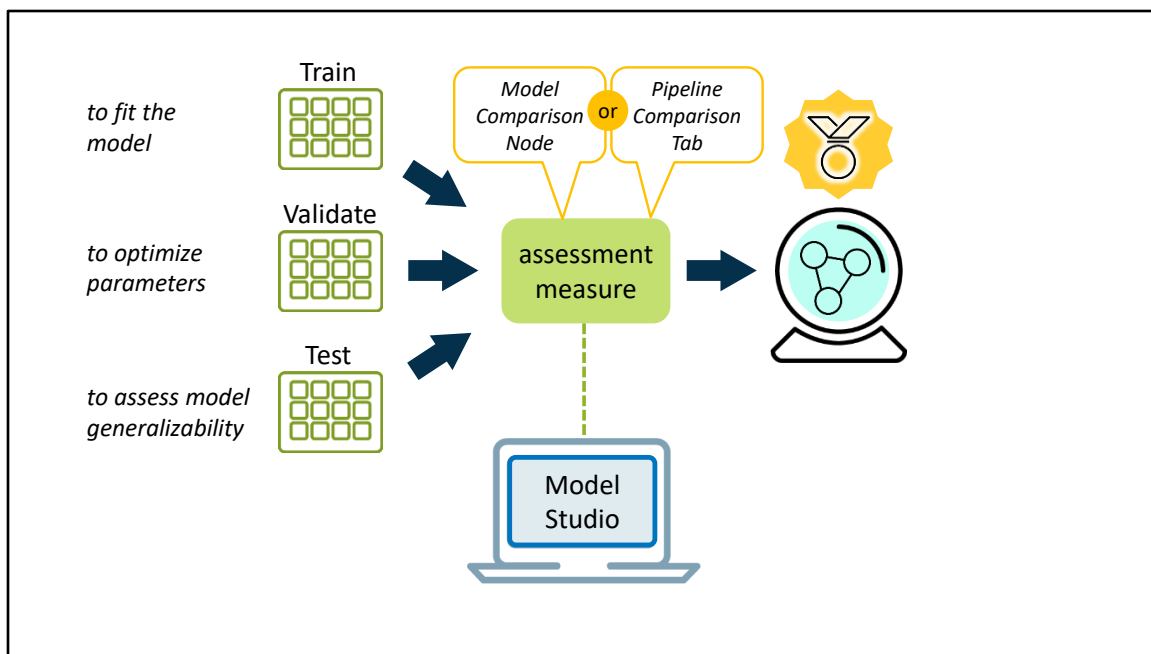
In Model Studio, you can compare models in two ways: within a pipeline and across pipelines.



To compare models within a pipeline, you use the Model Comparison node. Remember that when you add a node for any supervised learning model to a pipeline, the Model Comparison node is automatically added as well.



To compare models across pipelines, you use the Pipeline Comparison tab. By default, the Pipeline Comparison tab compares the champion models from each pipeline. However, if you want, you can add other models to the Pipeline Comparison tab.



To select a champion model either within a pipeline or between pipelines, Model Studio uses a default assessment measure, which varies by the type of target. If you want, you can specify a different assessment measure.

Model Studio computes all assessment measures for each available data partition (train, validate, and test). By default, Model Studio selects a champion model based on the validation data set unless a test data set is available. If you want, you can specify a different data set.

Question: Model Comparison Node



Copyright © SAS Institute Inc. All rights reserved.

Question: Model Comparison Node

In Model Studio, when you add any node to a pipeline, the Model Comparison node is added automatically.

- a. True
- b. False

Answer: Model Comparison Node

In Model Studio, when you add any node to a pipeline, the Model Comparison node is added automatically.

a. True

☒ b. False

CA: b

The Model Comparison node is automatically added only when you add a **supervised learning** node to a pipeline.

Demo: Comparing Models within a Pipeline

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Comparing Models across Pipelines

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Score Holdout Data (Out-of-Time Testing)

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Reviewing a Project Summary Report on the Insights Tab

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Registering the Champion Model

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Exploring the Settings for Model Selection

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Model Comparison Node Properties



Copyright © SAS Institute Inc. All rights reserved.

Question: Model Comparison Node Properties

Which of the following can you specify in the properties panel of the Model Comparison node? (Select all that apply.)

- a. the depth to use when applying a lift-based measure
- b. the cutoff to use when applying an ROC-based measure
- c. the data partition to use for selecting the champion

Answer: Model Comparison Node Properties

Which of the following can you specify in the properties panel of the Model Comparison node? (Select all that apply.)

- ☒ a. the depth to use when applying a lift-based measure
- ☒ b. the cutoff to use when applying an ROC-based measure
- ☒ c. the data partition to use for selecting the champion

CA: a,b,c

You can specify all these things in the properties panel of the Model Comparison node.

Model Selection Statistics by Target Type

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Model Selection Statistics



Copyright © SAS Institute Inc. All rights reserved.

Question: Model Selection Statistics

When you have an interval target, which of the following fit statistics can you use to select the champion model?

- a. area under the curve (C-statistic)
- b. average square error
- c. cumulative lift
- d. misclassification

Answer: Model Selection Statistics

When you have an interval target, which of the following fit statistics can you use to select the champion model?

- a. area under the curve (C-statistic)
- ☒ b. average square error
- c. cumulative lift
- d. misclassification

CA: b

You can use the average square error for both categorical (class) and interval targets. The other fit statistics shown here are applicable only to categorical targets.

6.2 Model Deployment



Scoring and Managing the Champion Model



Copyright © SAS Institute Inc. All rights reserved.

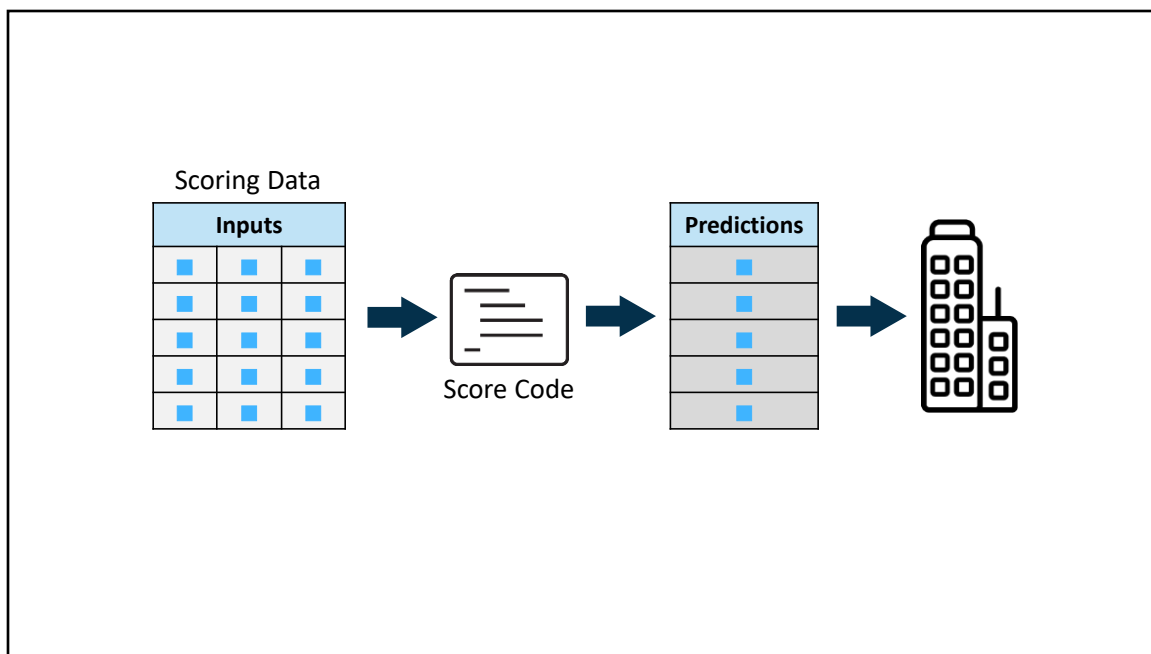
Scoring and Managing the Champion Model



Essential Deployment Tasks

- ✓ Assess models.
- ✓ Compare models.
- **Score the champion model.**
- Assess model bias.
- Monitor model performance over time.
- Update the model as needed.

After selecting a champion model, it's time to deploy the model -- that is, put it to use in a production environment. All tasks performed earlier in the analytics life cycle lead to this task: generating predictions through scoring.



For scoring, the model is first translated into another format (typically, score code). In the SAS Viya environment, score code is a SAS program that you can easily run on a scoring data set. The model is then applied to the scoring data set to obtain predicted outcomes. Based on the predictions from the model, the enterprise makes business decisions and takes action.

Score Code and Model Deployment

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Internally Scored Data Sets in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Introducing SAS Model Manager

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Demo: Viewing the Score Code and Running a Scoring Test

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



SAS Model Manager Tabs

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Assessing Model Bias



Copyright © SAS Institute Inc. All rights reserved.

Assess Model Bias



Essential Deployment Tasks

- ✓ Assess models.
- ✓ Compare models.
- ✓ Score the champion model.
 - Assess model bias.
 - Monitor model performance over time.
 - Update the model as needed.

Fairness and Bias

Every model application has the potential to affect separate segments of the user population differently. It's important for you to understand the biases present in your models to take proper mitigating actions. Bias assessment in Model Studio comes in the form of a new tab in the Results of a supervised learning model called Fairness and Bias. To view these reports for a particular classification variable, such as Sex, you must select the "Assess this variable for bias" option in the Data tab of a Model Studio project.

See the next page and the resources mentioned therein for more details.

Assessing Model Bias and Relevant Resources

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Monitoring and Updating the Model



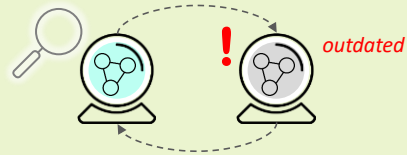
Copyright © SAS Institute Inc. All rights reserved.

Monitoring and Updating the Model

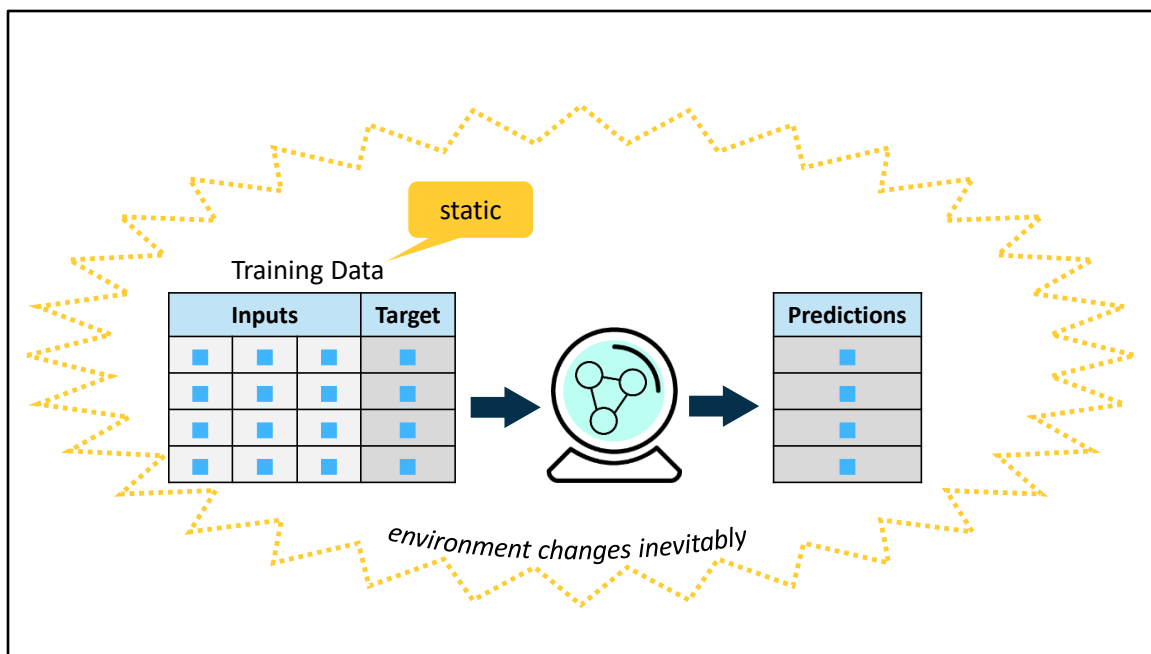


Essential Deployment Tasks

- ✓ Assess models.
- ✓ Compare models.
- ✓ Score the champion model.
- ✓ Assess model bias.
- Monitor model performance over time.
- Update the model as needed.

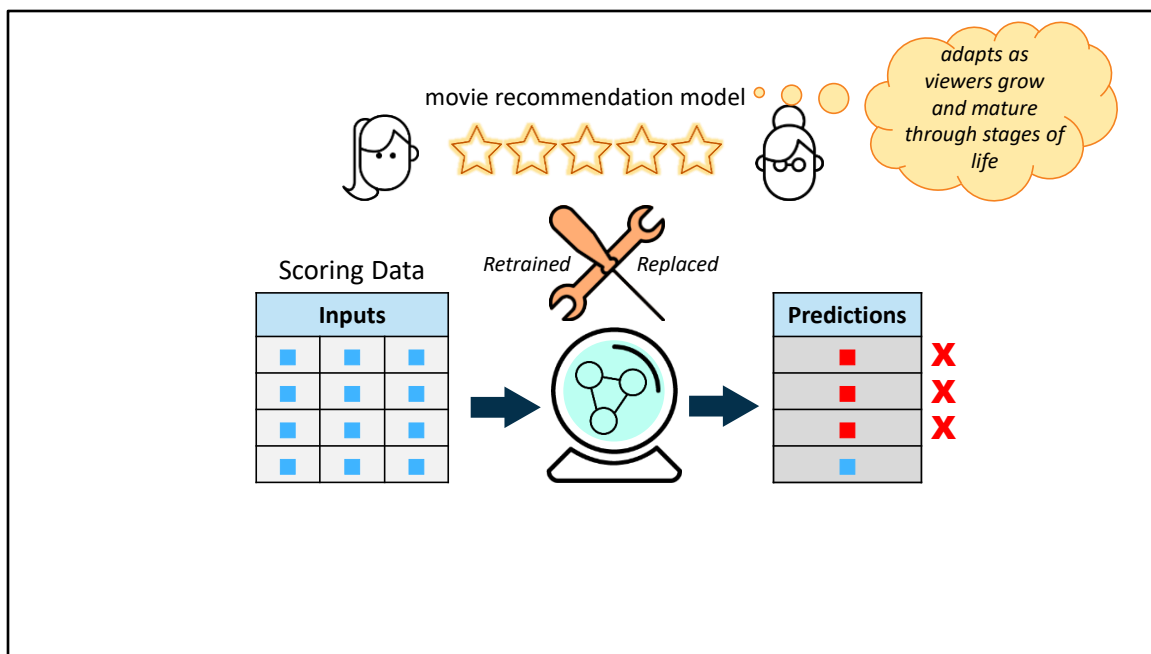


After a model is deployed, you must closely monitor its accuracy and performance over time. At some point, the model might become outdated. It is then that we would likely update the model so that it can continue to perform well. Let's discuss how this might happen.



Models are trained on static snapshots of data.

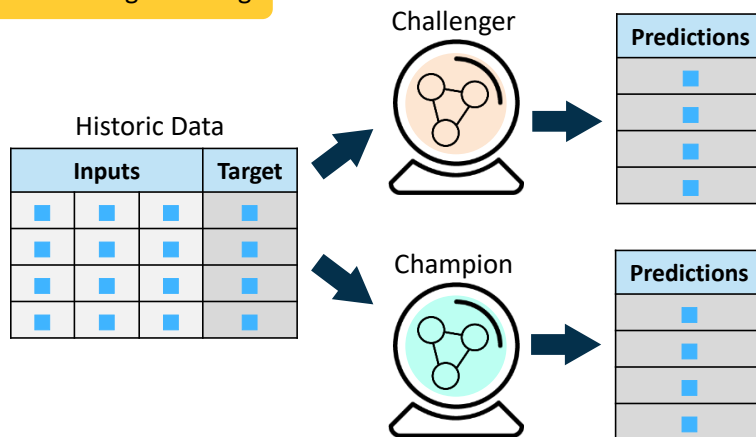
After the model is deployed, the environment inevitably changes and becomes less and less like the conditions that were captured in the training data.



As a result, the model's predictions typically become less accurate. For example, consider a movie recommendation model that must adapt as viewers grow and mature through stages of life.

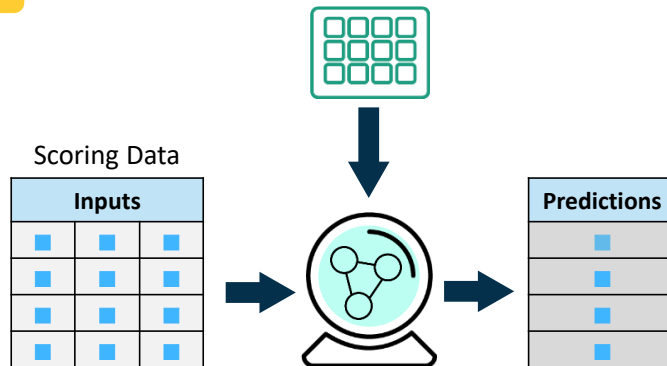
After a certain period, the error rate on new data surpasses a predefined threshold, and the model must be retrained or replaced.

Champion-Challenger Testing



Champion-challenger testing is a common model deployment practice. This method compares the performance of a new, challenger model with the performance of the deployed model on a historic data set at regular time intervals. If the challenger model outperforms the deployed model, the challenger model replaces it. The champion-challenger process is then repeated.

Online Updates



Another approach to refreshing a trained model is through online updates. Online updates continuously change the value of model parameters or rules based on the values of new, streaming data. It is prudent to assess the trustworthiness of real-time data streams before implementing an online modeling system.

Question: Scoring



Copyright © SAS Institute Inc. All rights reserved.

Question: Scoring

Which of the following statements about scoring are true?
(Select all that apply.)

- a. In the SAS Viya environment, score code is a SAS program.
- b. You use the validation data again for scoring.
- c. You can export score code from SAS Model Manager.

Answer: Scoring

Which of the following statements about scoring are true?
(Select all that apply.)

- ☒ a. In the SAS Viya environment, score code is a SAS program.
- ☐ b. You use the validation data again for scoring.
- ☒ c. You can export score code from SAS Model Manager.

CA: a,c

For scoring, you use scoring data (which is new data), not validation data (which was used during model building).

6.3 Lesson Conclusion



Lesson Quiz

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Lesson 7: Additional Nodes



7.0 Lesson Overview



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



7.1 Exploring Additional Nodes in Model Studio



Additional Nodes in Model Studio

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Save Data Node

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



SAS Code Node

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Open Source Code Node

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Open Source Code Node

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Resources



References



References

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.

